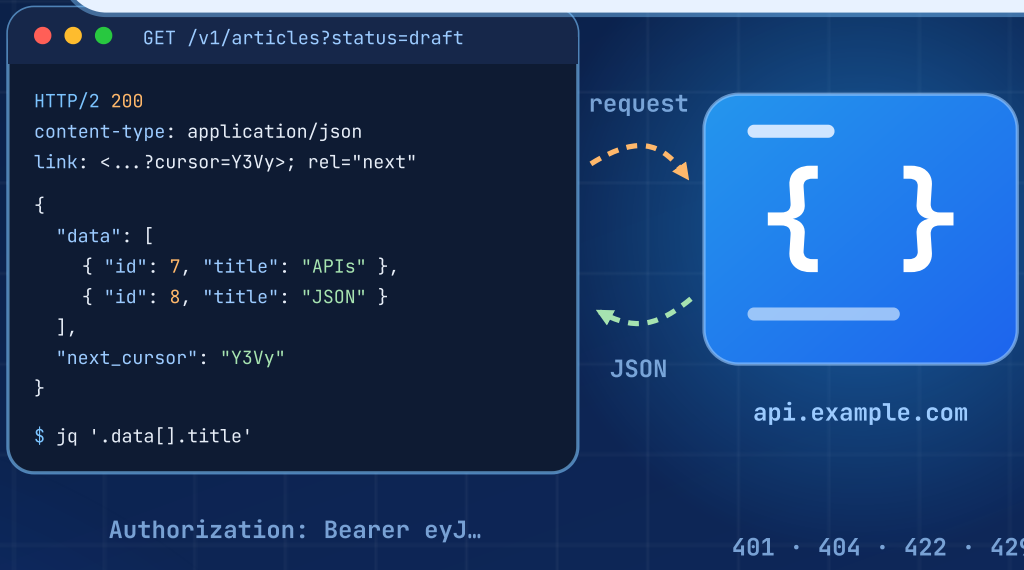
كتاب مذاكرة عملي · بالعامية المصري

APIs و JSON

إزاي تفهم أي API من جوه كمستخدم... وبعدين تفكّر زي اللي بيبنيه، قبل ما تكتب أول API في Node

من "يعني إيه API أصلاً" لحد ما تقرا أي دوكيومنتيشن وتجرّب أي Endpoint بـ curl و HTTPie و fetch، وتفصّص أي JSON بـ jq، وتفهم الـ Tokens والـ JWT و OAuth، وتعرف تصمّم REST API صح: الأسماء والـ Methods والـ Status codes والأخطاء والـ Pagination والـ Versioning والـ Rate limits والـ Webhooks، وتكتب Spec بـ OpenAPI. كل Header وكل Field وكل رمز متشرّح.

على الـ Mac · من الـ Specs والدوكيومنتيشن الرسمية (أكتوبر 2026)



problem+json · 201 Created · jq · { "json": true } · GET /v1/articles · webhooks · OpenAPI 3.2 · cursor · OAuth 2.0 · Bearer · JWT

كتابك للمذاكرة

من المستخدم للمصمّم

24 فصل · APIs حقيقية اتجربت فعلاً

+ Cheat Sheet وقاموس ولينكات الـ Specs الرسمية

المحتويات

الكتاب ده ماشي معاك على 7 مراحل. الأول هنفهم الـ API يعني إيه وأنواعه، وهنشرّح طلب API حقيقي. بعدين JSON بالتفصيل: القواعد والأخطاء و jq و JSON Schema. وبعدها هتتكلم مع APIs حقيقية كمستخدم بكل الأدوات، وتقرا الدوكيومنتيشن، وتفهم الـ Auth من API keys لحد OAuth. وفي النص التاني هتلبس طاقية اللي بيبني الـ API: تصميم REST كامل، والـ Webhooks، و OpenAPI. وفي الآخر جسر لـ Express ومشروعين: تستكشف API حقيقي من الأول للآخر، وتصمم Spec لـ API بتاعك.

إزاي تستخدم الكتاب ده؟ دليل الألوان

- **معلومة مهمة**: حاجة لازم تفضل فاكرها
- **خلي بالك**: غلطة شائعة أو فخ
- **حاجة مشتركة**: فكرة أو Flag بيتكرر في أكتر من مكان
- **إحالة**: رقم الجزء والصفحة اللي فيها الشرح الأصلي
- **مشكلة شائعة**: العَرَض والسبب والحل
- **ملخص + صح ولا غلط + أسئلة**: آخر كل فصل
- **Best Practice**: الطريقة الصح في الشغل الحقيقي
- **مثال عملي**: جرّبه بإيدك
- **لو حابب تتعمق**: تفاصيل زيادة تقدر تعدّيها أول مرة
- **تشريح**: كل حتة في الأمر أو الهيدر عليها رقم وشرح
- **تحديات الفصل**: سهل / متوسط / صعب، والحلول في الملحق

01 الحكاية من الأول: الـ API اتعمل ليه أصلاً؟ 8
1.1 قبل الـ APIs: النسخ واللزق 8
1.2 فاتطور حل: عقد بين برنامجين 9
1.3 الـ API موجود في كل حتة 10
1.4 أول طلب API بإيدك 10
1.5 الـ Consumer والـ Provider: طاقيتين 11
1.6 لو حابب تتعمق: حكاية الكلمات اللي هتسمعها 12

02 أنواع الـ APIs: REST و GraphQL و Webhooks وأصحابهم 14
2.1 ليه في أكتر من نوع؟ 14
2.2 قبل REST: SOAP و XML 14
2.3 REST: كل حاجة ليها عنوان 15
2.4 GraphQL: اسأل على اللي عايزه بالظبط 15
2.5 Webhooks: السيرفر هو اللي بيكلّمك 17
2.6 لمحة: WebSockets و SSE و gRPC 18
2.7 تختار إيه؟ 18

03 تشريح طلب API حقيقي: كل سطر رايح وكل سطر جاي 21
3.1 فكرة سريعة من كتاب الإنترنت 21
3.2 الطلب: 5 سطور رايحين 21
3.3 الرد: الـ Headers اللي تهم الـ API 23
3.4 الـ Body: إزاي تمشي في JSON كبير 24
3.5 طلب Conditional: "اتغير حاجة؟" 26

04 JSON: الحكاية والقواعد من المواصفة نفسها 29
4.1 الحكاية: من XML لـ JSON 29
4.2 المواصفتين: RFC 8259 و ECMA-404 30
4.3 الست أنواع، ومفيش غيرهم 30
4.4 الـ Strings: علامة التنصيص والـ Escapes 31
4.5 الأرقام: المسموح والممنوع 32
4.6 الـ Objects: حاجات المواصفة سايباها مفتوحة 32
4.7 الـ Encoding والـ MIME type 33
4.8 حاجات مش في المواصفة بس هتقابلها 33

05 JSON ضد JS Object، و JSON.parse و stringify، والأخطاء الشائعة 35
5.1 أول فرق: JSON نص، والـ Object في الذاكرة 35
5.2 الفروق في الصيغة جنب بعض 36
5.3 JSON.stringify(value, replacer, space) 37
5.4 JSON.parse(text, reviver) 38
5.5 الأخطاء الشائعة ورسايلها الحقيقية 39

06 jq على الماك: تفصّص أي JSON من الـ Terminal 42
6.1 ليه jq؟ 42
6.2 التسطيب على الماك 42
6.3 النقطة: أبسط فلتر 43
6.4 الوصول للحقول: key. و [n]. و []. 44
6.5 الـ Pipe جوه jq، وتبني شكل جديد 45
6.6 map و select وأصحابهم 46
6.7 الـ Flags وبناء JSON صح 47
6.8 تطلّع CSV و TSV ونصوص 48

07 JSON Schema: العقد مكتوب كـ JSON 51
7.1 المشكلة: if 50 51
7.2 أول Schema، سطر سطر 52
7.3 Validation حقيقي في Node بـ Ajv 54
7.4 $ref و $defs$: متكررش نفسك 55

08 الأدوات: curl للـ APIs و HTTPie و Bruno/Postman وتابة Network 58
8.1 curl للـ APIs: الـ Flags اللي هتعيش بيها 58
8.2 HTTPie: curl بس معمول للـ APIs 61
8.3 البرامج بالواجهة: Bruno و Postman و Insomnia 62
8.4 تابة Network: تصطاد الطلب وتعيده 63
8.5 تختار أنهي أداة؟ 64

09 APIs مجانية حقيقية تتمرّن عليها (واتجرّبت كلها) 66
9.1 الجدول الكبير 66
9.2 الـ Echo: شوف طلبك بعين السيرفر 68
9.3 CRUD وهمي: JSONPlaceholder و DummyJSON 69
9.4 Pagination بطريقتين: PokeAPI و Rick and Morty 70
9.5 Query parameters وأخطاء مفيدة: Open-Meteo 70
9.6 NASA :API key 71
9.7 درس حقيقي: لما API يقفل نسخة 71

10 الـ API من JavaScript: fetch صح من أول مرة 74
10.1 الفخ الأشهر: fetch مبيقعش على 404 74
10.2 الـ Response فيه إيه 75
10.3 POST بـ JSON 77
10.4 أنواع الفشل التلاتة 77
10.5 URLSearchParams: Query string صح 78
10.6 Helper واحد لكل مشروع 79
10.7 Node ضد المتصفح 81

11 تقرا أي API Docs، وتفهم ملف OpenAPI كمستخدم 83
11.1 ترتيب القراية: 8 محطات 83
11.2 تشريح صفحة Reference 84
11.3 OpenAPI: الدوكيومنتيشن ملف JSON 85
11.4 الصفحات اللي بتتولد من الملف 87

12 الـ Auth الجزء 1: API keys و Basic و Bearer، والـ JWT متشرّح 90
12.1 كلمتين لازم يفضلوا منفصلين 90
12.2 API keys: أبسط حاجة 90
12.3 Basic auth: Base64 مش تشفير 91
12.4 Bearer tokens 92
12.5 Sessions بالكوكيز ضد Tokens 93
12.6 الـ JWT: تذكرة عليها ختم 93
12.7 نعمل JWT بإيدنا ونحاول نزوّره 95

13 الـ Auth الجزء 2: OAuth 2.0 ببساطة، والـ CORS من ناحية السيرفر 99
13.1 الحكاية: التطبيقات اللي كانت بتطلب باسوردك 99
13.2 الأدوار الأربعة 100
13.3 PKCE + Authorization Code: الـ Flow الأساسي 100
13.4 باقي الـ Flows 103
13.5 CORS من ناحية الـ API 104

14 REST: الفكرة، والـ Resources، والـ Endpoints، والتسمية 107
14.1 API بالأفعال: الوجع 107
14.2 REST من المصدر: 6 قيود 108
14.3 قواعد التسمية 109
14.4 العمليات اللي مش CRUD 110
14.5 الـ IDs 111
14.6 نصمم: Content Library API 111

15 الـ Methods والـ Idempotency 113
15.1 الطلب اللي ضاع في النص 113
15.2 Safe و Idempotent 113
15.3 PUT ضد PATCH: تجربة 114
15.4 PATCH بإيه؟ Merge Patch ضد JSON Patch 115
15.5 Idempotency-Key: نخلّي POST آمن 117
15.6 الـ Custom method: :publish 118
15.7 QUERY: البحث اللي محتاج Body 119

16 الـ Status codes وتصميم الـ Body 121
16.1 "OK 200" وجوه Error 121
16.2 الأكواد اللي هتحتاجها 121
16.3 الأزواج اللي بتلخبط 123
16.4 تصميم الـ Body 124

17 شكل الـ Errors: Problem Details (RFC 9457) 128
17.1 كل API بلغة 128
17.2 الشكل 129
17.3 الـ Extensions: errors لكل Field 130
17.4 قراية الـ Problem من الـ Client 130
17.5 الـ 500: اللي ميطلعش برّه 131
17.6 Checklist لأي Error 132

18 الفلترة والترتيب والـ Pagination 134
18.1 ليه من أول يوم 134
18.2 الفلترة والترتيب والـ Fields 134
18.3 Page و Offset/Limit 135
18.4 الـ Cursor 136
18.5 الـ Link header (RFC 8288) 137

19 الـ Versioning والـ Rate limits والـ Concurrency 140
19.1 إيه اللي بيكسر؟ 140
19.2 طرق للـ Versioning 141
19.3 Deprecation و Sunset: تقفل بأدب 142
19.4 الـ Rate limits 143
19.5 الـ Lost update: ETag و If-Match 144

20 الـ Webhooks: لما الـ API هو اللي يكلمك 147
20.1 السؤال كل 5 ثواني 147
20.2 شكل الـ Delivery 147
20.3 التوقيع: ليه وإزاي 148
20.4 نبني الـ Receiver 149
20.5 ليه الـ Raw body؟ التجربة 150
20.6 التكرار والإعادة والسرعة 151
20.7 تجرّب على جهازك إزاي؟ 151

21 تكتب OpenAPI بإيدك 153
21.1 Design-first 153

21.2 الهيكل 153
21.3 Operation كامل 155
21.4 الـ Components 156
21.5 الـ Lint بـ Redocly CLI 157
21.6 ملف واحد، حاجات كتير 159
21.7 Contract test: مين بيكدب؟ 159

22 الجسر لـ Express: كل اللي فات في كود 162
22.1 الصورة الكبيرة 162
22.2 الـ req والـ res 163
22.3 Express 5: اللي اتغير 164
22.4 الـ Error middleware 165
22.5 الخريطة: الكتاب ← الكود 165
22.6 إيه اللي ناقص علشان Production؟ 166
22.7 الكود كامل: library.js 166

23 المشروع 1: تستكشف API حقيقي من الأول للآخر 173
23.1 الخطة: 8 خطوات 173
23.2 الخطوة 1: الـ Docs والـ Spec 173
23.3 الخطوات 2 و 3: أول طلب وشكل البيانات 174
23.4 الخطوة 7 بدري: الحدود مختلفة 174
23.5 مفاجأة: الـ Issues فيها PRs 175
23.6 الخطوة 8: الـ Script 175
23.7 ملف الملاحظات اللي هتسلّمه 177
23.8 دورك 177

24 المشروع 2: تصمم Content Library API 179
24.1 الـ Brief 179
24.2 الـ Endpoints 180
24.3 القرارات الصعبة 180
24.4 الـ Spec 181
24.5 الـ Checklist: 30 نقطة لأي API 183
24.6 التسليم 184

أ Cheat Sheet وجدول الـ Flags وجدول الرموز 186
أ.1 جدول الـ Flags 187
أ.2 جدول الرموز 188

ب قاموس المصطلحات 190

ج ملحق الروابط الرسمية 193
ج.1 IETF: RFCs والـ Drafts 193
ج.2 JSON و JSON Schema و jq 195
ج.3 OpenAPI والأدوات حواليه 195
ج.4 MDN 196
ج.5 Node و Express 197
ج.6 GitHub 197
ج.7 Stripe 199
ج.8 أدوات ومكتبات 199
ج.9 APIs عامة ومراجع تانية 200

د المصادر ومنهجية التحقق 202
د.1 تاريخ اللقطة 202
د.2 النسخ اللي اتجربت عليها 202
د.3 إيه اللي اتجرب وإزاي 202
د.4 حاجات لسه Draft أو جديدة 202
د.5 ملاحظات لقيناها وإحنا بنجرّب 203

ه الإجابات والحلول 204

خريطة الكتاب

بص على الطريق كله مرة واحدة قبل ما تبدأ. أول نص الكتاب انت **المستخدم** (Consumer): بتبعت طلبات وبتقرا ردود. وتاني نص انت **المصمّم**: بتقرر شكل الـ URLs والردود والأخطاء. والترتيب ده مقصود: مش هتعرف تصمم API كويس غير لما تكون اتعذبت من API وحش كمستخدم. الأرقام اللي جوه الدوايز أرقام المراحل، وفوقها أرقام الفصول.



الطريق من "يعني إيه API" لحد ما تصمم Spec كامل لـ API بتاعك، وانت جاهز تبنيه في Node.

1 · الفكرة

الـ API اتعمل ليه (الحكاية)، وأنواعه: Library و Web و REST و GraphQL و Webhooks، ولمحة عن gRPC و WebSockets، وتشريح طلب API حقيقي Header بـ Header.

2 · JSON بالتفصيل

القواعد من RFC 8259، والأنواع الست، والأخطاء الشائعة ورسايلها الحقيقية، و JSON ضد JS Object، و JSON.parse/stringify، و jq على الماك، و JSON Schema.

3 · تتكلم مع API

curl و HTTPie و Bruno/Postman/Insomnia وتابة Network، و APIs مجانية حقيقية للتمرين، و fetch من JS، وإزاي تقرا الدوكيومنتيشن وملف OpenAPI.

4 · الـ Auth

API keys و Basic و Bearer، والـ JWT متشرّح حتة حتة، و OAuth 2.0 بالـ Flows بتاعته ببساطة، والـ CORS من ناحية السيرفر.

5 · تصميم REST

الـ Resources والأسماء، والـ Methods والـ Idempotency، والـ Status codes والـ Bodies، والأخطاء بـ RFC 9457، والـ Filtering والـ Pagination، والـ Versioning والـ Rate limits.

6 · OpenAPI و Webhooks

السيرفر يكلّمك هو: التوقيع بـ HMAC والـ Retries والتكرار. وبعدين تكتب Spec بـ OpenAPI بإيدك وتعمله Lint.

7 · الجسر والمشروع

نفس الأفكار في Express على مستوى الصورة الكبيرة، ومشروع استكشاف API حقيقي من الأول للآخر، ومشروع تصميم REST API Spec كامل.

Best Practice

متقراش الكتاب وبس. افتح الـ Terminal جنبك وشغّل كل `curl` بإيدك على الـ APIs الحقيقية اللي في الكتاب: كلها مجانية وأغلبها من غير تسجيل. وخلي الـ **Cheat Sheet** و**قاموس المصطلحات** و**لينكات الـ Specs** جنبك وانت بتذاكر.

الكتاب ده مبني على كتابين قبله

أساسيات HTTP نفسها (الـ URL والـ Methods والـ Status codes والـ Headers والـ Cookies والـ CORS من ناحية المتصفح والـ Cache وتابة Network) اتشرحت في **كتاب الإنترنت و DevTools**، فهنا هنفكّرك بيها بسطر ونحيلك للفصل، ونركّز على الزاوية بتاعة الـ APIs. وكلام Python مع الـ APIs (`requests` والـ Retries والـ Rate limits في اسكريبتات الـ AI) اتشرح في **كتاب Python for AI**. ولما نقول "كتاب الإنترنت فصل 9" يبقى نقصد الفصل ده بالظبط.

خلي بالك: الأرقام دي صورة للحظة

الـ Specs (الـ RFCs و JSON و OpenAPI) ثابتة تقريباً. لكن الـ APIs العامة وحدودها وأسعار الأدوات وأرقام النسخ بتتغير. كل حاجة زي دي في الكتاب متاخدة من المصدر الرسمي ومتجرّبة يوم **23 أكتوبر 2026**، وجنبها سطر **المصدر** علشان تراجع بنفسك.

01

المرحلة 1 · الفكرة

الفصل الأول

الحكاية من الأول: الـ API اتعمل ليه أصلاً؟

قبل ما نتكلم عن REST و JSON و Tokens، لازم نفهم المشكلة اللي الـ API جه يحلّها. هنبدأ بحكاية فريق محتوى بينسخ ويلزق أرقام كل يوم، وبعدين برنامج بيعمل Scraping لصفحة ويتكسر، وبعدين الحل: باب معمول مخصوص للبرامج، وعليه عقد مكتوب.

بعد الفصل ده هتكون عارف

- المشكلة اللي قبل الـ APIs: النسخ واللزق والـ Scraping
- يعني إيه API بجد: عقد بين برنامجين
- الـ API موجود في كل حتة: Library و DOM و Web
- ليه كلمة Web APIs في MDN معناها حاجة تانية
- تبعت أول طلب API بإيدك وتفهم كل حتة فيه
- الفرق بين الـ Consumer والـ Provider، وليه الكتاب بيبدأ بالأول

1.1 قبل الـ APIs: النسخ واللزق

تخيل إنك في فريق محتوى بيكتب مقالات عن الأسعار، وكل مقالة لازم يبقى فيها سعر الدولار النهارده. فكل يوم الصبح، حد في الفريق بيفتح موقع البنك، ويدوّر على الرقم، وينسخه، ويلزقه في الـ CMS. ولو نسي يوم، المقالة بتطلع بسعر قديم. ولو بقوا 50 مقالة بـ 10 عملات، الموضوع بقى شغلانة لوحده.

فواحد من الفريق قال: "أنا هكتب اسكريبت يعمل ده لوحده". والاسكريبت بيعمل اللي الإنسان كان بيعمله بالظبط: يفتح الصفحة، وياخد الـ HTML، ويدوّر جواه على الرقم. والطريقة دي اسمها **Scraping**.

الصفحة اللي الاسكريبت بيدوّر فيها

```
<div class="rates-box">
  <span class="label">USD</span>
  <span class="value big-green">54.87</span>
</div>
```

والاسكريبت بيقول: "هات أول `span` عليه `class="value"` جوه `rates-box`". واشتغل أسبوعين. وبعدين **وهنا كانت المشكلة**: الموقع عمل تصميم جديد، والـ class بقى اسمه `rate-amount`، فالاسكريبت رجّع فاضي، والمقالات كلها طلعت من غير سعر. ومحدش في البنك غلطان: الصفحة دي معمولة **للبني آدمين**، وهم حرّين يغيّروا شكلها في أي وقت. محدش وعدك إن الـ HTML هيفضل زي ما هو.

مشاكل الـ Scraping

- بيتكسر مع أي تغيير في التصميم.
- بتنزّل صفحة كاملة (صور و CSS و JS) علشان رقم واحد.
- الرقم جاي كنص `"54.87"` وسط كلام، ومحتاج تنضّفه.
- كتير من المواقع بتمنعه في شروط الاستخدام، أو بتعمل Block للـ IP.

اللي كنا محتاجينه

- باب **معمول للبرامج** مش للعين.
- بيرجّع الرقم بس، في شكل ثابت وسهل.
- صاحبه **واعد** إن الشكل ده مش هيتغير فجأة.
- ومكتوب له دوكيومنتيشن: تبعت إيه وترجعلك إيه.

1.2 فاتطور حل: عقد بين برنامجين

الحل إن صاحب البيانات يفتح **باب تاني** مخصوص للبرامج: عنوان تبعتله سؤال بشكل متفق عليه، ويرجّعلك الإجابة بشكل متفق عليه. والباب ده اسمه **API**: اختصار **Application Programming Interface**، يعني "واجهة برمجة التطبيقات". وخلينا نفصّص الاسم:

`Application`	أي برنامج: اسكريبتك، أو موقع، أو تطبيق موبايل، أو سيرفر تاني.
`Programming`	بيستخدمه **كود**، مش إنسان بيدوس بالماوس.
`Interface`	"واجهة": نقطة التلاقي بين حاجتين. زي فيشة الكهربا: انت مش محتاج تعرف المحطة شغالة إزاي، محتاج بس تعرف شكل الفيشة.



شكل 1.1 الـ API هو الباب اللي في النص. انت مش شايف السيرفر من جوه (بيكلم Database ولا ملف ولا موديل AI)، وده بالظبط المقصود: صاحبه يقدر يغيّر كل اللي جوه من غير ما يكسرك، طالما الباب نفسه ثابت.

والكلمة المهمة هنا هي **العقد** (Contract). الـ API الكويس بيقولك بوضوح:

1. **تبعت فين**: العنوان، زي `/v2/rates`.
2. **تبعت إيه**: الـ Parameters، زي `base=USD`، وأنهي منها إجباري.
3. **هيرجعلك إيه**: شكل الرد بالظبط، وكل Field نوعه إيه.
4. **لو حصل غلط**: هيرجّع إيه، وإزاي تعرف السبب.
5. **الحدود**: تبعت كام طلب في الدقيقة، ومحتاج تسجّل ولا لأ.
6. **الوعد**: إمتى ممكن يتغير، وإزاي هيبلّغك (الـ Versioning، فصل 19).

> **تشبيه المطعم (بس متعلقش فيه)**
>
> انت قاعد في مطعم: المنيو هو **الدوكيومنتيشن**، والجرسون هو **الـ API**، والمطبخ هو **السيرفر**. انت مبتدخلش المطبخ، وبتطلب من الحاجات اللي في المنيو بس، بالطريقة اللي الجرسون فاهمها. والمطبخ ممكن يغيّر الطباخ أو الفرن من غير ما تحس. التشبيه حلو للبداية، بس الـ API الحقيقي أدق بكتير: كل كلمة في "الطلب" ليها شكل محدد، والرد بيرجع بـ Status code بيقولك نجح ولا لأ.

1.3 الـ API موجود في كل حتة

الكلمة مش خاصة بالإنترنت. أي حتة فيها **كود بيكلم كود تاني من خلال واجهة متفق عليها** فيها API. وانت بتستخدم APIs من أول يوم في كورس Jonas من غير ما تاخد بالك:

النوع	مثال	العقد فيه إيه
Library API (اللغة)	`[3, 1, 2].map(n => n * 2)`	`map` بتاخد Function وترجّع Array جديدة بنفس الطول.
Browser API	`document.querySelector(".btn")`	بتاخد CSS selector وترجّع أول Element أو `null`.
Node API	`fs.readFile(path, "utf8")`	المسار والـ Encoding، وبترجّع النص أو Error.
Python library API	`Path("a.md").read_text()`	من كتاب Python for AI.
Web API	`GET https://api.github.com/users/octocat`	عبر HTTP: عنوان و Method و Headers، والرد JSON. **ده موضوع الكتاب.**

جدول 1.1 APIs انت بتستخدمها أصلاً

الفرق الوحيد إن الـ Library API بيشتغل جوه نفس البرنامج (Function call)، أما الـ **Web API** فبيعدّي على الشبكة: طلبك بيخرج من جهازك كـ HTTP request، ويوصل لسيرفر في حتة تانية في الدنيا، والرد بيرجع. وعلشان كده كل اللي اتعلمته في كتاب الإنترنت (الـ DNS و TLS والـ Headers والـ Status codes) هيبان هنا تاني.

> **خلي بالك: كلمة "Web APIs" في MDN**
>
> لما تفتح MDN وتلاقي قسم اسمه **Web APIs**، ده **مش** معناه APIs على سيرفرات. MDN بيقصد الـ APIs اللي **المتصفح** بيدّيهالك في JavaScript: `fetch` و DOM و `localStorage` و Geolocation... أما اللي احنا بنتكلم عنه (سيرفر بيرد بـ JSON) فالناس بتسميه "Web API" أو "HTTP API" أو "REST API". نفس الكلمة، معنيين، فخد بالك من السياق.

المصدر MDN Glossary · API MDN · Web APIs (بتوع المتصفح)

1.4 أول طلب API بإيدك

كفاية كلام. خلينا نعمل نفس اللي الاسكريبت بتاع الفريق كان عايزه، بس من الباب الصح. في API مجاني اسمه **Frankfurter** بيدّي أسعار العملات، من غير تسجيل ولا Key. افتح الـ Terminal واكتب:

```
Terminal
user@MacBook ~ % curl -s "https://api.frankfurter.dev/v2/rates?base=USD&quotes=EGP,EUR"
[{"date":"2026-10-23","base":"USD","quote":"EGP","rate":54.868},{"date":"2026-10-
23","base":"USD","quote":"EUR","rate":0.89062}]
```

ده خرج حقيقي يوم كتابة الكتاب. بص على الفرق: مفيش HTML، ومفيش تصميم، ومفيش صور. الرقم نفسه `54.868` جاي كـ **رقم** مش نص، وجنبه التاريخ والعملة. وده شكل اسمه **JSON**، وهنقعد معاه 4 فصول. وخلينا نفصّص الأمر:

تشريح الأمر

```
curl -s "https://api.frankfurter.dev/v2/rates?base=USD&quotes=EGP,EUR"
```

1. `curl` برنامج بيبعت HTTP requests من الـ Terminal. جاي مع الماك جاهز، واتشرح بالتفصيل في كتاب الإنترنت (فصل 10)، وهنزوّد عليه حاجات الـ APIs في فصل 8.
2. `-s` "Silent": متطبعش شريط التقدم والرسايل. عايزين الرد بس.
3. `"…"` علامات التنصيص حوالين الـ URL مهمة جداً: من غيرها الـ Shell هيفهم `&` إنها "شغّل الأمر في الخلفية" ويقطع الـ URL.
4. `https://api.frankfurter.dev` الـ Host. وكتير من الشركات بتحط الـ API على Subdomain اسمه `api.` علشان تفصله عن الموقع العادي.
5. `/v2` رقم النسخة (Version) من الـ API. يعني في `v1` قديمة لسه شغالة. ودي أول مرة تشوف الـ Versioning، ولينا فيه فصل 19.
6. `/rates` الـ **Resource**: الحاجة اللي بتسأل عليها. هنا "الأسعار".
7. `?base=USD` أول **Query parameter**: العملة الأساسية. والـ `?` بتفصل المسار عن الـ Parameters.
8. `&quotes=EGP,EUR` تاني Parameter، والـ `&` بتفصل بين الـ Parameters. والقيمة فيها كذا عملة مفصولين بـ `,` : ودي طريقة اختارها صاحب الـ API ومكتوبة في الدوكيومنتيشن بتاعته، مش قاعدة عامة.

مثال عملي: نفس الطلب في المتصفح

الصق نفس الـ URL في Chrome. هتلاقي نفس الـ JSON ظاهر كنص. وده لأن أي request `GET` ممكن يتعمل من شريط العنوان. وجرّب تغيّر `quotes=EGP` لـ `quotes=SAR`، أو تشيل `base` خالص وتشوف هو بيفترض إيه. انت دلوقتي بتستكشف API، وده بالظبط اللي هنعمله في المشروع.

المصدر curl · man page Frankfurter · Docs

1.5 الـ Consumer والـ Provider: طاقيتين

في أي API في طرفين: الـ **Consumer** (أو Client): اللي بيستخدم الـ API (اسكريبتك أو موقعك). والـ **Provider**: اللي بانيه وشغّال على السيرفر بتاعه. وكورس Maximilian الجاي هيخليك **Provider**: هتكتب سيرفر Node بيرد على طلبات. بس الكتاب ده بيبدأ بيك **Consumer** عن قصد:

النص الأول: انت Consumer

- تبعت طلبات لـ APIs حقيقية (GitHub و PokeAPI و Open-Meteo...).
- تقرا الرد Header بـ Header و Field بـ Field.
- تقرا الدوكيومنتيشن وتعرف تـ Authenticate.
- وتحس بنفسك إيه اللي بيخلي API مريح وإيه اللي بيخليه مزعج.

النص التاني: انت Provider

- تختار أسماء الـ URLs والـ Methods.
- تقرر الـ Status code لكل حالة، وشكل الأخطاء.
- تصمم الـ Pagination والـ Versioning والـ Rate limits.
- وتكتب العقد بـ OpenAPI قبل ما تكتب سطر Express.

ليه الترتيب ده؟

أحسن مصممين APIs هم اللي اتعذبوا كتير كـ Consumers: API بيرجّع 200 وجواه `"error"`، أو Pagination مش واضحة، أو رسالة غلط بتقول "Something went wrong" وبس. ولما تحس بالمعاناة دي بنفسك، هتصمم API محدش يتعذب معاه.

1.6 لو حابب تتعمق: حكاية الكلمات اللي هتسمعها

REST و JSON جم منين؟

تقدر تعدّيه في أول قراية

REST: اسم اخترعه **Roy Fielding** في رسالة الدكتوراه بتاعته سنة **2000**. وهو نفسه من اللي كتبوا مواصفات HTTP/1.1. والرسالة كانت بتوصف "إيه اللي خلّى الويب ينجح ويكبر"، وطلّعت منه مجموعة قواعد سماها *Representational State Transfer*. وهنفصّصها في فصل 14.

JSON: اشتهر بإنه "اكتشفه" أوائل الألفينات: هو قال إنه مخترعهوش، اكتشفه، لأنه أصلاً جزء من JavaScript. وبقى Standard رسمي في **RFC 8259** و **ECMA-404**. وهنقرا الاتنين في فصل 4.

وقبل ده كان في APIs بـ **XML** و **SOAP**، وكانت تقيلة ومعقدة، وده جزء من سبب إن JSON + REST كسبوا. وهنقارن في الفصل الجاي.

المصدر RFC 8259 · JSON · json.org · Fielding (2000) · Architectural Styles… (REST)

ملخص الفصل

- قبل الـ APIs: نسخ ولزق يدوي، أو Scraping بيتكسر مع أي تغيير، لأن الـ HTML معمول للعين.
- الـ API = باب مخصوص للبرامج، وعليه عقد: تبعت فين وإيه، وهيرجع إيه، والأخطاء والحدود والوعد بالثبات.
- الـ APIs في كل حتة: `map` و DOM و `fs`، والـ **Web API** هو اللي بيعدّي على HTTP.
- "Web APIs" في MDN = APIs المتصفح، مش APIs السيرفرات.
- في أي URL لـ API: Host و Version و Resource و Query parameters (`?` و `&`).
- الكتاب: الأول Consumer، وبعدين Provider.

صح ولا غلط؟

1. الـ Scraping أضمن من الـ API لأنه بياخد نفس اللي المستخدم شايفه. صح / غلط
2. `document.querySelector` تعتبر API. صح / غلط
3. قسم Web APIs في MDN بيشرح إزاي تبني REST API على السيرفر. صح / غلط

؟ أسئلة مراجعة

1. اكتب 4 حاجات لازم تكون موجودة في "عقد" أي API.
2. ليه علامات التنصيص حوالين الـ URL في `curl` ضرورية لما يكون فيه `&` ؟
3. إيه الفرق بين Library API و Web API؟ وإدّي مثال على كل واحد.
4. إيه اللي يخلّي الـ Provider يقدر يغيّر السيرفر كله من غير ما يكسر الـ Consumers؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ابعت نفس طلب Frankfurter بس بـ `base=EUR` و `quotes=EGP,USD,SAR` ، وقارن الأرقام.
2. متوسط افتح `https://api.github.com/users/octocat` في المتصفح، واكتب 5 Fields في الرد وخمّن كل واحد معناه إيه.
3. صعب شيل `/v2` من الـ URL الـ Frankfurter وجرّب `/v1/latest?base=USD` . الشكل اختلف إزاي؟ ودوّر في صفحة الـ Docs على اللي مكتوب عن v1.

02

المرحلة 1 · الفكرة

الفصل التاني

أنواع الـ APIs: REST و GraphQL و Webhooks وأصحابهم

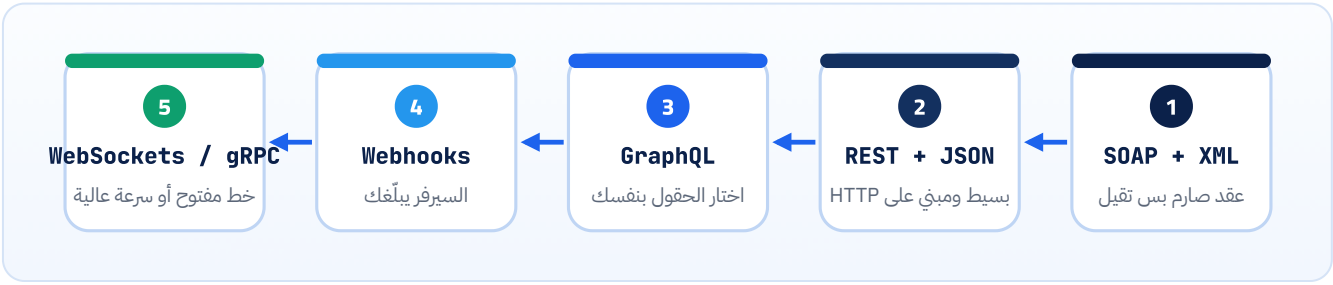
مش كل الـ Web APIs شكلها واحد. في اللي بتسأله على حاجة بعنوانها (REST)، وفي اللي بتكتبله سؤال بالظبط على الحقول اللي عايزها (GraphQL)، وفي اللي هو بيكلمك لما حاجة تحصل (Webhooks)، وفي اللي الخط بيفضل مفتوح بينكم (WebSockets). كل واحد اتعمل علشان يحل مشكلة معينة، وده اللي هنحكيه.

بعد الفصل ده هتكون عارف

- الفرق بين Library API و Web API تاني بس بالتفصيل
- SOAP و XML: ليه الناس هربت منهم
- REST: الـ Resources بالعنوان والـ Methods بتاعة HTTP
- GraphQL: Endpoint واحد وانت تختار الحقول، وتجربة على API حقيقي
- Webhooks: السيرفر هو اللي بيكلّمك
- لمحة عن gRPC و WebSockets و SSE وإمتى كل واحد
- جدول يختار لك النوع المناسب

2.1 ليه في أكتر من نوع؟

كل نوع من دول طلع لأن النوع اللي قبله كان بيوجع في حتة معينة. فبدل ما تحفظهم كقايمة، خلينا نمشي ورا الوجع: إيه المشكلة، وإيه اللي اتعمل علشانها.



شكل 2.1 مش خط زمني صارم ولا كل واحد بيلغي اللي قبله: النهارده هتلاقي الخمسة شغالين جنب بعض، وأغلب الـ APIs العامة JSON + REST، ومعاها Webhooks.

2.2 قبل REST: SOAP و XML

في أوائل الألفينات، الشركات الكبيرة كانت بتكلم بعض بـ **SOAP**. كل طلب عبارة عن "ظرف" XML كبير، وكل حاجة ليها تعريف صارم في ملف اسمه WSDL. والميزة إن العقد كان واضح جداً، والعيب إن طلب بسيط زي "هات سعر الدولار" كان بيبقى كده:

SOAP request (للفرجة بس)

```xml
<soap:Envelope xmlns:soap="http://www.w3.org/2003/05/soap-envelope">
  <soap:Body>
    <GetRate xmlns="http://example.com/rates">
      <Base>USD</Base>
      <Quote>EGP</Quote>
    </GetRate>
  </soap:Body>
</soap:Envelope>
```

كل الطلبات بتروح لنفس الـ URL بـ `POST`، واسم العملية جوه الـ XML. يعني HTTP كان مجرد "عربية نقل"، ومش مستفيدين من أي حاجة فيه (لا الـ Methods ولا الـ Status codes ولا الـ Cache). وهتقابل SOAP النهارده في أنظمة بنوك وحكومات قديمة، بس مش هتبني بيه حاجة جديدة.

2.3 REST: كل حاجة ليها عنوان

REST قال: "HTTP نفسه فيه كل اللي محتاجينه، استخدموه صح". فبدل عملية اسمها `GetRate` جوه XML، كل **حاجة** (Resource) ليها **عنوان** (URL)، والـ **Method** بتقول عايز تعمل فيها إيه:

HTTP

```
GET    /articles          ← هات كل المقالات
GET    /articles/42       ← هات المقالة 42
POST   /articles          ← اعمل مقالة جديدة
PATCH  /articles/42       ← عدّل جزء من المقالة 42
DELETE /articles/42       ← امسح المقالة 42
```

والرد بيرجع بـ **Status code** حقيقي (200 و 201 و 404...)، فأي أداة (المتصفح، و curl، والـ Cache، والـ Proxy) فاهمة اللي حصل من غير ما تقرا الـ Body. وده سر انتشاره: **مفيش حاجة جديدة تتعلمها فوق HTTP**. ومن فصل 14 لـ 19 كلهم عن تصميم REST صح.

> ⚠ **خلي بالك: مش كل اللي اسمه REST API هو REST**
>
> الناس بتقول "REST API" على أي API بيرجّع JSON على HTTP. بس REST الأصلي عند Fielding فيه قواعد أكتر بكتير (زي إن الرد نفسه يحتوي لينكات للخطوة الجاية، فصل 14). فلما تشوف API عنده `POST /getUserById`، ده "HTTP API بـ JSON" مش REST. والكتاب ده هيستخدم الكلمة زي ما الصناعة بتستخدمها، وهيقولك فين الفرق لما يهم.

2.4 GraphQL: اسأل على اللي عايزه بالظبط

القصة: فيسبوك سنة 2012 كان عنده تطبيق موبايل بيعرض Feed. وعلشان يرسم بوست واحد، كان محتاج البوست، وصاحبه، وصورته، وعدد اللايكات، وأول كومنتين. وبـ REST ده معناه طلبات كتير (`/posts/1` وبعدين `/users/7` وبعدين `/posts/1/comments`...)، وكل واحد بيرجّع Fields كتير مش محتاجها. والموبايل بنت ساعتها كان بطيء. والمشكلتين دول ليهم أسماء:

`Over-fetching`	بتاخد Fields أكتر من اللي محتاجه. زي ما شفنا في PokeAPI: `/pokemon/pikachu` بيرجّع **309 KB** و 21 Field في أول مستوى، وانت يمكن عايز الاسم والنوع بس.
`Under-fetching`	طلب واحد مش كفاية، فبتعمل طلبات ورا بعض (ودي اسمها مشكلة N+1 لو عملت طلب لكل عنصر في قايمة).

فاتطور حل: **GraphQL**. Endpoint **واحد** (غالباً `POST /graphql`)، وانت بتبعتله **Query** مكتوب فيها شكل الرد اللي عايزه بالظبط. والسيرفر عنده **Schema** بيوصف كل الأنواع والحقول المتاحة. جرّبها حالاً على API عام للدول:

Terminal

```
user@MacBook ~ % curl -s https://countries.trevorblades.com/graphql \
    -H 'Content-Type: application/json' \
    -d '{"query":"{ country(code: \"EG\") { name capital currency languages { name } } }"}'
{"data":{"country":{"capital":"Cairo","currency":"EGP","languages":
[{"name":"Arabic"}],"name":"Egypt"}}}
```

`-H 'Content-Type: application/json'`	بنقول للسيرفر إن الـ Body اللي جاي JSON. والـ `-H` = Header (كتاب الإنترنت فصل 9).
`-d '{...}'`	الـ Body. ووجود `-d` بيخلي curl يبعت `POST` لوحده. وحطيناه بين `'` علشان الـ Shell ميلعبش في الـ `"` اللي جوه.
`"query": "{ … }"`	نص الـ GraphQL query كـ String جوه JSON. وعلشان جواه `"EG"`، اضطرينا نكتب `\"EG\"` (فصل 4 هيشرح الـ Escaping).
`country(code: "EG")`	Field بياخد Argument، زي استدعاء فانكشن.
`{ name capital … }`	الـ **Selection set**: الحقول اللي عايزها. والرد رجع بيهم هم بس، بنفس الشكل.
`languages { name }`	علاقة: لغات الدولة، ومن كل لغة عايز الاسم بس. ده اللي كان هياخد طلب تاني في REST.

ولو سألت على Field مش موجود في الـ Schema، السيرفر بيرفض قبل ما ينفّذ. وبص على حاجة مهمة في الرد ده:

Terminal

```
user@MacBook ~ % curl -s https://countries.trevorblades.com/graphql -H 'Content-Type:
application/json' \
    -d '{"query":"{ country(code: \"EG\") { name population } }"}' -w ' %{http_code}'
{"errors":[{"message":"Cannot query field \"population\" on type \"Country\".","locations":
[{"line":4,"column":5}],"extensions":{"code":"GRAPHQL_VALIDATION_FAILED"}}]} 200
```

الـ Status code **200** مع إن الطلب فشل! والغلط جوه `"errors"`. وده من أشهر الفروق بين GraphQL و REST: كتير من سيرفرات GraphQL بترجّع 200 دايماً وتحط الأخطاء في الـ Body، فلازم تقرا الـ Body دايماً. (ومواصفة GraphQL-over-HTTP

الأحدث بتسمح بـ 4xx في حالات زي دي، بس متعتمدش عليه.)

بدل ما تلزق القيم في النص Variables

```
{
  "query": "query($c: ID!) { country(code: $c) { name currency } }",
  "variables": { "c": "JP" }
}
```

`query($c: ID!)`	بنعرّف Variable اسمه `$c` نوعه `ID`، والـ `!` = "إجباري، مينفعش null".
`"variables": {…}`	القيم نفسها في JSON منفصل. آمن وأنضف من إنك تركّب نص الـ Query بإيدك (زي SQL injection بالظبط).

GraphQL كويس في

- شاشات محتاجة بيانات من حاجات كتير مرتبطة.
- Clients كتير مختلفين (ويب وموبايل) كل واحد عايز Fields مختلفة.
- Schema بنوع واضح لكل حاجة، وأدوات بتكمّلك وانت بتكتب.

وبيوجع في

- الـ HTTP cache: كله `POST` لنفس الـ URL، فالـ CDN مش فاهم حاجة.
- الـ Rate limiting: Query واحدة ممكن تبقى تقيلة جداً.
- الأخطاء جوه الـ Body مع 200.
- أصعب في البداية من REST API صغير.

المصدر GitHub GraphQL API · graphql.org · Serving over HTTP · GraphQL Specification · graphql.org · Learn

2.5 Webhooks: السيرفر هو اللي بيكلّمك

في كل اللي فات، **انت** اللي بتسأل. طب لو عايز تعرف "أول ما حد يدفع"، أو "أول ما حد يفتح Issue على GitHub"؟ الحل الساذج إنك تسأل كل دقيقة: "حد دفع؟ حد دفع؟". ودي اسمها **Polling**، وأغلب الإجابات بتبقى "لأ"، يعني طلبات على الفاضي.

فاتطور حل: انت تدّي الخدمة **URL عندك**، وهي لما الحدث يحصل تبعتلك `POST` عليه فيه التفاصيل. يعني الأدوار اتقلبت: الخدمة بقت Client، وانت بقيت Server. ودي الـ **Webhooks**، وليها فصل كامل (20) لأن فيها مشاكل أمان وتكرار مهمة.



شكل 2.2 Webhook: الحدث بيحصل عند الخدمة، والخدمة بتبعت HTTP request لسيرفرك انت. ومن غير سيرفر شغّال على الإنترنت مش هتستقبل حاجة.

2.6 لمحة: WebSockets و SSE و gRPC

دول مش موضوع الكتاب، بس هتسمعهم كتير، فلازم تعرف كل واحد بيحل إيه:

الاسم	بيحل إيه	إزاي	هتقابله فين
WebSockets	محادثة في الاتجاهين بسرعة، من غير طلب جديد كل مرة	بيبدأ كـ HTTP request فيه `Upgrade: websocket`، والسيرفر يرد `101 Switching Protocols`، وبعدين الخط يفضل مفتوح والاتنين يبعتوا رسايل في أي وقت (`wss://`)	شات، ألعاب، Google Docs، الأسعار اللحظية
SSE (Server-Sent Events)	السيرفر يبعت تحديثات ورا بعض في اتجاه واحد	HTTP عادي بـ `Content-Type: text/event-stream`، والرد مبيخلصش، وكل حدث سطر `data: …`	الـ Streaming في الـ APIs الـ AI (كتاب Python for AI)، والإشعارات
gRPC	سيرفرات بتكلم بعض بسرعة جداً وبعقد صارم	العقد في ملف `.proto`، والبيانات Binary (Protocol Buffers) مش JSON، وفوق HTTP/2	جوه الشركات الكبيرة بين الـ Microservices، ونادراً لـ API عام

جدول 2.1 التلاتة باختصار

rates.proto (شكل عقد gRPC)

```
syntax = "proto3";

service Rates {
  rpc GetRate (RateRequest) returns (RateReply);
}

message RateRequest { string base = 1; string quote = 2; }
message RateReply   { double rate = 1; string date = 2; }
```

بص الفكرة: `rpc GetRate` = "استدعي فانكشن على سيرفر تاني (Remote Procedure Call)، و`= 1` و`= 2` مش قيم، دي **أرقام الحقول** في الـ Binary. قريب من SOAP في الفكرة (عملية باسمها) بس أخف وأسرع بكتير.

المصدر RFC 6455 · WebSocket · gRPC · Introduction · MDN · Using server-sent events · MDN · WebSockets API

2.7 تختار إيه؟

لو عايز...	استخدم
API عام بسيط، أو Backend لموقع، أو أول API في Node	**JSON + REST** (الكتاب ده)

لو عايز...	استخدم
شاشات معقدة بتجمع بيانات مرتبطة كتير، وفريق Frontend كبير	GraphQL
تعرف لما حاجة تحصل عند خدمة تانية	Webhooks (أو تعمل Polling لو مفيش)
تحديثات لحظية من السيرفر بس	SSE
محادثة لحظية في الاتجاهين	WebSockets
سيرفرات داخلية بتكلم بعض بسرعة	gRPC

جدول 2.2 جدول الاختيار السريع

حاجة مشتركة: كله فوق HTTP تقريباً

REST و GraphQL و Webhooks و SSE كلهم HTTP requests عادية: نفس الـ Headers ونفس الـ TLS ونفس تابة Network في DevTools. و WebSockets بيبدأ HTTP ويتحوّل، و gRPC فوق HTTP/2. فكل اللي في كتاب الإنترنت شغال هنا. ولما تفتح تابة Network هتلاقي فلتر اسمه **Fetch/XHR** للأولانيين، وفلتر **WS** لـ WebSockets.

ملخص الفصل

- SOAP: ظرف XML تقيل، كله `POST` لنفس الـ URL، و HTTP مجرد ناقل.
- REST: كل Resource ليه URL، والـ Method بتقول الفعل، والـ Status code بيقول النتيجة.
- GraphQL: Endpoint واحد، والـ Query بتحدد الحقول، وبيحل Over/Under-fetching، والأخطاء ممكن ترجع مع 200.
- Webhooks: الخدمة بتبعتلك `POST` لما حدث يحصل، بدل الـ Polling.
- WebSockets للاتجاهين، و SSE من السيرفر بس، و gRPC بين السيرفرات بـ Protobuf.

صح ولا غلط؟

1. في GraphQL كل Resource ليه URL مختلف. صح / غلط
2. رد GraphQL بـ 200 معناه إن الـ Query نجحت أكيد. صح / غلط
3. في الـ Webhook، الخدمة التانية هي اللي بتبعت الـ HTTP request لسيرفرك. صح / غلط

أسئلة مراجعة

1. اشرح Over-fetching و Under-fetching بمثال من عندك.
2. ليه الـ HTTP cache بيشتغل مع REST أحسن من GraphQL؟
3. إيه الفرق بين Polling و Webhook؟ وإمتى تضطر تعمل Polling؟
4. لو عايز تعرض ردّ موديل AI كلمة بكلمة، تختار WebSockets ولا SSE؟ وليه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل عدّل الـ GraphQL query بتاعة مصر وخليها ترجّع `name` و `continent { name }` .
2. متوسط استخدم `variables` واسأل عن 3 دول مختلفة بنفس الـ Query.
3. صعب اكتب نفس السؤال ("اسم الدولة وعاصمتها ولغاتها") بشكل REST: إيه الـ URLs اللي كنت هتعملها؟ وكام طلب محتاج؟

03

المرحلة 1 · الفكرة

الفصل التالت

تشريح طلب API حقيقي: كل سطر رايح وكل سطر جاي

في كتاب الإنترنت شرّحنا طلب HTTP لصفحة ويب. هنا هنعمل نفس الحكاية بس على API حقيقي: GitHub. هنبعت طلب واحد ونقرا كل Header رايح وكل Header راجع، ونفهم ليه الـ API محتاج كل واحد فيهم، ونمشي على الـ JSON اللي رجع Field بـ Field.

بعد الفصل ده هتكون عارف

- تقرا ناتج curl -v لـ API سطر سطر
- Accept و Content-Type و User-Agent و Authorization من ناحية الـ API
- Headers خاصة بالـ API: X-GitHub-Api-Version و x-ratelimit-* و link و etag
- تمشي على JSON كبير وتفهم شكله من غير ما تتوه
- طلب Conditional بـ If-None-Match ورد 304
- خطة التلات أسئلة لأي رد: نجح؟ إيه الشكل؟ فاضلّي قد إيه؟

3.1 فكرة سريعة من كتاب الإنترنت

أي HTTP request فيه: **سطر الطلب** (Method + Path + Version)، و**Headers**، و**Body** اختياري. وأي رد فيه: **سطر الحالة** (Version + Status code)، و**Headers**، و**Body**. ده كله اتشرح بالتفصيل في كتاب الإنترنت (فصول 7 و 8 و 9)، فمش هنعيده. اللي جديد هنا: **إيه اللي بيفرق طلب API عن طلب صفحة؟**

طلب صفحة (المتصفح)

- `Accept: text/html,…`
- الرد HTML للعرض.
- Cookies بتتبعت لوحدها.
- بيتبعه عشرات الطلبات (CSS و JS وصور).

طلب API (كود)

- `Accept: application/json` أو نوع خاص بالـ API.
- الرد JSON للكود.
- الهوية غالباً في `Authorization` header.
- طلب واحد بيرجّع بيانات وبس، ومعاه Headers بتقولك الحدود والصفحات.

3.2 الطلب: 5 سطور رايحين

GitHub عنده API عام تقدر تجرّبه من غير تسجيل (بحد 60 طلب في الساعة). هنسأله عن Repository بتاع Express، اللي هتشتغل بيه في كورس Maximilian:

Terminal

```
user@MacBook ~ % curl -sv -H "Accept: application/vnd.github+json" \
    -H "X-GitHub-Api-Version: 2026-03-10" \
    https://api.github.com/repos/expressjs/express -o repo.json
```

تشريح الأمر

```
curl -sv -H "Accept: application/vnd.github+json" -H "X-GitHub-Api-Version: 2026-03-10" https://api.github.com/repos/expressjs/express -o repo.json
```

1. `-sv` اتنين Flags مع بعض: `-s` اسكت عن شريط التقدم، و `-v` "Verbose": اطبع الطلب والرد كاملين بالـ Headers (بـ `>` للرايح و `<` للجاي).
2. `-H "Accept: …"` بنقول للسيرفر عايزين الرد بأنهي شكل. و GitHub بيطلب في الدوكيومنتيشن `application/vnd.github+json` . والـ `vnd.` = "Vendor": نوع خاص بشركة معينة، والـ `+json` = بس هو في الآخر JSON.
3. `-H "X-GitHub-Api-Version: …"` نسخة الـ API اللي عايزها، بتاريخ. ولو مبعتّوش، GitHub بيفترض النسخة القديمة `2022-11-28` (فصل 19).
4. `https://api.github.com` الـ Host بتاع الـ API، منفصل عن `github.com` بتاع الموقع.
5. `/repos/expressjs/express` الـ Resource: "الـ Repo اللي صاحبه `expressjs` واسمه `express`". اتنين Path parameters ورا بعض.
6. `-o repo.json` "Output": احفظ الـ Body في ملف بدل ما يتطبع في الشاشة، علشان نشوف الـ Headers لوحدها ونفصّص الـ JSON بعدين بـ jq.

الرايح (> في ناتج curl -v)

```
> GET /repos/expressjs/express HTTP/2
> Host: api.github.com
> User-Agent: curl/8.14.1
> Accept: application/vnd.github+json
> X-GitHub-Api-Version: 2026-03-10
```

`GET … HTTP/2`	`GET` = "هات" ومبيغيّرش حاجة. والمسار من غير الـ Host. و HTTP/2 اتفق عليه curl مع السيرفر أثناء TLS (كتاب الإنترنت فصل 11).
`Host`	إجباري. السيرفر الواحد ممكن يخدم كذا Domain.
`User-Agent`	مين بيكلّم. curl بيحطه لوحده. و**GitHub بيرفض أي طلب من غيره**: جرّبنا نشيله بـ `-H 'User-Agent:'` ورجع **403** ورسالة: *"Please make sure your request has a User-Agent header."*. وفي اسكريبتاتك حط اسم تطبيقك فيه.

`Accept`	الـ Content negotiation: "أنا بفهم الشكل ده". والسيرفر يرد بالشكل ده أو يقولك `406`.
`X-GitHub-Api-Version`	Header خاص بـ GitHub (الـ `X-` في الأول عُرف قديم للـ Headers الغير رسمية).

ومفيش `Authorization` هنا، فاحنا "مجهولين"، وده اللي مخلّي الحد 60 في الساعة. ولو بعتنا Token (فصل 12) الحد بيبقى 5000.

المصدر GitHub · Getting started with the REST API · GitHub · Troubleshooting (User-Agent required)

3.3 الرد: الـ Headers اللي تهم الـ API

الجاي (< في ناتج curl -v، مختصر)

```
< HTTP/2 200
< content-type: application/json; charset=utf-8
< cache-control: public, max-age=60, s-maxage=60
< vary: Accept,Accept-Encoding, Accept, X-Requested-With
< etag: W/"db10cba91645251533a85da8678ac26b20a3aeb72adfc7474f274afeabee7811"
< last-modified: Fri, 23 Oct 2026 10:17:04 GMT
< x-github-api-version-selected: 2026-03-10
< access-control-allow-origin: *
< x-ratelimit-limit: 60
< x-ratelimit-remaining: 47
< x-ratelimit-used: 13
< x-ratelimit-resource: core
< x-ratelimit-reset: 1792754886
< content-length: 6899
< x-github-request-id: 6E3C:2BED:12792C7:3C98199:6ADB436A
```

شلنا Headers الأمان العامة (`strict-transport-security` وأصحابها، كتاب الإنترنت فصل 12). وخلينا نقسم الباقي لـ 4 أسئلة:

Header	السؤال	القيمة هنا يعني إيه
`HTTP/2 200`	نجح؟	أيوه، والـ Body فيه اللي طلبته.
`content-type`	الـ Body إيه؟	JSON بـ UTF-8. **اقراه قبل ما تعمل parse**: لو جالك `text/html` يبقى غالباً صفحة غلط مش JSON.
`content-length`	حجمه؟	6899 byte.
`cache-control`	أقدر أخزنه؟	أيوه 60 ثانية (كتاب الإنترنت فصل 15). يعني متسألش أكتر من مرة في الدقيقة على نفس الحاجة.
`etag` و `last-modified`	اتغير ولا لأ؟	"بصمة" النسخة دي. هنستخدمها تحت في طلب Conditional. والـ `W/` = Weak: "نفس المعنى" مش لازم نفس البايتات.

Header	السؤال	القيمة هنا يعني إيه
`vary`	الرد بيختلف حسب إيه؟	حسب `Accept` وغيره. فالـ Cache لازم يخزّن نسخة لكل `Accept` مختلف.
`x-github-api-version-selected`	اتعاملت بأنهي نسخة؟	اللي طلبناها. لو مكناش بعتنا كان هيقول `2022-11-28` .
`access-control-allow-origin: *`	الـ JS في أي موقع يقدر يقراه؟	أيوه، أي Origin (CORS، فصل 13).
`x-ratelimit-limit`	حدّي كام؟	60 طلب في الساعة لـ `core` .
`x-ratelimit-remaining` / `used`	فاضل كام؟	47 فاضلين، و 13 استخدمناهم (كنا بنجرّب كتير).
`x-ratelimit-reset`	الحد هيتصفّر إمتى؟	رقم ثواني من 1970 (Unix time) = 11:28 UTC = **2:28 م بتوقيت القاهرة**. وتحوّله على الماك بـ `date -r 1792754886` .
`x-github-request-id`	لو اشتكيت؟	رقم الطلب عندهم. ابعته للدعم الفني لو في مشكلة.

جدول 3.1 كل Header والسؤال اللي بيجاوبه

date على الماك غير Linux

على الماك: `date -r 1792754886` (الـ `-r` بتاخد ثواني). وعلى Linux نفس الحاجة بـ `date -d @1792754886` . والفرق ده لأن `date` في الماك نسخة BSD، وفي Linux نسخة GNU. ونفس الكلام هيقابلك مع `sed` و `base64` .

Best Practice: خطة التلات أسئلة لأي رد

قبل ما تبص في الـ Body: (1) **نجح؟** بص على الـ Status. (2) **إيه الشكل؟** بص على `content-type` . (3) **فاضلّي قد إيه؟** دوّر على الـ Rate limit Headers. وبعدين بس افتح الـ JSON.

المصدر MDN · ETag · GitHub · Best practices · GitHub · Rate limits for the REST API

3.4 الـ Body: إزاي تمشي في JSON كبير

الملف فيه **85 Field** في أول مستوى. متقراهوش كله. اختار اللي يهمك (الأمر ده بـ jq، وهنشرحه حتة حتة في فصل 6):

Terminal

```
user@MacBook ~ % jq '{id, name, full_name, private, owner: {login: .owner.login, type:
.owner.type}, description, stargazers_count, open_issues_count, default_branch, topics:
.topics[:3], created_at, license: .license.spdx_id}' repo.json
```

النتيجة


```
{
  "id": 237159,
  "name": "express",
  "full_name": "expressjs/express",
  "private": false,
  "owner": {
    "login": "expressjs",
    "type": "Organization"
  },
  "description": "Fast, unopinionated, minimalist web framework for node.",
  "stargazers_count": 69577,
  "open_issues_count": 248,
  "default_branch": "master",
  "topics": [
    "express",
    "javascript",
    "nodejs"
  ],
  "created_at": "2009-06-26T18:56:01Z",
  "license": "MIT"
}
```


بـ Field: إيه اللي تتعلمه من كل واحد كمصمّم Field

1 **رقم ثابت** للـ Repo. الاسم ممكن يتغير (لو حد عمل Rename)، الـ `id` لأ. فلو هتخزّن مرجع في Database، خزّن الـ `id`.	`"id": 237159`
2 نفس المعلومة بشكلين للراحة. الـ `full_name` بيوفّر عليك تركّب `owner/name`.	`"name" / "full_name"`
3 Boolean حقيقي، مش `"false"` نص ولا `0`.	`"private": false`
4 **Object جوه Object** (Nested). صاحب الـ Repo نفسه Resource تاني، فـ GitHub حاطط ملخص منه هنا علشان متعملش طلب تاني.	`"owner": { … }`
5 الأسماء بـ `snake_case` (شرطة تحتية). APIs تانية بتستخدم `camelCase`. مفيش صح وغلط، المهم الثبات (فصل 16).	`"stargazers_count": 69577`
6 Array من Strings. ولو مفيش Topics هترجع `[]` مش `null`: ودي عادة كويسة.	`"topics": [ … ]`

7 التاريخ كـ String بصيغة **RFC 3339 / ISO 8601**. والـ `Z` = UTC. JSON معندوش نوع تاريخ (فصل 4)، فده العرف.	`"created_at": "2009-06-26T18:56:01Z"`
8 في الحقيقة دي Object كامل (`.license` فيها `key` و `name` و `spdx_id`...)، واحنا اللي اخترنا `spdx_id` بس. ولو الـ Repo من غير License، القيمة `null`، فخلي كودك جاهز.	`"license": "MIT"`

المصدر RFC 3339 · Date and Time on the Internet GitHub · Get a repository

3.5 طلب Conditional: "اتغير حاجة؟"

قلنا إن الـ `etag` بصمة. لو عايز تسأل كل شوية "الـ Repo اتغير؟"، متنزّلش الـ 6899 byte كل مرة. ابعت البصمة اللي معاك في `If-None-Match`:

Terminal

```
user@MacBook ~ % curl -s -i -H 'If-None-Match:
W/"7641f364f03df15614e181458c8ebae5a96dac49d08ccd9ddaead72f01911a0d"' \
    https://api.github.com/repos/expressjs/express
HTTP/2 304
x-ratelimit-remaining: 59
```

304 Not Modified ومفيش Body خالص: "اللي معاك لسه صح". وده بيوفر عليك وعلى السيرفر. وفي جملة مهمة في دوكيومنتيشن GitHub: الـ 304 **مبيتحسبش من الحد** لو الطلب كان بـ `Authorization`. ومن غيره ممكن يتحسب، فمتعتمدش على ده وانت مجهول.

> **خلي بالك: الـ ETag مربوط بالـ Accept**
>
> أول مرة جربنا، بعتنا ETag جاي من طلب بـ `Accept: application/vnd.github+json` في طلب من غيره، فرجع **200** مش 304. ليه؟ الـ `vary: Accept` بيقول إن الرد بيختلف حسب `Accept`، فكل واحد ليه ETag. فابعت نفس الـ Headers في الطلبين.

> **حاجة مشتركة: نفس الطلب من JS**
>
> كل اللي فوق بالظبط من Console المتصفح أو Node 18+:
>
> `sole.log(res.status, res.headers.get("x-ratelimit-remaining")); const repo = await res.json();`
>
> والـ `fetch` هنشرحه بالتفصيل في فصل 10.
>
> > **تشغيل JavaScript من الـ Terminal بـ Node**
> >
> > من هنا ورايح هنشغّل أمثلة كتير بـ **Node**: ده برنامج بيشغّل JavaScript برّه المتصفح، من الـ Terminal (لو مش متسطب عندك، شوف كتاب الـ Terminal فصل 11: `brew install fnm` ثم `fnm install 24`). اللي محتاج تعرفه دلوقتي بس:

- `node file.mjs` : شغّل الملف ده. الامتداد `.mjs` معناه إن الملف **ES Module**، فينفع جواه تكتب `import` وتكتب `await` في أول الملف على طول من غير دالة `async` .
- `import { readFileSync } from "node:fs"` : هات دالة جاهزة من مكتبة. اللي أولها `node:` جاية مع Node نفسه، والباقي محتاج `npm install اسمها` الأول، وبتتحط في `node_modules` .
- `node -e '...'` : شغّل سطر JavaScript واحد من غير ملف.
- `process.env.X` : اقرا متغير بيئة (كتاب الـ Terminal فصل 13).

والـ `await` نفسها اتشرحت في صندوق "Promise و async/await في دقيقة" في كتاب الإنترنت (فصل 7). والـ Modules و npm بالتفصيل في كتاب عالم الفرونت إند (فصول 13 و 17).

ملخص الفصل

- طلب الـ API = نفس HTTP، بس `Accept` بيطلب JSON، والهوية في `Authorization`، والرد بيانات بس.
- GitHub بيرفض أي طلب من غير `User-Agent` بـ 403.
- Headers الـ API المهمة: `content-type` و `etag` و `cache-control` و `x-ratelimit-*` و Header النسخة و Request ID.
- الـ `x-ratelimit-reset` = Unix time، وتحوّله بـ `date -r` على الماك.
- JSON الكبير: اختار الحقول بـ jq، وخزّن الـ `id` مش الاسم، والتواريخ ISO 8601 بـ `Z` .
- `If-None-Match` + ETag = 304 من غير Body لو مفيش تغيير.

صح ولا غلط؟

1. رد `304` بيبقى فيه نفس الـ Body بتاع 200. صح / غلط
2. `x-ratelimit-reset` بيقولك عدد الثواني الفاضلة لحد ما الحد يتصفّر. صح / غلط
3. الأحسن تخزّن `full_name` كمرجع لـ Repo بدل الـ `id` . صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `Accept` و `Content-Type` ؟ وكل واحد بيتبعت في الطلب ولا الرد؟
2. ليه ممكن تاخد 200 بدل 304 مع إنك باعت ETag صح؟
3. لو `content-type` رجع `text/html` من API المفروض بيرجّع JSON، تتوقع إيه حصل؟
4. إيه فايدة `x-github-request-id` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ابعت نفس الطلب لـ `https://api.github.com/users/octocat` واقرا الـ `x-ratelimit-remaining` قبل وبعد.
2. متوسط اعمل Conditional request بـ ETag للـ `users/octocat` وتأكد إنه رجّع 304.
3. صعب جرّب `-H "Accept: application/xml"`. GitHub رد بإيه؟ وإيه الـ Status code اللي كنت تتوقعه حسب HTTP؟

04

المرحلة 2 · JSON بالتفصيل

الفصل الرابع

JSON: الحكاية والقواعد من المواصفة نفسها

JSON شكله سهل لدرجة إن ناس كتير بتكتبه غلط وهي مش واخدة بالها. في الفصل ده هنقرا القواعد من المصدر: RFC 8259. الست أنواع، وكل حرف مسموح بيه، والـ Escaping، والأرقام وحدودها، والمسافات، والـ UTF-8، والحاجات اللي المواصفة سايباها مفتوحة وبتعمل مشاكل.

بعد الفصل ده هتكون عارف

- JSON جه منين وليه كسب XML
- RFC 8259 و ECMA-404: المواصفتين وعلاقتهم ببعض
- الست أنواع وشكل كل واحد بالظبط
- الـ Strings: علامات التنصيص والـ Escapes و \u
- الأرقام: المسموح والممنوع وحد الـ 2^53
- المسافات والـ UTF-8 والـ Keys المتكررة
- JSON Lines والتواريخ: حاجات مش في المواصفة بس في كل حتة

4.1 الحكاية: من XML لـ JSON

سنة Gmail 2004 طلع وعمل حاجة غريبة ساعتها: الصفحة بتجيب الإيميلات الجديدة **من غير ما تعمل Reload**. والتكنيك ده اتسمى **AJAX**: Asynchronous JavaScript **and XML**. يعني البيانات كانت بتيجي XML. والمشكلة إن الـ JS لازم يفصّص الـ XML ده: يدوّر على Tags، ويطلّع النص منها، ويحوّل `"42"` لرقم بنفسه. كتير وبطيء.

وهنا **Douglas Crockford** لاحظ حاجة بسيطة: JavaScript عندها أصلاً طريقة تكتب بيها بيانات: الـ Object literal `{ name: "Omar", age: 30 }`. طب ما نبعت البيانات بالشكل ده؟ والـ JS هيفهمها على طول. فأخد جزء صغير من صيغة JS، وشال منه أي حاجة فيها كود أو غموض، وسماه **JSON**: JavaScript Object Notation. وعمل له موقع من صفحة واحدة: json.org. والمواصفة كلها بتتكتب على كارت.

نفس البيانات بـ XML

```xml
<user>
  <name>Omar</name>
  <age>30</age>
  <admin>false</admin>
  <tags><tag>api</tag><tag>json</tag>
</tags>
</user>
```

كل حاجة نص. `30` نص، و `false` نص، والـ List محتاجة Tag لكل عنصر.

وبـ JSON

```json
{
  "name": "Omar",
  "age": 30,
  "admin": false,
  "tags": ["api", "json"]
}
```

الأنواع جوه الصيغة نفسها: رقم ورقم، و Boolean و Boolean، و List و List.

والنتيجة إن JSON بقى لغة البيانات بتاعة الويب. و**مش خاص بـ JavaScript**: Python و Go و Java وأي لغة عندها Parser ليه. وانت شفته في كتاب Python for AI بـ `json.loads` و `json.dumps`.

4.2 المواصفتين: RFC 8259 و ECMA-404

المواصفة	مين	فيها إيه
RFC 8259 (ديسمبر 2017)	IETF	الصيغة + قواعد التبادل على الشبكة: UTF-8 إجباري، والـ MIME type `application/json`، ونصايح عن الأرقام والـ Keys المتكررة. وهي **Internet Standard** (STD 90)، أعلى درجة في الـ RFCs.
ECMA-404	Ecma International (نفس اللي بتعمل JavaScript)	الصيغة بس (Grammar). و RFC 8259 بيقول صراحة إن الاتنين متفقين على نفس الصيغة.
json.org	Crockford	نفس الصيغة برسومات. أسهل حاجة تبص عليها.

جدول 4.1 مين بيعرّف JSON

المصدر `json.org` `ECMA-404` `RFC 8259 · The JSON Data Interchange Format`

4.3 الست أنواع، ومفيش غيرهم

أي قيمة في JSON لازم تكون واحدة من دول. ومفيش تاريخ، ولا `undefined`، ولا Function، ولا تعليقات:

النوع	شكله	مثال	ملاحظة
object	`{ "key": value, … }`	`{"id": 7}`	الـ Key لازم **String** بـ `"`.
array	`[ value, … ]`	`[1, "a", null]`	مرتّب، ومسموح أنواع مختلفة جوه.

النوع	شكله	مثال	ملاحظة
string	`"…"`	`"APIs و JSON"`	`"` بس، مش `'`.
number	عشري	`-12.5e3`	مفيش NaN ولا Infinity.
boolean	`true` / `false`	`true`	حروف صغيرة بس.
null	`null`	`null`	"مفيش قيمة" عن قصد.

جدول 4.2 أنواع القيم في JSON



شكل 4.1 الـ JSON text كله قيمة واحدة. ومن RFC 7159 (2014) القيمة دي ممكن تبقى أي نوع، حتى رقم لوحده أو String لوحده. قبلها كان لازم Object أو Array، وهتلاقي أدوات قديمة لسه فاكرة كده.

والـ **Structural characters** ستة بس: `[` و `]` و `{` و `}` و `:` و `,`. والمسافات حواليهم مسموحة، بس **أربع أنواع مسافات بس**: Space، و Tab، و سطر جديد (LF)، و Carriage return (CR). يعني الـ Non-breaking space اللي بيتنسخ ساعات من Word أو من صفحة ويب بيبوّظ الـ JSON.

4.4 الـ Strings: علامة التنصيص والـ Escapes

الـ String بيبدأ وبيخلص بـ `"`. طب لو عايز `"` جواه؟ أو سطر جديد؟ هنا بييجي الـ **Escape**: الـ `\` بيقول "الحرف اللي بعدي ليه معنى خاص":

`\"`	علامة تنصيص جوه النص: `"he said \"hi\""`.
`\\`	الـ Backslash نفسه. ومشهورة في مسارات Windows: `"C:\\Users"`.
`\/`	Slash. **مسموح بس مش إجباري**: `"/"` و `"\/"` نفس القيمة. وبعض الـ APIs القديمة (PHP) بتطلّعها كده.
`\n \r \t`	سطر جديد، و Carriage return، و Tab.
`\b \f`	Backspace و Form feed (نادراً).
`\u0639`	أي حرف برقمه في Unicode (4 أرقام Hex). `"\u0639\u0645\u0631"` = `"عمر"` (جربناها في Node). والإيموجي أكبر من 4 أرقام، فبيتكتب كـ "Surrogate pair": `"\uD83D\uDE00"`.

> **خلي بالك: السطر الجديد الحقيقي ممنوع جوه String**
>
> الـ Control characters (من U+0000 لـ U+001F، ومنهم الـ Enter والـ Tab الحقيقيين) **لازم** يتكتبوا Escaped. فلو لزقت نص فيه Enter جوه `"..."`، الـ Parser هيقع. وفي Node بيطلع: `SyntaxError: Bad control character in string literal in JSON at position 12`. وده بيحصل كتير لما حد يركّب JSON بإيده من نص منسوخ.

4.5 الأرقام: المسموح والممنوع

الصيغة: سالب اختياري، وبعدين جزء صحيح، وبعدين كسر اختياري، وبعدين أُس اختياري:

number (مبسّط، من RFC 8259)

```
number = [ "-" ] int [ "." 1*DIGIT ] [ ("e" / "E") [ "+" / "-" ] 1*DIGIT ]
int    = "0" / ( digit1-9 *DIGIT )
```

`[ "-" ]`	الأقواس المربعة في الـ Grammar = اختياري. فالسالب مسموح، بس **الموجب بـ `+` ممنوع**: `+1` غلط.
`"0" / ( digit1-9 *DIGIT )`	يا صفر لوحده، يا رقم يبدأ من 1 لـ 9 وبعده أي أرقام. يعني `012` **ممنوع** (Leading zero).
`"." 1*DIGIT`	النقطة لازم يبقى بعدها رقم واحد على الأقل (`1*` = واحد أو أكتر). **وقبلها لازم رقم**: `.5` غلط، اكتب `0.5`. و `5.` غلط برضه.
`e / E`	الأُس العلمي: `1.5e3` = 1500.

وممنوع كمان: `NaN` و `Infinity` و Hex زي `0x1F`. ومفيش فرق في JSON بين Integer و Float: كله "number".

> **حد الـ 2^53: ليه IDs بتيجي كـ String**
>
> تقدر تعدّيه في أول قراية
>
> المواصفة نفسها مش حاطة حد للأرقام، بس بتقول إن أغلب البرامج بتخزّنها كـ IEEE 754 double، وإن الأرقام الصحيحة الآمنة هي من $-(2^{53}-1)$ لـ $2^{53}-1$، يعني `9007199254740991` (ده `Number.MAX_SAFE_INTEGER` في JS). جربنا في Node: `JSON.parse('{"big": 12345678901234567890}').big` رجّع `12345678901234567000` : آخر أرقام اتغيرت من غير أي Error! وعلشان كده Twitter زمان كان بيبعت كل ID مرتين: `id` رقم، و `id_str` نص. والقاعدة: **IDs كبيرة؟ ابعتها String.**

4.6 الـ Objects: حاجات المواصفة سايباها مفتوحة

- **الـ Keys المتكررة**: المواصفة بتقول الـ Keys "SHOULD be unique"، يعني "ينبغي" مش "لازم". ولو اتكررت، السلوك مش مضمون. `JSON.parse('{"a":1,"a":2}')` في Node رجّع `{"a":2}` (الأخير بيكسب)، ولغات تانية ممكن تاخد الأول أو ترمي Error. فمتعتمدش عليه أبداً.
- **الترتيب**: ترتيب الـ Keys "مش مهم" في المواصفة. يعني `{"a":1,"b":2}` و `{"b":2,"a":1}` نفس البيانات، ومتكتبش كود بيعتمد على الترتيب.

- **Array فاضية و Object فاضي**: `[]` و `{}` مسموحين.

4.7 الـ Encoding والـ MIME type

- RFC 8259: الـ JSON اللي بيتبادل بين أنظمة مختلفة **MUST** يبقى **UTF-8**. وممنوع تحط BOM في أوله. فالعربي يتكتب عادي: `"عمر"`، ومش محتاج `\u`.
- الـ MIME type الرسمي: `application/json`. والـ `charset=utf-8` اللي بتشوفه في Headers كتير (زي GitHub) ملوش معنى رسمياً لـ JSON (لأنه UTF-8 دايماً)، بس مبيضرش.
- أنواع تانية مبنية على JSON بـ `+json` في الآخر: `application/problem+json` (فصل 17)، و `application/vnd.github+json` (فصل 3)، و `application/merge-patch+json` (فصل 15).

> **I-JSON: نسخة أصرم للـ APIs**
>
> في RFC اسمه **RFC 7493 (I-JSON)** بيقول: لو بتصمم API، التزم بالقيود دي: UTF-8، ومفيش Keys متكررة، والأرقام في حدود الـ double، والتواريخ بصيغة RFC 3339. وده بالظبط اللي هنعمله في نص الكتاب التاني.

المصدر RFC 8259 · §6 Numbers · RFC 8259 · §8.1 Character Encoding · RFC 7493 · I-JSON

4.8 حاجات مش في المواصفة بس هتقابلها

التواريخ

مفيش نوع تاريخ. العرف: String بصيغة **RFC 3339** زي `"2026-10-23T11:00:00Z"` (الـ `Z` = UTC) أو `"2026-10-23T14:00:00+03:00"`. وأحياناً رقم Unix time (زي `x-ratelimit-reset`). والمهم: **نوع واحد في الـ API كله**، ومع Timezone دايماً.

(NDJSON) JSON Lines

ملف كل سطر فيه JSON كامل لوحده. مش JSON واحد صحيح كملف، بس سهل تضيف عليه سطر وتقراه سطر سطر. وانت استخدمته في كتاب Python for AI للـ Resume والـ Batch API. وبيتسمى `.jsonl`.

results.jsonl

```
{"file": "a.md", "ok": true}
{"file": "b.md", "ok": false, "error": "timeout"}
```

المصدر RFC 3339 · §5.6 Internet Date/Time Format · jsonlines.org

ملخص الفصل

- JSON = جزء صغير من صيغة JS بيانات بس، وبقى Standard في RFC 8259 و ECMA-404.
- ست أنواع: object و array و string و number و boolean و null. والـ Keys Strings بـ `"` .
- الـ Escapes: `\" \\ \/ \n \r \t \b \f \uXXXX`، والـ Control characters لازم Escaped.
- الأرقام: مفيش `+` ولا Leading zero ولا `.5` ولا NaN. وأكبر من `2^53-1` ممكن يتغير بهدوء.
- الـ Keys المتكررة مش مضمونة، والترتيب مش مهم.
- UTF-8 إجباري، و `application/json`، والتواريخ RFC 3339 كـ String.

صح ولا غلط؟

1. `"42"` لوحده JSON text صحيح. صح / غلط
2. `{"price": .99}` JSON صحيح. صح / غلط
3. الـ JSON لازم يتكتب فيه العربي بـ `\u` . صح / غلط

أسئلة مراجعة

1. اكتب 5 أرقام ممنوعة في JSON، وصلّح كل واحد.
2. ليه API بيبعت ID كبير كـ String؟
3. إيه اللي بيحصل في Node لو JSON فيه نفس الـ Key مرتين؟ وليه متعتمدش على ده؟
4. إيه الفرق بين ملف `.json` وملف `.jsonl` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب JSON بيوصف كتاب: العنوان، والسنة، والمؤلفين (List)، ومتاح ولا لأ، والسعر، والـ ISBN (ليه String؟).
2. متوسط اكتب String فيه: علامة تنصيص، وسطر جديد، و Backslash، وحرف عربي بـ `\u` . وتأكد منه بـ `JSON.parse` في الـ Console.
3. صعب افتح RFC 8259 §9 (Parsers) واقرا إيه اللي مسموح للـ Parser يرفضه. واكتب جملتين بالعامية عن اللي فهمته.

05

المرحلة 2 · JSON بالتفصيل

الفصل الخامس

JSON ضد JS Object، و JSON.parse و stringify، والأخطاء الشائعة

في كورس Jonas بتكتب Objects كل يوم، فطبيعي تفتكر إن JSON هو نفس الحاجة. هو قريب جداً، بس الفروق الصغيرة دي هي اللي بتعمل 80% من أخطاء الـ JSON. هنحطهم جنب بعض، وبعدين نشوف JSON.parse و JSON.stringify بالـ Parameters بتاعتهم، وبعدين جدول بكل غلطة ورسالتها الحقيقية في Node 24.

بعد الفصل ده هتكون عارف

- الفرق بين JSON text و JS Object في الذاكرة
- 7 فروق في الصيغة جدول واحد
- JSON.parse و reviver، و JSON.stringify و replacer و space
- إيه اللي بيضيع في stringify: undefined و Functions و Date و NaN و BigInt
- رسايل الأخطاء الحقيقية في Node 24 وتفهم كل واحدة
- غلطة Unexpected token '<' من res.json() وسببها

5.1 أول فرق: JSON نص، والـ Object في الذاكرة

ده أهم سطر في الفصل: **JSON دايماً String**. نص بيتبعت على الشبكة أو بيتحفظ في ملف. أما الـ Object في JS فحاجة عايشة في الذاكرة، ليها Methods و Prototype، وممكن يبقى فيها Functions. والمترجم بينهم اتنين:



شكل 5.1 الرحلة الكاملة لأي بيانات في API: الـ Client بيعمل stringify وبيبعت، والسيرفر بيعمل parse ويشتغل، وبعدين stringify للرد، والـ Client يعمل parse. وفي Express الكلام ده بيحصل لوحده في express.json() و res.json() (فصل 22).

Console · Chrome DevTools

```
> const user = { name: "Omar", age: 30 }
> typeof user
'object'
> const text = JSON.stringify(user)
> text
'{"name":"Omar","age":30}'
> typeof text
'string'
> JSON.parse(text).name
'Omar'
```

5.2 الفروق في الصيغة جنب بعض

الحاجة	JS Object	JSON
الـ Keys	`{ name: "x" }` من غير تنصيص عادي	**لازم** `{ "name": "x" }`
علامة التنصيص	`'` أو `"` أو `` ` ``	`"` بس
الفاصلة في الآخر	`[1, 2, 3,]` مسموحة	ممنوعة
التعليقات	`// …` و `/* … */`	مفيش تعليقات خالص
القيم	`undefined` و Functions و `NaN` و Date و `Map` …	الست أنواع بس
الأرقام	`.5` و `0x1F` و `1_000` و `10n`	عشري بالصيغة الصارمة بس
Keys محسوبة / مختصرة	`{ [k]: 1, name }`	لأ: النص مكتوب حرفياً

جدول 5.1 JS Object literal ضد JSON

ليه JSON صارم كده؟

علشان أي لغة تقدر تكتب له Parser صغير وسريع ومن غير غموض. كل فاصلة زيادة أو تعليق بيفتح سؤال "طب ده معناه إيه؟". وعلشان كده ملفات زي `package.json` مينفعش فيها تعليقات. (في صيغ تانية زي JSONC اللي VS Code بيستخدمه في `settings.json`، و JSON5، بيسمحوا بتعليقات، بس دول مش JSON، ومينفعش تبعتهم لـ API.)

5.3 JSON.stringify(value, replacer, space)

Console · Chrome DevTools

```
> JSON.stringify({ name: "عمر", tags: ["a", "b"] }, null, 2)
{
  "name": "عمر",
  "tags": [
    "a",
    "b"
  ]
}
> JSON.stringify({ id: 1, password: "x", name: "a" }, ["id", "name"])
'{"id":1,"name":"a"}'
> JSON.stringify({ a: 1, b: "x" }, (key, value) => typeof value === "number" ? value * 10 :
value)
'{"a":10,"b":"x"}'
```

`value`	الحاجة اللي هتتحوّل.
`replacer = null`	مش عايز أغيّر حاجة. و `null` هنا بس علشان نوصل للـ Parameter التالت.
`replacer = ["id", "name"]`	Array = "الـ **Keys دي بس**". طريقة سريعة تشيل حاجة حساسة زي `password` قبل ما تطبع أو تبعت. (بس في سيرفر حقيقي، الأحسن تبني الرد من الأول بالحقول اللي عايزها.)
`replacer = (key, value) => …`	Function بتتنادى على كل Key، واللي ترجّعه هو اللي يتكتب. ولو رجّعت `undefined` الـ Key بيختفي.
`space = 2`	Pretty print بمسافتين. من غيره الناتج سطر واحد (وده اللي بيتبعت على الشبكة عادة، أصغر). وممكن تحط `"\t"` . والعربي بيطلع عادي من غير `\u` .

اللي بيضيع أو بيتغير

Console · Chrome DevTools

```
> JSON.stringify({ a: undefined, b: () => 1, c: Symbol("x"), d: NaN, e: Infinity, f: new
Date(Date.UTC(2026, 9, 23, 11)), g: [undefined, () => 1], h: null, i: -0 })
'{"d":null,"e":null,"f":"2026-10-23T11:00:00.000Z","g":[null,null],"h":null,"i":0}'
> JSON.stringify({ n: 10n })
Uncaught TypeError: Do not know how to serialize a BigInt
> const o = {}; o.self = o; JSON.stringify(o)
Uncaught TypeError: Converting circular structure to JSON
> JSON.stringify(new Map([["a", 1]]))
'{}'
```

القيمة	جوه Object	جوه Array
`undefined` و Function و Symbol	الـ Key **بيختفي**	بيبقى `null`
`NaN` و `Infinity`	`null`	`null`
`Date`	String بصيغة ISO (لأن Date عنده `toJSON()`)، ويترجع من parse مش Date **String**	
`Map` و `Set`	`{}` فاضي! حوّلهم بنفسك: `Object.fromEntries(map)` و `[...set]`	
`BigInt`	TypeError	
Object بيشاور على نفسه	circular structure :TypeError	

جدول 5.2 stringify بيعمل إيه في كل حاجة مش JSON

> ⚠ **خلي بالك: undefined مش null**
>
> لو عندك `{ title: "x", summary: undefined }` وعملت stringify، الـ `summary` **بتختفي** من الـ JSON. فالسيرفر مش هيعرف إنك "مسحت" الـ Summary، هيفتكر إنك مبعتهاش. ولو عايز تقول "فاضية عن قصد"، ابعت `null` . وده هيفرق جداً في `PATCH` (فصل 15).

5.4 JSON.parse(text, reviver)

Console · Chrome DevTools

```
> const r = JSON.parse('{"d":"2026-10-23T11:00:00Z"}', (key, value) => key === "d" ? new
Date(value) : value)
> r.d instanceof Date
true
> JSON.parse('{"big": 12345678901234567890}', (key, value, ctx) => key === "big" ?
BigInt(ctx.source) : value).big
12345678901234567890n
```

`reviver (key, value)`	نفس فكرة الـ replacer بالعكس: بتتنادى على كل قيمة بعد ما تتقري، واللي ترجّعه بيتحط مكانها. هنا حوّلنا الـ String لـ `Date` .
`ctx.source`	Parameter تالت جديد (*JSON.parse source text access*)، شغال في Node 24: بيدّيك **النص الأصلي** للرقم قبل ما يتحوّل لـ double. فقدرنا نعمل `BigInt` بالرقم الصح من غير ما آخره يضيع. ولسه مش موجود في كل المتصفحات، فاتأكد قبل ما تستخدمه في Frontend.

> 🔍 **لو حابب تتعمق: __proto____ في JSON**
>
> تقدر تعدّيه في أول قراية
>
> جربنا `JSON.parse('{"__proto__": {"admin": true}}').admin` ورجّع `undefined` : الـ parse بيعمل `__proto__` كـ Key عادي، مش بيغيّر الـ Prototype. بس لو خدت الناتج ده وعملت له Merge بكود ساذج (

`target[key] = source[key]` بشكل Recursive)، ممكن تلوّث `Object.prototype` كله. ودي ثغرة مشهورة اسمها **Prototype pollution**، وعلشان كده متعملش Merge لـ JSON جاي من برّه من غير مكتبة موثوقة.

المصدر `MDN · JSON` `MDN · JSON.parse()` `MDN · JSON.stringify()`

5.5 الأخطاء الشائعة ورسايلها الحقيقية

كل الرسايل دي طالعة فعلاً من `JSON.parse` في **Node 24.22** (نفس محرك V8 اللي في Chrome، فالرسايل قريبة جداً في الـ Console). و`position` = رقم الحرف من الصفر، و`line/column` من واحد:

اللي اتكتب	الرسالة	الحل
`{"name": "Omar",}`	Expected double-quoted property name in JSON at position 16	شيل الفاصلة الأخيرة: بعدها كان مستني Key.
`[1, 2, 3,]`	Unexpected token ']', "[1, 2, 3," is not valid JSON	نفس الحكاية في Array.
`{'name': 'Omar'}`	Expected property name in JSON at position 1 (or '{')	`'` ممنوعة: استخدم `"` .
`{name: "Omar"}`	Expected property name in JSON at position 1 (or '{')	الـ Key محتاج `"` .
`{"a": 1 // c}`	Expected property value in JSON at position 8 (or '{' or ',') after	شيل التعليق.
`{"n": NaN}`	Unexpected token 'N', ... is not valid JSON	`null` أو شيل الـ Key.
`{"n": 012}`	Unexpected number in JSON at position 7	`12` أو `"012"` لو كود بصفر.
`{"x": .5}`	Unexpected token '.', ... is not valid JSON	`0.5` .
Enter حقيقي جوه String	Bad control character in string literal in JSON at position 12	`\n` .
`{"a":1}{"b":2}`	Unexpected non-whitespace character after JSON at position 7	اتنين JSON ورا بعض: حطهم في Array، أو ده JSONL واقراه سطر سطر.

اللي اتكتب	الرسالة	الحل
نص فاضي `""`	Unexpected end of JSON input	الرد فاضي (غالباً 204 أو Error). اتأكد من الـ Status قبل الـ parse.
`undefined`	is not valid "undefined" JSON	بتعمل parse لمتغير قيمته `undefined` (بيتحوّل للنص `"undefined"`). اتأكد إن المتغير اتملا. (و `JSON.parse(null)` بالمناسبة بيرجّع `null` من غير Error، لأن `"null"` JSON صحيح.)
`<!DOCTYPE html>…`	Unexpected token '<', "<!" is not valid ... " DOCTYPE JSON	الأشهر! تحت.

جدول 5.3 الغلطة ← الرسالة ← الحل

مشكلة 1 **SyntaxError: Unexpected token '<', "<!" **

العَرَض: كتبت `await res.json()` ووقع بالرسالة دي.

السبب: السيرفر رجّع **صفحة HTML** مش JSON، والحرف الأول فيها `<`. ده بيحصل لما: الـ URL غلط فالسيرفر رجّع صفحة 404 بتاعته، أو الـ Dev server (زي Vite) رجّع `index.html` لأي مسار مش موجود، أو في Proxy/Login page في النص، أو السيرفر وقع ورجّع صفحة 500.

الحل: قبل `res.json()` بص على `res.status` و `res.headers.get("content-type")`. ولو مش متأكد، اعمل `const text = await res.text(); console.log(text.slice(0, 200))` وشوف السيرفر رجّع إيه فعلاً. وفي تابة Network افتح الطلب وبص على Response (كتاب الإنترنت فصل 19).

Best Practice: متكتبش JSON بإيدك في الكود

لو بتبعت JSON من كود، متركّبوش بـ Template string زي `` `{"title": "${title}"}` ``: لو الـ `title` فيه `"` أو سطر جديد، الـ JSON هيبوظ (أو أسوأ: حد يحقن Fields). ابني Object واعمل `JSON.stringify`. ونفس الكلام في الـ Terminal: استخدم `jq -n` (فصل 6) أو `http` (فصل 8) بدل ما تكتب الـ JSON بإيدك جوه `'…'`.

حاجة مشتركة: نفس الكلام في Python

`json.dumps` = `JSON.stringify`، و `json.loads` = `JSON.parse`. والـ `indent=2` = الـ `space`، و `ensure_ascii=False` علشان العربي يطلع عادي (في JS العربي بيطلع عادي من الأول). و Python بتسمح بـ `NaN` افتراضياً في الاتجاهين (`allow_nan=True`)، يعني ممكن تطلّع JSON غلط من غير ما تاخد بالك! وده من كتاب Python for AI فصل 10.

ملخص الفصل

- JSON = String. والـ Object في الذاكرة. و `stringify` / `parse` المترجمين.
- JSON أصرم: Keys بـ `"`، ومفيش `'` ولا فاصلة أخيرة ولا تعليقات ولا NaN.
- `stringify(value, replacer, space)`: الـ Array فلتر Keys، والـ Function تحويل، والرقم Pretty print.
- `undefined` بيختفي من Objects، و Date بيبقى String، و Map بيبقى `{}`، و BigInt بيقع.
- `parse(text, reviver)`: حوّل التواريخ، و `ctx.source` للأرقام الكبيرة في Node 24.
- `Unexpected token '<'` = جالك HTML. اتأكد من الـ Status والـ Content-Type.

صح ولا غلط؟

1. `JSON.stringify({ a: undefined })` بيرجّع `'{"a":null}'`. صح / غلط
2. `JSON.parse(JSON.stringify(new Date()))` بيرجّع Date. صح / غلط
3. الـ `package.json` ينفع يبقى فيه تعليقات `//`. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `{ summary: undefined }` و `{ summary: null }` بعد stringify؟ وليه ده مهم للسيرفر؟
2. إزاي تشيل `password` من Object قبل ما تطبعه بـ stringify في سطر واحد؟
3. جالك `Unexpected end of JSON input` من `res.json()`. إيه أول حاجة تشك فيها؟
4. ليه بناء JSON بـ Template string فكرة وحشة؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في Console المتصفح، جرّب كل سطر في جدول الأخطاء، وقارن الرسايل بالجدول (Chrome و Node نفس المحرك).
2. متوسط اكتب Function اسمها `safeJson(res)` بتتأكد إن الـ content-type فيه `json` قبل `res.json()`، ولو لأ ترمي Error فيه أول 100 حرف من النص.
3. صعب اكتب reviver بيحوّل أي String شكلها تاريخ ISO (Regex) لـ `Date` أوتوماتيك. وفكّر: إيه الخطر لو Field اسمه `title` قيمته بالصدفة شكلها تاريخ؟

06

المرحلة 2 · JSON بالتفصيل

الفصل السادس

jq على الماك: تفصّص أي JSON من الـ Terminal

PokeAPI رجّعلك 309 KB عن Pikachu، وانت عايز الاسم والنوع بس. ممكن تفتحه في المتصفح وتدوّر بعينك، أو تكتب اسكريبت JS. أو تكتب سطر jq واحد. في الفصل ده هنتعلم jq حتة حتة على بيانات حقيقية: من النقطة لوحدها لحد select و map و group_by وتطليع CSV، وكل رمز متشرّح.

بعد الفصل ده هتكون عارف

- [x] jq موجود على الماك ولا تسطّبه، ونسخة مين
- [x] الفلتر . والـ Paths: a.b و .[0] و .[]
- [x] الـ Pipe
- [x] جوه jq (مش بتاع الـ Shell)
- [x] تبني Objects و Arrays جديدة بـ { } و [ ]
- [x] map و select و length و keys و sort_by و unique و group_by
- [x] -r و -c و -s و -n و -e و --arg و --argjson
- [x] التعامل مع null و // و ?
- [x] تطلّع CSV و TSV من JSON

6.1 ليه jq؟

JSON جاي من الـ API سطر واحد طويل، أو مئات السطور متداخلة. وانت عايز تجاوب أسئلة بسيطة: "كام عنصر؟"، "هات الأسماء بس"، "مين اللي حالته draft؟". و**jq** لغة صغيرة معمولة للحكاية دي بالظبط: بتاخد JSON، وتعدّيه على **فلتر**، وتطلّع JSON (أو نص). وأحلى حاجة فيه إنه بيتركب مع `curl` بـ Pipe، فتستكشف أي API من غير ما تكتب كود.

6.2 التسطيب على الماك

Terminal

```
user@MacBook ~ % which -a jq
/opt/homebrew/bin/jq
/usr/bin/jq
user@MacBook ~ % /usr/bin/jq --version
jq-1.7.1-apple
user@MacBook ~ % brew install jq
user@MacBook ~ % jq --version
jq-1.8.2
```

- من **macOS 15 Sequoia** وطالع، أبل بقت حاطة jq جاهز في `/usr/bin/jq`، بس نسخة قديمة (1.7.1 في macOS 26). ومينفعش تمسحها ولا تحدّثها (System volume).
- `brew install jq` بيدّيك أحدث نسخة (**1.8.2** في أكتوبر 2026) في `/opt/homebrew/bin`، وده بيبقى أول في الـ `PATH` فبيكسب.

- `which -a` : الـ `-a` = "All": اطبع كل نسخة لقيتها في الـ `PATH` بالترتيب، مش أول واحدة بس.
- كل الأمثلة هنا اتجربت على 1.8.2. وأغلبها شغال زي ما هو على 1.7.1، بس Functions جديدة زي `trim` مش موجودة في القديمة.

المصدر formulae.brew.sh · jq jq 1.8 Manual jqlang.org · Download

6.3 النقطة: أبسط فلتر

نزّل بيانات حقيقية الأول علشان نلعب عليها:

Terminal

```
user@MacBook ~ % curl -s https://pokeapi.co/api/v2/pokemon/pikachu -o pika.json
user@MacBook ~ % curl -s https://jsonplaceholder.typicode.com/users -o users.json
user@MacBook ~ % jq . pika.json | head -5
```

تشريح الأمر

```
1  2    3
jq . pika.json
```

1. `jq` البرنامج.
2. `.` الفلتر: "**اللي داخل زي ما هو**" (Identity). بس jq بيطبعه منسّق وملوّن، فده أسرع Pretty printer. ومن غير اسم ملف بياخد من الـ Pipe: `curl -s URL | jq .`
3. `pika.json` الملف اللي هيقرا منه.

خلي بالك: حط الفلتر بين ''

الفلاتر فيها رموز الـ Shell بيحب يلعب فيها (`|` و `[ ]` و `$` و `*` و `>`). فاكتب الفلتر دايماً بين علامتين `'…'`: `jq '.types[].type.name'`. والـ zsh على الماك تحديداً بيعتبر `[ ]` من غير تنصيص Pattern للملفات، ويطلّع `zsh: no matches found`.

6.4 الوصول للحقول: key. و .[n] و .[]

Terminal

```
user@MacBook ~ % jq '.name, .id, .height' pika.json
"pikachu"
25
4
user@MacBook ~ % jq '.[0].address.geo' users.json
{
  "lat": "-37.3159",
  "lng": "81.1496"
}
user@MacBook ~ % jq -r '.types[].type.name' pika.json
electric
```

`.name`	قيمة الـ Key `name`. زي `obj.name` في JS.
`.a, .b`	الفاصلة = "طلّع الاتنين" ورا بعض كنتايج منفصلة.
`.[0]`	أول عنصر في Array. و `.[-1]` آخر عنصر، و `.[2:5]` Slice من 2 لحد قبل 5.
`.address.geo`	تدخل جوه Objects متداخلة. وخد بالك: `lat` جاي **String** مش رقم، وده قرار تصميم وحش من JSONPlaceholder هتتكلم عنه في فصل 16.
`.[]`	**"فك"** الـ Array: كل عنصر يطلع لوحده كنتيجة منفصلة. ده أهم رمز في jq.
`.types[].type.name`	لكل عنصر في `types`، ادخل في `type` وهات `name`.
`-r`	"Raw output": اطبع النص من غير `"`. لازم لو هتستخدم الناتج في أمر تاني.

لو الـ Key مش موجود

Terminal

```
user@MacBook ~ % jq '.nope' pika.json
null
user@MacBook ~ % jq '.nope.deeper' pika.json
null
user@MacBook ~ % jq '.name.x' pika.json
jq: error (at pika.json:0): Cannot index string with string ("x")
user@MacBook ~ % jq '.name.x?' pika.json
user@MacBook ~ % jq '.license.spdx_id // "none"' repo.json
"MIT"
user@MacBook ~ % echo '{"license": null}' | jq '.license.spdx_id // "none"'
"none"
```

`.nope`	Key مش موجود = `null` من غير Error. و `null` نفسه لو دخلت جواه برضه `null`.
`Cannot index string`	لكن لو حاولت تدخل جوه String أو رقم، ده Error حقيقي، والـ Exit code بيبقى **5**.
`?`	"لو فيه Error، اسكت ومطلعش حاجة". زي `?.` في JS تقريباً.
`//`	الـ **Alternative operator**: "لو الشمال `null` أو `false` أو مطلعش حاجة، خد اليمين". زي `??` في JS. ومش تعليق!

6.5 الـ Pipe جوه jq، وتبني شكل جديد

الـ `|` جوه الفلتر بتاع jq زي الـ Pipe بتاع الـ Shell (كتاب الإنترنت فصل 3): ناتج اللي على الشمال بيدخل للي على اليمين. بس هي **جوه** jq، وده سبب التنصيص.

Terminal

```
user@MacBook ~ % jq '{name, weight, types: [.types[].type.name]}' pika.json
{
  "name": "pikachu",
  "weight": 60,
  "types": [
    "electric"
  ]
}
user@MacBook ~ % jq -c '.stats[] | {stat: .stat.name, base: .base_stat}' pika.json
{"stat":"hp","base":35}
{"stat":"attack","base":55}
{"stat":"defense","base":40}
{"stat":"special-attack","base":50}
{"stat":"special-defense","base":50}
{"stat":"speed","base":90}
user@MacBook ~ % jq '[.stats[] | .base_stat] | add' pika.json
320
```

حتة حتة

`{name, weight}`	1 بناء Object جديد. `{name}` اختصار لـ `{name: .name}` (زي الـ Shorthand في JS).
`types: [.types[].type.name]`	2 الـ `[ … ]` حوالين فلتر بيطلّع كذا نتيجة = "**لمّهم في Array**". عكس `.[]`.
`-c`	3 "Compact": كل نتيجة في سطر واحد. مفيد مع النتايج الكتير، وبيطلّع JSON Lines.

4 فك الـ Array، وكل عنصر يدخل للـ Pipe لوحده، ويتبني منه Object صغير. وجوه الـ Pipe النقطة بقت معناها "العنصر الحالي".	`.stats[] | {stat: .stat.name, base: .base_stat}`
5 لمّ الأرقام في Array، و `add` تجمعهم: 35+55+40+50+50+90 = 320.	`[ … ] | add`

6.6 map و select وأصحابهم

Terminal

```
user@MacBook ~ % jq 'length' users.json
10
user@MacBook ~ % jq -c 'keys[:6]' pika.json
["abilities","base_experience","cries","forms","game_indices","height"]
user@MacBook ~ % jq -c '.[] | select(.id == 3) | {id, name}' users.json
{"id":3,"name":"Clementine Bauch"}
user@MacBook ~ % jq -c 'map(select(.address.city | startswith("S"))) | map({name, city:
.address.city})' users.json
[{"name":"Patricia Lebsack","city":"South Elvis"},{"name":"Mrs. Dennis Schulist","city":"South
Christy"}]
user@MacBook ~ % jq 'map(select(.completed)) | length' todos.json
90
user@MacBook ~ % jq -c 'group_by(.userId) | map({user: .[0].userId, done:
map(select(.completed)) | length}) | .[:3]' todos.json
[{"user":1,"done":11},{"user":2,"done":8},{"user":3,"done":7}]
user@MacBook ~ % jq -c '[.stats[] | {name: .stat.name, base: .base_stat}] | sort_by(-.base) |
.[:2]' pika.json
[{"name":"speed","base":90},{"name":"attack","base":55}]
```

jq	بتعمل	في JS
`length`	عدد عناصر Array أو Keys أو حروف String	`.length`
`keys`	الـ Keys مترتبة أبجدياً	`Object.keys()` (من غير ترتيب)
`map(f)`	طبّق `f` على كل عنصر = `[.[] | f]`	`.map()`
`select(cond)`	عدّي العنصر لو الشرط صح، وامسحه لو لأ	`.filter()` (مع map)
`add`	اجمع أرقام / ادمج Strings أو Arrays	`.reduce((a, b) => a + b)`
`sort_by(f)`	رتّب حسب قيمة. و `-.base` = تنازلي	`.sort((a, b) => …)`
`group_by(f)`	Array of Arrays، كل مجموعة ليها نفس القيمة	`Object.groupBy()`
`unique`	شيل المتكرر (ورتّب)	`[...new Set(arr)]`
`startswith("S")`	String يبدأ بـ	`.startsWith()`

jq	بتعمل	في JS
`from_entries` / `to_entries`	Array of ↔ Object `{key, value}`	`Object.entries` / `fromEntries`
`type`	اسم النوع: `"object"` و `"array"` و...	`typeof` (بس أدق)

جدول 6.1 أهم الـ Functions وما يقابلها في JS

`select(.id == 3)`	`==` مقارنة (مفيش `===` في jq). و `and` و `or` و `not` كلمات: `select(.ok | not)`.
`.address.city | startswith("S")`	جوه `select` تقدر تعمل Pipe عادي، والنتيجة لازم تبقى true/false.
`map({user: .[0].userId, …})`	بعد `group_by` كل عنصر بقى Array، فـ `.[0].userId` = الـ userId بتاع أول واحد في المجموعة.

6.7 الـ Flags وبناء JSON صح

Terminal

```
user@MacBook ~ % jq -c 'select(.ok | not)' results.jsonl
{"file":"b.md","ok":false,"error":"timeout"}
user@MacBook ~ % jq -s -c 'map(.file)' results.jsonl
["a.md","b.md"]
user@MacBook ~ % jq -n --arg t 'He said "hi"' --argjson v 10 '{title: $t, views: $v}' -c
{"title":"He said \"hi\"","views":10}
user@MacBook ~ % jq -e '.missing' pika.json; echo "exit=$?"
null
exit=1
```

ملف `.jsonl`	jq بيقرا أي عدد JSON ورا بعض، وبيطبّق الفلتر على كل واحد لوحده. فـ JSON Lines شغال لوحده.
`-s`	"Slurp": اقرا كل الـ JSONs ولمّهم في Array واحدة قبل الفلتر. فـ `-s 'length'` = عدد السطور.
`-n`	"Null input": متقراش أي حاجة، ابدأ من `null`. علشان **تبني** JSON من الصفر.
`--arg t '…'`	متغير `$t` قيمته **String**. وjq بيعمله Escape صح لوحده: الـ `"` اللي جوه بقت `\"`. وده الطريقة الصح تبني Body لـ curl.
`--argjson v 10`	نفس الحكاية بس القيمة بتتقري كـ **JSON**، فـ `10` رقم مش `"10"`.
`-e`	"Exit status": لو آخر نتيجة `null` أو `false`، الـ Exit code يبقى 1. مفيد في اسكريبتات Shell: `if curl … | jq -e '.ok' >/dev/null; then …`.

`$?`	Exit code بتاع آخر أمر في الـ Shell (0 = نجح).

```
Terminal
# تبني Body صح وتبعته
user@MacBook ~ % jq -n --arg title "APIs و JSON" '{title: $title, userId: 1}' \
    | curl -s --json @- https://jsonplaceholder.typicode.com/posts
{
  "title": "APIs و JSON",
  "userId": 1,
  "id": 101
}
```

الـ `@-` معناها "خد الـ Body من الـ stdin" (اللي جاي من الـ Pipe)، و `--json` هنشرحه في فصل 8.

6.8 تطلّع CSV و TSV ونصوص

```
Terminal
user@MacBook ~ % jq -r '.[] | [.id, .name, .email] | @csv' users.json | head -3
1,"Leanne Graham","Sincere@april.biz"
2,"Ervin Howell","Shanna@melissa.tv"
3,"Clementine Bauch","Nathan@yesenia.net"
user@MacBook ~ % curl -s "https://api.github.com/repos/expressjs/express/releases?per_page=3"
\
    | jq -r '.[] | "\(.tag_name)  \(.published_at[:10])"'
v5.3.0  2026-10-09
v4.22.2  2026-05-11
v5.2.1  2025-12-01
user@MacBook ~ % jq -rn '"q=\("APIs و JSON" | @uri)"'
q=APIs%20%D9%88%20JSON
```

`[.id, .name, .email] | @csv`	ابني Array بالترتيب اللي عايزه، و `@csv` تحوّله لسطر CSV بتنصيص صح. وفي `@tsv` بـ Tab. ومحتاج `-r` وإلا السطر هيطلع جوه `"…"`. وتحفظه بـ `> users.csv` وتفتحه في Excel.
`"\(.tag_name)"`	**String interpolation**: زي `${…}` في الـ Template strings.
`.published_at[:10]`	الـ Slice شغال على الـ Strings كمان: أول 10 حروف = التاريخ من غير الوقت.
`@uri`	Percent-encoding للـ URL (كتاب الإنترنت فصل 6): المسافة `%20` و"و" بقت بايتاتها في UTF-8.

مثال عملي: سؤال واحد، سطر واحد

"إيه أكتر 3 Repos عليها Stars عند منظمة expressjs؟":

```
_page=100" | jq -r 'sort_by(-.stargazers_count) | .[:3][] | "\(.stargazers_count)\t\(.name)"'
```

رجّع وقت الكتابة: `69577 express` و `12087 multer` و `8200 morgan`. وفصّص كل حتة بالجداول اللي فوق: `sort_by(-…)` تنازلي، و `.[:3]` أول 3، و `[]` فكّهم.

حاجة مشتركة: نفس الفكرة في كل حتة

الـ Path بالنقط (`.owner.login`) هتلاقيه في JSONPath (**RFC 9535**: `$.owner.login`)، وفي JSON Pointer (**RFC 6901**: `/owner/login`) اللي هيظهر في JSON Schema (فصل 7) وفي أخطاء الـ Validation (فصل 17) وفي JSON Patch (فصل 15). تلات صيغ لنفس الفكرة: "إزاي أشاور على حتة جوه JSON".

المصدر

RFC 6901 · JSON Pointer
jq Manual · @csv @uri
jq Manual · Builtin functions
jq Manual · Basic filters
RFC 9535 · JSONPath

ملخص الفصل

- macOS 15+ فيه jq قديم في `/usr/bin`، و `brew install jq` = 1.8.2.
- الفلتر دايماً بين `'…'`. و `.` = زي ما هو، و `a.b.` و `[0].` و `[].` للوصول.
- `|` جوه jq Pipe، و `{ }` تبني Object، و `[ ]` تلمّ نتايج.
- `map` و `select` و `length` و `keys` و `sort_by` و `group_by` و `add`.
- `//` قيمة بديلة، و `?` يسكّت الأخطاء.
- `-r` نص، و `-c` سطر، و `-s` لمّ، و `-n` بناء، و `--arg` / `--argjson` متغيرات آمنة، و `-e` Exit code.

صح ولا غلط؟

1. `//` في jq معناها تعليق. صح / غلط
2. `jq '.nope'` على Key مش موجود بيرجّع Error. صح / غلط
3. `--arg n 5` بيخلي `$n` رقم. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `[].` و `[ … ]` ؟
2. اكتب فلتر بيطلّع أسماء الـ Users اللي الـ email بتاعهم بيخلص بـ `.biz` .
3. ليه `jq -n --arg` أحسن من إنك تكتب الـ JSON بإيدك في `curl -d` ؟
4. إمتى تحتاج `-r` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل من `pika.json` طلّع أسماء الـ abilities كنص، كل واحدة في سطر.
2. متوسط من `todos.json` طلّع لكل User نسبة المهام المخلّصة كـ `{user, percent}`.
3. صعب من `https://api.github.com/repos/expressjs/express/issues?per_page=30` طلّع CSV فيه رقم الـ Issue والعنوان واسم صاحبها والـ Labels مفصولة بـ `;` (تلميح: `[.labels[].name] | join(";")`).

07

المرحلة 2 · JSON بالتفصيل

الفصل السابع

JSON Schema: العقد مكتوب كـ JSON

الـ API بتاعك مستني مقالة فيها title و status، وجاله id كـ String، و status قيمته done، و Field زيادة محدش طلبه. تكتب if لكل حالة؟ هتبقى 50 if، والدوكيومنتيشن هتقول حاجة والكود حاجة. JSON Schema بيحل الاتنين: وصف واحد للبيانات، تعمل بيه Validation، وتكتب بيه الدوكيومنتيشن، وهو نفسه اللي جوه OpenAPI.

بعد الفصل ده هتكون عارف

- المشكلة: Validation يدوي ودوكيومنتيشن بتكدب
- شكل الـ Schema: $schema و type و properties و required
- قيود الـ Strings والأرقام والـ Arrays
- enum و const و additionalProperties
- $ref و $defs لإعادة الاستخدام، و oneOf/anyOf/allOf
- format و default: annotations مش أوامر
- Validation حقيقي في Node بـ Ajv وتقرا الأخطاء
- علاقته بـ OpenAPI و Pydantic و zod

7.1 المشكلة: 50 if

تخيل إن الـ API بتاعك (اللي هتبنيه في Express) بيستقبل مقالة جديدة. ولازم تتأكد إن الـ `title` موجود و String ومش قصير، وإن `status` واحدة من 3 قيم، وإن `tags` List من Strings من غير تكرار... فبتكتب:

```javascript
if (typeof body.title !== "string") return error("title must be a string");
if (body.title.length < 3) return error("title too short");
if (!["draft", "review", "published"].includes(body.status)) return error("bad status");
if (body.tags && !Array.isArray(body.tags)) return error("tags must be an array");
// ... و 40 سطر كمان
```

وبعد شهر، زميلك زوّد حالة `"archived"` في الكود ونسي يحدّث الدوكيومنتيشن. **وهنا كانت المشكلة**: العقد مكتوب في مكانين (الكود والدوكيومنتيشن)، والاتنين بيبعدوا عن بعض. **فاتطور حل**: تكتب العقد **مرة واحدة** كـ JSON، وأدوات تعمل منه Validation، وأدوات تانية تعمل منه دوكيومنتيشن. والوصف ده اسمه **JSON Schema**.



شكل 7.1 نفس ملف الـ Schema بيخدم أكتر من حاجة. وانت شفت آخر واحدة في كتاب Python for AI فصل 21: الـ Structured outputs في OpenAI و Gemini بتاخد JSON Schema بالظبط.

والنسخة الحالية من المواصفة اسمها **2020-12** (والموقع الرسمي بيقول كده لحد أكتوبر 2026)، وهي اللي OpenAPI 3.1 و 3.2 مبنيين عليها.

المصدر Understanding JSON Schema · json-schema.org · Specification (2020-12)

7.2 أول Schema، سطر سطر

article.schema.json

```json
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$id": "https://example.com/schemas/article.json",
  "title": "Article",
  "type": "object",
  "properties": {
    "id": { "type": "integer", "minimum": 1 },
    "title": { "type": "string", "minLength": 3, "maxLength": 120 },
    "status": { "enum": ["draft", "review", "published"] },
    "tags": {
      "type": "array",
      "items": { "type": "string" },
      "uniqueItems": true,
      "maxItems": 10
    },
    "author": {
      "type": "object",
      "properties": {
        "name": { "type": "string" },
        "email": { "type": "string", "format": "email" }
      },
      "required": ["name"]
    },
    "publishedAt": { "type": ["string", "null"], "format": "date-time" }
  },
  "required": ["id", "title", "status"],
  "additionalProperties": false
}
```

كل Keyword ومعناه	
1 "الـ Schema ده مكتوب بأنهي نسخة من المواصفة". الـ `$` في أول الاسم معناها Keyword أساسي في المواصفة نفسها (Core). من غيره كل أداة هتفترض نسخة على مزاجها.	`"$schema": "…/2020-12/schema"`
2 اسم فريد للـ Schema على شكل URL. **مش لازم يفتح** في المتصفح، ده مجرد معرّف، وبيستخدم لما Schema تشاور على Schema تانية.	`"$id": "https://example.com/…"`
3 اسم للبني آدمين والدوكيومنتيشن. ومعاه `"description"` لو عايز تشرح. مالهمش أي تأثير على الـ Validation.	`"title"`
4 نوع القيمة: `object` و `array` و `string` و `number` و `integer` و `boolean` و `null`. و `integer` مش نوع في JSON نفسه، ده "number من غير كسر".	`"type": "object"`
5 لكل Key متوقّع، Schema صغيرة بتوصفه. وخد بالك: `properties` لوحدها **مش بتقول إن الـ Key إجباري**. بتقول بس "لو موجود، يبقى شكله كده".	`"properties": { … }`
6 أقل قيمة مسموحة (شامل). وفي `maximum` ، و `exclusiveMinimum` (أكبر من، مش يساوي)، و `multipleOf` .	`"minimum": 1`
7 طول الـ String بعدد الحروف (Code points)، مش البايتات. فـ "عمر" طوله 3.	`"minLength" / "maxLength"`
8 القيمة لازم تبقى واحدة من دول بالظبط. ولاحظ إن مفيش `type` : الـ enum كفاية. ولو قيمة واحدة بس: `"const": "ar"` .	`"enum": [ … ]`
9 كل عنصر في الـ Array لازم يطابق الـ Schema دي.	`"items": { "type": "string" }`
10 مفيش عنصرين زي بعض.	`"uniqueItems": true`
11 Schema جوه Schema: نفس الكلام على Object متداخل، وليه `required` لوحده.	`"author": { "type": "object", … }`
12 Array من الأنواع = "واحد من دول". كده `publishedAt` ممكن يبقى تاريخ أو `null` (لسه متنشرتش).	`"type": ["string", "null"]`
13 شكل معروف للـ String: `date-time` (RFC 3339) و `email` و `uri` و `uuid` … بس خد بالك من الـ Box اللي تحت.	`"format": "date-time"`
14 هنا بس بتقول مين **إجباري**. Array من أسماء الـ Keys.	`"required": [ … ]`
15 "أي Key مش مذكور في `properties` ممنوع". من غيره، أي Field زيادة بيعدّي عادي (الافتراضي `true`). وده بيحميك من الـ Typos: `"titel"` بدل `"title"` هيتمسك.	`"additionalProperties": false`

7.3 Validation حقيقي في Node بـ Ajv

Ajv أشهر Validator لـ JSON Schema في JavaScript، وهو اللي أدوات كتير في Node بتستخدمه من جوه. خلينا نجرّبه على ملفين: واحد صح وواحد فيه كل الغلطات:

bad.json

```
{ "id": "7", "title": "Hi", "status": "done", "tags": ["api", "api"], "author": { "email": "not-an-email" }, "views": 10 }
```

validate.mjs

```
import { readFileSync } from "node:fs";
import Ajv2020 from "ajv/dist/2020.js";
import addFormats from "ajv-formats";

const ajv = new Ajv2020({ allErrors: true });
addFormats(ajv);

const schema = JSON.parse(readFileSync("article.schema.json", "utf8"));
const validate = ajv.compile(schema);

for (const file of process.argv.slice(2)) {
  const data = JSON.parse(readFileSync(file, "utf8"));
  if (validate(data)) {
    console.log(`${file}: valid`);
  } else {
    console.log(`${file}: ${validate.errors.length} errors`);
    for (const err of validate.errors) {
      console.log(`  ${err.instancePath || "/"} ${err.message}`);
    }
  }
}
```

Terminal

```
user@MacBook ~ % npm install ajv ajv-formats
user@MacBook ~ % node validate.mjs good.json bad.json
good.json: valid
bad.json: 7 errors
  / must NOT have additional properties
  /id must be integer
  /title must NOT have fewer than 3 characters
  /status must be equal to one of the allowed values
  /tags must NOT have duplicate items (items ## 1 and 0 are identical)
  /author must have required property 'name'
  /author/email must match format "email"
```

سطر سطر	
1 Ajv فيه Class لكل نسخة من المواصفة. والـ Import العادي `"ajv"` بيستخدم draft-07 القديمة، فلو الـ Schema بتاعك `2020-12` لازم الـ Class ده. (اتجرب على ajv 8.20.)	`import Ajv2020 from "ajv/dist/2020.js"`
2 افتراضياً Ajv بيقف عند **أول** غلطة. و `allErrors` بيخليه يجمعهم كلهم، وده اللي عايزه في رد API علشان المستخدم يصلّح كله مرة واحدة.	`new Ajv2020({ allErrors: true })`
3 من غير السطر ده، `format` **مبيتشيكش** خالص (تحت).	`addFormats(ajv)`
4 بيحوّل الـ Schema لـ Function سريعة. اعملها **مرة واحدة** بره أي Loop أو Route، مش مع كل طلب.	`const validate = ajv.compile(schema)`
5 بترجّع `true` أو `false`، والأخطاء بتتحط في `validate.errors`.	`validate(data)`
6 مكان الغلطة في البيانات كـ **JSON Pointer** (`/author/email`). و `""` = الجذر نفسه. ودي بالظبط اللي هنرجّعها للمستخدم في رد الـ Error (فصل 17).	`err.instancePath`

> ⚠ **خلي بالك: format و default مش أوامر**
>
> في 2020-12، الـ `format` افتراضياً **Annotation**: معلومة بس، مش شرط. جربنا Ajv من غير `ajv-formats` على `"nope"` مع `format: "email"` : رجّع **true** (وطبع تحذير `unknown format "email" ignored`). وكمان `default` **مبيحطش القيمة** في البيانات: جربنا `views: { default: 0 }` والـ `views` فضلت مش موجودة بعد الـ Validation. لو عايز تطبيقها، Ajv عنده Option اسمها `useDefaults`، أو حطها انت في الكود.

المصدر Understanding JSON Schema · type · Ajv · JSON Schema keywords · Ajv · Getting started

7.4 $ref و $defs: متكررش نفسك

الـ Author والـ Editors والـ Reviewers كلهم "شخص". بدل ما توصف الشخص 3 مرات:

JSON

```
{
  "$schema": "https://json-schema.org/draft/2020-12/schema",
  "$defs": {
    "person": {
      "type": "object",
      "properties": { "name": { "type": "string" } },
      "required": ["name"]
    }
  },
  "type": "object",
  "properties": {
    "author":  { "$ref": "#/$defs/person" },
```

```
    "editors": { "type": "array", "items": { "$ref": "#/$defs/person" } },
    "price":   { "type": "number", "exclusiveMinimum": 0 },
    "slug":    { "type": "string", "pattern": "^[a-z0-9-]+$" },
    "lang":    { "const": "ar" }
  }
}
```

`"$defs"`	مكان تحط فيه Schemas بأسماء علشان تستخدمها تاني. مبيعملش Validation لوحده.
`"$ref": "#/$defs/person"`	"استخدم الـ Schema اللي هناك". والـ `#` = "الملف ده نفسه"، والباقي JSON Pointer. وممكن يشاور على ملف تاني: `"person.json"`.
`"pattern"`	Regex لازم الـ String يطابقه. و `^` و `$` أول وآخر النص، فده "حروف صغيرة وأرقام وشرطة بس" (شكل الـ Slug في الـ URL).

```
Terminal
/author must have required property 'name'
/editors/1 must have required property 'name'
/price must be > 0
/slug must match pattern "^[a-z0-9-]+$"
/lang must be equal to constant
```

(ناتج حقيقي لما بعتنا `author: {}` و editor تاني فيه `nam` بدل `name` و `price: 0` و `slug: "Hello World"` و `lang: "en"`.)

oneOf و anyOf و allOf

Keyword	معناه	مثال استخدام
`allOf`	لازم يطابق **كلهم**	Schema أساسي + زيادات
`anyOf`	يطابق **واحد على الأقل**	ID ممكن رقم أو Slug
`oneOf`	يطابق **واحد بس بالظبط**	دفع بكارت **أو** محفظة، مش الاتنين
`not`	ميطابقش	نادراً

جدول 7.1 التركيب

حاجة مشتركة: JSON Schema في كل حتة

OpenAPI 3.1 و 3.2: الـ `schema` جوه أي Request أو Response **هو** JSON Schema 2020-12 (فصل 21). **Pydantic** في Python: `Model.model_json_schema()` بيطلّع JSON Schema (كتاب Python for AI فصل 21). **zod** في JS/TypeScript: بتكتب الـ Schema كـ كود، وفي zod 4 تقدر تطلّعه JSON Schema بـ `z.toJSONSchema()`. يعني نفس الفكرة بتلاقيها في كل أداة، والـ Keywords هي هي.

Best Practice: Schema للـ Input والـ Output

كتير من الناس بيعملوا Schema للي داخل بس. اعمل كمان للرد اللي خارج، وشغّله في الـ Tests: كده لو حد غيّر شكل الرد بالغلط (شال Field أو غيّر نوعه)، الـ Test يقع قبل ما الـ Clients تقع.

ملخص الفصل

- JSON Schema = العقد مكتوب مرة واحدة، للـ Validation والدوكيومنتيشن والأنواع والـ AI.
- `$schema` النسخة (2020-12)، و`type` و`properties` و`required` الأساس.
- `properties` مش بتقول إجباري: `required` هي اللي بتقول.
- قيود: `minLength` و`pattern` و`minimum` و`enum` و`const` و`items` و`uniqueItems`.
- `additionalProperties: false` يمسك الـ Fields الزيادة والـ Typos.
- `$ref` + `$defs` لإعادة الاستخدام، و`oneOf/anyOf/allOf` للتركيب.
- `format` و`default` Annotations: محتاجين `ajv-formats` و`useDefaults`.

صح ولا غلط؟

1. أي Key مذكور في `properties` بيبقى إجباري. صح / غلط
2. من غير `additionalProperties: false`، الـ Fields الزيادة بتعدّي. صح / غلط
3. `"default": 0` بيضيف القيمة في البيانات لوحده. صح / غلط

أسئلة مراجعة

1. إيه المشكلة اللي JSON Schema بيحلّها غير الـ Validation؟
2. اكتب Schema لـ `email` إجباري و`age` اختياري رقم صحيح من 13 لـ 120.
3. ليه `allErrors: true` مهمة في API؟
4. إيه الفرق بين `anyOf` و`oneOf`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل نزّل `ajv` و`ajv-formats` في فولدر تجارب، وشغّل `validate.mjs` على الملفين.
2. متوسط اكتب Schema لـ Todo من JSONPlaceholder (`userId` و`id` و`title` و`completed`)، واعمل Validation على الـ 200 Todo كلهم (تلميح: `"type": "array", "items": {"$ref": …}`).
3. صعب اعمل Schema لـ `users.json` وخلّي `address.geo.lat` `"type": "number"`. الـ Validation هيقول إيه؟ وده بيقولك إيه عن تصميم JSONPlaceholder؟

08

المرحلة 3 · تتكلم مع API

الفصل التامن

الأدوات: curl للـ APIs و HTTPie و Bruno/Postman وتابة Network

عرفت curl في كتاب الإنترنت كأداة تشوف بيها HTTP. هنا هنستخدمه كأداة شغل مع الـ APIs: تبعت JSON صح، وتضيف Auth، وتركّب Query parameters بأمان، وتفشل بصوت عالي. وبعدين HTTPie اللي معمول للـ APIs مخصوص، وبرامج زي Bruno و Postman و Insomnia، وإزاي تاخد أي طلب من تابة Network وتعيده في الـ Terminal.

بعد الفصل ده هتكون عارف

- curl: -X و -H و -d و --json و -G و --data- urlencode و -u و -f و -w
- الغلطة الأشهر: -d من غير Content-Type
- HTTPie: = و := و == و : و -a و --offline
- Bruno و Postman و Insomnia: الفرق وإمتى كل واحد
- تابة Network: فلتر Fetch/XHR و Copy as cURL و Copy as fetch
- جدول يختار لك الأداة

8.1 curl للـ APIs: الـ Flags اللي هتعيش بيها

في كتاب الإنترنت (فصل 10) استخدمنا curl علشان **نتفرج**: `-v` و `-I` و `-o`. مع الـ APIs هنحتاج **نبعت**: Methods و JSON و Tokens. وأول حاجة لازم تعرفها:

مشكلة 1 **السيرفر مش شايف الـ JSON اللي بعته (null :json)**

العَرَض: بعت `curl -d '{"a":1}' https://httpbin.org/post` والسيرفر رد كإنه مستلمش حاجة، أو رجّع 400 أو 415.

السبب: `-d` لوحدها بتبعت `Content-Type: application/x-www-form-urlencoded` (شكل الـ HTML forms القديم). فالسيرفر بيحاول يقرا الـ Body كـ `a=1&b=2`. وده الرد الحقيقي من httpbin:

```JSON
{"json":null,"form":{"{\"a\":1}":""},"ct":"application/x-www-form-urlencoded"}
```

الـ JSON كله اتقرا كـ **اسم Field** في Form، وقيمته فاضية!

الحل: يا إما `-H 'Content-Type: application/json'` مع `-d`، يا إما الأحسن: `--json`.

```
Terminal
user@MacBook ~ % curl -s --json '{"a":1}' -X PUT https://httpbin.org/put | jq -c '{json, ct: .headers["Content-Type"], accept: .headers.Accept}'
{"json":{"a":1},"ct":"application/json","accept":"application/json"}
```

تشريح الأمر

```
curl -s --json '{"a":1}' -X PUT https://httpbin.org/put
                         (2)      (3)
```

1. `--json '…'` — Flag جديد نسبياً (curl 7.82، سنة 2022) بيعمل 3 حاجات مرة واحدة: يبعت الـ Body، ويحط `Content-Type: application/json`، ويحط `Accept: application/json`. وزي `-d`، بيخلي الـ Method `POST` لوحده. وتأكد من نسختك بـ `curl --version`.
2. `-X PUT` — "Request method": غيّر الـ Method. هنا محتاجينها لأن `--json` بيختار `POST`. ومتستخدمش `-X GET` أو `-X POST` من غير داعي: curl بيختارهم لوحده، و`-X` بتغيّر **الكلمة** بس مش السلوك.
3. `httpbin.org/put` — httpbin سيرفر بيرجّعلك اللي بعته بالظبط، فأحسن مكان تتأكد إن الطلب طالع صح (فصل 9).

باقي الـ Flags، مجمّعة حسب الشغلانة

الشغلانة	الـ Flag	التفاصيل
Header	`-H 'Name: value'`	تتكرر لكل Header. و`-H 'Name:'` (فاضي) يشيل Header curl بيحطه لوحده.
Body من ملف	`--json @body.json` / `-d @body.json`	الـ `@` = "اقرا من الملف ده". و`@-` = من الـ stdin (الـ Pipe).
Body زي ما هو	`--data-binary @file`	`-d` بتشيل الـ Newlines من الملفات. `--data-binary` بتبعت البايتات زي ما هي.
Query بأمان	`-G --data-urlencode 'q=…'`	`-G` = "حوّل كل `-d` لـ Query string في GET". و`--data-urlencode` بتعمل Percent-encoding صح للمسافات والعربي.
Basic auth	`-u user:pass`	بيبني `Authorization: Basic …` لوحده (فصل 12).
Bearer token	`-H "Authorization: Bearer $TOKEN"`	أو `--oauth2-bearer "$TOKEN"`. والـ Token من Variable مش مكتوب في الأمر.
شوف الـ Headers	`-i` / `-v`	`-i` بيطبع Headers الرد قبل الـ Body. و`-v` الطلب والرد.
افشل بصوت عالي	`-f` / `--fail-with-body`	لو الـ Status ≥ 400، الـ Exit code بيبقى **22**. `-f` بتخفي الـ Body، و`--fail-with-body` بتطبعه (أحسن لـ APIs لأن رسالة الغلط جواه).

الشغلانة	الـ Flag	التفاصيل
Status بس	`-o /dev/null -w '%{http_code}\n'`	ارمي الـ Body واطبع الـ Status. و `-w` ليه متغيرات كتير (تحت).
الـ Redirects	`-L`	امشي ورا 301/302. من غيرها curl بيقف عند الـ Redirect.
الهدوء الذكي	`-sS`	اسكت (`-s`) بس اطبع الأخطاء (`-S`). أحسن من `-s` لوحدها في الاسكريبتات.
Retries	`--retry 3`	يعيد على أخطاء مؤقتة (منها 429 و xx5) ويحترم `Retry-After`.

جدول 8.1 curl للـ APIs

Terminal

```
user@MacBook ~ % curl -s -G https://httpbin.org/get --data-urlencode "q=APIs و JSON" -d
limit=2 | jq -c '{args, url}'
{"args":{"limit":"2","q":"APIs و JSON"},"url":"https://httpbin.org/get?q=APIs+و+JSON&limit=2"}
user@MacBook ~ % curl -s --fail-with-body https://api.github.com/repos/nope-xx/nope; echo "
exit=$?"
{
  "message": "Not Found",
  "documentation_url": "https://docs.github.com/rest/repos/repos#get-a-repository",
  "status": "404"
}
 exit=22
user@MacBook ~ % curl -s -o /dev/null -w 'status=%{http_code} type=%{content_type} time=%
{time_total}s size=%{size_download}\n' https://pokeapi.co/api/v2/pokemon/pikachu
status=200 type=application/json; charset=utf-8 time=0.100613s size=309106
```

`-G … -d limit=2`	اتنين `-d` (واحد Encoded وواحد عادي) اتلمّوا في الـ Query بـ `&`، والـ Method فضلت GET.
`q=APIs+و+JSON`	المسافة بقت `+` (طريقة الـ Forms القديمة)، و httpbin عرضها لنا متفكوكة. والسيرفر هيفكها لـ "APIs و JSON".
`echo " exit=$?"`	الـ "HTTP error" = Exit code 22. ده اللي بيخلّي `curl -f … && echo ok` يقف لو الـ API رجّع غلط.
`%{http_code} %{content_type} %{time_total} %{size_download}`	متغيرات `-w`: الـ Status، والـ Content-Type، والوقت الكلي بالثواني، والحجم بالبايت. وفي كمان `%{time_namelookup}` و `%{time_connect}` و `%{time_appconnect}` (TLS) و `%{time_starttransfer}` (أول بايت) لو بتقيس السرعة.

Best Practice: الـ Token من Variable

`export GITHUB_TOKEN=…` مرة في الجلسة، وبعدين `curl -H "Authorization: Bearer $GITHUB_TOKEN" …` بـ " مش ' (علشان الـ `$` يتفك). كده الـ Token مش مكتوب في الأمر، وميظهرش لو عملت Copy للأمر لحد. وخلّي بالك إن `-v` بيطبع الـ `Authorization` header، فمتلزقش ناتج `-v` في Slack.

المصدر `curl man · --json` `curl man · -G` `curl man · --fail-with-body` `curl man · --write-out` `Everything curl`

8.2 HTTPie: curl بس معمول للـ APIs

curl عام لكل البروتوكولات، فكتابة JSON فيه متعبة. **HTTPie** اتعمل مخصوص للـ APIs: JSON افتراضي، والرد ملوّن ومنسّق، والصيغة قريبة من الكلام. تسطّبه بـ `brew install httpie` (3.2.4 في أكتوبر 2026)، والأمر اسمه `http` (و `https`).

Terminal

```
user@MacBook ~ % http --offline POST api.example.com/v1/articles title="Hello" views:=10
tags:='["api","json"]' status=draft Authorization:"Bearer abc123" lang==ar
POST /v1/articles?lang=ar HTTP/1.1
Accept-Encoding: gzip, deflate
Connection: keep-alive
Content-Length: 75
User-Agent: HTTPie/3.2.4
Accept: application/json, */*;q=0.5
Content-Type: application/json
Authorization: Bearer abc123
Host: api.example.com

{"title": "Hello", "views": 10, "tags": ["api", "json"], "status": "draft"}
```

تشريح الأمر

```
http --offline POST api.example.com/v1/articles title="Hello" views:=10
tags:='["api","json"]' Authorization:"Bearer abc123" lang==ar
```

1. `--offline` — "متبعتش، اطبعلي الطلب اللي كنت هتبعته". أحسن طريقة تتعلم بيها HTTPie وتتأكد من الطلب قبل ما يروح لـ API حقيقي.
2. `POST` — الـ Method في الأول. ولو مكتبتهاش: GET، أو POST لو فيه Body.
3. `api.example.com/…` — من غير `https://` : الأمر `http` بيفترض `http://`، و `https` بيفترض `https://`. فمع APIs حقيقية اكتب `https api.github.com/…` أو الـ URL كامل.
4. `title="Hello"` — الـ `=` = Field في الـ JSON قيمته **String**.
5. `views:=10` — الـ `:=` = Field قيمته **JSON خام**: رقم أو Boolean أو Array أو Object. فـ `10` طلعت رقم مش `"10"`.
6. `tags:='[…]'` — Array كاملة كـ JSON. وبين `'…'` علشان الـ Shell.

7 `Authorization:"Bearer …"` الـ `:` لوحدها = **Header**.

8 `lang==ar` الـ `==` = **Query parameter**. وطلع في سطر الطلب: `?lang=ar` .

الرمز	بيعمل	مثال
`:`	Header	`X-Trace:abc`
`==`	Query parameter	`limit==2`
`=`	(String) JSON field	`title=Hello`
`:=`	(خام) JSON field	`done:=true`
`@`	ملف (مع `--form`)	`file@photo.jpg`
`-a user:pass`	Basic auth	يبني `Authorization: Basic dXNlcjpwYXNz`
`-A bearer -a TOKEN`	Bearer auth	يبني `Authorization: Bearer TOKEN`
`--print=HhBb`	اطبع إيه	H/B = Headers/Body الطلب، h/b = بتوع الرد
`--form`	Form بدل JSON	`application/x-www-form-urlencoded`

جدول 8.2 رموز HTTPie

> **خلي بالك: HTTPie في الاسكريبتات**
>
> لو شغّلت `http` جوه اسكريبت أو Cron (مفيش Terminal)، ممكن يقع بـ *"Request body and request data (...from stdin) cannot be mixed (key=value)"*، لأنه بيفتكر إن في Body جاي من الـ stdin. الحل `--ignore-stdin`. (حصلت معانا بالظبط وإحنا بنجرّب للكتاب.)

المصدر: `formulae.brew.sh · httpie` · `HTTPie · Request items` · `HTTPie CLI docs`

8.3 البرامج بالواجهة: Bruno و Postman و Insomnia

لما يبقى عندك 30 طلب لنفس الـ API، ومتغيرات زي `baseUrl` و `token`، وعايز تحفظهم وتشاركهم، الـ Terminal بيتعب. البرامج دي بتديك **Collections** (طلبات محفوظة في فولدرات)، و**Environments** (نفس الطلبات على local و production)، وتعرض الرد حلو، وتكتب Tests صغيرة.

	Bruno	Postman	Insomnia
Homebrew	`brew install --cask bruno` (4.3.0)	`brew install --cask postman` (12.33)	`all --cask insomnia` (13.3)
فين بيتحفظ	**ملفات نص في فولدر عندك** (صيغة Bru / YAML)، تعملها Commit في Git مع الكود	في حسابك على السحابة أساساً	Local أو Cloud أو Git

	Bruno	Postman	Insomnia
محتاج حساب؟	لأ	أيوه للمزايا الأساسية	اختياري
المجاني	Open source، والأساسي كله مجاني	خطة Free: **مستخدم واحد**	في خطة مجانية
OpenAPI	Import و Export	Import ومحرر Specs	Import ومحرر Specs
هتقابله فين	فرق صغيرة بتحب Git	**الأشهر** في الشركات والكورسات	ناس بتحبه أخف

جدول 8.3 التلاتة جنب بعض (أكتوبر 2026)

> **هتختار إيه؟**
>
> كورس Maximilian (زي أغلب الكورسات) غالباً هيستخدم Postman، فاعرف شكله. بس لشغلك الشخصي جرّب **Bruno**: الـ Collection بيبقى فولدر جوه الـ Repo بتاع الـ API نفسه، فأي حد يعمل Clone يلاقي الطلبات جاهزة. والفكرة في التلاتة واحدة: Method و URL و Headers و Body و Auth، يعني اللي في الكتاب ده كله.

المصدر Bruno docs · Postman docs · Postman pricing · Insomnia docs

8.4 تابة Network: تصطاد الطلب وتعيده

كل موقع حديث بيكلم API بـ `fetch`. يعني أي موقع بتستخدمه هو "دوكيومنتيشن حي" للـ API بتاعه. وتابة Network اتشرحت في كتاب الإنترنت (فصول 18 و 19)، فهنا الحاجات الخاصة بالـ APIs بس:

1. **فلتر Fetch/XHR**: من الشريط اللي فوق، علشان يختفي كل الـ CSS والصور ويفضل طلبات الـ API بس.
2. اضغط على أي طلب: **Headers** (الـ Method والـ Status والـ Headers)، و**Payload** (الـ Query string والـ Body اللي اتبعت)، و**Preview** (الـ JSON كشجرة تفتحها وتقفلها)، و**Response** (النص الخام).
3. كليك يمين على الطلب ← **Copy** ← **Copy as cURL**: الطلب كله بكل الـ Headers والـ Cookies جاهز تلزقه في الـ Terminal. ومنه تشيل الـ Headers اللي مش محتاجها لحد ما توصل لأقل طلب شغال.
4. أو **Copy as fetch** (أو **Copy as fetch (Node.js)**): نفس الطلب كـ كود JS تلزقه في الـ Console أو في ملف.
5. من نفس القايمة **Replay XHR**: يعيد الطلب زي ما هو، مفيد وانت بتعدّل في السيرفر.

> **خلي بالك: Copy as cURL فيه أسرارك**
>
> الطلب المنسوخ فيه الـ Cookies بتاعة الـ Session و الـ `Authorization` header. يعني أي حد معاه الأمر ده يقدر يتصرف كأنه انت لحد ما الـ Session تخلص. متلزقهوش في Issue ولا شات، وامسح الـ Headers الحساسة قبل ما تشاركه.

Safari

في Safari، تابة Network في Web Inspector، وكليك يمين على الطلب فيها **Copy as cURL** برضه (كتاب الإنترنت فصل 21). بس فلاتر Chrome وتابة Payload أوضح للشغل ده.

المصدر Chrome DevTools · Network features reference

8.5 تختار أنهي أداة؟

الموقف	الأداة
طلب سريع، أو جوه اسكريبت Shell، أو في Docs رسمية	**curl** (موجود في كل حتة)
بتجرّب API بإيدك وعايز JSON سهل ورد ملوّن	**HTTPie**
Collection كاملة ومتغيرات وتشاركها مع فريق	Insomnia / Postman / **Bruno**
عايز تعرف الموقع ده بيكلم إيه	Copy as cURL + **تابة Network**
هتكرر الطلب ده كتير في كود	`fetch` (فصل 10) أو Python `requests`

جدول 8.4 الأداة حسب الموقف

ملخص الفصل

- `-d` لوحدها = Form مش JSON. استخدم `--json` (POST + Accept + Content-Type + Body).
- `-X` للـ Method، و`-H` للـ Headers، و`@file` و`@-` للـ Body من ملف أو Pipe.
- `-G --data-urlencode` لـ Query آمن، و`--fail-with-body` Exit 22 مع رسالة الغلط، و`-w` للـ Status والوقت.
- HTTPie: `:` Header، و`==` Query، و`=` String، و`:=` JSON خام، و`--offline` تشوف قبل ما تبعت.
- Bruno ملفات في Git، و Postman الأشهر (Free لمستخدم واحد)، و Insomnia الأخف.
- Network: فلتر Fetch/XHR، و fetch / Copy as cURL، وخلي بالك من الأسرار.

صح ولا غلط؟

1. `curl -d '{"a":1}' URL` بيبعت `Content-Type: application/json`. صح / غلط
2. في HTTPie، `count=5` بيبعت `5` كرقم. صح / غلط
3. `-f` في curl بتخلي الـ Exit code 22 لو الـ Status 404. صح / غلط

أسئلة مراجعة

1. اكتب نفس الطلب (POST بـ JSON فيه `title` و `userId` رقم) بـ curl وبـ HTTPie.
2. إيه الفرق بين `-f` و `--fail-with-body`؟ وليه التانية أحسن مع APIs؟
3. ليه `-G --data-urlencode` أحسن من إنك تكتب `?q=…` في الـ URL بإيدك؟
4. إيه الخطر في مشاركة ناتج Copy as cURL؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل سطّب HTTPie وابعت `https httpbin.org/post title=Hi done:=false` وقارن الرد بنفس الطلب بـ curl.
2. متوسط افتح أي موقع بتستخدمه (مثلاً GitHub وانت عامل Login)، وفلتر Fetch/XHR، واختار طلب، واعمله Copy as cURL، وشيل Headers لحد ما توصل لأقل طلب شغال.
3. صعب سطّب Bruno، واعمل Collection فيها 3 طلبات لـ JSONPlaceholder (GET و POST و DELETE) بمتغير `baseUrl`، وبص على الملفات اللي اتعملت في الفولدر.

09

المرحلة 3 · تتكلم مع API

الفصل التاسع

APIs مجانية حقيقية تتمرّن عليها (واتجرّبت كلها)

أحسن طريقة تفهم بيها الـ APIs إنك تكلّم كتير منها وتقارن. كل API في الفصل ده اتبعتله طلب حقيقي يوم 23 أكتوبر 2026 ورد. هنعرف كل واحد بيعلّمك إيه بالظبط: CRUD وهمي، أو Pagination، أو Auth، أو أخطاء، أو GraphQL، وهنشوف بعين المصمّم إيه اللي حلو فيهم وإيه اللي وحش.

بعد الفصل ده هتكون عارف

- جدول بـ 15 API مجاني: محتاج Key ولا لأ، وحدوده، وبيعلّمك إيه
- APIs الـ Echo: httpbin و Postman Echo علشان تشوف طلبك
- JSONPlaceholder و DummyJSON: تجرّب POST و PUT و DELETE من غير ما تبوّظ حاجة
- PokeAPI و Rick and Morty: Pagination بطريقتين
- Open-Meteo و Frankfurter: Query parameters وأخطاء 400 مفيدة
- NASA: API key وإيه اللي بيحصل من غيره
- درس حقيقي: REST Countries قفلت نسخة قديمة

9.1 الجدول الكبير

كل اللي في الجدول اتجرّب بـ `curl` ورجّع رد صح يوم **23 أكتوبر 2026**. والـ APIs العامة ممكن تتغير أو تقفل (زي ما هتشوف في آخر الفصل)، فلو حاجة وقفت، ارجع للـ Docs بتاعتها.

الـ API	Key؟	بيعلّمك إيه	جرّب
httpbin	لأ	Echo: بيرجّعلك طلبك. Auth و Status codes على الطلب	`httpbin.org/anything`
Postman Echo	لأ	Echo تاني	`postman-echo.com/get?x=1`
JSONPlaceholder	لأ	CRUD وهمي: posts و users و todos و comments	`jsonplaceholder.typicode.com/posts/1`

الـ API	Key؟	بيعلّمك إيه	جرّب
DummyJSON	لأ	Login بـ JWT، وبحث، و `limit` / `skip`، و `select`، وترتيب	`dummyjson.com/products?limit=2`
GitHub REST	اختياري (60/ ساعة من غيره)	API حقيقي بكل حاجة: Versioning و Link و pagination و Rate limit headers	`api.github.com/users/octocat`
PokeAPI	لأ (GET بس)	بيانات متداخلة كبيرة، و Pagination بـ `offset` / `limit`	`pokeapi.co/api/v2/pokemon/pikachu`
Rick and Morty	لأ	Pagination بـ `page`، وفلترة، و 404 بـ JSON	`rickandmortyapi.com/api/character?page=2`
Open-Meteo	لأ (غير تجاري: أقل من 10,000 طلب في اليوم)	Query parameters كتير، ووحدات، وأخطاء 400 واضحة	`api.open-meteo.com/v1/forecast?…`
Frankfurter	لأ	أسعار العملات، و v1 جنب v2	`api.frankfurter.dev/v2/rates?base=USD`
(GraphQL) Countries	لأ	GraphQL حقيقي (فصل 2)	`countries.trevorblades.com/graphql`
Open Library	لأ	بحث، و `fields` لاختيار الحقول	`openlibrary.org/search.json?q=dune`
NASA APOD	**أيوه** (`DEMO_KEY` للتجربة)	API key في الـ Query، وحد صغير	`api.nasa.gov/planetary/apod?api_key=DEMO_KEY`

الـ API	Key؟	بيعلّمك إيه	جرّب
Hacker News	لأ	API على Firebase: IDs وبعدين تجيب كل Item	`hacker-news.firebaseio.com/v0/item/8863.json`
Random User	لأ	بيانات وهمية لأشخاص (للـ UI)	`randomuser.me/api/`
Dog CEO	لأ	أبسط API ممكن: صورة عشوائية	`dog.ceo/api/breeds/image/random`

جدول 9.1 APIs للتمرين

> **الـ URLs في الجدول من غير https://**
>
> علشان الجدول يبقى مقروء، شلنا `https://` من الأول. كلهم HTTPS، فاكتبه وانت بتجرّب.

9.2 الـ Echo: شوف طلبك بعين السيرفر

أول ما تتلخبط "أنا بعت إيه بالظبط؟"، ابعت نفس الطلب لـ httpbin. هو بيرجّعلك الـ Headers والـ Query والـ Body زي ما وصلوه:

Terminal

```
user@MacBook ~ % curl -s "https://httpbin.org/anything?lang=ar" --json '{"title":"Hi"}' | jq
'{method, args, json, ct: .headers["Content-Type"]}'
{
  "method": "POST",
  "args": {
    "lang": "ar"
  },
  "json": {
    "title": "Hi"
  },
  "ct": "application/json"
}
```

`/anything`	بيقبل أي Method وأي Path بعدها، ويرجّع كل حاجة.
`/status/429`	بيرد بالـ Status اللي تطلبه، علشان تجرّب كودك على الأخطاء من غير ما تستنى API حقيقي يقع.
`/basic-auth/user/pass`	بيطلب Basic auth باليوزر والباسورد اللي في المسار (فصل 12).

`/bearer`	بيتأكد إن في `Authorization: Bearer …` .
`/delay/3`	بيستنى 3 ثواني قبل ما يرد: تجرّب الـ Timeouts (فصل 10).

المصدر `Postman Echo` `httpbin.org`

9.3 CRUD وهمي: JSONPlaceholder و DummyJSON

عايز تجرّب `POST` و `PUT` و `DELETE` من غير ما تبوّظ بيانات حد؟ JSONPlaceholder بيرد كإنه نفّذ، بس مبيحفظش حاجة. والدوكيومنتيشن بتاعته بتقولها صراحة: *"The resource is not really created on the server, but the response is faked as it."*

Terminal

```
user@MacBook ~ % curl -s -i --json '{"title":"Hello","userId":1}'
https://jsonplaceholder.typicode.com/posts | grep -iE '^HTTP|^location'
HTTP/2 201
location: https://jsonplaceholder.typicode.com/posts/101
user@MacBook ~ % curl -s -X DELETE https://jsonplaceholder.typicode.com/posts/1 -w ' %
{http_code}\n'
{} 200
user@MacBook ~ % curl -s https://jsonplaceholder.typicode.com/posts/999 -w ' %{http_code}\n'
{} 404
```

بعين المصمّم: **201** و `Location` بعد الإنشاء = ممتاز (فصل 16). بس الـ **404 بـ Body فاضي** `{}` مش مفيد: مفيش أي رسالة بتقولك إيه اللي مش موجود. قارنه بـ GitHub اللي بيرجّع `message` و `documentation_url` .

و**DummyJSON** أغنى: فيه Login حقيقي بيرجّع JWT (هنفصّصه في فصل 12)، وبحث، وتحكم في الحقول والترتيب:

Terminal

```
user@MacBook ~ % curl -s "https://dummyjson.com/products?
limit=2&skip=10&select=title,price&sortBy=price&order=desc" | jq -c .
{"products":[{"id":192,"title":"Rolex Datejust Women","price":10999.99},
{"id":95,"title":"Rolex Cellini Date Black
Dial","price":8999.99}],"total":194,"skip":10,"limit":2}
```

`limit=2&skip=10`	Offset pagination: عدّي أول 10 وهات 2 (فصل 18).
`select=title,price`	Field selection: هات الحقول دي بس، والـ `id` بييجي دايماً. حل بسيط لـ Over-fetching من غير GraphQL.
`sortBy=price&order=desc`	ترتيب. APIs تانية بتكتبها `sort=-price` (فصل 18).
`"total": 194`	العدد الكلي، علشان تعرف فاضل كام صفحة.

المصدر `DummyJSON · Docs` `JSONPlaceholder · Guide`

9.4 Pagination بطريقتين: PokeAPI و Rick and Morty

Terminal

```
user@MacBook ~ % curl -s "https://pokeapi.co/api/v2/pokemon?limit=3&offset=6" | jq '{count,
next, previous}'
{
  "count": 1351,
  "next": "https://pokeapi.co/api/v2/pokemon?offset=9&limit=3",
  "previous": "https://pokeapi.co/api/v2/pokemon?offset=3&limit=3"
}
user@MacBook ~ % curl -s "https://rickandmortyapi.com/api/character?page=2" | jq .info
{
  "count": 826,
  "pages": 42,
  "next": "https://rickandmortyapi.com/api/character?page=3",
  "prev": "https://rickandmortyapi.com/api/character?page=1"
}
user@MacBook ~ % curl -s "https://rickandmortyapi.com/api/character?page=99" -w ' %
{http_code}\n'
{"error":"There is nothing here"} 404
```

الاتنين بيدّوك **URL الصفحة الجاية جاهز** في `next`، فمش محتاج تحسب. ودي عادة ممتازة: الـ Client يمشي ورا `next` لحد ما يبقى `null`. والفرق: PokeAPI بالـ `offset` (بتعدّي كام عنصر) و Rick and Morty بالـ `page` (رقم الصفحة)، ووصلنا لـ 404 لما عدّينا آخر صفحة. وكلام كتير عن ده في فصل 18.

المصدر Rick and Morty API · Docs · PokeAPI · Docs v2

9.5 Query parameters وأخطاء مفيدة: Open-Meteo

Terminal

```
user@MacBook ~ % curl -s "https://api.open-meteo.com/v1/forecast?
latitude=30.04&longitude=31.24&current=temperature_2m,wind_speed_10m&timezone=Africa%2FCairo"
| jq '{timezone, current_units, current}'
{
  "timezone": "Africa/Cairo",
  "current_units": {
    "time": "iso8601",
    "interval": "seconds",
    "temperature_2m": "°C",
    "wind_speed_10m": "km/h"
  },
  "current": {
    "time": "2026-10-23T14:30",
    "interval": 900,
    "temperature_2m": 29.9,
    "wind_speed_10m": 11.9
  }
```

```
}
user@MacBook ~ % curl -s "https://api.open-meteo.com/v1/forecast?latitude=300&longitude=31" -w
' %{http_code}\n'
{"error":true,"reason":"Latitude must be in range of -90 to 90°. Given: 300.0."} 400
```

`latitude / longitude`	إحداثيات القاهرة. والـ API قرّب لأقرب نقطة عنده (`30.03`).
`current=a,b`	اختار المتغيرات اللي عايزها مفصولة بـ `,`.
`timezone=Africa%2FCairo`	الـ `/` في اسم المنطقة اتعملها Encode لـ `%2F` علشان متتلخبطش مع المسار.
`"current_units"`	الوحدات **جنب** الأرقام في Object منفصل: تصميم حلو، فمحدش يخمّن إن 29.9 سيلزيوس ولا فهرنهايت.
`"reason": "…Given: 300.0."`	رسالة الغلط بتقول القاعدة والقيمة اللي بعتها. ده المستوى اللي عايزه في الـ API بتاعك (فصل 17).

المصدر Open-Meteo · Terms (حدود الاستخدام المجاني) Open-Meteo · Docs

9.6 NASA :API key

Terminal

```
user@MacBook ~ % curl -s "https://api.nasa.gov/planetary/apod" -w ' %{http_code}\n'
{
  "error": {
    "code": "API_KEY_MISSING",
    "message": "No api_key was supplied. Get one at https://api.nasa.gov:443"
  }
} 403
user@MacBook ~ % curl -sI "https://api.nasa.gov/planetary/apod?api_key=DEMO_KEY" | grep -i
ratelimit
x-ratelimit-limit: 10
x-ratelimit-remaining: 9
```

حاجتين للمصمّم: الـ `code` ثابت (`API_KEY_MISSING`) تقدر تكتب عليه `if`، والـ `message` للبني آدم وفيه الحل. بس الـ Status **403**، والأدق حسب HTTP كان **401** ("مين انت؟") مش 403 ("عارفك بس ممنوع"). فصل 16 هيفصّل الفرق. و `DEMO_KEY` حدّه صغير جداً (الـ Header قال 10)، فسجّل Key مجاني لو هتستخدمه بجد.

المصدر api.nasa.gov

9.7 درس حقيقي: لما API يقفل نسخة

وإحنا بنجرّب للكتاب، بعتنا لـ REST Countries (API مشهور جداً في الكورسات، ومنها كورس Jonas!) على النسخة القديمة `v3.1`:

```
Terminal
user@MacBook ~ % curl -sL "https://restcountries.com/v3.1/name/egypt?fields=name,capital"
{
    "success": false,
    "data": null,
    "errors": [
        {
            "message": "This API version has been deprecated. Please visit
https://restcountries.com/docs/countries/legacy-api-deprecation to migrate to our new version
(v5)."
        }
    ]
}
```

يعني كل كود قديم بيستخدم `v3.1` وقف. ولو بتذاكر من فيديو قديم بيستخدم REST Countries، هتقابل ده. والدرس للمصمّم (فصل 19): لما تقفل نسخة، قول ده بوضوح، وحط لينك للـ Migration، والأحسن تبعت قبلها بفترة Headers زي `Deprecation` و `Sunset` علشان الناس تلحق.

Best Practice: تمرين يومي 10 دقايق

كل يوم اختار API من الجدول، واقرا صفحة الـ Docs بتاعته 5 دقايق، وابعت 3 طلبات: واحد صح، وواحد ناقص Parameter، وواحد بقيمة غلط. واكتب في ورقة: الـ Status كام؟ وشكل الغلط إيه؟ ده اللي هيبني إحساسك كمصمّم.

ملخص الفصل

- httpbin و Postman Echo: شوف طلبك زي ما وصل، وجرّب Status codes و Auth و Delays.
- JSONPlaceholder: CRUD وهمي (201 + Location حلو، 404 فاضي وحش). DummyJSON: JWT و `select` و `limit/skip`.
- PokeAPI (`offset`) و Rick and Morty (`page`) بيدّوك `next` جاهز.
- Open-Meteo: الوحدات جنب الأرقام، وأخطاء 400 بتقول القاعدة والقيمة.
- NASA: API key في الـ Query، و `code` ثابت، بس 403 مكان 401.
- REST Countries قفلت v3.1: الـ Versioning مش رفاهية.

صح ولا غلط؟

1. JSONPlaceholder بيحفظ الـ Posts اللي بتعملها بـ POST. صح / غلط
2. PokeAPI و Rick and Morty بيستخدموا نفس طريقة الـ Pagination. صح / غلط
3. httpbin مفيد علشان تشوف الطلب بتاعك زي ما السيرفر استلمه. صح / غلط

؟ أسئلة مراجعة

1. إيه اللي حلو في رد الغلط بتاع Open-Meteo؟ وإيه اللي ناقص في 404 بتاع JSONPlaceholder؟
2. ليه `next` كـ URL كامل أحسن من إنك تحسب الـ offset بنفسك؟
3. NASA رجّعت 403 من غير Key. كنت هترجّع إيه انت؟ وليه؟
4. إيه اللي كان ممكن REST Countries تعمله علشان الناس متتفاجئش؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل هات درجة الحرارة الحالية في الإسكندرية من Open-Meteo بـ `timezone=Africa/Cairo` .
2. متوسط اكتب Loop في الـ Shell أو JS بيمشي ورا `next` في Rick and Morty لحد الآخر، ويعد كام شخصية بتاعها `status` `Dead` .
3. صعب اختار 3 APIs من الجدول، وابعت لكل واحد طلب غلط، وقارن شكل الأخطاء في جدول: الـ Status، والـ Fields، وفيه `code` ثابت ولا لأ.

10

المرحلة 3 · تتكلم مع API

الفصل العاشر

الـ API من JavaScript: fetch صح من أول مرة

في كورس Jonas هتكتب fetch و async/await. بس أغلب الكود اللي بتشوفه بيقع في نفس الفخ: fetch مبيرميش Error لما الـ API يرجّع 404 أو 500. الفصل ده بيشرح fetch من ناحية الـ API: الـ Response وفيه إيه، و res.ok، والـ Body اللي بيتقري مرة واحدة، و POST بـ JSON، والـ Timeouts، وبعدين نبني Helper صغير تستخدمه في كل مشروع، ونمشي بيه ورا Pagination حقيقية.

بعد الفصل ده هتكون عارف

- [x] fetch يرجّع Promise لـ Response، ومبيقعش على xx/5xx4
- [x] res.status و res.ok و res.headers.get و res.json() و res.text()
- [x] POST بـ JSON: method و headers و body=JSON.stringify
- [x] أنواع الفشل التلاتة: الشبكة، والـ Timeout، والـ HTTP
- [x] AbortSignal.timeout و URLSearchParams
- [x] Helper اسمه api() بيجمع كل ده، ولوب بيمشي ورا next
- [x] fetch في Node 24 وفي المتصفح: الفرق (CORS)

> **محتاج تفتكر async/await؟**
>
> كل كود الفصل ده فيه `await`. لو مش فاكر هي بتعمل إيه: `fetch` بترجّع Promise (وعد إن الرد جاي)، و `await` = "استنى الوعد ده يتحقق وهات النتيجة"، ومكانها جوه دالة `async` أو في أول ملف `.mjs`. الشرح بالتشبيه في كتاب الإنترنت (فصل 7)، وتشغيل ملفات `.mjs` في صندوق فصل 3 هنا.

10.1 الفخ الأشهر: fetch مبيقعش على 404

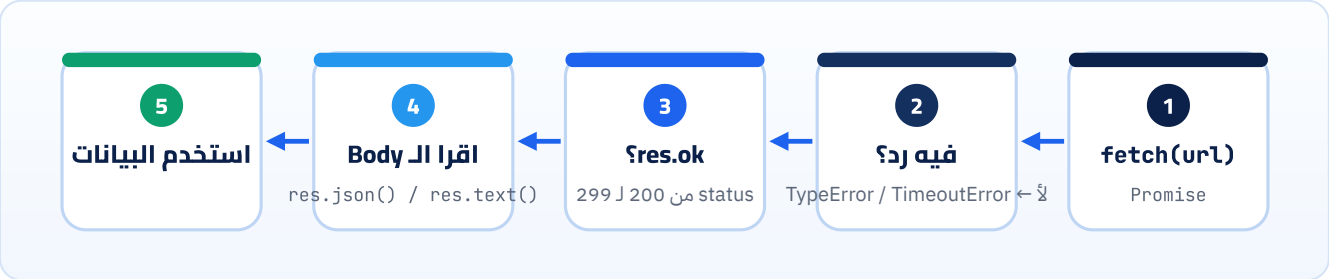
جرّب ده في Node 24 (أو Console المتصفح):

f1.mjs

```
const r404 = await fetch("https://api.github.com/repos/nope-xx/nope");
console.log(r404.status, r404.ok);
console.log(await r404.json());
```

```
Terminal
user@MacBook ~ % node f1.mjs
404 false
{
  message: 'Not Found',
  documentation_url: 'https://docs.github.com/rest/repos/repos#get-a-repository',
  status: '404'
}
```

مفيش Error! الـ `fetch` بيعتبر إن "السيرفر رد" يبقى نجاح، حتى لو الرد "مش لاقي". وده مقصود في المواصفة: الـ Promise بتتـ Reject بس لو **مفيش رد خالص** (الشبكة وقعت، أو الـ DNS فشل، أو اتلغى). فلو كتبت `const data = await (await fetch(url)).json()` من غير ما تبص على الـ Status، كودك هيكمّل بـ `{message: "Not Found"}` كأنه البيانات، ويقع بعدها بـ 10 سطور في حتة ملهاش علاقة.



شكل 10.1 فيه مرحلتين فشل منفصلين: الأولى fetch بترميها لوحدها (مفيش رد). والتانية انت اللي لازم تشيكها بـ res.ok (فيه رد بس مش نجاح). ومعظم الـ Bugs من نسيان التانية.

10.2 الـ Response فيه إيه

```
JavaScript
const res = await fetch("https://api.github.com/repos/expressjs/express", {
  headers: { Accept: "application/vnd.github+json", "User-Agent": "apis-json-book" },
});
console.log(res.status, res.ok, res.statusText);
console.log(res.headers.get("content-type"));
console.log(res.headers.get("x-ratelimit-remaining"));
const repo = await res.json();
console.log(repo.full_name, repo.stargazers_count);
```

```
Terminal
200 true OK
application/json; charset=utf-8
52
expressjs/express 69577
```

`fetch(url, options)`	بترجّع **Promise** لـ `Response`، فلازم `await`. والـ `options` Object فيه `method` و`headers` و`body` و`signal` ...
`headers: { … }`	Object عادي: الـ Key اسم الـ Header. واللي فيه شرطة لازم بين `"…"`: `"User-Agent"`.
`res.status`	الرقم (200). و`res.statusText` الكلمة (`"OK"`)، وفي HTTP/2 ممكن تبقى فاضية، فاعتمد على الرقم.
`res.ok`	`true` لو الـ Status من 200 لـ 299. ده الشيك اللي لازم تعمله.
`res.headers.get("…")`	قيمة Header. ومش فارق Capital ولا Small. ولو مش موجود: `null`.
`await res.json()`	اقرا الـ Body كله واعمله `JSON.parse`. Promise برضه، لأن الـ Body ممكن يكون لسه بيوصل.

مشكلة 1 **Body has already been read :Body is unusable :TypeError**

العَرَض: ناديت `res.json()` مرتين، أو `res.text()` وبعدين `res.json()`.

السبب: الـ Body Stream بيتقري **مرة واحدة بس**. وده الخطأ الحقيقي اللي طلع في Node 24.

الحل: اقراه مرة في متغير: `const text = await res.text()` وبعدين `JSON.parse(text)` لو محتاج الاتنين. أو `res.clone()` قبل القراية.

الـ Method	بترجّع	إمتى
`res.json()`	Array / Object	الـ `content-type` فيه json
`res.text()`	String	HTML أو نص، أو لما مش متأكد (Debug)
`res.blob()` / `res.arrayBuffer()`	بايتات	صور وملفات
`res.body`	Stream	Streaming (زي ردود الـ AI)

جدول 10.1 طرق قراية الـ Body

المصدر MDN · Using the Fetch API · MDN · Response · MDN · Response.ok

10.3 POST بـ JSON

```javascript
const created = await fetch("https://jsonplaceholder.typicode.com/posts", {
  method: "POST",
  headers: { "Content-Type": "application/json" },
  body: JSON.stringify({ title: "Hello", userId: 1 }),
});
console.log(created.status, created.headers.get("location"), await created.json());
// 201 https://jsonplaceholder.typicode.com/posts/101 { title: 'Hello', userId: 1, id: 101 }
```

`method: "POST"`	الافتراضي `"GET"`. ومينفعش GET يبقى فيه `body` (fetch بيرمي TypeError).
`"Content-Type": "application/json"`	نفس درس `curl -d` في فصل 8: من غيره، الـ `body` اللي هو String هيتبعت `text/plain;charset=UTF-8`، وسيرفر Express بـ `express.json()` **هيتجاهله** ويبقى `req.body` فاضي.
`body: JSON.stringify({ … })`	لازم String. لو حطيت الـ Object نفسه، هيتحوّل للنص `"[object Object]"`!

10.4 أنواع الفشل التلاتة

```javascript
try {
  await fetch("https://httpbin.org/delay/5", { signal: AbortSignal.timeout(2000) });
} catch (e) {
  console.log(e.name, e.message);   // TimeoutError The operation was aborted due to timeout
}

try {
  await fetch("https://does-not-exist.example.invalid/");
} catch (e) {
  console.log(e.name, e.message, e.cause?.code);   // TypeError fetch failed …
}
```

الحالة	بيظهر إزاي	تعمل إيه
مفيش شبكة، أو Domain غلط، أو السيرفر قافل	`TypeError: fetch failed`، والتفاصيل في `e.cause.code` (زي `ENOTFOUND` أو `ECONNREFUSED` في Node). وفي المتصفح: `TypeError: Failed to fetch` من غير تفاصيل	Retry بعد شوية، أو بلّغ المستخدم إنه Offline
السيرفر أبطأ من الحد	`TimeoutError` (مع `AbortSignal.timeout`)	Retry محدود
السيرفر رد بغلط	**مفيش Error**: `res.ok === false`	اقرا الـ Body: 4xx صلّح الطلب، 429/5xx Retry (كتاب Python for AI فصل 25 فيه الـ Backoff)

جدول 10.2 إزاي تعرف إيه اللي حصل

`AbortSignal.timeout(2000)`	Signal بيلغي الطلب بعد 2000 ميلّي ثانية. **fetch معندوش Timeout افتراضي** تتحكم فيه من الكود، فحط واحد دايماً في الكود اللي بيشتغل لوحده.
`e.cause?.code`	في Node، الـ Error الحقيقي جوه `cause`. والـ `?.` علشان لو `cause` مش موجود ميقعش.

الـ Error اللي طلع عندنا

في بيئة الكتابة، الـ Domain الغلط طلّع `TypeError fetch failed` و`cause.code` = `ECONNRESET` بسبب الـ Network بتاع البيئة. على الماك عندك المتوقع `ENOTFOUND` (الـ DNS ملقاش الاسم). الفكرة واحدة: `TypeError` = مفيش رد.

10.5 صح Query string :URLSearchParams

```JavaScript
const params = new URLSearchParams({ q: "APIs و JSON", limit: 2 });
params.toString();                    // 'q=APIs+%D9%88+JSON&limit=2'
await fetch(`https://httpbin.org/get?${params}`).then((r) => r.json());
// args: { q: 'APIs و JSON', limit: '2' }
```

نفس فكرة `--data-urlencode`: بيعمل الـ Encoding للمسافات والعربي والـ `&` لوحده. ومتركّبش `?q=${q}` بإيدك: لو `q` فيها `&` هتقسم الـ Query لاتنين. ولاحظ إن الأرقام بتوصل String (`'2'`): الـ Query string كله نص، والسيرفر هو اللي يحوّل (فصل 22).

المصدر `MDN · AbortSignal.timeout()` `MDN · URLSearchParams` `Node.js 24 · fetch`

10.6 Helper واحد لكل مشروع

بدل ما تكرر الشيكات دي في كل حتة، اعمل Function واحدة. ودي اللي هتستخدمها في المشروع (فصل 23):

api.mjs

```
export class ApiError extends Error {
  constructor(status, body, url) {
    super(`HTTP ${status} for ${url}`);
    this.name = "ApiError";
    this.status = status;
    this.body = body;
  }
}

export async function api(url, { method = "GET", body, token, timeout = 10_000 } = {}) {
  const headers = { Accept: "application/json" };
  if (body !== undefined) headers["Content-Type"] = "application/json";
  if (token) headers.Authorization = `Bearer ${token}`;

  const res = await fetch(url, {
    method,
    headers,
    body: body === undefined ? undefined : JSON.stringify(body),
    signal: AbortSignal.timeout(timeout),
  });

  const type = res.headers.get("content-type") ?? "";
  const data = res.status === 204 ? null
    : type.includes("json") ? await res.json()
    : await res.text();

  if (!res.ok) throw new ApiError(res.status, data, url);
  return data;
}
```

سطر سطر

1 Error خاص بينا شايل الـ `status` والـ `body`، علشان اللي بينادي يقدر يقرر (يعيد على 429، يعرض رسالة على 422...). وبـ `instanceof ApiError` يفرّق بينه وبين غلطة شبكة.	`class ApiError extends Error`
2 Destructuring بقيم افتراضية (كورس Jonas). والـ `{}` في الآخر علشان لو ناديت `api(url)` من غير Options ميقعش. و `10_000` = 10000 (الـ `_` للقراية).	`{ method = "GET", body, token, timeout = 10_000 } = {}`
3 نحط `Content-Type` بس لو فيه Body فعلاً. والـ Bracket notation لأن الاسم فيه شرطة.	`if (body !== undefined) headers["Content-Type"] = …`

4 شكل الـ Bearer token (فصل 12). كلمة `Bearer` ومسافة وبعدين الـ Token.	`headers.Authorization = `Bearer ${token}``
5 **No Content 204** مفيهوش Body، و `res.json()` عليه بيقع بـ *Unexpected end of JSON input* (فصل 5).	`res.status === 204 ? null : …`
6 بيغطي `application/json` و `application/problem+json` و `…+json`. وأي حاجة تانية (زي صفحة HTML غلط) نقراها نص بدل ما `json()` يقع بـ `Unexpected token '<'`.	`type.includes("json") ? await res.json() : await res.text()`
7 هنا بنحوّل "رد غلط" لـ Exception، فبقى سلوكه زي غلطة الشبكة: لازم حد يمسكه.	`if (!res.ok) throw new ApiError(…)`

use.mjs

```
import { api, ApiError } from "./api.mjs";

try {
  await api("https://api.github.com/repos/nope-xx/nope");
} catch (err) {
  if (err instanceof ApiError) console.log(err.message, "|", err.body.message);
  else throw err;
}

let url = "https://rickandmortyapi.com/api/character?status=dead";
let dead = 0, pages = 0;
while (url) {
  const page = await api(url);
  dead += page.results.length;
  pages++;
  url = page.info.next;
}
console.log({ pages, dead });
```

Terminal

```
user@MacBook ~ % node use.mjs
HTTP 404 for https://api.github.com/repos/nope-xx/nope | Not Found
{ pages: 15, dead: 287 }
```

أي Error مش بتاعنا (شبكة أو Bug) نرميه تاني زي ما هو. متبلعش الأخطاء.	`else throw err`
امشي ورا `next` لحد ما يبقى `null` (اللوب يقف لوحده). 15 طلب جابوا 287 شخصية `dead` وقت الكتابة. وفي API حقيقي حط `await` صغير بين الطلبات أو حد أقصى للصفحات علشان الـ Rate limit.	`while (url) { … url = page.info.next }`

10.7 Node ضد المتصفح

fetch في Node 24

- موجود جاهز (من Node 18)، من غير `npm install`.
- مفيش CORS: أي سيرفر تقدر تكلّمه.
- بيبعت `User-Agent: node` لوحده.
- الأسرار (Tokens) آمنة لأنها على جهازك أو السيرفر.

fetch في المتصفح

- نفس الـ API بالظبط.
- **CORS**: الـ Response بيتقري بس لو السيرفر سمح (فصل 13، وكتاب الإنترنت فصل 14).
- بعض الـ Headers ممنوعة (زي `User-Agent` و `Cookie`).
- **أي Token في الكود ظاهر لأي حد**: افتح DevTools وهتلاقيه.

حاجة مشتركة: نفس الـ Helper في Python

في كتاب Python for AI (فصل 11) عملنا نفس الفكرة بـ `requests`: `r = requests.get(url, timeout=10)` وبعدين `r.raise_for_status()` اللي بترمي Error لو الـ Status ≥ 400. يعني `raise_for_status()` = `if (!res.ok) throw …` و`timeout=10` = `AbortSignal.timeout(10_000)`. ونفس القاعدة: من غير Timeout، الاسكريبت ممكن يستنى للأبد.

ملخص الفصل

- `fetch` مبيرميش Error على xx/5xx4. شيك `res.ok` دايماً.
- `res.status` و`res.headers.get` و`res.json()` / `res.text()`، والـ Body بيتقري مرة واحدة.
- POST: `method` + `Content-Type: application/json` + `body: JSON.stringify(…)`.
- تلات أنواع فشل: `TypeError` (مفيش رد)، و`TimeoutError`، و`!res.ok`.
- `AbortSignal.timeout(ms)` دايماً، و`URLSearchParams` للـ Query.
- Helper واحد يرجّع البيانات أو يرمي `ApiError` فيه الـ Status والـ Body.

صح ولا غلط؟

1. `await fetch(url)` يرمي Error لو الـ API رجّع 500. صح / غلط
2. `body: { title: "x" }` من غير stringify بيتبعت JSON صح. صح / غلط
3. تقدر تنادي `res.json()` مرتين على نفس الـ Response. صح / غلط

؟ أسئلة مراجعة

1. إمتى الـ Promise بتاعة fetch بتتـ Reject؟ وإمتى لأ؟
2. ليه الـ Helper بيشيك على 204 قبل `res.json()` ؟
3. إيه اللي هيحصل في سيرفر Express لو نسيت `Content-Type: application/json` ؟
4. ليه متحطش API key سري في كود Frontend؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اكتب `fetch` لـ `https://httpbin.org/status/503` واطبع `status` و `ok` ، واتأكد إن مفيش Error.
2. متوسط زوّد على الـ Helper: لو الـ Status 429 أو 503 وفيه `Retry-After` ، استنى الثواني دي وأعد مرة واحدة بس.
3. صعب اكتب `paginate(url)` كـ `async function*` (Async generator) بيعمل `yield` لكل عنصر، وتستخدمه بـ `for await (const c of paginate(url))` .

11

المرحلة 3 · تتكلم مع API

الفصل الحادي عشر

تقرا أي API Docs، وتفهم ملف OpenAPI كمستخدم

أول ما تفتح دوكيومنتيشن API كبير زي GitHub أو Stripe، بتلاقي مئات الصفحات. تبدأ منين؟ الفصل ده بيديك ترتيب قراية ثابت، ويشرّح صفحة Reference واحدة حتة حتة، وبعدين يفتح ملف OpenAPI حقيقي (بتاع Frankfurter اللي استخدمناه في أول فصل) ويقراه بـ jq، علشان تعرف إن الصفحات الحلوة دي كلها طالعة من ملف JSON واحد.

بعد الفصل ده هتكون عارف

- ترتيب قراية ثابت لأي API: من Getting started لحد Changelog
- تشريح صفحة Endpoint: Method و Path و Parameters و Body و Responses
- الفرق بين Path و Query و Header و Body parameters
- يعني إيه OpenAPI و Swagger، وإيه الفرق
- تقرا ملف OpenAPI حقيقي بـ jq: servers و paths و components و parameters
- Swagger UI و Redoc و Scalar: الدوكيومنتيشن اللي بتتولد من الملف
- توليد Client و Types من الملف

11.1 ترتيب القراية: 8 محطات

الغلطة الشائعة إنك تروح على طول لصفحة الـ Endpoint اللي عايزه وتنسخ المثال. وبعدين تتفاجئ بـ 401، أو تاخد أول 30 عنصر بس ومتعرفش إن في صفحات، أو تخبط في الـ Rate limit. فاقرا بالترتيب ده، ولو بسرعة:

1. **Getting started / Overview**: الـ API بيعمل إيه، والـ **Base URL** (زي `https://api.github.com`)، وأول طلب.
2. **Authentication**: محتاج Key؟ Token؟ OAuth؟ وبيتبعت فين (فصل 12 و 13).
3. **Versioning**: في `/v2`؟ ولا Header؟ ولو مبعتّوش إيه اللي بيحصل (فصل 19).
4. **Errors**: شكل الغلط، والـ Codes الثابتة (فصل 17).
5. **Rate limits**: الحد كام، والـ Headers اسمها إيه، وبتعمل إيه لما تخبط.
6. **Pagination**: `page`؟ `cursor`؟ `Link` header؟ والحد الأقصى للصفحة.
7. **Reference**: صفحة الـ Endpoint اللي عايزه (تحت).
8. **Changelog**: إيه اللي اتغير قريب، وإيه اللي هيتشال.

Best Practice: اعمل كارت للـ API

بعد ما تقرا، اكتب كارت من 8 سطور: الـ Base URL، والـ Auth Header، والنسخة، وشكل الغلط، والحد، والـ Pagination، وأهم 3 Endpoints. الكارت ده هو اللي هتحطه في `README` بتاع مشروعك، وهو بالظبط نفس الأقسام اللي هتكتبها لما تعمل API بتاعك.

11.2 تشريح صفحة Reference

خلينا ناخد صفحة "Get a repository" في GitHub (اللي جربناها في فصل 3). أي صفحة Reference محترمة فيها نفس الأجزاء دي، حتى لو الترتيب مختلف:

رأس الصفحة

```
GET /repos/{owner}/{repo}
```

الجزء	فيه إيه	في المثال
Path + Method	الفعل والعنوان. والـ `{...}` = Placeholder تحط مكانه قيمة	`GET /repos/{owner}/{repo}`
الوصف	بيعمل إيه، وأي حالة خاصة	"The parent and source objects are present when the repository is a fork"
الصلاحيات	محتاج Token؟ بأنهي Scopes أو Permissions؟	Public repos من غير Token. Private محتاج Permission "read :Metadata"
Headers	الـ Headers المطلوبة أو المقترحة	`accept`: يُفضّل `application/vnd.github+json`
Path parameters	جوه المسار، **إجبارية دايماً**	`owner` و `repo` (و GitHub بيقول case-insensitive)
Query parameters	بعد `?`، غالباً اختيارية وليها Default	مفيش هنا. في `List issues` هتلاقي `state` و `per_page` و `page`
Body parameters	في `POST` / `PATCH`: الـ Fields ونوعها وإجباري ولا لأ	في `Update a repository`: `name` و `description` ...
Responses	كل Status ممكن، ومعاه شكل الـ Body	`200` و `301` (اتنقل) و `403` و `404`
Examples	طلب ورد جاهزين بـ curl و JS وغيره	زر Copy

جدول 11.1 أجزاء صفحة الـ Endpoint

`Path parameter`	جزء من هوية الـ Resource: `/repos/expressjs/express`. لو اتغير، بقى Resource تاني.
`Query parameter`	تعديل على **طريقة العرض**: فلترة وترتيب وصفحات. `?state=open&per_page=50`.

`Header parameter`	معلومات عن الطلب نفسه مش عن الـ Resource: الـ Auth والنسخة والشكل.
`Body parameter`	البيانات اللي بتبعتها علشان تعمل أو تعدّل. JSON غالباً.

> ⚠ **خلي بالك: الـ Responses اللي مش مكتوبة**
>
> حتى لو الصفحة كاتبة `200` و`404` بس، كودك لازم يتعامل مع **أي** Status: 401 لو الـ Token انتهى، و429 لو خبطت في الحد، و500 و503 لو السيرفر عنده مشكلة. والـ Helper بتاع فصل 10 بيعمل كده لوحده.

المصدر GitHub · List repository issues · GitHub · Get a repository

11.3 OpenAPI: الدوكيومنتيشن ملف JSON

القصة: كل شركة كانت بتكتب الدوكيومنتيشن بإيدها في صفحات HTML. فالصفحات بتبعد عن الكود، ومفيش أداة تقدر "تفهم" الـ API لوحدها. سنة 2011 ظهر مشروع اسمه **Swagger**: ملف JSON/YAML بيوصف الـ API بالكامل، وأداة بتعرضه كصفحة حلوة فيها زر "Try it out". واتبرع بيه سنة 2015 لمؤسسة اسمها OpenAPI Initiative، واتغير اسم المواصفة لـ **OpenAPI**.

OpenAPI

المواصفة نفسها: إزاي تكتب ملف وصف الـ API. آخر نسخة **3.2.1** (سبتمبر 2026)، و 3.2.0 نزلت سبتمبر 2025، وأغلب الملفات اللي هتشوفها 3.0 أو 3.1.

Swagger

النهارده: **اسم مجموعة أدوات** من شركة SmartBear: Swagger UI و Swagger Editor. يعني "ملف Swagger" بالمعنى القديم = ملف OpenAPI 2.0. والناس لسه بتقول الكلمتين مكان بعض.

وأحلى حاجة: كتير من الـ APIs بتنشر الملف ده. Frankfurter (أول API جربناه) بينشره على `/v2/openapi.json`. نزّله واقراه بـ jq:

```
Terminal
user@MacBook ~ % curl -s https://api.frankfurter.dev/v2/openapi.json -o fr.json
user@MacBook ~ % jq -r '.openapi, .info.title, .info.version' fr.json
3.1.2
Frankfurter API
2.1.1
user@MacBook ~ % jq -c '.servers' fr.json
[{"url":"https://api.frankfurter.dev/v2"}]
user@MacBook ~ % jq -r '.paths | keys[]' fr.json
/coverage
/currencies
/currency/{code}
/providers
/providers/{provider}
```

```
/providers/{provider}/rate/{base}/{quote}
/providers/{provider}/rates
/rate/{base}/{quote}
/rates
```

`.openapi`	نسخة **المواصفة** اللي الملف مكتوب بيها (3.1.2).
`.info.version`	نسخة **الـ API نفسه** (2.1.1). حاجتين مختلفين خالص، وناس كتير بتلخبط بينهم.
`.servers`	الـ Base URL. وكل الـ Paths بتتحط بعده: `https://api.frankfurter.dev/v2` + `/rates`.
`.paths`	كل Endpoints الـ API. كل Key مسار، وجواه Method لكل عملية.

عملية واحدة من جوه

Terminal

```
user@MacBook ~ % jq '.paths["/rates"].get | {summary, operationId, params:
[.parameters[]."$ref"], responses: (.responses | keys)}' fr.json
{
  "summary": "Get exchange rates",
  "operationId": "getRates",
  "params": [
    "#/components/parameters/date",
    "#/components/parameters/from",
    "#/components/parameters/to",
    "#/components/parameters/base",
    "#/components/parameters/quotes",
    "#/components/parameters/providers",
    "#/components/parameters/group",
    "#/components/parameters/expand"
  ],
  "responses": [
    "200",
    "404",
    "422",
    "503"
  ]
}
user@MacBook ~ % jq -c '.components.parameters.base' fr.json
{"name":"base","in":"query","schema":{"type":"string","default":"EUR","example":"USD"}}
```

بنقرا العقد

`.paths["/rates"].get`	1 الـ `["…"]` لأن الـ Key فيه `/`. والـ `get` = العملية بتاعة `GET /rates`.

2 اسم فريد للعملية. الأدوات اللي بتولّد كود بتعمل منه اسم Function: `client.getRates()`.	`"operationId": "getRates"`
3 نفس `$ref` بتاع JSON Schema (فصل 7): الـ Parameter متعرّف مرة واحدة تحت في `components`، وكل العمليات بتشاور عليه.	`"$ref": "#/components/parameters/base"`
4 الـ Parameter ده بيتبعت فين: `query` أو `path` أو `header` أو `cookie`. (الـ Body ليه مكان لوحده اسمه `requestBody`.)	`"in": "query"`
5 لو مبعتّوش. وده جواب التحدي الصعب في فصل 1: لو شلت `base`، السعر بيتحسب بالـ EUR.	`"default": "EUR"`
6 الـ Statuses الموثّقة. و422 = Parameter قيمته غلط: جربنا `base=XYZ` ورجع `422` و `{"status":422,"message":"invalid currency: XYZ"}`.	`"responses": ["200", "404", "422", "503"]`

Terminal

```
user@MacBook ~ % jq '.components.schemas.Rate | {required, props: (.properties | keys)}' -c fr.json
{"required":["date","base","quote","rate"],"props":["base","date","providers","quote","rate"]}
```

وده JSON Schema عادي جداً (فصل 7)! بيقولك كل عنصر في الرد فيه 4 حقول مضمونين، و`providers` اختياري. يعني تقدر تكتب كودك وانت متأكد إن `rate` موجود.

المصدر Frankfurter · openapi.json · OpenAPI Specification (latest) · OpenAPI Initiative · What is OpenAPI

11.4 الصفحات اللي بتتولد من الملف

نفس الملف ده بيتحوّل لدوكيومنتيشن تفاعلية بأدوات زي:

الأداة	بتعمل
Swagger UI	الصفحة الكلاسيكية: كل Endpoint ليه زر **Try it out** يبعت طلب حقيقي من المتصفح. وأي API Express بيستخدم `swagger-ui-express` بيعرضها على `/docs`.
Redoc	صفحة دوكيومنتيشن من 3 أعمدة، أشيك للقراية، من غير Try it out في النسخة المجانية.
Scalar	أحدث، وفيه API client جوه الصفحة.
Insomnia / Postman / Bruno	Import للملف = Collection جاهزة بكل الطلبات (فصل 8).

جدول 11.2 أدوات العرض

مثال عملي: Petstore

افتح `https://petstore3.swagger.io/` : ده المثال الرسمي من Swagger، ملفه OpenAPI **3.0.4** فيه 13 Path (`/pet` و `/pet/{petId}` و `/store/order` و...). افتح `GET /pet/{petId}` ← Try it out ← حط 1 ← Execute. هتشوف الـ curl اللي اتبعت، والرد، والـ Headers. وبعدين حط `abc` وشوف الـ 400.

توليد Client و Types من الملف

لأن الملف دقيق، في أدوات بتكتبلك كود منه: **openapi-typescript** بيطلّع Types لـ TypeScript لكل Request و Response، و**OpenAPI Generator** بيطلّع Client كامل بأكتر من 50 لغة. و GitHub نفسه بينشر ملف OpenAPI للـ REST API كله في Repo اسمه `github/rest-api-description`، ومنه بيتعمل SDK بتاعهم `octokit`. وده اللي هتعمله بعدين في React: Types من ملف الـ API بدل ما تخمّن.

المصدر `Swagger UI` `Redoc` `Scalar` `openapi-typescript` `github/rest-api-description`

ملخص الفصل

- اقرا بالترتيب: Getting started ← Auth ← Versioning ← Errors ← Rate limits ← Pagination ← Reference ← Changelog.
- صفحة الـ Endpoint: Path + Method، وصلاحيات، و Parameters (path و query و header و body)، و Responses، وأمثلة.
- Path = هوية الـ Resource، و Query = طريقة العرض، و Header = عن الطلب، و Body = البيانات.
- OpenAPI = المواصفة (3.2.1 الأحدث)، و Swagger = أدوات. والملف فيه `openapi` و `info` و `servers` و `paths` و `components`.
- `openapi.` نسخة المواصفة، و `info.version` نسخة الـ API.
- من الملف: Swagger UI و Redoc و Scalar، و Collections، و Types و Clients.

صح ولا غلط؟

1. Swagger و OpenAPI حاجتين ملهمش علاقة ببعض. صح / غلط
2. الـ Path parameters عادة اختيارية. صح / غلط
3. الـ `info.version` في ملف OpenAPI هو نسخة مواصفة OpenAPI. صح / غلط

أسئلة مراجعة

1. ليه تقرا صفحة الـ Errors والـ Rate limits قبل صفحة الـ Endpoint؟
2. إمتى تخلّي حاجة Path parameter وإمتى Query parameter؟ إدّي مثال.
3. إيه فايدة `operationId` ؟
4. اكتب أمر jq يطلّع كل الـ `operationId` في ملف `fr.json` .

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل "كارت" بالـ 8 سطور لـ Frankfurter من صفحة الـ Docs بتاعته.
2. متوسط من `fr.json` : إيه الـ Parameters بتاعة `/rate/{base}/{quote}` ؟ وأنهي منهم `in: path` ؟ وجرّب العملية بـ curl.
3. صعب اعمل Import لـ `https://api.frankfurter.dev/v2/openapi.json` في Bruno أو Postman، وشغّل 3 طلبات من الـ Collection اللي اتولدت.

12

المرحلة 4 · الـ Auth

الفصل التاني عشر

الـ Auth الجزء 1: API keys و Basic و Bearer، والـ JWT متشرّح

أي API مش عام لازم يعرف مين بيكلّمه. في الفصل ده هنمشي على طرق إثبات الهوية من الأبسط للأشهر: الـ API key، و Basic auth، و Bearer tokens، وبعدين أشهر Token في الدنيا: الـ JWT. هناخد JWT حقيقي من DummyJSON ونفكّه حتة حتة، وبعدين نعمل واحد بإيدنا في Node ونحاول نزوّره ونشوف إزاي التوقيع بيمسكنا.

بعد الفصل ده هتكون عارف

- الفرق بين Authentication و Authorization، و 401 و 403
- API keys: فين تتبعت، والـ Publishable ضد الـ Secret
- Basic auth: Base64 مش تشفير
- Bearer tokens و WWW-Authenticate
- الـ Sessions بالكوكيز ضد الـ Tokens
- JWT: الـ Header والـ Payload والـ Signature، وكل Claim
- تعمل JWT بإيدك في Node، وتزوّره، وتشوف التوقيع بيرفض
- أخطاء JWT المشهورة: alg none، وأسرار في الـ Payload، وتخزين الـ Token

12.1 كلمتين لازم يفضلوا منفصلين

(AuthN) Authentication

"**انت مين؟**" إثبات الهوية: باسورد، أو Key، أو Token. ولو فشل: **401 Unauthorized** (الاسم مضلل، معناه الحقيقي "Unauthenticated").

(AuthZ) Authorization

"**مسموحلك تعمل ده؟**" عرفنا انت مين، بس هل ليك صلاحية تمسح المقالة دي؟ ولو لأ: **403 Forbidden**.

الفصل ده عن الأولى أساساً. والتانية قرار جوه الـ API بتاعك (Roles و Permissions)، وهتشوفها في Express. وفي فصل 16 هنرجع للفرق بين 401 و 403 بالتفصيل.

12.2 API keys: أبسط حاجة

الـ API key = String طويل عشوائي، الـ Provider بيدّيهولك لما تسجّل، وبتبعته مع كل طلب. والسيرفر بيدوّر عليه في الـ Database: "الـ Key ده بتاع مين؟ وحدّه كام؟". وبيتبعت في مكان من تلاتة:

المكان	مثال	رأيك فيه
Header خاص	`x-api-key: abc123`	كويس، وشائع جداً
`Authorization`	`Authorization: Bearer sk-…` (OpenAI)	الأحسن: الـ Header الرسمي، والأدوات عارفة إنه سر فبتخفيه في اللوجز
Query string	`?api_key=DEMO_KEY` (NASA)	**أوحش واحد**: الـ URL بيتسجّل في لوجز السيرفرات والـ Proxies والـ History بتاع المتصفح، وبيتنسخ مع اللينك

جدول 12.1 فين بيتبعت الـ Key

Publishable key

معمول إنه يبقى في كود الـ Frontend، وصلاحياته محدودة جداً (زي Stripe `pk_…` اللي بيعمل Token للكارت بس، أو مفتاح Google Maps المربوط بـ Domain بتاعك).

Secret key

صلاحيات كاملة (Stripe `sk_…`، و OpenAI `sk-…`، و GitHub `ghp_…` و `github_pat_…`). **مكانه السيرفر بس**، في Environment variable، وعمره ما يتحط في Git ولا في كود المتصفح.

ليه الـ Keys ليها Prefix؟

GitHub بيبدأ كل نوع Token بـ Prefix ثابت: `ghp_` للـ Personal access token الكلاسيكي، و `github_pat_` للـ Fine-grained، و `gho_` لـ OAuth. الفايدة: أدوات الـ Secret scanning (في GitHub نفسه وغيره) بتلاقي أي Token اتعمله Push بالغلط في أي Repo وتبلّغ عنه أو تلغيه. وده سبب إنك لو عملت Push لـ `.env` فيه Key، ممكن يتلغي في دقايق.

المصدر: Stripe · API keys، GitHub · Token prefixes

12.3 Basic auth: Base64 مش تشفير

أقدم طريقة في HTTP: اليوزر والباسورد في Header. والسيرفر لو عايزها بيقولك كده في الـ 401:

Terminal

```
user@MacBook ~ % curl -si https://httpbin.org/basic-auth/user/pass | grep -iE '^HTTP|www-auth'
HTTP/2 401
www-authenticate: Basic realm="Fake Realm"
user@MacBook ~ % curl -s -u user:pass https://httpbin.org/basic-auth/user/pass
{
  "authenticated": true,
  "user": "user"
}
user@MacBook ~ % echo -n 'user:pass' | base64
dXNlcjpwYXNz
user@MacBook ~ % echo dXNlcjpwYXNz | base64 -d
user:pass
```

`www-authenticate: Basic realm="…"`	السيرفر بيقول: "أنا عايز Basic auth". والـ `realm` اسم للمنطقة المحمية. والمتصفح لما يشوف الـ Header ده بيطلّع نافذة اليوزر والباسورد الرمادي المشهورة.
`-u user:pass`	curl بيعمل `user:pass`، ويحوّله Base64، ويبعت `Authorization: Basic dXNlcjpwYXNz`.
`echo -n`	الـ `-n` = "متحطش سطر جديد في الآخر". من غيرها الـ Base64 هيبقى لـ `user:pass\n` وهيطلع غلط.
`base64 -d`	"Decode". وفكّيناه في ثانية: يعني **Base64 = Encoding مش Encryption**. أي حد شاف الـ Header شاف الباسورد. فـ Basic auth من غير HTTPS = الباسورد مكشوف (كتاب الإنترنت فصل 12).

هتقابل Basic في: Stripe (الـ Secret key كيوزر وباسورد فاضي: `-u sk_test_…:`)، وأدوات داخلية، و Registries. ولـ API عام جديد، استخدم Tokens.

المصدر MDN · HTTP authentication · RFC 7617 · Basic Authentication

12.4 Bearer tokens

"Bearer" يعني "حامل": **أي حد شايل التوكن ده يتعامل كصاحبه**، زي تذكرة السينما: الكمسري مش بيسألك انت مين، بيبص على التذكرة. والشكل اتحدد في **RFC 6750**:

```http
GET /user HTTP/1.1
Host: api.github.com
Authorization: Bearer ghp_xxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxxx
```

`Authorization:`	نفس الـ Header بتاع Basic.
`Bearer`	اسم الـ Scheme. وبعده **مسافة واحدة**. وغلطة شائعة: `Bearer:` بنقطتين، أو نسيان الكلمة خالص.
`ghp_…`	الـ Token نفسه. ممكن يبقى String عشوائي (Opaque) السيرفر بيدوّر عليه، أو JWT جواه البيانات (تحت).

```
Terminal
user@MacBook ~ % curl -s https://api.github.com/user -H "Authorization: Bearer ghp_fake" -i |
grep -E '^HTTP' ; curl -s https://api.github.com/user -H "Authorization: Bearer ghp_fake"
HTTP/2 401
{
  "message": "Bad credentials",
  "documentation_url": "https://docs.github.com/rest",
  "status": "401"
}
```

و RFC 6750 بيقول إن السيرفر **المفروض** يرجّع مع الـ 401 Header زي `WWW-Authenticate: Bearer error="invalid_token"`. GitHub مبيعملش ده (جربنا)، وكتير من الـ APIs كده. فلما تبني API بتاعك، رجّعه: الأدوات بتفهمه.

المصدر MDN · WWW-Authenticate RFC 6750 · Bearer Token Usage

12.5 Sessions بالكوكيز ضد Tokens

في كتاب الإنترنت (فصل 13) شفنا الـ Session: بتعمل Login، والسيرفر يحفظ "اليوزر ده عامل Login" عنده، ويدّيك Cookie فيها ID. والمتصفح يبعتها لوحده مع كل طلب. والـ Token فكرة مختلفة:

	Cookie + Session	(Bearer) Token
مين يبعته	المتصفح لوحده	كودك بيحطه في الـ Header
السيرفر بيحفظ حاجة؟	أيوه (Session store)	Opaque: أيوه. JWT: لأ، البيانات جوه التوكن
مناسب لـ	موقع ويب نفس الـ Domain	Mobile و CLI و سيرفر لسيرفر و APIs عامة
الخطر الأشهر	CSRF (المتصفح بيبعتها لوحده)	السرقة (XSS لو في `localStorage`)

جدول 12.2 الاتنين جنب بعض

12.6 الـ JWT: تذكرة عليها ختم

المشكلة: عندك 10 سيرفرات، وكل طلب جاي بـ Token. كل سيرفر لازم يسأل الـ Database: "التوكن ده بتاع مين؟ وصلاحياته إيه؟". ده ضغط كبير على الـ Database. **فاتطور حل**: نحط المعلومات دي **جوه التوكن نفسه**، ونختمه بختم محدش يقدر يزوّره غير السيرفر. أي سيرفر يقدر يتأكد من الختم من غير ما يسأل حد. والتوكن ده اسمه **JSON Web Token** (RFC 7519)، وبيتنطق "jot".

اعمل Login حقيقي على DummyJSON:

```
Terminal
user@MacBook ~ % curl -s https://dummyjson.com/auth/login --json
'{"username":"emilys","password":"emilyspass","expiresInMins":30}' | jq -r .accessToken
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJpZCI6MSwidXNlcm5hbWUiOiJlbWlseXMiLCJlbWFpbCI6ImVtaWx5L
mpvaG5zb25AeC5kdW1teWpzb24uY29tIiwiZmlyc3ROYW1lIjoiRW1pbHkiLCJsYXN0TmFtZSI6IkpvaG5zb24iLCJnZW5
kZXIiOiJmZW1hbGUiLCJpbWFnZSI6Imh0dHBzOi8vZHVtbXlqc29uLmNvbS9pY29uL2VtaWx5cy8xMjgiLCJpYXQiOjE3O
TI3NTM5NTksImV4cCI6MTc5Mjc1NTc1OX0.XgA2LhaGESyVHmT4EaYxHlb-lLO-vuQLMR5LO8D5RHI
```

بص كويس: في **نقطتين** بيقسموه **3 حتت**. وكل حتة Base64url. والـ `eyJ` اللي في الأول دايماً هي `{"` بعد الـ Encoding، فأول ما تشوف `eyJ…` اعرف إنه JWT.



شكل 12.1 تشريح الـ JWT. الحتتين الأولانيين مجرد JSON متعمله Base64url، يعني أي حد يقدر يقراهم. التالتة هي اللي بتثبت إن محدش لعب فيهم.

```
Terminal
user@MacBook ~ % node -e 'const [h, p] = process.argv[1].split(".");
console.log(JSON.parse(Buffer.from(h, "base64url"))); console.log(JSON.parse(Buffer.from(p,
"base64url")))' "$TOKEN"
{ alg: 'HS256', typ: 'JWT' }
{
  id: 1,
  username: 'emilys',
  email: 'emily.johnson@x.dummyjson.com',
  firstName: 'Emily',
  lastName: 'Johnson',
  gender: 'female',
  image: 'https://dummyjson.com/icon/emilys/128',
  iat: 1792753959,
  exp: 1792755759
}
```

`process.argv[1]`	مع `node -e`، أول Argument بعد الكود هو `argv[1]`. وبعتنا التوكن من Variable `"$TOKEN"`.
`.split(".")`	قسّم على النقط، وخد أول اتنين بـ Destructuring.
`Buffer.from(h, "base64url")`	فك الـ **Base64url**: نسخة من Base64 آمنة في الـ URLs (RFC 4648): `-` بدل `+`، و `_` بدل `/`، ومن غير `=` في الآخر. وعلشان كده `base64 -d` العادي ساعات يقع عليه.

كل Claim ومعناه

1 الخوارزمية اللي اتعمل بيها الختم: HAS + MAC**H** **256**-. يعني ختم بـ **سر واحد** عند السيرفر (Symmetric). البديل `RS256` أو `ES256` : مفتاح خاص يختم ومفتاح عام يتأكد (Asymmetric)، علشان سيرفرات تانية تقدر تتأكد من غير ما تعرف السر.	`"alg": "HS256"`
2 نوع التوكن. اختياري.	`"typ": "JWT"`
3 Claims خاصة بـ DummyJSON (Private claims). والمعيار بيقترح `sub` (Subject) لهوية اليوزر بدل `id` .	`"id": 1, "username": …, "email": …`
4 **Issued At**: اتعمل إمتى (Unix time بالثواني) = 2:12 م بتوقيت القاهرة.	`"iat": 1792753959`
5 **Expiration**: بينتهي إمتى. الفرق 1800 ثانية = **30 دقيقة**، بالظبط اللي طلبناه في `expiresInMins` . وجربنا توكن بدقيقة واحدة: اشتغل (200)، وبعد 70 ثانية رجع **401** و `{"message":"Token Expired!"}` . وتوكن متلخبط خالص بيرجّع `{"message":"Invalid/Expired Token!"}` .	`"exp": 1792755759`

Claim	اسمه	معناه
`iss`	Issuer	مين عمل التوكن (`https://auth.example.com`)
`sub`	Subject	التوكن عن مين (ID اليوزر، String)
`aud`	Audience	معمول لأنهي API. والـ API لازم يرفض توكن مش ليه
`exp`	Expiration	بعده مرفوض
`nbf`	Not Before	قبله مرفوض
`iat`	Issued At	اتعمل إمتى
`jti`	JWT ID	ID فريد للتوكن (علشان تقدر تلغيه أو تمنع إعادة استخدامه)

جدول 12.3 الـ Registered claims في RFC 7519

المصدر RFC 7519 · JWT · RFC 7519 · §4.1 Registered Claims · RFC 4648 · §5 base64url · DummyJSON · Auth

12.7 نعمل JWT بإيدنا ونحاول نزوّره

علشان تفهم الختم بجد، خلينا نكتبه من الصفر بـ `node:crypto` من غير مكتبات. (في الشغل الحقيقي هتستخدم مكتبة، تحت.)

jwt.mjs

```
import { createHmac, timingSafeEqual } from "node:crypto";

const b64url = (obj) => Buffer.from(JSON.stringify(obj)).toString("base64url");
```

```
function sign(payload, secret) {
  const header = { alg: "HS256", typ: "JWT" };
  const data = `${b64url(header)}.${b64url(payload)}`;
  const sig = createHmac("sha256", secret).update(data).digest("base64url");
  return `${data}.${sig}`;
}

function verify(token, secret) {
  const [h, p, s] = token.split(".");
  const expected = createHmac("sha256", secret).update(`${h}.${p}`).digest("base64url");
  if (s.length !== expected.length || !timingSafeEqual(Buffer.from(s), Buffer.from(expected)))
{
    throw new Error("bad signature");
  }
  const payload = JSON.parse(Buffer.from(p, "base64url"));
  if (payload.exp && payload.exp < Date.now() / 1000) throw new Error("expired");
  return payload;
}
```

سطر سطر

1 الـ **HMAC**: "بصمة" للنص `data` محسوبة بالسر. لو حرف واحد في النص اتغير، أو السر مختلف، البصمة بتختلف خالص. وده نفس اللي هنستخدمه في توقيع الـ Webhooks (فصل 20).	`createHmac("sha256", secret).update(data).digest("base64url")`
2 الختم بيتحسب على **الحتتين الأولانيين مع النقطة**، مش على الـ Payload لوحده.	`` `${b64url(header)}.${b64url(payload)}` ``
3 مقارنة بتاخد نفس الوقت مهما كان أول حرف مختلف فين. الـ `===` العادية بتقف عند أول اختلاف، والفرق في الوقت ممكن يتقاس ويستخدم لتخمين الختم حرف حرف (Timing attack). ولازم الطولين يبقوا زي بعض الأول وإلا الـ Function بترمي Error.	`timingSafeEqual(…)`
4 `Date.now()` بالميلّي، و `exp` بالثواني، فبنقسم على 1000.	`if (payload.exp && payload.exp < Date.now() / 1000)`

Terminal

```
user@MacBook ~ % node jwt.mjs
eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJzdWIiOiI0MiIsInJvbGUiOiJlZGl0b3IiLCJpYXQiOjE3OTI3NTU0M
TIsImV4cCI6MTc5Mjc1NjMxMn0.kagj3bF95VTxmgTm8M_K20hAOznHmal2Io8A-WrMkjw
{ sub: '42', role: 'editor', iat: 1792755412, exp: 1792756312 }
tampered: bad signature
wrong secret: bad signature
```

في السطر التالت، خدنا الـ Payload وغيّرنا `role` من `editor` لـ `admin` وحطيناه مكان القديم بنفس الختم: **اترفض**. ومن غير السر، مستحيل تعمل ختم جديد صح. ده كل سحر الـ JWT: **أي حد يقرا، محدش يعدّل**.

خلي بالك: أخطاء JWT المشهورة

1. **أسرار في الـ Payload**: الـ Payload مقروء لأي حد (فكّيناه بسطر). متحطش باسورد ولا بيانات حساسة.
2. `alg: "none"` : المواصفة بتسمح بتوكن "من غير ختم". ومكتبات قديمة كانت بتقبله لو المهاجم غيّر الـ Header! فدايماً حدد الـ Algorithms المسموحة في الـ Verify (`algorithms: ["HS256"]`). وده أول تحذير في **RFC 8725** Best practices للـ JWT).
3. **سر ضعيف**: HS256 بسر زي `"secret"` بيتكسر بالتخمين. استخدم 32 بايت عشوائي على الأقل من Environment variable.
4. **مفيش طريقة "تلغي" JWT** قبل الـ `exp` من غير ما ترجع تسأل Database. علشان كده الـ Access token بيبقى قصير (5 لـ 15 دقيقة) ومعاه **Refresh token** أطول بيجيب واحد جديد (DummyJSON رجّع `refreshToken` جنب الـ `accessToken`).

Best Practice: في Express هتستخدم مكتبة

أشهر اتنين في Node: `jsonwebtoken` (9.0.3، وهتلاقيه في كورسات كتير): `jwt.sign(payload, secret, { expiresIn: "15m" })` و `jwt.verify(token, secret, { algorithms: ["HS256"] })`. و `jose` (6.2) الأحدث وبيشتغل في Node والمتصفح و Edge. وفي المتصفح: الأأمن تخلّي التوكن في **Cookie** عليها `HttpOnly` و `Secure` و `SameSite` (كتاب الإنترنت فصل 13) بدل `localStorage`، علشان أي XSS ميقدرش يقراه.

المصدر RFC 8725 · JWT Best Current Practices · Node.js · crypto.createHmac · jsonwebtoken · jose

ملخص الفصل

- AuthN = انت مين (401)، و AuthZ = مسموحلك (403).
- API key في Header (`Authorization` أو `x-api-key`) مش في الـ Query. والـ Secret key مكانه السيرفر بس.
- Basic = `user:pass` بـ Base64، يعني مكشوف من غير HTTPS.
- Bearer: `Authorization: Bearer <token>`، وأي حد شايله = صاحبه.
- JWT = `header.payload.signature` بـ Base64url. مقروء لأي حد، ومحدش يعدّله من غير السر.
- Claims: `iss` و `sub` و `aud` و `exp` و `iat`. وحدد الـ `algorithms`، ومتحطش أسرار، و Access قصير + Refresh.

صح ولا غلط؟

1. Basic auth بيشفّر الباسورد. صح / غلط
2. أي حد معاه JWT يقدر يقرا الـ Payload بتاعه. صح / غلط
3. تقدر تغيّر `role` في JWT وتخلّي الختم القديم صالح. صح / غلط

أسئلة مراجعة

1. ليه API key في الـ Query string فكرة وحشة؟ اذكر 3 أماكن ممكن يتسرّب فيها.
2. اشرح الفرق بين HS256 و RS256 في جملتين.
3. ليه الـ Access token بيبقى قصير؟ وإيه دور الـ Refresh token؟
4. إيه هجوم `alg: none` ؟ وإزاي تمنعه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل اعمل Login على DummyJSON بـ `expiresInMins: 1` ، وابعت التوكن لـ `/auth/me` فوراً وبعد دقيقتين. قارن الردين.
2. متوسط استخدم `jwt.mjs` واعمل توكن بـ `exp` في الماضي، واتأكد إن الـ `verify` بيقول `expired` .
3. صعب زوّد على `verify` : شيك على `aud` (لازم يساوي `"articles-api"`) وعلى `alg` في الـ Header (لازم `"HS256"` بالظبط).

13

المرحلة 4 · الـ Auth

الفصل التالت عشر

الـ Auth الجزء 2: OAuth 2.0 ببساطة، والـ CORS من ناحية السيرفر

زمان، علشان تطبيق يجيب أصحابك من Gmail، كان بيطلب منك باسورد Gmail نفسه! OAuth اتعمل علشان يحل المصيبة دي: تدّي التطبيق مفتاح محدود ومؤقت بدل المفتاح الرئيسي. هنمشي على الـ Flows الأربعة اللي هتقابلها خطوة بخطوة بالـ URLs والـ Parameters، وبعدين نلبس طاقية السيرفر ونفهم الـ CORS: إيه الـ Headers اللي الـ API بتاعك لازم يرجّعها علشان موقعك يقدر يكلّمه.

بعد الفصل ده هتكون عارف

- المشكلة: Password anti-pattern، والحل: مفتاح الـ Valet
- الأدوار الأربعة: Resource owner و Client و Authorization server و Resource server
- PKCE + Authorization Code خطوة بخطوة، وكل Parameter
- Client Credentials و Device flow و Refresh
- ليه Implicit و Password اتشالوا (RFC 9700 و OAuth 2.1)
- OpenID Connect و id_token في سطرين
- CORS من ناحية الـ API: Allow-Origin و Methods و Headers و Credentials و Expose و Max-Age
- الـ Preflight بيحصل إمتى، وتجربة حقيقية على Express

13.1 الحكاية: التطبيقات اللي كانت بتطلب باسوردك

حوالي 2007، مواقع كتير كان فيها زرار "اعزم أصحابك من Gmail". وكانت بتعمل كده إزاي؟ **بتطلب منك إيميلك وباسورد Gmail**، وتدخل بيهم بالنيابة عنك وتسحب الـ Contacts. **وهنا كانت المشكلة**: الموقع ده بقى معاه مفتاح بيتك كله: يقرا إيميلاتك، ويبعت باسمك، ويغيّر الباسورد. ولو عايز تلغي ده، لازم تغيّر باسوردك نفسه في كل حتة. والاسم اللي الناس سمّته للحكاية دي: **Password anti-pattern**.

فاتطور حل: زي "مفتاح الـ Valet" في العربيات الغالية: مفتاح بيشغّل العربية ويسوقها مسافة صغيرة، بس مبيفتحش الشنطة ولا الدرج. **OAuth 2.0** (RFC 6749، سنة 2012) بيخلّي Google نفسها تسألك: "التطبيق ده عايز يقرا الـ Contacts بس. موافق؟". ولو وافقت، Google تدّي التطبيق **Access token** محدود (Scopes) ومؤقت، وتقدر تلغيه في أي وقت من غير ما تغيّر باسوردك. والتطبيق **عمره ما يشوف الباسورد**.

13.2 الأدوار الأربعة

الدور	يعني	في مثال "Login with GitHub" لموقع مقالات
Resource owner	صاحب البيانات	انت
Client	التطبيق اللي عايز يوصل	موقع المقالات
Authorization server	اللي بيسألك ويدّي التوكن	`github.com/login/oauth/…`
Resource server	الـ API اللي فيه البيانات	`api.github.com`

جدول 13.1 مين مين في OAuth

13.3 PKCE + Authorization Code: الـ Flow الأساسي

ده اللي بيحصل لما تدوس "Sign in with Google" أو "Login with GitHub" في أي موقع أو تطبيق. 6 خطوات:



شكل 13.1 الـ code اللي بيعدّي في المتصفح (خطوة 3) لوحده مالوش قيمة: لازم يتبدّل في خطوة 4 من التطبيق نفسه، ومعاه إثبات إنه نفس اللي بدأ الطلب (PKCE). والـ access_token عمره ما بيظهر في شريط العنوان.

خطوة 1: الـ Redirect للـ Authorization server

```http
GET https://auth.example.com/authorize
  ?response_type=code
  &client_id=articles-web
  &redirect_uri=https%3A%2F%2Farticles.example.com%2Fcallback
  &scope=articles%3Aread%20profile
  &state=x7Fq2kLp
  &code_challenge=E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM
  &code_challenge_method=S256
```

`response_type=code`	"عايز Authorization code" (مش توكن على طول).
`client_id`	ID التطبيق، بتاخده لما تسجّل التطبيق عند الـ Provider. **مش سر.**
`redirect_uri`	يرجّعك فين بعد الموافقة. ولازم يطابق **بالظبط** واحد من اللي مسجّلين مسبقاً، وإلا أي حد يقدر يخلّي الـ code يروحله. واتعمله Percent-encoding لأنه URL جوه URL.
`scope=articles:read profile`	الصلاحيات المطلوبة، مفصولة بمسافة (`%20`). ودي اللي بتظهرلك في شاشة الموافقة. والـ `:` بقت `%3A`.
`state`	رقم عشوائي التطبيق بيحفظه، وبيرجع زي ما هو في خطوة 3. لو مرجعش نفسه = حد بيحاول يدخّلك في Login مش بتاعك (CSRF)، فارفض.
`code_challenge`	الـ **PKCE** (بتتنطق "Pixy"، RFC 7636): التطبيق بيعمل سر عشوائي اسمه `code_verifier`، ويبعت هنا **بصمته** بس: `base64url(sha256(verifier))`. جربناها على المثال الرسمي في RFC 7636: `dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk` بصمته `E9Melhoa2OwvFrEMTJguCHaoeK1t8URWbuGJSstw-cM` بالظبط.
`code_challenge_method=S256`	البصمة معمولة بـ SHA-256. (في `plain` بس متستخدمهاش.)

خطوة 3 و 4: الـ code ويتبدّل

```http
GET https://articles.example.com/callback?code=SplxlOBeZQQYbYS6WxSbIA&state=x7Fq2kLp

POST /token HTTP/1.1
Host: auth.example.com
Content-Type: application/x-www-form-urlencoded

grant_type=authorization_code&code=SplxlOBeZQQYbYS6WxSbIA&redirect_uri=https%3A%2F%2Farticles.example.com%2Fcallback&client_id=articles-web&code_verifier=dBjftJeZ4CVP-mB92K27uhbUJU1p1r_wW1gFWFOEjXk
```

`code=…`	قصير العمر (دقايق) ويستخدم مرة واحدة.
`Content-Type: application/x-www-form-urlencoded`	طلب الـ Token في OAuth بيتبعت **Form** مش JSON، حسب المواصفة. (اللي اتعلمناه في فصل 8 عن `-d` بيبان هنا: ده المكان اللي `-d` العادية صح فيه.)
`grant_type=authorization_code`	نوع الطلب. وكل Flow ليه `grant_type` .
`code_verifier`	السر الأصلي. الـ Server يحسب بصمته ويقارنها بالـ `code_challenge` من خطوة 1. فلو حد سرق الـ code من المتصفح، ميقدرش يبدّله لأنه معندوش الـ verifier.

خطوة 5: الرد

```json
{
  "access_token": "eyJhbGciOiJSUzI1NiIsInR5cCI6ImF0K2p3dCJ9…",
  "token_type": "Bearer",
  "expires_in": 900,
  "refresh_token": "tGzv3JOkF0XG5Qx2TlKWIA",
  "scope": "articles:read profile"
}
```

`access_token`	التوكن اللي هيتبعت للـ API. ممكن JWT (زي هنا، و RFC 9068 بيحدد شكله) أو Opaque.
`token_type: "Bearer"`	يعني يتبعت `Authorization: Bearer …` (فصل 12).
`expires_in: 900`	صالح 900 ثانية = 15 دقيقة. **بالثواني من دلوقتي**، مش Unix time زي `exp` .
`refresh_token`	يجيب Access token جديد من غير ما المستخدم يعمل Login تاني: `grant_type=refresh_token&refresh_token=…` . ويتحفظ كويس جداً.

`scope`	الصلاحيات اللي اتدّت فعلاً. ممكن تبقى أقل من اللي طلبتها لو المستخدم رفض حاجة.

المصدر RFC 6749 · §4.1 Authorization Code Grant · RFC 7636 · PKCE · oauth.net/2

13.4 باقي الـ Flows

Flow	لمين	بيشتغل إزاي
PKCE + Authorization Code	أي تطبيق فيه مستخدم: ويب و SPA و Mobile	اللي فوق
Client Credentials	سيرفر بيكلّم سيرفر، **مفيش مستخدم**	`POST /token` بـ `grant_type=client_credentials` و `client_id` و `client_secret`، ويرجع Access token على طول
(RFC 8628) Device Authorization	أجهزة من غير متصفح أو كيبورد كويس: CLI و TV	الجهاز ياخد `user_code` قصير زي `WDJB-MJHT`، ويقولك "افتح `github.com/login/device` واكتبه"، ويفضل يسأل كل `interval` ثواني لحد ما توافق. ده اللي `gh auth login` بيعمله
Refresh Token	تجديد Access token	`grant_type=refresh_token`

جدول 13.2 تختار أنهي Flow

> **خلي بالك: Flows اتشالت**
>
> هتلاقي في مقالات قديمة **Implicit flow** (التوكن بيرجع في الـ URL على طول، كان للـ SPAs) و**Password grant** (التطبيق ياخد اليوزر والباسورد ويبدّلهم بتوكن). الاتنين **متستخدمهمش**: **RFC 9700** (Best practices لأمان OAuth، يناير 2025) بيقول Implicit "SHOULD NOT" والـ Password "MUST NOT". ومسودة **OAuth 2.1** (لسه Draft، آخر نسخة -16 في سبتمبر 2026) شالتهم خالص، وخلّت PKCE إجباري لكل الـ Authorization Code flows.

> **OpenID Connect: OAuth للـ Login**
>
> OAuth لوحده بيقول "التطبيق ده مسموحله يوصل لـ API". مبيقولش "المستخدم ده مين". فاتعمل فوقه **OpenID Connect (OIDC)**: بتضيف `openid` في الـ `scope`، فيرجعلك جنب الـ `access_token` واحد كمان اسمه `id_token` : JWT فيه `sub` و`email` و`name`. ده اللي ورا "Sign in with Google". وفي Express مش هتكتب الـ Flow بإيدك: هتستخدم مكتبة (زي `openid-client` أو Passport) أو خدمة (Auth0 و Clerk و Supabase Auth...).

المصدر RFC 9700 · OAuth 2.0 Security BCP · OAuth 2.1 (draft) · RFC 8628 · Device Grant
GitHub · Authorizing OAuth apps · OpenID · How Connect works

13.5 CORS من ناحية الـ API

في كتاب الإنترنت (فصل 14) شفنا CORS من ناحية المتصفح: موقع على `http://localhost:5173` بيعمل `fetch` لـ API على `http://localhost:3000` (Port مختلف = Origin مختلف)، فالمتصفح بيمنع الـ JS يقرا الرد **إلا لو السيرفر قال إنه موافق**. ولما تبني الـ API في Express (وهتعمل React على Port تاني)، انت اللي هتقول "موافق". يعني الحل **دايماً على السيرفر**، مش في كود الـ Frontend.

> ⚠ **CORS مش حماية لـ API بتاعك**
>
> curl و Postman و أي سيرفر **مبيهتموش بـ CORS خالص**. CORS بيحمي **المستخدم** من إن موقع شرير مفتوح في المتصفح يقرا بياناته من موقع تاني بالكوكيز بتاعته. فالـ API بتاعك لازم يكون محمي بالـ Auth، و CORS مجرد "مين من المتصفحات يقدر يقرا".

الـ Preflight: بيحصل إمتى

لو الطلب "بسيط" (GET أو POST بـ Content-Type من نوع Form أو text/plain، ومن غير Headers خاصة)، المتصفح بيبعته على طول ويشيك على الرد. أما لو فيه أي حاجة من دول، بيبعت قبله طلب `OPTIONS` يسأل الأول (الـ **Preflight**):

- Method غير `GET` و`HEAD` و`POST` (زي `PATCH` و`DELETE` و`PUT`).
- `Content-Type: application/json`. يعني **أي POST بـ JSON بيعمل Preflight**.
- أي Header زي `Authorization` أو `X-Api-Key`.

جربنا الـ Preflight حقيقي على سيرفر Express 5.3 فيه `cors({ origin: "http://localhost:5173" })` (هنبنيه في فصل 22):

```
Terminal
user@MacBook ~ % curl -si -X OPTIONS http://localhost:3000/v1/articles \
    -H 'Origin: http://localhost:5173' \
    -H 'Access-Control-Request-Method: POST' \
    -H 'Access-Control-Request-Headers: content-type'
HTTP/1.1 204 No Content
X-Powered-By: Express
Access-Control-Allow-Origin: http://localhost:5173
Vary: Origin, Access-Control-Request-Headers
Access-Control-Allow-Methods: GET,HEAD,PUT,PATCH,POST,DELETE
Access-Control-Allow-Headers: content-type
Content-Length: 0
```

Header	معناه	ملاحظات المصمّم
`Access-Control-Allow-Origin`	مين يقرا: Origin واحد بالظبط أو `*`	`*` للـ APIs العامة من غير كوكيز (زي GitHub). غير كده حدد Origins بعينها من قايمة
`Vary: Origin`	الرد بيختلف حسب الـ Origin	لازم لو بترجّع Origin مختلف لكل طلب، علشان الـ Cache ميدّيش رد Origin لـ Origin تاني

Header	معناه	ملاحظات المصمّم
`Access-Control-Allow-Methods`	الـ Methods المسموحة (في الـ Preflight)	مكتبة `cors` بتحط الكل افتراضياً
`Access-Control-Allow-Headers`	الـ Headers المسموحة (في الـ Preflight)	لازم يبقى فيه `authorization` لو بتبعت Bearer token. والمكتبة بترجّع اللي اتطلب
`Access-Control-Allow-Credentials: true`	مسموح بالكوكيز	**مينفعش مع** `Allow-Origin: *` . لازم Origin محدد
`Access-Control-Expose-Headers`	الـ Headers الرد اللي الـ JS يقدر يقراها	افتراضياً الـ JS مبيشوفش غير Headers أساسية. GitHub بيعمل Expose لـ `ETag` و `Link` و `X-RateLimit-*` و `Retry-After` (فصل 3)، علشان الـ Frontend يعرف يعمل Pagination ويحترم الحد
`Access-Control-Max-Age`	الـ Preflight يتخزّن كام ثانية	بيقلل طلبات الـ OPTIONS

جدول 13.3 الـ Headers الـ CORS اللي الـ API بيرجّعها

Best Practice: قايمة Origins، مش مراية

غلطة خطيرة ومشهورة: سيرفر بياخد أي `Origin` جاي في الطلب ويرجّعه زي ما هو في `Access-Control-Allow-Origin` مع `Allow-Credentials: true` . ده معناه إن **أي موقع في الدنيا** يقدر يقرا بيانات المستخدم بالكوكيز بتاعته. اعمل قايمة ثابتة:

```
cors({ origin: ["http://localhost:5173", "https://articles.example.com"], credentials: true })
```

.

المصدر MDN · CORS · Fetch Standard · CORS protocol · Express · cors middleware

ملخص الفصل

- OAuth = مفتاح محدود ومؤقت بدل الباسورد، والتطبيق عمره ما يشوف باسوردك.
- 4 أدوار: Resource owner و Client و Authorization server و Resource server.
- PKCE + Authorization Code: `/authorize` بـ `state` و `code_challenge` ← `code` ← `POST /token` بـ `code_verifier` ← `access_token` + `refresh_token` .
- Client Credentials لسيرفر-سيرفر، و Device flow للـ CLI والـ TV. و Implicit و Password ممنوعين.
- OIDC فوق OAuth: `scope=openid` ← `id_token` .
- CORS قرار السيرفر، ومش حماية للـ API. والـ JSON والـ Authorization بيعملوا Preflight.
- `Allow-Origin` من قايمة + `Vary: Origin` ، ومفيش `*` مع Credentials، و `Expose-Headers` للـ Link والـ Rate limit.

صح ولا غلط؟

1. في OAuth الـ `client_id` سر لازم يتخبّى. صح / غلط
2. POST بـ `Content-Type: application/json` من المتصفح بيعمل Preflight. صح / غلط
3. لو الـ API مش راجع Headers الـ CORS، curl مش هيقدر يكلّمه. صح / غلط

؟ أسئلة مراجعة

1. إيه المشكلة اللي `state` بيحلّها؟ وإيه اللي PKCE بيحلّه؟
2. ليه الـ code بيتبدّل بتوكن في طلب منفصل بدل ما التوكن يرجع على طول في الـ URL؟
3. عندك Cron job على سيرفر بيكلّم API كل ساعة. أنهي Flow؟ وليه؟
4. ليه `Access-Control-Expose-Headers` مهم لـ API فيه Pagination بالـ `Link` header؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل في Node، اعمل `code_verifier` عشوائي بـ `crypto.randomBytes(32).toString("base64url")` واحسب الـ `code_challenge` بتاعه.
2. متوسط افتح أي موقع فيه "Login with Google" أو GitHub، وراقب تابة Network وانت بتعمل Login: لاقي طلب `/authorize` واقرا الـ Parameters بتاعته (`scope` و `state` و `code_challenge`).
3. صعب من Console صفحة `https://example.com` ، اعمل `fetch("https://api.github.com/users/octocat")` واقرا `res.headers.get("x-ratelimit-remaining")` . اشتغل؟ ليه؟ وجرّب نفس الكلام مع `https://httpbin.org/get` بـ Header `Authorization` ، وراقب الـ Preflight في Network.

14

المرحلة 5 · تصميم REST

الفصل الرابع عشر

REST: الفكرة، والـ Resources، والـ Endpoints، والتسمية

من هنا بتلبس طاقية المصمّم. أول قرار في أي API: الـ URLs شكلها إيه. هنبدأ بـ API متصمم بالأفعال وبيوجع، وبعدين نفهم REST من مصدره (رسالة Fielding)، ونطلّع قواعد تسمية واضحة: جمع ولا مفرد، و Nesting لحد فين، والـ IDs، والعمليات اللي مش CRUD. وفي الآخر نصمم الـ Resources بتاعة الـ API اللي هيمشي معانا لآخر الكتاب.

بعد الفصل ده هتكون عارف

- المشكلة: API بالأفعال (/getArticles و /deleteArticle)
- قيود REST الست من رسالة Fielding بلغة بسيطة
- Resource ضد Representation
- Richardson Maturity Model: المستويات من 0 لـ 3
- قواعد التسمية: جمع، حروف صغيرة، شرطة، من غير أفعال
- الـ Nesting: /articles/42/comments لحد مستوى واحد
- العمليات اللي مش CRUD: publish و archive
- الـ IDs: أرقام ولا UUID، وليه
- تصميم الـ Resources الـ Content Library API

14.1 API بالأفعال: الوجع

تخيل إنك ورثت API لموقع مقالات، واللي عمله كان بيضيف Endpoint لكل حاجة بيحتاجها:

```http
GET  /getArticles
GET  /getArticleById?id=42
POST /createArticle
POST /updateArticle
GET  /deleteArticle?id=42
POST /articles/publishArticle
GET  /fetchCommentsForArticle?articleId=42
POST /Article/Comment/Add
```

كل Endpoint جديد لازم تفتح الدوكيومنتيشن علشان تعرف اسمه. ومفيش قاعدة: مرة `get` ومرة `fetch`، ومرة `Article` ومرة `articles`. وأخطر حاجة: `GET /deleteArticle?id=42` . ليه خطر؟ لأن `GET` المفروض "Safe" (فصل 15)، فأي برنامج بيعمل Prefetch للينكات، أو Crawler، أو Cache، ممكن **يمسح مقالات** وهو بيتفرج! وده حصل فعلاً لمواقع زمان.

14.2 REST من المصدر: 6 قيود

في الفصل الخامس من رسالة Fielding (2000)، REST مش "API بـ JSON". هو **مجموعة قيود** (Constraints)، ولو التزمت بيها السيستم بتاعك بيكسب نفس المميزات اللي خلّت الويب يكبر: يتوسّع، ويتخزّن، ويتغير من غير ما يكسر حد. بلغة بسيطة:

القيد	يعني	بيدّيك إيه
Client-Server	الواجهة منفصلة عن البيانات	React و Mobile و CLI على نفس الـ API
Stateless	كل طلب فيه كل اللي السيرفر محتاجه (التوكن، والصفحة...). السيرفر مبيفتكرش "انت كنت فين"	أي سيرفر من 10 يقدر يرد، فتكبّر بسهولة
Cacheable	الرد بيقول يتخزّن ولا لأ	`Cache-Control` و `ETag` (فصل 3 و 19)
Uniform interface	نفس الطريقة لكل حاجة: Resources ليها URLs، وبتتعامل معاها بـ Representations، والرسايل بتشرح نفسها، واللينكات بتوديك للخطوة الجاية	تتعلم مرة وتستخدم في كل حتة
Layered system	الـ Client ميعرفش في Proxy ولا CDN في النص	تحط Cache و Load balancer من غير ما تغيّر حاجة
Code on demand (اختياري)	السيرفر ممكن يبعت كود يتنفذ	الـ JS في صفحات الويب

جدول 14.1 قيود REST

Resource

"الحاجة" نفسها كفكرة: المقالة رقم 42. ليها **عنوان واحد**: `/articles/42`. ممكن تكون في Database أو ملف، مش فارق.

Representation

"شكل" الحاجة اللي بيتبعت: JSON، أو HTML، أو CSV. نفس المقالة ممكن ليها أكتر من Representation حسب `Accept`. وده الـ **R** في REST: *Representational* State Transfer.

لو حابب تتعمق: Richardson Maturity Model

تقدر تعدّيه في أول قراية

طريقة مشهورة (من Leonard Richardson، وشرحها Martin Fowler) تقيس بيها "الـ API ده REST قد إيه":

المستوى 0: URL واحد و POST لكل حاجة (SOAP، و JSON-RPC).

المستوى 1: Resources: كل حاجة ليها URL (`/articles/42`) بس لسه كله POST.

المستوى 2: HTTP Methods و Status codes صح (`GET` و `DELETE` و 201 و 404). **أغلب الـ APIs الكويسة هنا، وده هدفنا.**

المستوى 3: **Hypermedia** (اسمها HATEOAS): الرد فيه لينكات للخطوات الجاية، زي `next` في PokeAPI، أو `"_links": {"publish": "/articles/42:publish"}`. ده اللي Fielding بيسميه REST بجد، وقليل اللي بيوصله كامل.

المصدر Fielding · Chapter 5: REST Martin Fowler · Richardson Maturity Model

14.3 قواعد التسمية

نفس الـ API بتاع فوق بعد ما نطبّق القواعد:

```http
GET    /v1/articles              ← القايمة
POST   /v1/articles              ← مقالة جديدة
GET    /v1/articles/42           ← مقالة واحدة
PATCH  /v1/articles/42           ← تعديل جزء
DELETE /v1/articles/42           ← مسح
POST   /v1/articles/42:publish   ← عملية مش CRUD (تحت)
GET    /v1/articles/42/comments  ← تعليقات المقالة
POST   /v1/articles/42/comments  ← تعليق جديد عليها
```

القاعدة	صح	غلط	ليه
أسماء مش أفعال	`/articles`	`/getArticles`	الفعل هو الـ Method. الـ L = "إيه"، والـ Method = "تعمل إيه"
جمع للـ Collections	`/articles/42`	`/article/42`	"المقالة 42 من مجموعة المقالات" وثابت: متخلطش جمع ومفرد
حروف صغيرة	`/articles`	`/Articles`	الـ Path حساس للحروف (بعكس الـ Domain)، فـ `/Articles` و `/articles` ممكن يبقوا حاجتين
شرطة بين الكلمات	`/reading-lists`	`/readingLists` أو `/reading_lists`	عُرف الـ URLs (Kebab-case). والـ `_` بتستخبى تحت خط اللينك
من غير امتداد	`/articles/42`	`/articles/42.json`	الشكل بيتحدد بـ `Accept` و `Content-Type`
من غير `/` في الآخر	`/articles`	`/articles/`	عند سيرفرات كتير دول URLاين مختلفين. اختار واحد وحوّل التاني

القاعدة	صح	غلط	ليه
Nesting مستوى واحد	`/articles/42/comments`	`/authors/7/articles/42/comments/9`	التعليق 9 ليه ID لوحده: `comments/9` المسارات الطويلة بتكسر لو العلاقة اتغيرت
الفلترة في الـ Query	`/articles?status=draft`	`/articles/drafts`	الـ Drafts مش Resource تاني، نفس المقالات بفلتر (فصل 18)

جدول 14.2 القواعد وسبب كل واحدة

> **مفيش Standard رسمي للأسماء**
>
> مفيش RFC بيقول "لازم جمع" أو "لازم Kebab-case". دي أعراف اتفقت عليها الصناعة ومكتوبة في Style guides كبيرة: **Google AIP** (aip.dev)، و**Microsoft REST API Guidelines**، و**Zalando RESTful API Guidelines**. والأهم من أي قاعدة: **الثبات**. API فيه قاعدة واحدة "مش مثالية" أحسن من API فيه 3 قواعد مثالية.

14.4 العمليات اللي مش CRUD

"انشر المقالة"، و"ابعت الإيميل تاني"، و"إلغي الطلب". دول مش Create ولا Update بالظبط. عندك 3 حلول:

الطريقة	شكلها	إمتى
تعديل Field	`PATCH /articles/42` بـ `{"status": "published"}`	لو العملية فعلاً مجرد تغيير حالة من غير آثار جانبية كبيرة
Custom method	`POST /articles/42:publish`	لو فيها شغل (إيميلات، Cache، Validation خاص). ده أسلوب Google AIP-136: `:` وبعدها الفعل، ودايماً `POST`
Resource للعملية	`POST /articles/42/publications`	لو العملية نفسها ليها تاريخ وحالة (اتعملت إمتى وبواسطة مين)

جدول 14.3 طرق للعمليات

والـ `:publish` ده ببان غريب، بس ميزته إنه واضح إنه "فعل" مش Resource فرعي اسمه `publish`. وكتير من الـ APIs بتكتبه `/articles/42/publish` بـ `/`. الاتنين مقبولين، المهم تختار واحد.

المصدر Google AIP-122 · Resource names · Google AIP-136 · Custom methods · Microsoft · API Guidelines · Zalando · RESTful API Guidelines

14.5 الـ IDs

أرقام متسلسلة (1، 2، 3...)

- قصيرة وسهلة في الـ Debug.
- بس بتكشف حجمك (`id: 101` = عندك 100 مقالة).
- وبتسهّل التخمين: لو `/invoices/1043` بتاعتك، جرّب `1044` . (لازم Authorization على كل طلب في الحالتين، بس متسهلهاش.)

IDs / ULID / UUID بـ Prefix

- `"7f9c2a1e-…"` : مستحيل تخمّنه، وتقدر تعمله في الـ Client قبل ما تبعت.
- Stripe بيعمل IDs بـ Prefix: `cus_…` و `ch_…` ، فتعرف النوع من شكله.
- أطول وأصعب في القراية.

Best Practice: الـ ID في الـ JSON دايماً String

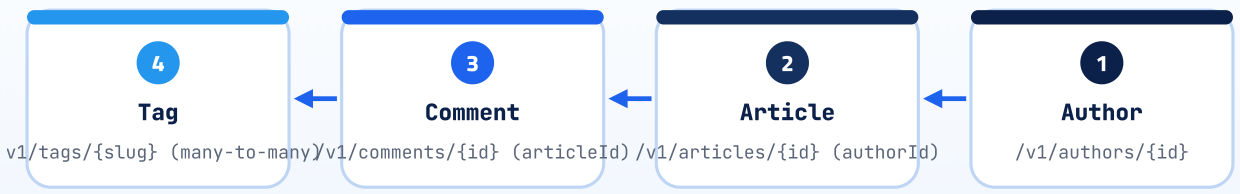
حتى لو الـ ID رقم في الـ Database، ابعته `"42"` . ده بيحميك من حد الـ 2^{53} (فصل 4)، وبيخليك تغيّر لـ UUID بعدين من غير ما تكسر الـ Clients (لأن النوع مش هيتغير). وفي Google AIP-122 أسماء الـ Resources كلها Strings. و GitHub نفسه بقى عنده `node_id` String (للـ GraphQL) جنب الـ `id` الرقمي اللي في `repo.json` .

14.6 نصمم: Content Library API

من هنا لآخر الكتاب، هنصمم API لمكتبة محتوى (الحاجة اللي بتشتغل فيها كـ Content Engineer): مقالات، وكتّاب، وتاجات، وتعليقات. وفي فصل 24 هنكتبه Spec كامل. أول خطوة: الـ Resources والعلاقات.

Resource	Collection	Item
Article	`/v1/articles`	`/v1/articles/{articleId}`
Author	`/v1/authors`	`/v1/authors/{authorId}`
Comment	`/v1/articles/{articleId}/comments`	`/v1/comments/{commentId}`
Tag	`/v1/tags`	`/v1/tags/{slug}`
المستخدم الحالي	—	`/v1/me`
النشر	—	`POST /v1/articles/{articleId}:publish`

جدول 14.4 الـ Resources



شكل 14.1 العلاقات في الـ Content Library API: كاتب ليه مقالات كتير، ومقالة ليها تعليقات كتير، ومقالة ليها تاجات كتير والعكس. والعلاقة بتتمثّل كـ ID جوه الـ JSON (authorId) مش كـ مسار طويل.

ملخص الفصل

- API بالأفعال = Endpoint لكل حاجة ومفيش قاعدة، و `GET` بيمسح = كارثة.
- REST = قيود: Client-Server و Stateless و Cacheable و Uniform interface و Layered.
- Resource = الحاجة، و Representation = شكلها (JSON). وهدفنا مستوى 2 في Richardson، ولينكات زي `next` كبداية للمستوى 3.
- أسماء جمع بحروف صغيرة وشرطة، ومن غير أفعال ولا امتدادات، و Nesting مستوى واحد، والفلترة في الـ Query.
- العمليات: `PATCH` لحالة بسيطة، و `POST …:publish` لعملية فيها شغل.
- الـ IDs Strings، ومتسهّلش التخمين.

صح ولا غلط؟

1. في REST، الـ URL المفروض فيه الفعل: `/createArticle` . صح / غلط
2. Stateless معناها إن السيرفر مبيخزّنش بيانات في Database. صح / غلط
3. `/articles?status=draft` أحسن من `/articles/drafts` . صح / غلط

أسئلة مراجعة

1. ليه `GET /deleteArticle?id=42` خطر حتى لو محدش بيدوس عليه بالغلط؟
2. اشرح الفرق بين Resource و Representation بمثال.
3. إمتى تستخدم `PATCH` بـ `status` وإمتى `POST …:publish` ؟
4. ليه `/v1/comments/{id}` أحسن من `/v1/authors/7/articles/42/comments/9` ؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل صلّح أسماء الـ 8 Endpoints اللي في أول الفصل من غير ما تبص على الحل.
2. متوسط صمّم Resources لـ API "قوايم قراية": مستخدم يعمل قوايم، ويحط فيها مقالات، ويرتّبهم.
3. صعب افتح GitHub REST docs وطلّع 5 Endpoints بتكسر قاعدة من القواعد اللي فوق، وفكّر ليه GitHub اختار كده.

15

المرحلة 5 · تصميم REST

الفصل الخامس عشر

الـ Methods والـ Idempotency

في كتاب الإنترنت (فصل 7) عرفت الـ Methods كمستخدم. هنا هتشوفها كمصمّم: إيه اللي Safe، وإيه اللي Idempotent، وليه ده مش كلام نظري (ده اللي بيحدد مين يقدر يعيد الطلب لو النت قطع). وهنشوف الفرق بين PUT و PATCH على سيرفر حقيقي، و Merge Patch ضد JSON Patch، و Idempotency-Key اللي بيحمي من الدفع مرتين، و QUERY الجديد.

بعد الفصل ده هتكون عارف

- Safe و Idempotent من RFC 9110 بمثال
- ليه الـ Idempotency بتفرق لما النت يقطع
- PUT بيستبدل و PATCH بيعدّل: تجربة حقيقية
- JSON Merge Patch (RFC 7396) و null بيمسح
- JSON Patch (RFC 6902): العمليات op و path
- Idempotency-Key: نفس المفتاح = نفس النتيجة
- الـ Custom method: :publish و 409
- DELETE مرتين = 204 و 204
- QUERY (RFC 10008): بحث بـ Body وهو Safe

15.1 الطلب اللي ضاع في النص

الـ Client بعت `POST /payments`. السيرفر استلم، وخصم الفلوس، وبدأ يبعت الرد... والنت قطع. الـ Client شايف Timeout. **هل الدفع حصل؟** مش عارف. يعيد؟ لو أعاد ممكن العميل يدفع مرتين. ميعيدش؟ ممكن الدفع ميكونش حصل. ودي مشكلة حقيقية في أي شبكة، وحلها بيبدأ من سؤال: **"لو الطلب ده اتبعت مرتين، هيحصل إيه؟"**



شكل 15.1 Timeout مش معناه إن الطلب فشل. معناه إنك مش عارف.

15.2 Safe و Idempotent

Safe

الطلب **مبيغيرش** حاجة في السيرفر (من وجهة نظر الـ Client). قراية بس. فأي حد (Browser، أو Crawler، أو Prefetch) يقدر يبعته براحته. لو السيرفر عمل Log أو عدّاد، ده مش "تغيير" يهم الـ Client.

Idempotent

إنك تبعته **مرة أو عشرة** النتيجة في السيرفر واحدة. `DELETE /articles/7` عشر مرات = المقالة ممسوحة. فلو النت قطع، **تعيد وانت مطمّن**. وكل Safe هو Idempotent أوتوماتيك.

Method	Safe	Idempotent	Body في الطلب	استخدامه في API
`GET`	✓	✓	لأ (ملوش معنى)	اقرا Resource أو Collection
`HEAD`	✓	✓	لأ	زي GET من غير Body: موجود؟ حجمه؟ اتغير؟
`OPTIONS`	✓	✓	نادراً	الـ Preflight بتاع CORS (فصل 13)
`QUERY`	✓	✓	أيوه	بحث معقد بـ Body (تحت)
`PUT`	✗	✓	أيوه	استبدل الـ Resource كله
`DELETE`	✗	✓	لأ	امسح
`POST`	✗	✗	أيوه	اعمل جديد، أو عملية (`:publish`)
`PATCH`	✗	✗ (مش مضمون)	أيوه	عدّل جزء

جدول 15.1 الـ Methods (RFC 9110، و PATCH من RFC 5789، و QUERY من RFC 10008)

> **Idempotent مش معناه نفس الرد**
>
> `DELETE` أول مرة على مقالة موجودة، والتانية على مقالة ممسوحة: الرد ممكن يختلف (204 ثم 404)، بس **حالة السيرفر** واحدة: المقالة مش موجودة. الـ Idempotency عن الأثر، مش عن الرد. وفي السيرفر بتاعنا اخترنا 204 في المرتين، والاتنين صح.

library.js مرتين على DELETE

```
user@MacBook ~ % curl -si -X DELETE localhost:3001/v1/articles/7
HTTP/1.1 204 No Content
user@MacBook ~ % curl -si -X DELETE localhost:3001/v1/articles/7
HTTP/1.1 204 No Content
user@MacBook ~ % curl -s localhost:3001/v1/articles/7
{"type":"https://example.com/problems/article-not-found","title":"Article not
found","status":404,"detail":"No article with id '7'"}
```

> **السيرفر اللي هنجرب عليه**
>
> من الفصل ده لحد فصل 22 هنجرب على `library.js` : سيرفر Express 5 صغير (حوالي 200 سطر) لـ Content Library API اللي صممناه في فصل 14، بيشتغل على `localhost:3001` . الكود كامل في فصل 22. وكل الردود اللي في الكتاب طالعة منه فعلاً.

المصدر RFC 9110 §9.2 · Common Method Properties — RFC 5789 · PATCH

15.3 PUT ضد PATCH: تجربة

JSONPlaceholder بيقلّد السلوك ده بالظبط. المقالة رقم 1 فيها `userId` و `id` و `title` و `body` . هنبعت `{"title":"New"}` بالطريقتين:

Terminal

```
user@MacBook ~ % curl -s -X PATCH https://jsonplaceholder.typicode.com/posts/1 --json
'{"title":"New"}'
{
  "userId": 1,
  "id": 1,
  "title": "New",
  "body": "quia et suscipit\nsuscipit recusandae consequuntur ..."
}
user@MacBook ~ % curl -s -X PUT https://jsonplaceholder.typicode.com/posts/1 --json
'{"title":"New"}'
{
  "title": "New",
  "id": 1
}
```

PATCH = "غيّر ده بس"

الـ `body` و `userId` فضلوا زي ما هما. بعتّ اللي عايز تغيره بس.

PUT = "ده الشكل الجديد كله"

اللي مبعتهوش **اتمسح**. الـ PUT بيقول "خلي الـ Resource ده يبقى بالظبط اللي معايا". وعلشان كده Idempotent: نفس الـ Body عشر مرات = نفس النتيجة.

وفي `library.js`، الـ PUT بيطلب كل الـ Fields الإجبارية، واللي ليه Default بيرجع للـ Default. المقالة 5 كانت `published`، وبعد PUT من غير `status`:

Terminal

```
user@MacBook ~ % curl -s -X PUT localhost:3001/v1/articles/5 --json '{"title":"Only title"}'
{"type":"https://example.com/problems/validation","title":"Validation failed","status":422,
 "errors":[{"pointer":"/authorId","detail":"Invalid input: expected string, received
undefined"}]}
user@MacBook ~ % curl -s -X PUT localhost:3001/v1/articles/5 --json '{"title":"Only
title","authorId":"2"}'
{"id":"5","title":"Only title","status":"draft","authorId":"2","tags":[],"createdAt":"2026-10-
05T09:00:00.000Z"}
```

⚠ الفخ: PUT من فورم ناقص

لاحظ: `status` رجع `draft` و `tags` بقت فاضية. لو الواجهة بتاعتك بتبعت PUT من فورم فيه العنوان بس، **هتمسح بيانات من غير ما تاخد بالك**. علشان كده أغلب الـ APIs الحديثة بتستخدم `PATCH` للتعديل، و `PUT` بس لما الـ Client فعلاً معاه الـ Resource كله.

15.4 PATCH بإيه؟ Merge Patch ضد JSON Patch

PATCH بيقول "عدّل"، بس مبيقولش **إزاي** توصف التعديل. فيه Standardين:

(RFC 7396) JSON Merge Patch

بتبعت JSON بشكل الـ Resource، فيه اللي عايز تغيره بس. وقاعدة واحدة غريبة: `null` **معناها امسح الـ Field**. والـ Content-Type: `application/merge-patch+json`.

```
Terminal
user@MacBook ~ % curl -si -X PATCH localhost:3001/v1/articles/4 \
    -H 'Content-Type: application/merge-patch+json' \
    -H 'If-Match: "39fqqzHpm_icenp6"' \
    -d '{"title":"Article four","tags":null}'
HTTP/1.1 200 OK
ETag: "kgfYo1GjspvVpE60"
Content-Type: application/json; charset=utf-8

{"id":"4","title":"Article four","status":"published","authorId":"1","tags":
[],"createdAt":"2026-10-04T09:00:00.000Z"}
```

الـ `title` اتغير، و`tags: null` مسح الـ Field، وبعدين الـ Validation رجّعه للـ Default `[]`. (والـ `If-Match` ده هنشرحه في فصل 19.)

> **حدود الـ Merge Patch**
>
> (1) مينفعش تحط `null` كقيمة حقيقية، لأنها معناها "امسح". (2) الـ Arrays بتتبدل كلها: مينفعش تقول "زوّد تاج واحد"، لازم تبعت الـ Array كامل. ولو محتاج الحاجات دي، استخدم JSON Patch.

(RFC 6902) JSON Patch

بدل ما تبعت الشكل، بتبعت **قايمة عمليات**، وكل عملية بتشاور على مكان بـ JSON Pointer (RFC 6901، اللي شفناه في الـ Errors بتاعة فصل 7):

تشريح الأمر

```
[
{ "op": "test",    "path": "/status",  "value": "draft" },
{ "op": "replace", "path": "/title", "value": "Article four" },
{ "op": "add", "path": "/tags/-",  "value": "rest" },
{ "op": "remove", "path": "/summary" }
]
```

1. `op` العملية: `add` و`remove` و`replace` و`move` و`copy` و`test`.
2. `path` JSON Pointer: `/` بيفصل المستويات، و`/tags/0` أول عنصر.
3. `/tags/-` الـ `-` معناها "آخر الـ Array": زوّد في الآخر.

4 `test` اتأكد إن القيمة كده، ولو لأ **كل** العمليات تتلغي (Atomic).

5 `Content-Type` `application/json-patch+json`

	Merge Patch	JSON Patch
سهولة	سهل جداً، شبه الـ Resource	أصعب، لغة عمليات
Arrays	تبدّل كلها	تزوّد وتمسح عنصر
`null`	معناها امسح	قيمة عادية
شروط	لأ	`test`
الاستخدام	أغلب الـ APIs (وأي PATCH بـ JSON عادي هو تقريباً كده)	محررات، و Kubernetes، وحالات معقدة

جدول 15.2 تختار إيه؟

المصدر RFC 7396 · JSON Merge Patch · RFC 6902 · JSON Patch · RFC 6901 · JSON Pointer

15.5 Idempotency-Key: نخلّي POST آمن

نرجع لمشكلة الدفع. الحل اللي Stripe عمله (وبقى Draft في IETF): الـ Client يعمل **مفتاح عشوائي** (UUID) لكل عملية، ويبعته في هيدر. والسيرفر يحفظ "المفتاح ده نتيجته كانت إيه". لو نفس المفتاح جه تاني، يرجّع نفس النتيجة من غير ما ينفّذ تاني.

Terminal

```
# المرة الأولى: اتعملت مقالة 26
user@MacBook ~ % curl -si -X POST localhost:3001/v1/articles \
    -H 'Idempotency-Key: 8e03978e-40d5-43e8-bc93-6894a57f9324' \
    --json '{"title":"REST naming","authorId":"2"}'
HTTP/1.1 201 Created
Location: /v1/articles/26

{"id":"26","title":"REST naming","status":"draft","authorId":"2","tags":[],"createdAt":"2026-
10-23T11:43:20.999Z"}

# إعادة بنفس المفتاح ونفس الـ Body: نفس المقالة، مفيش 27
user@MacBook ~ % curl -si -X POST localhost:3001/v1/articles \
    -H 'Idempotency-Key: 8e03978e-40d5-43e8-bc93-6894a57f9324' \
    --json '{"title":"REST naming","authorId":"2"}'
HTTP/1.1 201 Created
Idempotent-Replayed: true
Location: /v1/articles/26

{"id":"26","title":"REST naming",...,"createdAt":"2026-10-23T11:43:20.999Z"}

# نفس المفتاح بـ Body مختلف: غلط من الـ Client
user@MacBook ~ % curl -s -X POST localhost:3001/v1/articles \
```

```
    -H 'Idempotency-Key: 8e03978e-40d5-43e8-bc93-6894a57f9324' \
    --json '{"title":"Other","authorId":"2"}'
{"type":"https://example.com/problems/idempotency-key-reused","title":"Idempotency-Key
reused","status":422,"detail":"Same key was used with a different body"}
```

تشريح الأمر

```
Idempotency-Key: 8e03978e-40d5-43e8-bc93-6894a57f9324
```

1. **المفتاح** — الـ Client بيعمله (`crypto.randomUUID()`)، **مرة لكل عملية** مش لكل محاولة. الإعادة بنفس المفتاح.
2. **`Idempotent-Replayed`** — هيدر زيادة اخترناه في السيرفر بتاعنا علشان الـ Client يعرف إن ده رد محفوظ.
3. **`Body` مختلف** — الـ Draft بيقول 422. ولو المفتاح ناقص في Endpoint بيطلبه: 400. ولو الطلب الأول لسه شغال وجه التاني: 409.
4. **Stripe** — المفتاح لحد 255 حرف، وممكن يتمسح بعد 24 ساعة على الأقل، ولو نفس المفتاح جه بـ Parameters مختلفة بيرجّع Error.

Best Practice

كـ Client: أي POST فيه فلوس أو إيميلات أو حاجة متتعملش مرتين، ابعت `Idempotency-Key` لو الـ API بيدعمه، وأعد بنفس المفتاح. وكـ Provider: احفظ المفاتيح في Database أو Redis مش في الذاكرة زي `library.js` (لو السيرفر عمل Restart هتضيع)، وحط لها مدة صلاحية.

المصدر IETF Draft · The Idempotency-Key HTTP Header Field · Stripe · Idempotent requests

15.6 الـ publish: :Custom method

من فصل 14: النشر عملية فيها شغل، فعملناها `POST …:publish`. وهي مش Idempotent بطبيعتها (POST)، بس احنا نقدر نقرر سلوكها: المرة التانية بترجع **Conflict 409** لأن الحالة مش مناسبة.

Terminal

```
user@MacBook ~ % curl -s -X POST 'localhost:3001/v1/articles/3:publish'
{"id":"3","title":"Article 3","status":"published",...,"publishedAt":"2026-10-
23T11:43:36.535Z"}
user@MacBook ~ % curl -s -X POST 'localhost:3001/v1/articles/3:publish'
{"type":"https://example.com/problems/already-published","title":"Already
published","status":409,"detail":"Article '3' is already published"}
```

لو حابب تتعمق: الـ : في Express 5

تقدر تعدّيه في أول قراية

في Express، الـ `:` معناها Parameter (`:id`). فعلشان تكتب `:publish` حرفياً لازم تعمله Escape: `"/v1/articles/:id\\:publish"` (الـ `\\` في الـ JS String = `\` واحدة). جربناها في `library.js` واشتغلت. ودي حاجة صغيرة بس بتضيّع ساعة لو متعرفهاش.

15.7 QUERY: البحث اللي محتاج Body

قبل كده: عندك بحث معقد (فلاتر كتير، وشروط متداخلة). تحطه في `GET ?…`؟ الـ URL هيبقى طويل جداً وفيه حدود، وبيظهر في الـ Logs. تبعته `POST /search` بـ Body؟ كده خسرت الـ Safe والـ Idempotent: الـ Cache مش هيخزّنه، والأدوات مش هتعيده لوحدها. وكانت المشكلة إن مفيش Method "بيقرا بـ Body".

فاتطور حل: **RFC 10008** (يونيو 2026) عرّف `QUERY`: زي POST في إنه بياخد Body، بس **Safe و Idempotent** رسمياً، فينفع يتخزّن ويتعاد.

```http
QUERY /v1/articles HTTP/1.1
Host: api.example.com
Content-Type: application/json
Accept: application/json

{ "status": ["draft", "review"], "tags": { "any": ["api", "json"] }, "createdAfter": "2026-10-01T00:00:00Z" }
```

والسيرفر يقدر يعلن إنه بيدعمه وبأي صيغة بهيدر `Accept-Query`. و OpenAPI 3.2 فيه `query` كعملية جنب `get` و `post`.

> **لسه جديد**
> QUERY اتنشر في يونيو 2026، فالدعم في الـ Frameworks والـ Proxies والـ Browsers لسه بيتبني. اتأكد إن كل حاجة في السكة بتدعمه قبل ما تعتمد عليه. والحل الشائع لحد دلوقتي: `POST /v1/articles:search` وتوثّق إنه مبيغيرش حاجة.

المصدر RFC 10008 · The HTTP QUERY Method

ملخص الفصل

- Safe = مبيغيرش. Idempotent = مرة زي عشرة. GET و HEAD و OPTIONS و QUERY الاتنين، و PUT و DELETE Idempotent بس، و POST و PATCH ولا ده ولا ده.
- الـ Idempotency هي اللي بتقولك "تعيد وانت مطمّن" لما يحصل Timeout.
- PUT بيستبدل كله (اللي مبعتهوش بيضيع)، و PATCH بيعدّل جزء.
- Merge Patch: شكل الـ Resource و `null` = امسح. JSON Patch: قايمة `op` و `path` و `test`.
- `Idempotency-Key`: نفس المفتاح = نفس الرد، و Body مختلف = 422.
- `:publish` مرتين = 409، و QUERY = بحث بـ Body وهو Safe.

صح ولا غلط؟

1. DELETE مرتين لازم يرجّع نفس الـ Status code علشان يبقى Idempotent. صح / غلط
2. في JSON Merge Patch، `{"summary": null}` بيحط القيمة null في الـ Field. صح / غلط
3. الـ Idempotency-Key بيتعمل جديد مع كل إعادة للطلب. صح / غلط

؟ أسئلة مراجعة

1. ليه PUT Idempotent و POST لأ؟ وضّح بمثال "إنشاء مقالة".
2. عايز تزوّد تاج واحد لمقالة فيها 5 تاجات. هتعملها إزاي بـ Merge Patch، وإزاي بـ JSON Patch؟
3. السيرفر استلم Idempotency-Key وهو لسه بيعالج الطلب الأول، وجه طلب تاني بنفس المفتاح. يرد بإيه؟
4. إيه المشكلة اللي QUERY بيحلها وكانت موجودة في GET و POST؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل جرّب PATCH و PUT على `jsonplaceholder.typicode.com/users/1` وقارن الرد.
2. متوسط اكتب JSON Patch بيغيّر العنوان بس لو الـ `status` لسه `draft`، ويمسح أول تاج.
3. صعب في `library.js` المفاتيح في `Map`. اكتب على ورق: هتخزّنها فين في Production، ومدة صلاحيتها كام، وهتعمل إيه في حالة الـ 409.

16

المرحلة 5 · تصميم REST

الفصل السادس عشر

الـ Status codes وتصميم الـ Body

في كتاب الإنترنت (فصل 8) اتعلمت تقرا الـ Status codes. هنا هتختارها: أنهي كود لأنهي موقف، والفرق بين 400 و 422، و 401 و 403، و 404 و 410. وبعدين نصمم شكل الـ JSON نفسه: Envelope ولا لأ، وأسماء الـ Fields، والتواريخ، والفلوس، والـ null.

بعد الفصل ده هتكون عارف

- القاعدة: الكود للآلة والـ Body للإنسان
- الأكواد اللي هتحتاجها في أي API وإمتى كل واحد
- 201 + Location، و 202 للشغل الطويل، و 204
- 400 ضد 422، و 401 ضد 403، و 404 ضد 403
- ليه 200 مع error جوه = غلط
- Envelope: {data} ولا Array على طول
- camelCase ولا snake_case: اختار واحد
- التواريخ RFC 3339، والفلوس أعداد صحيحة
- null ضد مش موجود ضد فاضي
- الـ Enums كـ Strings

16.1 "OK 200" وجوه Error

API قديم كان بيرجّع كل حاجة 200:

```http
HTTP/1.1 200 OK

{ "success": false, "error": "User not found" }
```

وكانت المشكلة إن كل حاجة في السكة بتصدّق الكود مش الـ Body: `fetch` بيقول `res.ok === true`، والـ Cache بيخزّن الـ Error على إنه رد ناجح، والـ Monitoring بيقول "مفيش أخطاء"، والـ Retry مبيشتغلش. (وشفنا GraphQL بيعمل كده عن قصد في فصل 2، وده من عيوبه.) فالقاعدة:

> **القاعدة**
>
> **الـ Status code للآلة، والـ Body للإنسان.** الكود بيقول "إيه اللي حصل بشكل عام" علشان أي برنامج يتصرف صح من غير ما يفهم الـ Body. والـ Body بيقول التفاصيل (فصل 17).

16.2 الأكواد اللي هتحتاجها

الكود	إمتى في API	معناه
OK 200	GET ناجح، أو PATCH/PUT يرجّع الـ Resource الجديد	الـ Resource

الكود	إمتى في API	معاه
Created 201	POST عمل حاجة جديدة	`Location: /v1/articles/26` والـ Resource
Accepted 202	"استلمت وهشتغل بعدين": تصدير، أو فيديو، أو AI طويل	`Location` لـ Resource بيقول الحالة: `/v1/jobs/91`
No Content 204	DELETE، أو عملية مفيش حاجة ترجعها	مفيش Body خالص
Not Modified 304	الـ ETag زي ما هو (فصل 3 و 19)	مفيش Body
Permanent Redirect 308	الـ Endpoint اتنقل، والـ Method والـ Body يفضلوا زي ما هم	`Location`

جدول 16.1 النجاح والتحويل

الكود	إمتى	مثال حقيقي في الكتاب
Bad Request 400	الطلب نفسه بايظ: JSON مكسور، أو Parameter مش مفهوم	`library.js`: JSON بفاصلة زيادة، و `sort=views`
Unauthorized 401	مين انت؟ مفيش توكن أو التوكن بايظ أو منتهي	DummyJSON بعد ما التوكن خلص (فصل 12)
Forbidden 403	عارف انت مين، بس مش مسموحلك	GitHub من غير User-Agent (فصل 3)
Not Found 404	مش موجود (أو مش عايز أقولك إنه موجود)	`/v1/articles/999`
Method Not Allowed 405	الـ URL موجود بس مش بالـ Method ده. لازم هيدر `Allow`	Express بيرجّع 404 هنا افتراضياً، لازم تعملها بإيدك
415 / 406	406: مش هقدر أرجّع الشكل اللي طلبته في `Accept`. 415: مش فاهم الـ `Content-Type` اللي بعتّه	GitHub بـ `Accept: application/xml` رجّع 415، و `library.js` بـ `-d` رجّع 415
Conflict 409	الطلب سليم بس متعارض مع الحالة: موجود قبل كده، أو منشور فعلاً	`:publish` مرتين
Gone 410	كان موجود واتشال نهائي (أوضح من 404)	GitHub: لو طلبت API version اتقفل (حسب الـ Docs بتاعته، فصل 19)
428 / 412	412: الشرط في `If-Match` فشل. 428: لازم تبعت شرط (RFC 6585)	PATCH بـ ETag قديم (فصل 19)
Content Too Large 413	الـ Body أكبر من المسموح	Express: الـ Default لـ `express.json()` هو 100kb
Unprocessable Content 422	الـ JSON سليم، بس المحتوى مش مقبول: Validation	`{"title":"Hi"}` في `library.js`

الكود	إمتى	مثال حقيقي في الكتاب
Too Many Requests 429	عدّيت الحد	مع `Retry-After` (فصل 19)

جدول 16.2 أخطاء الـ Client (xx4): "انت اللي غلطت، صلّح وابعت تاني"

الكود	إمتى
Internal Server Error 500	Bug. ومتطلّعش الـ Stack trace للـ Client أبداً.
504 / 502	الـ Proxy أو الـ Gateway معرفش يكلم السيرفر (502)، أو استنى ومحدش رد (504).
Service Unavailable 503	واقف مؤقتاً (Maintenance، أو ضغط). معاه `Retry-After`.

جدول 16.3 أخطاء السيرفر (xx5): "أنا اللي غلطت، ممكن تحاول تاني"

المصدر RFC 6585 · 428 / 429 · MDN · HTTP response status codes · RFC 9110 §15 · Status Codes

16.3 الأزواج اللي بتلخبط

400 ضد 422

400: "مش قادر أقرا اللي بعتّه" (JSON بايظ، أو نوع غلط في الـ Query). **422**: "قريته وفهمته، بس مش مقبول" (العنوان قصير، والـ status مش من الاختيارات). بعض الـ APIs بتستخدم 400 للاتنين، وده مقبول، بس **اثبت على قاعدة واحدة**. واحنا في `library.js` فصلناهم.

401 ضد 403

401 = "مش عارف انت مين" (Authentication). الحل: سجّل دخول أو جدّد التوكن. والمفروض معاه `WWW-Authenticate` (GitHub مبيبعتوش، شفناها في فصل 12). **403** = "عارفك، بس لأ" (Authorization). تسجيل الدخول تاني مش هيفرق.

403 ضد 404

لو مستخدم بيحاول يفتح فاتورة واحد تاني، 403 بتقوله "الفاتورة دي **موجودة** بس مش بتاعتك". فكتير من الـ APIs (منها GitHub في الـ Private repos) بترجّع **404** علشان متكشفش إن الحاجة موجودة.

200 بـ [] ضد 404

`GET /v1/articles?status=archived` ومفيش نتايج = **200** و `{"data": []}`. القايمة موجودة بس فاضية.
الـ 404 للـ Resource اللي مش موجود (`/v1/articles/999`) أو URL غلط.



شكل 16.1 ترتيب منطقي للأسئلة اللي السيرفر بيسألها. أول «لأ» بتحدد الكود.

16.4 تصميم الـ Body

Envelope ولا Array على طول؟

Array على طول

```
JSON
[{"id": "1", ...}, {"id": "2", ...}]
```

GitHub بيعمل كده، والبيانات الإضافية (الصفحة الجاية) في الهيدرز (`Link`). بسيط، بس لو حبيت تزوّد حاجة جنب القايمة مش هتقدر من غير ما تكسر الـ Clients.

Envelope (غلاف)

```
JSON
{"data": [...], "nextCursor": "MQ"}
```

اللي عملناه في `library.js`، وزي PokeAPI (`count` و `next` و `results`). تقدر تزوّد `meta` بعدين من غير ما تكسر حاجة. **ده الأأمن للـ Collections.**

وللـ Resource الواحد (`GET /v1/articles/4`) رجّعه على طول من غير غلاف، زي GitHub و Stripe و `library.js`.

الأسماء

الأسلوب	مين بيستخدمه	مثال
snake_case	GitHub، و Stripe، و Python عموماً	`"created_at"` في `repo.json`
camelCase	Google AIP، و Microsoft، و JS عموماً	`"createdAt"` في `library.js`

جدول 16.4 camelCase ولا snake_case؟

مفيش صح وغلط. الـ Client بتاعك JS؟ `camelCase` هيوفر عليك تحويل. المهم **واحد بس في كل الـ API**، ومتغيرهوش بعد ما تنشر.

القيم

تشريح الأمر

```
{
"id": "4",
"title": "Article four",
"status": "published",
"wordCount": 1250,
"price": { "amount": 4999, "currency": "EGP" },
"isFeatured": false,
"createdAt": "2026-10-04T09:00:00.000Z",
"publishedAt": null,
"tags": []}
```

1. `"id": "4"` String دايماً (فصل 14).
2. `"status"` Enum كـ String مقروء، مش رقم (`2` معناها إيه؟). وقول في الـ Docs إن ممكن تتزود قيم جديدة بعدين، علشان الـ Client ميقعش لو شاف قيمة مش عارفها.
3. `"amount": 4999` الفلوس **Integer بأصغر وحدة** (قروش). `49.99` كـ Number ممكن يبقى `49.989999…` في الحسابات. Stripe بيعمل كده بالظبط: `1000` = 10 دولار. ومعاها العملة دايماً.
4. `"isFeatured"` Boolean حقيقي، مش `"yes"` ولا `1`.
5. `"createdAt"` RFC 3339 بالـ UTC و `Z` في الآخر (فصل 4). مش `"04/10/2026"` (أكتوبر ولا أبريل؟) ومش Unix timestamp (مش مقروء).
6. `"publishedAt": null` الـ Field موجود والقيمة "مفيش" = مش منشورة لسه.
7. `"tags": []` Array فاضية، مش `null`: الـ Client يعمل `.map` على طول من غير ما يقلق.

null ضد مش موجود

اتفق على معنى كل حالة وثبّته: **موجود بقيمة** = القيمة. `null` = "معروف إنه مفيش" (مش منشورة). **مش موجود خالص** = "مش متاح في الرد ده" (مثلاً طلبت `fields=id,title` ، أو مش مسموحلك تشوفه). وفي الـ Merge Patch، `null` معناها امسح (فصل 15)، فخلّي بالك من التعارض ده لما تصمم.

Best Practice: Field جديد = مش كسر

صمّم وانت فاكر إنك هتزوّد Fields بعدين. زوّد براحتك (الـ Clients الكويسة بتتجاهل اللي متعرفهوش)، بس **متشيلش** Field، و**متغيرش نوعه**، و**متغيرش معناه**. دول بيكسروا الناس (فصل 19).

المصدر Google AIP-140 · Field names · RFC 3339 · Date and Time on the Internet · Stripe · Currencies (minor units)

ملخص الفصل

- الكود للآلة والـ Body للإنسان. و 200 بـ Error جوه بيبوّظ الـ Cache والـ Monitoring و `res.ok` .
- 201 + Location للإنشاء، و 202 للشغل الطويل، و 204 من غير Body.
- 400 = مش قادر أقرا، و 422 = قريت ومش مقبول. 401 = مين انت، و 403 = عارفك بس لأ، و 404 ممكن تخبّي 403.
- قايمة فاضية = 200 و `[]` مش 404.
- Envelope لـ Collections، والـ Resource لوحده من غير غلاف.
- أسلوب أسماء واحد، و IDs Strings، و Enums Strings، وفلوس Integer بالعملة، وتواريخ RFC 3339، و Arrays فاضية مش null.

صح ولا غلط؟

1. لو مستخدم بعت توكن سليم بس ملوش صلاحية، الرد 401. صح / غلط
2. البحث اللي ملوش نتايج المفروض يرجّع 404. صح / غلط
3. تخزين السعر `49.99` كـ JSON Number فكرة كويسة. صح / غلط

أسئلة مراجعة

1. اكتب الكود المناسب: (أ) إيميل متسجل قبل كده. (ب) `Content-Type: text/plain` . (ج) طلب تصدير هيخلص بعد 5 دقايق. (د) الـ Body مش JSON سليم.
2. ليه 202 لازم يبقى معاه `Location` ؟
3. إيه ميزة الـ Envelope لو حبيت تزوّد `totalCount` بعدين؟
4. اشرح الفرق بين `"publishedAt": null` وإن الـ Field مش موجود.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل جرّب 5 طلبات غلط على `library.js` وسجّل الكود اللي رجع لكل واحد.
2. متوسط صمّم JSON لـ "كاتب" في الـ Content Library API فيه كل أنواع القيم اللي في الفصل.
3. صعب افتح `repo.json` بتاع GitHub وطلّع 3 قرارات تصميم (أسماء، و null، و Nested objects) واكتب هتعملها إزاي في API بتاعك.

17

المرحلة 5 · تصميم REST

الفصل السابع عشر

شكل الـ Errors: RFC) Problem Details 9457)

الـ Status code بيقول «حصل إيه» بشكل عام. بس الـ Client محتاج يعرف بالظبط: أنهي Field غلط؟ أعمل إيه؟ وكل API كان بيخترع شكل لوحده. هنا هنشوف الفوضى دي على GitHub و Stripe بجد، وبعدين الـ Standard اللي حلها: application/problem+json، Field بـ Field، ونقرا الـ Errors من الـ Client صح، ونحمي السيرفر من إنه يطلّع أسراره.

بعد الفصل ده هتكون عارف

- الفوضى: GitHub و Stripe وكل واحد بشكل
- application/problem+json: الـ Fields الخمسة
- type هو المفتاح، و about:blank وقاعدة الـ title
- الـ Extensions: errors[] بـ pointer لكل Field
- قراية الـ Problem من الـ Client بـ type مش بـ detail
- الـ 500: متطلّعش Stack trace، ورجّع ID للـ Logs
- Checklist لأي Error بتصممه

17.1 كل API بلغة

جربنا نغلط في 2 من أشهر الـ APIs:

```
Terminal
user@MacBook ~ % curl -s https://api.github.com/repos/expressjs/nope-nope -H 'User-Agent:
book'
{"message":"Not Found","documentation_url":"https://docs.github.com/rest/repos/repos#get-a-
repository","status":"404"}

user@MacBook ~ % curl -s https://api.stripe.com/v1/charges -u sk_test_abc:
{
  "error": {
    "message": "Invalid API Key provided: sk_test_abc",
    "type": "invalid_request_error"
  }
}
```

GitHub: `message` فوق، و`status` **String**. Stripe: كله جوه `error`، و`type` كلمة. وغيرهم `{"error": "..."}` أو `{"errors": [...]}` أو `{"msg": ...}`. وكانت المشكلة إن أي Client (أو مكتبة، أو أداة Monitoring) لازم يتعلم شكل جديد مع كل API. فاتطور حل: **RFC 7807** سنة 2016، واتحدّث بـ **RFC 9457** سنة 2023: شكل واحد لأي Error في أي HTTP API.

17.2 الشكل

رد من library.js

1 نوع Content خاص بيقول "ده Problem". الـ Client يعرف من الهيدر قبل ما يقرا الـ Body.	`HTTP/1.1 404 Not Found` `Content-Type: application/problem+json`
2 **أهم Field.** URI بيحدد نوع المشكلة، والكود بيقرّر على أساسه. لو فتحته المفروض يوديك صفحة Docs بتشرح المشكلة. ولو مش موجود = `about:blank`.	`"type": "https://example.com/problems/article-not-found"`
3 ملخص قصير للإنسان، **ثابت لنفس الـ type** (ميتغيرش من مرة للتانية).	`"title": "Article not found"`
4 نسخة من الـ Status code (Number مش String زي GitHub)، علشان لو حد عدّى الـ JSON لوحده يفضل فاهم.	`"status": 404`
5 شرح **المرة دي بالذات** للإنسان، ويساعده يصلّح. الكود **ميعملش Parse** للنص ده أبداً.	`"detail": "No article with id '999'"`
6 (اختياري) URI لـ "الحدوتة دي بالذات"، زي ID تقدر تدوّر بيه في الـ Logs.	`"instance": "urn:uuid:f4c6…"`

Terminal

```
user@MacBook ~ % curl -s localhost:3001/v1/articles/999
{"type":"https://example.com/problems/article-not-found","title":"Article not found","status":404,"detail":"No article with id '999'"}
```

⚠ about:blank ليه قاعدة

لو الـ `type` هو `about:blank`، يبقى المشكلة "مفيش عنها حاجة غير الـ Status code"، والـ RFC بيقول الـ **المفروض** يبقى نفس الجملة الرسمية للكود ("Not Found" لـ 404). أول نسخة من `library.js` كانت بترجّع `about:blank` مع `"title": "Article not found"`، وده مخالف. فصلّحناها: أي مشكلة ليها معنى خاص أخدت `type` خاص بيها، و `about:blank` فضل للحاجات العامة:

Terminal

```
user@MacBook ~ % curl -s localhost:3001/v1/nothing
{"type":"about:blank","title":"Not Found","status":404,"detail":"No route for GET /v1/nothing"}
user@MacBook ~ % curl -s -X POST localhost:3001/v1/articles --json @/tmp/big.json
{"type":"about:blank","title":"Content Too Large","status":413,"detail":"request entity too large"}
```

المصدر IANA · HTTP Problem Types registry · RFC 9457 · Problem Details for HTTP APIs

17.3 الـ Extensions: errors لكل Field

الـ RFC بيسمحلك تزوّد أي Fields تانية (Extensions) حسب الـ `type`. وأشهرها لمشاكل الـ Validation: قايمة `errors`، كل واحد فيه `detail` و`pointer` (JSON Pointer للـ Field، فصل 7 و 15). الـ RFC نفسه بيدّي المثال ده (وبيكتب الـ Pointer بالشكل `"#/age"`)، واحنا استخدمنا الشكل `"/title"`:

Terminal

```
user@MacBook ~ % curl -s -X POST localhost:3001/v1/articles --json
'{"title":"Hi","status":"live"}'
```

JSON

```
{
  "type": "https://example.com/problems/validation",
  "title": "Validation failed",
  "status": 422,
  "errors": [
    { "pointer": "/title",    "detail": "Too small: expected string to have >=3 characters" },
    { "pointer": "/status",   "detail": "Invalid option: expected one of
\"draft\"|\"review\"|\"published\"" },
    { "pointer": "/authorId", "detail": "Invalid input: expected string, received undefined" }
  ]
}
```

الـ 3 مشاكل في رد واحد، فالفورم في React يقدر يحط كل رسالة تحت الـ Input بتاعها مرة واحدة، بدل ما المستخدم يصلّح غلطة ويكتشف التانية. والرسايل دي طالعة من zod (اللي شفناه في فصل 7) ومحولة لـ Pointers بسطرين في `library.js`.

Extensions شائعة

`errors` (Validation)، و`traceId` أو `instance` (للـ Logs)، و`retryAfter`، و`balance` (المثال اللي في الـ RFC: "رصيدك 30 والمطلوب 50"). والـ Client اللي ميعرفش Extension **يتجاهلها**.

17.4 قراية الـ Problem من الـ Client

نكمّل على `ApiError` بتاع فصل 10. القاعدة: **قرّر بالـ** `type`، واعرض الـ `detail` للإنسان، ومتعملش أبداً `if (p.detail.includes("not found"))` لأن النص ممكن يتغير أو يتترجم.

problem.mjs

```
const res = await fetch(BASE + path, init);
const type = res.headers.get("content-type") ?? "";
if (res.ok) return res.status === 204 ? null : res.json();
if (type.includes("application/problem+json")) {
  const p = await res.json();
  switch (p.type) {                    // stable machine key
    case "https://example.com/problems/validation":
```

```
      for (const e of p.errors) console.log(`  field ${e.pointer}: ${e.detail}`);
      break;
    case "https://example.com/problems/article-not-found":
      console.log("  show a 'not found' page");
      break;
    default:
      console.log(`  ${p.status} ${p.title}${p.detail ? " — " + p.detail : ""}`);
  }
}
```

Terminal

```
user@MacBook ~ % node problem.mjs
POST bad article:
  field /title: Too small: expected string to have >=3 characters
  field /authorId: Invalid input: expected string, received undefined
GET missing article:
  show a 'not found' page
POST publish twice:
  409 Already published — Article '6' is already published
```

17.5 الـ 500: اللي ميطلعش برّه

في `library.js` عملنا Route بيضرب Error فيه "سر" (باسورد Database وهمي). Express 5 بيمسك الـ Error من الـ `async` handler لوحده ويوديه للـ Error middleware:

Terminal

```
user@MacBook ~ % curl -si localhost:3001/v1/debug/crash
HTTP/1.1 500 Internal Server Error
Content-Type: application/problem+json; charset=utf-8

{"type":"about:blank","title":"Internal Server
Error","status":500,"instance":"urn:uuid:f4c64429-270c-43b2-a87f-7172512af3b6"}
```

الـ Log بتاع السيرفر (مش للـ Client)

```
f4c64429-270c-43b2-a87f-7172512af3b6 Error: DB password=hunter2 rejected
    at file:///workspace/book7/lab/library.js:201:9
    at Layer.handleRequest (.../node_modules/router/lib/layer.js:152:17)
```

اللي راح للـ Client

"حصل خطأ عندنا"، و ID. مفيش رسالة الـ Error، ولا Stack trace، ولا أسماء ملفات. الـ RFC نفسه بيحذّر من إن الـ Stack dump يطلع برّه.

اللي راح للـ Log

كل حاجة، ومعاها نفس الـ ID. المستخدم يبعتلك الـ ID في الـ Support، وانت تدوّر عليه في الـ Logs وتلاقي المشكلة بالظبط.

17.6 Checklist لأي Error

1. الـ Status code صح (فصل 16)، ومش 200 أبداً.
2. `Content-Type: application/problem+json` .
3. `type` ثابت لكل نوع مشكلة، وصفحة Docs ليه. والكود بتاع الـ Client بيعتمد عليه.
4. `title` ثابت، و `detail` بيساعد المستخدم يصلّح ("العنوان لازم 3 حروف على الأقل")، مش بيشرح الكود بتاعك.
5. Validation = كل الأخطاء مرة واحدة في `errors` بـ `pointer` .
6. 429 و 503: `Retry-After` . و 401: `WWW-Authenticate` . و 405: `Allow` .
7. 500: من غير أسرار، وبـ ID يربط بالـ Log.
8. متكشفش معلومات: "الإيميل ده مش مسجل" في Login بتساعد اللي بيخمّن الحسابات. قول "الإيميل أو الباسورد غلط".

ملخص الفصل

- كل API كان بشكل (GitHub و Stripe مختلفين)، و RFC 9457 وحّد الشكل.
- `application/problem+json` وفيه `type` و `title` و `status` و `detail` و `instance` .
- `type` هو اللي الكود بيقرر بيه. و `about:blank` = الـ title هو جملة الكود الرسمية.
- Extensions زي `errors[]` بـ `pointer` لكل Field.
- الـ 500: ID للـ Client، والتفاصيل في الـ Log بس.

صح ولا غلط؟

1. الـ Client يقدر يعتمد على نص `detail` علشان يعرف نوع المشكلة. صح / غلط
2. لو الـ `type` مش موجود في الـ Problem، قيمته الافتراضية `about:blank` . صح / غلط
3. إنك تبعت الـ Stack trace في الـ 500 بيساعد الـ Client وده Best practice. صح / غلط

أسئلة مراجعة

1. ليه الـ `status` موجود في الـ Body مع إنه في الـ Status line؟
2. إيه الفرق بين `title` و `detail` ؟ وأنهي واحد ثابت؟
3. اكتب Problem لـ "رصيدك مش كفاية" بـ Extension مناسب.
4. ليه بنرجّع ID في الـ 500؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل حوّل رد GitHub بتاع الـ 404 اللي فوق لشكل RFC 9457.
2. متوسط زوّد على `problem.mjs` حالة لـ 429 بتقرا `Retry-After` وتستنى وتعيد.
3. صعب صمّم 6 `type`s للـ Content Library API (URI، و title، و status، وإمتى) زي جدول Docs.

18

المرحلة 5 · تصميم REST

الفصل الثامن عشر

الفلترة والترتيب والـ Pagination

محدش بيرجّع 10 آلاف مقالة في رد واحد. هنا هنصمم الـ Query string: إزاي تفلتر وترتّب وتختار الـ Fields، وبعدين أهم قرار: Offset ولا Cursor. وهنشوف بتجربة حقيقية إزاي الـ Offset بيكرر عناصر لما حاجة جديدة تتزود، وإزاي GitHub و Stripe و PokeAPI حلوها، ونكتب Client بيمشي ورا Link header لحد الآخر.

بعد الفصل ده هتكون عارف

- ليه لازم Pagination من أول يوم
- الفلترة: ?authorId=7&status=draft وأشكالها
- الترتيب: ?title,sort=-createdAt
- اختيار الـ Fields: ?title,fields=id
- Offset/Limit و Page: البساطة والمشاكل
- تجربة: عنصر جديد بيكرر الصفحة في الـ Offset
- Cursor: معتم، وثابت، وسريع
- Link header (RFC 8288) و rel=next عند GitHub
- Client بيمشي ورا next لحد الآخر

18.1 ليه من أول يوم

أول نسخة من أي API بتبقى فيها 20 مقالة، فـ `GET /articles` بيرجّعهم كلهم وكله تمام. بعد سنة بقوا 40 ألف: الرد 30 ميجا، والـ Database بتتعب، والموبايل بيقفل. ولو جيت تزوّد Pagination دلوقتي، **هتكسر كل Client** كان متوقع إنه بياخد كله. علشان كده القاعدة: **أي Collection فيه Pagination من أول يوم**، و Default معقول (10 أو 20)، و Maximum (في `library.js`: 50).

18.2 الفلترة والترتيب والـ Fields

Terminal

```
user@MacBook ~ % curl -si 'localhost:3001/v1/articles?status=published&sort=-
createdAt&limit=3&fields=id,createdAt'
Link: </v1/articles?status=published&sort=-
createdAt&limit=3&fields=id%2CcreatedAt&cursor=MjI>; rel="next"

{"data":[{"id":"25","createdAt":"2026-10-25T09:00:00.000Z"},
         {"id":"23","createdAt":"2026-10-23T09:00:00.000Z"},
         {"id":"22","createdAt":"2026-10-22T09:00:00.000Z"}],"nextCursor":"MjI"}
```

تشريح الأمر

```
?status=published&sort=-createdAt&limit=3&fields=id,createdAt
 1               2          3          4
```

1. `status=published` فلتر بالمساواة: اسم الـ Field = القيمة. وأكتر من فلتر = AND.

2 `sort=-createdAt` الترتيب. الـ `-` قبل الاسم = تنازلي (الأحدث الأول). وأكتر من Field بفاصلة: `sort=-createdAt,title`. ده نفس أسلوب JSON:API.

3 `limit=3` حجم الصفحة. والسيرفر بيحط حد أقصى مهما طلبت.

4 `fields=id,createdAt` رجّعلي الـ Fields دي بس (Sparse fieldsets): رد أصغر للموبايل. والـ `,` اتعملها لـ Encoding `%2C` في الـ Link.

المحتاج	شكل شائع	ملاحظة
يساوي	`?status=draft`	الأبسط والأشهر
أكتر من قيمة	`?status=draft,review` أو `?status=draft&status=review`	اختار واحد ووثّقه. (`URLSearchParams.getAll` بيقرا التاني)
مدى	`?createdAfter=2026-10-01T00:00:00Z` أو `?filter[createdAt][gte]=…`	الأول أوضح، والتاني أعم (أسلوب الأقواس)
بحث نصي	`?q=express`	زي GitHub Search
علاقة	`?authorId=7`	بدل `/authors/7/articles` (فصل 14)

جدول 18.1 أشكال الفلترة الشائعة

> ⚠ **الترتيب والفلترة على أي Field = خطر**
>
> كل Field بتسمح بالترتيب أو الفلترة عليه محتاج Index في الـ Database، وإلا الطلب هيبقى بطيء جداً على بيانات كبيرة. علشان كده `library.js` بيسمح بـ `id` و`createdAt` و`title` بس، وأي حاجة تانية 400 واضحة:

```
Terminal
user@MacBook ~ % curl -s 'localhost:3001/v1/articles?sort=views'
{"type":"https://example.com/problems/invalid-sort","title":"Invalid sort
field","status":400,"detail":"Cannot sort by 'views'"}
```

المصدر JSON:API · Sorting · JSON:API · Sparse Fieldsets

18.3 Offset/Limit و Page

الأبسط: "هات 20 بعد ما تعدّي 40". وشفناها في فصل 9: PokeAPI بيستخدم `offset` و`limit`، و Rick and Morty بيستخدم `page`:

```
Terminal
user@MacBook ~ % curl -s 'https://pokeapi.co/api/v2/pokemon?limit=1'
{"count":1351,"next":"https://pokeapi.co/api/v2/pokemon?
offset=1&limit=1","previous":null,"results":
[{"name":"bulbasaur","url":"https://pokeapi.co/api/v2/pokemon/1/"}]}
```

مميزات

- سهل تفهمه وتعمله.
- تقدر تنط لأي صفحة: "صفحة 37".
- تعرف العدد الكلي (`count`) وتعمل "صفحة 3 من 68".

مشاكل

- **بطيء في الآخر**: الـ Database لازم تعدّ وترمي كل اللي قبل الـ Offset. `offset=900000` = تقرا 900 ألف صف علشان ترمّيهم.
- **بيكرر ويفوّت** لو البيانات اتغيرت بين الصفحات (التجربة تحت).
- `count` نفسه مكلف على جداول كبيرة.

التجربة: عنصر جديد في النص

قايمة بالأحدث الأول (5، 4، 3، 2، 1). خدنا صفحة 1 (عنصرين)، وبعدين حد نشر بوست جديد رقم 6، وطلبنا صفحة 2:

drift.mjs (الجزء المهم)

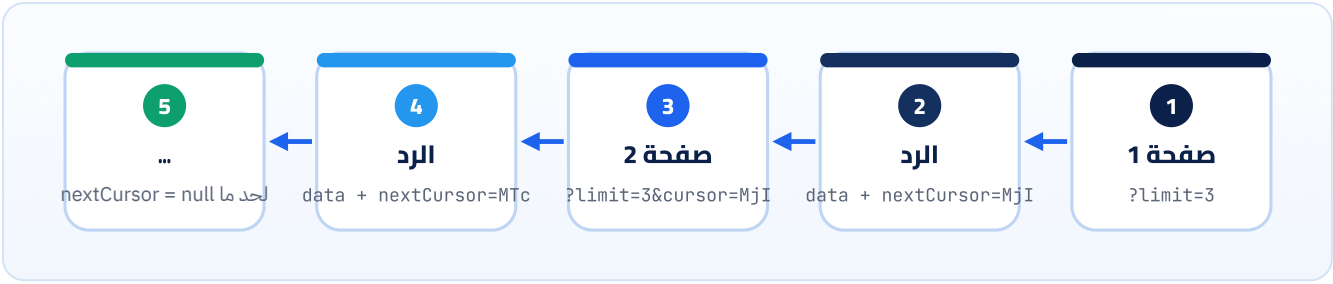
```
const byOffset = (offset, limit) => items.slice(offset, offset + limit).map((x) => x.id);
const byCursor = (afterId, limit) => {
  const start = afterId == null ? 0 : items.findIndex((x) => x.id === afterId) + 1;
  return items.slice(start, start + limit).map((x) => x.id);
};
```

Terminal

```
user@MacBook ~ % node drift.mjs
page 1   offset: [ 5, 4 ]  cursor: [ 5, 4 ]
-- new post 6 arrives --
page 2   offset: [ 4, 3 ]  cursor: [ 3, 2 ]
```

الـ Offset رجّع **4 تاني**: لأن 6 دخل في الأول فكل حاجة اتزحلقت خطوة. ولو كان عنصر **اتمسح**، كان هيحصل العكس: عنصر **يتفوّت** ومحدش يشوفه. والـ Cursor مكمّل صح من بعد 4، لأنه بيقول "**بعد العنصر ده**" مش "بعد 2 عنصر".

18.4 الـ Cursor



شكل 18.1 الـ Cursor بيشاور على «آخر عنصر شفته»، فالصفحة الجاية بتبدأ من بعده بالظبط مهما اتزود قبله.

الـ Cursor في `library.js` هو الـ ID بتاع آخر عنصر، متعمله Base64url: `MjI` = `"22"`. والـ Database بتعمل `WHERE id < 22 ORDER BY id DESC LIMIT 3`، وده سريع بالـ Index مهما كنت بعيد.

الـ Cursor معتم (Opaque)

الـ Client **ميفكّش** الـ Cursor ولا يألّفه، ياخده زي ما هو ويرجّعه. ليه؟ علشان السيرفر يقدر يغيّر جواه (يزوّد Field للترتيب، أو يشفّره) من غير ما يكسر حد. وده اللي GitHub بيعمله: الـ `after=Y3Vyc29yOnYyOpLP…` لو فكّيته أوله `cursor:v2:` وبعده Bytes. يعني حتى هو عامله Versions.

	Page / Offset	Cursor
تنط لصفحة 37	✓	✗ (التالي والسابق بس)
البيانات بتتغير	بيكرر ويفوّت	ثابت
السرعة في الآخر	بيبطأ	ثابت
مناسب لـ	جداول Admin صغيرة، ونتايج بحث بأرقام صفحات	Feeds، و Infinite scroll، و Sync، و APIs كبيرة
مين	PokeAPI، و Rick and Morty	Stripe (`starting_after` و `has_more`)، و GitHub (`after`)، و `library.js`

جدول 18.2 Offset ضد Cursor

المصدر Stripe · Pagination — GitHub · Using pagination in the REST API

18.5 الـ Link header (RFC 8288)

فين تحط لينك الصفحة الجاية؟ في الـ Body (`next` عند PokeAPI، و`nextCursor` عندنا) أو في هيدر `Link` اللي معرّف في RFC 8288. GitHub بيرجّع الـ Body Array على طول، فاللينكات في الهيدر (فصل 3):

تشريح الأمر

```
link: <https://api.github.com/repositories/237159/issues?per_page=2&page=3&after=Y3Vy…
>; rel="next",      <https://api.github.com/repositories/237159/issues?
per_page=2&page=1&before=Y3Vy…>; rel="prev"
```

1. `<…>` — الـ URL بين أقواس زاوية. وكامل جاهز: متركّبش URLs بإيدك.
2. `rel="next"` — نوع العلاقة: `next` و`prev` و`first` و`last`. (مش كلهم بييجوا دايماً، مثلاً مفيش `prev` في أول صفحة.)
3. `,` — أكتر من لينك بفاصلة.
4. `after= و page=` — لاحظ إن GitHub في الـ Issues بيرجّع Cursor (`after`) مع `page`. والـ Docs بتاعته بتقول: امشي ورا اللينكات ومتركبهاش بنفسك.

وده Client بيمشي ورا `rel="next"` لحد الآخر على `library.js` (اللي بيبعت الاتنين: `Link` و`nextCursor`):

walk.mjs

```
function nextLink(header) {
  if (!header) return null;
  for (const part of header.split(",")) {
    const m = part.match(/<([^>]+)>;\s*rel="next"/);
    if (m) return m[1];
  }
  return null;
}
let url = "/v1/articles?status=draft&limit=3&fields=id";
while (url) {
  const res = await fetch(BASE + url);
  const body = await res.json();
  ids.push(...body.data.map((a) => a.id));
  pages++;
  url = nextLink(res.headers.get("link"));
}
```

Terminal

```
user@MacBook ~ % node walk.mjs
{
  pages: 3,
  ids: [ '9', '12', '15', '18', '21', '24', '26' ]
}
```

الـ Parser البسيط ده

تقسيم بالـ `,` شغال هنا، بس الـ URL نفسه ممكن يبقى فيه `,` (زي `fields=id,title`) لو مكانتش اتعملها Encoding). في كود حقيقي استخدم مكتبة Parser للـ Link header، أو اعتمد على `next` اللي في الـ Body لو الـ API بيبعته.

المصدر RFC 8288 · Web Linking

ملخص الفصل

- Pagination من أول يوم، بـ Default و Maximum.
- فلترة بـ `?field=value`، وترتيب بـ `?sort=-createdAt`، و Fields بـ `?fields=`. ومسموح بس على Fields ليها Index.
- Offset: سهل وبينط، بس بيبطأ وبيكرر لما البيانات تتغير (شفناها بعينينا).
- Cursor: "بعد العنصر ده"، ثابت وسريع، ومعتم للـ Client.
- Link header (RFC 8288) بـ `rel="next"`، وامشي وراه متركبش URLs.

صح ولا غلط؟

1. `sort=-title` معناها ترتيب تنازلي بالعنوان. صح / غلط
2. الـ Client المفروض يفك الـ Cursor ويعدّل فيه علشان ينط صفحات. صح / غلط
3. الـ Cursor pagination بتخليك تروح لصفحة 37 على طول. صح / غلط

؟ أسئلة مراجعة

1. اشرح بالأرقام ليه الـ Offset كرر العنصر 4 في التجربة.
2. إمتى تختار Offset رغم عيوبه؟
3. إيه فايدة الـ Maximum على `limit` ؟
4. فين ممكن تحط لينك الصفحة الجاية؟ ومين بيستخدم أنهي طريقة؟

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل هات كل الـ monéPok بتوع الـ Generation الأولى (151) بـ 4 طلبات بالـ `offset` .
2. متوسط عدّل `drift.mjs` بحيث عنصر **يتمسح** بين الصفحتين، وشوف الـ Offset فوّت إيه.
3. صعب الـ Cursor في `library.js` بيعتمد على الـ `id` بس. لو الترتيب `-createdAt` وفيه مقالتين بنفس الوقت، إيه اللي ممكن يبوظ؟ وإزاي تصلّحه؟ (تلميح: Cursor فيه قيمتين)

19

المرحلة 5 · تصميم REST

الفصل التاسع عشر

الـ Versioning والـ Rate limits والـ Concurrency

الـ API بعد ما يتنشر بيبقى وعد. الفصل ده عن 3 حاجات بتحمي الوعد ده: إزاي تغيّر من غير ما تكسر الناس (Versioning و Deprecation و Sunset)، وإزاي تحمي السيرفر من الضغط (Rate limits و 429)، وإزاي تمنع اتنين يمسحوا شغل بعض (ETag و If-Match و 412). وكله بردود حقيقية من GitHub ومن library.js.

بعد الفصل ده هتكون عارف

- إيه اللي بيكسر الـ Clients وإيه اللي لأ
- 3 طرق للـ Versioning: URL و Header و Date
- GitHub: X-GitHub-Api-Version والـ 400 والـ 410
- Deprecation (RFC 9745) و Sunset (RFC 8594)
- درس REST Countries من تاني بعين المصمم
- Rate limits: x-ratelimit-* و RateLimit draft و Retry-After + 429
- Lost update: اتنين بيعدّلوا نفس المقالة
- ETag و If-Match و 412 بتجربة حقيقية

19.1 إيه اللي بيكسر؟

أول ما حد يبني على الـ API بتاعك، أي تغيير ممكن يوقّع App شغال عند ناس انت متعرفهمش. فلازم تفرّق:

آمن (Non-breaking)	بيكسر (Breaking)
تزوّد Endpoint جديد	تشيل Endpoint أو تغيّر الـ URL
تزوّد Field جديد في الرد	تشيل Field، أو تغيّر اسمه، أو نوعه (`"id": 4` لـ `"4"`)
تزوّد Parameter اختياري	تخلي Parameter إجباري، أو تضيّق القيم المقبولة
تزوّد قيمة Enum (لو قلت ده في الـ Docs)	تغيّر معنى Field أو Default (`limit` من 20 لـ 10)
رسالة `detail` أوضح	تغيّر `type` أو الـ Status code

جدول 19.1 Breaking ولا لأ

الحل الأول: متكسرش

أغلب التغييرات ممكن تتعمل بطريقة آمنة: Field جديد جنب القديم (`authorId` و `author`)، و Endpoint جديد جنب القديم. الـ Versioning هو آخر حل لما مفيش مفر.

19.2 3 طرق للـ Versioning

الطريقة	شكلها	مين	الرأي
في الـ URL	`/v1/articles`	Frankfurter (`/v2`)، و REST Countries، و `library.js`	واضحة جداً وسهلة في المتصفح والـ Logs. بس النسخة الجديدة = كل الـ URLs اتغيرت
في Header بتاريخ	`X-GitHub-Api-Version: 2026-03-10`	GitHub، و Stripe (`Stripe-Version`)	الـ URLs ثابتة، والتغييرات صغيرة ومتواريخ. أصعب تشوفها من المتصفح
في الـ Accept	`Accept: application/vnd.example.v2+json`	أقل انتشاراً	"صحيح" من ناحية HTTP، بس متعب للكل

جدول 19.2 فين تحط النسخة

GitHub على الطبيعة

Terminal

```
# من غير هيدر: الـ Default
user@MacBook ~ % curl -si https://api.github.com/zen -H 'User-Agent: book' | grep -i x-github-api-version
x-github-api-version-selected: 2022-11-28

# بالنسخة الجديدة
user@MacBook ~ % curl -s -D - -o /dev/null https://api.github.com/repos/expressjs/express \
    -H 'User-Agent: book' -H 'X-GitHub-Api-Version: 2026-03-10' | grep -i version
x-github-api-version-selected: 2026-03-10

# نسخة عمرها ما كانت موجودة
user@MacBook ~ % curl -s https://api.github.com/repos/expressjs/express -H 'User-Agent: book' -H 'X-GitHub-Api-Version: 2019-01-01'
{"message":"Bad Request","errors":"The version you specified in the \"X-GitHub-API-Version\" request header,
 \"2019-01-01\", is not a supported version. The following versions are currently supported:
 \"2026-03-10\" (most recent) and \"2022-11-28\".","documentation_url":"https://docs.github.com/rest","status":"400"}
```

تشريح الأمر

```
x-github-api-version-selected: 2026-03-10
```
(2)

1. `Default` من غير هيدر = `2022-11-28`. يعني الكود القديم مبيتكسرش لوحده.
2. `-selected` السيرفر بيقولك "اشتغلت بأنهي نسخة"، فتتأكد في الـ Logs.
3. `400` نسخة غلط خالص = 400، ورسالة بتقولك النسخ المتاحة. (وحسب الـ Docs، نسخة **اتقفلت** بعد فترة الدعم = **Gone 410**.)
4. `شهر 24` الـ Docs بتقول إن كل نسخة مدعومة 24 شهر بعد ما تنزل اللي بعدها، وقبل القفل بيبعت `Deprecation` و `Sunset`.

المصدر Stripe · Versioning GitHub · API Versions

19.3 Deprecation و Sunset: تقفل بأدب

في فصل 9 الكود القديم بتاع REST Countries وقف فجأة، والرد كان:

JSON

```
{ "success": false, "data": null,
  "errors": [{ "message": "This API version has been deprecated. Please visit
https://restcountries.com/docs/countries/legacy-api-deprecation to migrate to our new version
(v5)." }] }
```

والنهارده بقى بيعمل `301` لملف تاني. الرسالة كويسة، بس جت **بعد** ما الكود وقف، و Status code مش واضح. الطريقة الـ Standard إنك تحذّر **قبلها بفترة** بهيدرز، وأي أداة أو Middleware تقدر تقراها وتنبّه المطوّر. جربناها على `/v1/posts` (اسم قديم لـ `/v1/articles`):

curl -si localhost:3001/v1/posts

(1) **RFC 9745**: "الحاجة دي Deprecated من التاريخ ده". الشكل `@` + Unix timestamp (Structured field). 1790812800 = 1 أكتوبر 2026 الساعة 00:00 UTC. لسه شغال، بس ابدأ تنقل.	`Deprecation: @1790812800`
(2) **RFC 8594**: "هتقف خالص في التاريخ ده". بصيغة HTTP-date (مش @). بعدها المفروض 410.	`Sunset: Fri, 01 Jan 2027 00:00:00 GMT`
(3) البديل فين.	`Link: </v1/articles>; rel="successor-version"`
(4) صفحة بتشرح الـ Migration (الـ `rel="deprecation"` معرّف في RFC 9745).	`<https://example.com/docs/deprecations/posts>; rel="deprecation"; type="text/html"`

1. أعلن في الـ Changelog والإيميل، وابدأ تبعت `Deprecation` و `Link`.
2. حدد `Sunset` بمهلة كفاية (GitHub بيدّي 24 شهر للنسخ).
3. راقب مين لسه بيستخدم القديم (Logs بالـ API key) وكلّمهم.
4. بعد الـ Sunset: **Gone 410** و Problem فيه لينك الـ Migration، مش 404 ولا 200 بـ Error.

المصدر RFC 8594 · Sunset Header RFC 9745 · Deprecation Header

19.4 الـ Rate limits

من غير حد، Client واحد فيه Bug (Loop) أو Scraper ممكن يوقّع السيرفر على الكل. فكل API عام بيحط حد، والمهم إنه **يقولك** الحد وفاضل كام. GitHub (فصل 3) بيستخدم هيدرز `x-` خاصة بيه:

```http
x-ratelimit-limit: 60
x-ratelimit-remaining: 49
x-ratelimit-used: 11
x-ratelimit-resource: core
x-ratelimit-reset: 1792758101
```

وكل API بأسماء مختلفة شوية (NASA: `x-ratelimit-limit` و `x-ratelimit-remaining` بس). فـ IETF شغالة على Standard (لسه Draft، النسخة 11)، وطبقناه في `library.js` بحد 3 طلبات في الدقيقة علشان نشوف الـ 429:

```
user@MacBook ~ % for i in 1 2 3 4; do curl -s -o /dev/null -D - localhost:3002/v1/articles/1 |
grep -iE '^(HTTP|RateLimit|Retry)'; done
HTTP/1.1 200 OK
RateLimit-Policy: "default";q=3;w=60
RateLimit: "default";r=2;t=60
HTTP/1.1 200 OK
RateLimit-Policy: "default";q=3;w=60
RateLimit: "default";r=1;t=60
HTTP/1.1 200 OK
RateLimit-Policy: "default";q=3;w=60
RateLimit: "default";r=0;t=60
HTTP/1.1 429 Too Many Requests
RateLimit-Policy: "default";q=3;w=60
RateLimit: "default";r=0;t=60
Retry-After: 60

user@MacBook ~ % curl -s localhost:3002/v1/articles/1
{"type":"about:blank","title":"Too Many Requests","status":429,"detail":"Limit is 3 requests
per 60 seconds"}
```

تشريح الأمر

```
RateLimit-Policy: "default";q=3;w=60RateLimit: "default";r=0;t=60Retry-After: 60
```

1. `"default"` اسم السياسة (ممكن يبقى في أكتر من حد: في الدقيقة وفي اليوم).
2. `q=3` Quota: الحد (3 طلبات).
3. `w=60` Window: في 60 ثانية.
4. `r=0` Remaining: فاضل كام.
5. `t=60` بعد كام ثانية العدّاد يرجع (ثواني من دلوقتي، مش Timestamp زي GitHub).
6. `Retry-After` هيدر Standard من RFC 9110: استنى 60 ثانية قبل ما تجرب تاني.

كـ Client

اقرا `remaining` وهدّي قبل ما توصل للصفر. ولو جالك 429: استنى `Retry-After` (لو موجود) أو Backoff متزايد مع Jitter، زي ما عملنا في كتاب Python (فصل 25). ومتعيدش فوراً في Loop، ده بيخليك تتحظر.

كـ Provider

الـ Rate limiter اللي في `library.js` (Map في الذاكرة) للتعلم بس. لو عندك أكتر من سيرفر، كل واحد هيعدّ لوحده. في Production بيتعمل في Redis أو في الـ API Gateway أو مكتبة زي `express-rate-limit`. وحطه **قبل** الـ Body parser (زي ما عملنا) علشان حتى الطلبات البايظة تتعد.

المصدر MDN · Retry-After · GitHub · Rate limits · IETF Draft · RateLimit header fields for HTTP

19.5 الـ Lost update: ETag و If-Match

انت وزميلك فتحتوا نفس المقالة الساعة 10. انت غيرت العنوان وحفظت الساعة 10:05. هو غيّر الـ Status وحفظ الساعة 10:07، والفورم بتاعه فيه العنوان **القديم**. النتيجة: تعديلك اتمسح ومحدش خد باله. ودي اسمها **Lost update**.

الحل في HTTP نفسه: كل نسخة من الـ Resource ليها `ETag` (بصمة)، والتعديل بيقول "عدّل **بشرط** إن النسخة لسه دي" بـ `If-Match`. ولو حد عدّل قبلك، السيرفر يرفض بـ **412**:



شكل 19.1 نفس الـ ETag اللي بيوفّر Bandwidth في الـ Cache (304) هو اللي بيحمي من الـ Lost update.

Terminal

```
user@MacBook ~ % curl -si localhost:3001/v1/articles/4
HTTP/1.1 200 OK
ETag: "39fqqzHpm_icenp6"
{"id":"4","title":"Article 4","status":"published","authorId":"1","tags":["json"],...}

# الأولى: بنفس الـ ETag ← نجح، و ETag جديد
user@MacBook ~ % curl -si -X PATCH localhost:3001/v1/articles/4 -H 'Content-Type:
application/merge-patch+json' \
    -H 'If-Match: "39fqqzHpm_icenp6"' -d '{"title":"Article four","tags":null}'
HTTP/1.1 200 OK
ETag: "kgfYo1GjspvVpE60"

# التاني: لسه معاه الـ ETag القديم ← اترفض
user@MacBook ~ % curl -si -X PATCH localhost:3001/v1/articles/4 -H 'Content-Type:
application/merge-patch+json' \
    -H 'If-Match: "39fqqzHpm_icenp6"' -d '{"status":"review"}'
HTTP/1.1 412 Precondition Failed
Content-Type: application/problem+json; charset=utf-8

{"type":"about:blank","title":"Precondition Failed","status":412,"detail":"The article changed
since you read it"}

# وبنفس الـ ETag مع 304 ← GET (Cache، فصل 3)
user@MacBook ~ % curl -si localhost:3001/v1/articles/4 -H 'If-None-Match: "39fqqzHpm_icenp6"'
HTTP/1.1 304 Not Modified
```

If-None-Match (مع GET)

"ابعتلي بس لو **اتغير**". لو زي ما هو = 304 من غير Body. للـ Cache.

If-Match (مع PATCH/PUT/DELETE)

"نفّذ بس لو **متغيرش**". لو اتغير = 412. للـ Concurrency. ولو عايز تجبر الـ Clients يبعتوه: 428 Precondition Required.

ETag قوي ولا ضعيف

`"39fq…"` = Strong: البايتات نفسها. `W/"113-…"` = Weak: "تقريباً نفس المحتوى". Express بيحط Weak ETag أوتوماتيك على الردود (شفناه قبل ما نقفله في `library.js`)، وده كويس للـ 304 بس **مينفعش** مع `If-Match` (RFC 9110 بيقول المقارنة هناك Strong). علشان كده عملنا ETag بإيدنا من Hash للمحتوى.

المصدر MDN · 412 Precondition Failed · RFC 9110 · If-Match

ملخص الفصل

- زوّد براحتك، بس متشيلش ومتغيرش نوع أو معنى. الـ Versioning آخر حل.
- Versioning في الـ URL (`/v1`) أو Header بتاريخ (Stripe و GitHub). GitHub: Default قديم، و 400 لنسخة مش موجودة، و 410 لنسخة اتقفلت.
- اقفل بأدب: `Deprecation` (@timestamp) و`Sunset` (HTTP-date) و`Link`، وبعدها 410.
- Rate limits: قول الحد والفاضل، و 429 + `Retry-After`. والـ Client يهدّي ويعمل Backoff.
- ETag + `If-Match` = 412 بدل ما تعديل حد يضيع.

صح ولا غلط؟

1. إنك تزوّد Field جديد في الرد Breaking change. صح / غلط
2. في هيدر RateLimit، `t=30` معناها Unix timestamp. صح / غلط
3. 412 معناها إن الـ Resource اتغير بعد ما انت قريته. صح / غلط

أسئلة مراجعة

1. إيه الفرق بين `Deprecation` و`Sunset`؟ وكل واحد بصيغة إيه؟
2. ليه GitHub عامل Default قديم بدل أحدث نسخة؟
3. اشرح الـ Lost update بمثال، وإزاي `If-Match` بيحله.
4. ليه الـ Rate limiter في الذاكرة مش كفاية لو عندك 3 سيرفرات؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل ابعت طلب لـ GitHub واحسب الـ `x-ratelimit-reset` بتوقيت القاهرة بـ `date -r`.
2. متوسط اكتب Client بيعمل GET، وبعدين PATCH بـ `If-Match`، ولو جاله 412 يعمل GET تاني ويعرض الفرق.
3. صعب عايز تغيّر `authorId` لـ `author: {id, name}` في الـ Content Library API. اكتب خطة من غير ما تكسر حد: إيه اللي يتزود إمتى، وإمتى تبعت Deprecation، وإمتى Sunset.

20

المرحلة 6 · OpenAPI و Webhooks

الفصل العشرين

الـ Webhooks: لما الـ API هو اللي يكلمك

في فصل 2 عرفنا الـ Webhook كفكرة: بدل ما تسأل كل شوية، السيرفر التاني يبعتلك. هنا هنبني واحد بجد: Express بيستقبل Webhook شبه GitHub، ويتأكد من التوقيع بـ HMAC على الـ Raw body، ويرفض المزوّر، ويتجاهل المكرر، ويرد بسرعة. وهنشوف بعينينا ليه express.json() هنا بيبوّظ التوقيع.

بعد الفصل ده هتكون عارف

- الـ Polling ضد الـ Webhook: الحسبة
- شكل الـ Webhook: POST فيه Event و Delivery ID و Signature
- HMAC-SHA256 حرف بحرف، و test vector بتاع GitHub
- ليه لازم Raw body: تجربة بتفشل
- timingSafeEqual وليه مش ===
- الـ Duplicate والـ Retry: GitHub مبيعيدش لوحده و Stripe بيعيد 3 أيام
- رد في أقل من 10 ثواني والشغل في Queue
- إزاي تجرّب على جهازك: smee.io و stripe listen

20.1 السؤال كل 5 ثواني

عايز تعرف لما حد يفتح Issue في الـ Repo بتاعك. الحل الأول (Polling): تسأل GitHub كل دقيقة "في جديد؟". يعني 1440 طلب في اليوم، وأغلبها إجابته "لأ"، وكمان بتاكل من الـ Rate limit (60 في الساعة من غير توكن!)، والخبر بيوصلك متأخر لحد دقيقة. فاتطور حل: **انت تدّي GitHub عنوان**، وهو يبعتلك `POST` أول ما الحاجة تحصل. ده الـ Webhook: API بالعكس، وانت هنا السيرفر.



شكل 20.1 الـ Webhook = HTTP request عادي، بس اللي بيبعته هو الـ Provider واللي بيستقبله انت.

20.2 شكل الـ Delivery

Webhook شبه GitHub

1 طلب عادي، الـ Body JSON.

```
POST /webhooks/github HTTP/1.1
Content-Type: application/json
```

2 نوع الحدث، علشان تعرف تتعامل إزاي قبل ما تقرا الـ Body.

```
X-GitHub-Event: issues
```

3 ID فريد لكل Delivery. هو اللي هتمنع بيه التكرار.	`X-GitHub-Delivery: 72d3162e-cc78-11e3-81ab-4c9367dc0958`
4 التوقيع: `sha256=` + HMAC-SHA256 لـ Body بالـ Secret اللي انت وGitHub بس عارفينه.	`X-Hub-Signature-256: sha256=d3a7e3c6…ee68`
5 الـ Payload. و `action` بيحدد التفاصيل (opened و closed و edited).	`{"action": "opened", "issue": {...}}`

20.3 التوقيع: ليه وإزاي

الـ URL بتاعك عام. أي حد يقدر يبعتله `POST` بـ `{"action": "opened"}` مزوّر. فلازم تتأكد إن الطلب جاي من GitHub **ومحدش غيّر فيه**. الحل: **HMAC**. GitHub بياخد الـ Body بايت بايت، ويعمله Hash مع الـ Secret، ويبعت النتيجة. وانت تعمل نفس الحسبة: لو طلعت نفس النتيجة يبقى الـ Body ده بالظبط واللي بعته عارف الـ Secret.

hmac.mjs بالـ test vector في Docs بتاعة GitHub اللي

```
import { createHmac, timingSafeEqual } from "node:crypto";

function verify(secret, rawBody, header) {
  const expected = "sha256=" + createHmac("sha256", secret).update(rawBody).digest("hex");
  const a = Buffer.from(expected);
  const b = Buffer.from(header ?? "");
  return a.length === b.length && timingSafeEqual(a, b);
}
const secret = "It's a Secret to Everybody";
const body = "Hello, World!";
const header = "sha256=757107ea0eb2509fc211221cce984b8a37570b6d7586c22c46f4379c8b043e17";
console.log(verify(secret, body, header));          // الصح
console.log(verify(secret, body + " ", header));    // Body مسافة زيادة في الـ
console.log(verify("wrong", body, header));         // Secret غلط
```

Terminal

```
user@MacBook ~ % node hmac.mjs
true
false
false
```

تشريح الأمر

```
createHmac("sha256", secret).update(rawBody).digest("hex")
```

1. `createHmac` من `node:crypto` (مدمج، مفيش npm).
2. `"sha256"` الخوارزمية. GitHub بيبعت كمان `X-Hub-Signature` القديم بـ SHA-1 للتوافق، والـ Docs بتقول استخدم الـ 256.

3	`.update(rawBody)`	البايتات **زي ما وصلت بالظبط**.
4	`.digest("hex")`	النتيجة Hex (64 حرف)، وبعدين نزوّد `sha256=` قبلها.
5	`timingSafeEqual`	مقارنة بتاخد نفس الوقت مهما كان أول اختلاف فين. الـ `===` بتقف عند أول حرف مختلف، فالـ Attacker يقدر يقيس الوقت ويخمّن التوقيع حرف حرف (Timing attack). ولازم الطولين يبقوا متساويين قبلها وإلا بترمي Error.

المصدر Node.js · crypto.timingSafeEqual · GitHub · Validating webhook deliveries

20.4 نبني الـ Receiver

hook.js (الجزء المهم)

```
// express.raw keeps the exact bytes as a Buffer in req.body
app.post("/webhooks/github", express.raw({ type: "application/json" }), (req, res) => {
  if (!validSignature(req.body, req.get("X-Hub-Signature-256")))
    return res.status(401).json({ error: "bad signature" });

  const id = req.get("X-GitHub-Delivery");
  if (seen.has(id)) return res.status(200).json({ ok: true, duplicate: true });
  seen.add(id);

  const event = req.get("X-GitHub-Event");
  const payload = JSON.parse(req.body); // parse only AFTER verifying
  res.status(202).json({ ok: true });   // answer first...
  setImmediate(() => {                  // ...then do the slow work (a queue in production)
    console.log(`handled ${event} #${id}: ${payload.action} ${payload.issue?.title ?? ""}`);
  });
});
```

ونجرّب بـ curl: نعمل التوقيع بـ `openssl` (موجود على الـ Mac) زي ما GitHub بيعمله بالظبط:

Terminal

```
user@MacBook ~ % SIG="sha256=$(openssl dgst -sha256 -hmac 'dev-secret' -hex < payload.json |
awk '{print $NF}')"
user@MacBook ~ % echo "$SIG"
sha256=d3a7e3c6a80e15960db92f0cdf3368051d3504e870ced58c15275e4daa82ee68

# 1) Delivery سليم
user@MacBook ~ % curl -s -X POST localhost:4000/webhooks/github -H 'Content-Type:
application/json' \
    -H 'X-GitHub-Event: issues' -H 'X-GitHub-Delivery: 72d3162e-cc78-11e3-81ab-4c9367dc0958' \
    -H "X-Hub-Signature-256: $SIG" --data-binary @payload.json -w '\n%{http_code}\n'
{"ok":true}
202

# 2) نفس الـ Delivery تاني (إعادة)
{"ok":true,"duplicate":true}
```

```
200

# 3) Body تاني بنفس التوقيع (تزوير)
user@MacBook ~ % curl -s ... -H "X-Hub-Signature-256: $SIG" --data-binary
'{"action":"deleted"}'
{"error":"bad signature"}
401
```

السيرفر Log

```
> handled issues #72d3162e-cc78-11e3-81ab-4c9367dc0958: opened Docs typo
```

تشريح الأمر

```
openssl dgst -sha256 -hmac 'dev-secret' -hex < payload.json | awk '{print $NF}'
1            2           3          4            5
```

1. `dgst -sha256` — اعمل Hash بـ SHA-256.
2. `-hmac 'dev-secret'` — اعمله HMAC بالـ Secret ده.
3. `-hex` — النتيجة Hex.
4. `< payload.json` — اقرا الملف كـ Input.
5. `awk '{print $NF}'` — الـ openssl هنا كتب `SHA2-256(stdin)= d3a7…` (والشكل ممكن يختلف شوية في LibreSSL اللي على الـ Mac)، فبناخد آخر كلمة بس، وده شغال في الحالتين.
6. `--data-binary` — ابعت الملف **زي ما هو**. الـ `-d` بيشيل الـ Newlines، فالتوقيع هيبوظ!

20.5 ليه الـ Raw body؟ التجربة

عملنا Route تاني بالطريقة "الطبيعية": `express.json()` وبعدين نوقّع `JSON.stringify(req.body)`. نفس الـ Payload ونفس التوقيع:

Terminal

```
user@MacBook ~ % curl -s -X POST localhost:4000/webhooks/wrong -H 'Content-Type:
application/json' \
    -H "X-Hub-Signature-256: $SIG" --data-binary @payload.json -w '\n%{http_code}\n'
{"ok":false}
401
```

ليه؟ الملف كان متنسّق بمسافات وسطور. الـ `JSON.parse` ثم `JSON.stringify` طلّعوا `{"action":"opened","issue":{"number":7,"title":"Docs typo"}}` : نفس البيانات بس **بايتات مختلفة**، والـ HMAC بيشتغل على البايتات. وده أشهر Bug في الـ Webhooks، و Stripe بيقول نفس الكلام: لازم الـ Raw body.

الترتيب في Express

لو عامل `app.use(express.json())` على الـ App كله، هيقرا الـ Body قبل الـ Webhook route ويضيّع البايتات. الحل: `express.raw()` على الـ Route ده بالذات (زي `hook.js`)، أو حط الـ Webhook route **قبل** `app.use(express.json())`.

20.6 التكرار والإعادة والسرعة

	GitHub	Stripe
التوقيع	`X-Hub-Signature-256: sha256=…`	`Stripe-Signature: t=…,v1=…` (فيه Timestamp)
وقت الرد	xx2 خلال 10 ثواني، وإلا فشل	xx2 بسرعة قبل أي شغل معقد
لو فشل	**مبيعيدش لوحده**. انت تعمل Redeliver (من الـ UI أو الـ API)	بيعيد لحد 3 أيام بـ Exponential backoff (Live mode)
التكرار	استخدم `X-GitHub-Delivery`	سجّل الـ Event IDs اللي اتعالجت
Replay قديم	—	الـ `t=` جوه التوقيع، والمكتبة بترفض الأقدم من 5 دقايق افتراضياً

جدول 20.1 GitHub ضد Stripe (من الـ Docs)

Best Practice: الـ Receiver

1. اتأكد من التوقيع على الـ Raw body بـ `timingSafeEqual`، **قبل** أي Parse.
2. اتجاهل المكرر بالـ Delivery/Event ID (في Database، مش `Set` في الذاكرة).
3. رد xx2 فوراً، وحط الشغل التقيل في Queue.
4. متعتمدش على الترتيب: ممكن "closed" يوصل قبل "opened". ولو محتاج الحالة الحقيقية، اسأل الـ API.
5. Endpoint بـ HTTPS، والـ Secret في Environment variable.

المصدر
Stripe · Webhooks · GitHub · Redelivering webhooks · GitHub · Best practices for webhooks · Stripe · Signature verification

20.7 تجرّب على جهازك إزاي؟

المشكلة: GitHub مش هيقدر يوصل لـ `localhost:4000` على الـ Mac بتاعك. الحل: خدمة في النص بتستقبل وتحوّل لجهازك:

الأداة	بتعمل إيه
smee.io	اللي Docs بتاعة GitHub بتستخدمه: URL عام، و `smee-client` على جهازك بيحوّل لـ `localhost`
Stripe CLI	`stripe listen --forward-to localhost:4242/webhook` ، و `stripe trigger` يبعت Events تجريبية
openssl + curl	اللي عملناه فوق: تجرّب التوقيع والتكرار من غير أي خدمة
Redeliver في GitHub	من صفحة الـ Webhook: "Recent Deliveries" تشوف كل طلب ورده وتعيده

جدول 20.2 أدوات

المصدر Stripe CLI · GitHub · Handling webhook deliveries (smee)

ملخص الفصل

- Webhook = الـ Provider بيبعتلك POST بدل ما تسأل كل شوية.
- التوقيع HMAC-SHA256 على البايتات، والمقارنة بـ `timingSafeEqual`.
- `express.json()` قبل التوقيع = فشل (شفناه)، فاستخدم `express.raw()`.
- اتجاهل المكرر بالـ ID، ورد بسرعة، والشغل في Queue.
- GitHub مبيعيدش لوحده، و Stripe بيعيد لحد 3 أيام.

صح ولا غلط؟

1. لو الـ JSON بعد Parse ثم Stringify هو نفس البيانات، التوقيع هيطلع صح. صح / غلط
2. استخدام `===` لمقارنة التوقيعات آمن تماماً. صح / غلط
3. الـ Webhook endpoint المفروض يخلّص كل الشغل قبل ما يرد. صح / غلط

أسئلة مراجعة

1. احسب: Polling كل 30 ثانية لـ 5 Repos = كام طلب في الساعة؟ وقارنه بحد GitHub من غير توكن.
2. ليه الـ Secret لازم يفضل سر؟ إيه اللي يحصل لو اتسرب؟
3. إزاي الـ Timestamp في توقيع Stripe بيحمي من الـ Replay؟
4. ليه `--data-binary` مش `-d` في التجربة؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `hmac.mjs` وغيّر حرف واحد في الـ Body وشوف النتيجة.
2. متوسط زوّد على `hook.js` إنه يتعامل بس مع `issues` و `action === "opened"` ويتجاهل الباقي بـ 204.
3. صعب اعمل Repo تجريبي على GitHub، وحط Webhook بـ smee.io، وخلّي `hook.js` يستقبل Delivery حقيقي.

21

المرحلة 6 · OpenAPI و Webhooks

الفصل الحادي والعشرين

تكتب OpenAPI بإيدك

في فصل 11 قريت OpenAPI بتاع Frankfurter كمستخدم. دلوقتي هتكتبه للـ API بتاعك: نوصف library.js بـ OpenAPI 3.2، وكل جزء بنشرحه سطر بسطر، ونعمل Lint بـ Redocly CLI ونصلّح اللي طلّعه، وبعدين نطلّع منه Docs و TypeScript types، ونستخدمه كـ Contract test يمسك سيرفر بيكدب.

بعد الفصل ده هتكون عارف

- Design-first ضد Code-first
- الهيكل: openapi و info و servers و paths و components
- Path و Operation و operationId
- Parameters: query و path و header
- requestBody والـ Content types (merge-patch و problem+json)
- components: schemas و parameters و responses و headers و $ref
- Lint بـ Redocly CLI: 3 تحذيرات وتصليحها
- Docs HTML و TypeScript types من نفس الملف
- Contract test: السيرفر القديم فشل

21.1 Design-first

قبل كده: الفريق يكتب الكود، وبعد ما يخلص حد يكتب Docs في Word أو Wiki. بعد شهرين الكود اتغير والـ Docs لأ، والـ Frontend بيشتغل على معلومات غلط. وكانت المشكلة إن الـ Docs **نسخة تانية** من الحقيقة. فاتطور حل: الـ **Spec** يبقى ملف واحد تقدر الآلة تقراه، ومنه تطلع الـ Docs، والـ Types، والـ Mock server، والـ Tests.

Design-first

تكتب الـ Spec الأول، والفريق (وانت والـ Frontend) يتفقوا عليه، وبعدين تكتب الكود. التصميم بيتراجع وهو لسه رخيص يتغير. **ده اللي هنعمله في المشروع (فصل 24).**

Code-first

تكتب الكود، ومكتبة بتطلّع الـ Spec منه (من zod مثلاً، أو من Decorators في NestJS). أسرع في الأول، بس التصميم بيتعمل وانت بتكتب كود.

21.2 الهيكل

هنوصف `library.js` (فصل 15 لـ 19). الملف كامل 239 سطر YAML. (YAML = نفس JSON بس بالمسافات بدل الأقواس، وأسهل في الكتابة بإيدك. والاتنين مقبولين في OpenAPI.)

YAML في صفحة: هتحتاجها في Docker و Kubernetes

YAML هو JSON بس من غير أقواس ولا Quotes في الغالب. نفس البيانات بالشكلين:

article.yaml

```yaml
# comment: Kubernetes and Compose files look exactly like this
title: Hello YAML        # key: value  (string without quotes)
views: 120               # number
published: true          # boolean
tags:                    # a list: every item starts with "- "
  - api
  - json
author:                  # nested object: indent with 2 spaces
  name: Sara
  email: sara@example.com
body: |                  # | = multi-line text, kept as is
  First line.
  Second line.
```

- **المسافات هي القواعد**: اللي متزحزح لجوه تحت مفتاح = جواه. مسافات بس، والـ Tab ممنوع. وأشهر Error هتشوفه سببه مسافة زيادة أو ناقصة.
- `key: value` : لازم مسافة بعد الـ `:` .
- `-` : عنصر في List. ولو العنصر Object، المفاتيح بتاعته تيجي تحت الشرطة بنفس المسافة.
- النص من غير Quotes عادي، بس حط `"..."` لو فيه `:` أو `#` ، أو لو عايز `"8080"` يفضل نص مش رقم.

الشكل ده بالظبط هو اللي هتكتبه في `compose.yaml` في كتاب Docker، وفي كل ملفات Kubernetes.

spec/openapi.yaml: أول جزء

1 نسخة الـ **Spec** نفسه (مش الـ API). 3.2.0 نزلت سبتمبر 2025، وفيه 3.2.1 (سبتمبر 2026) تصليحات. ولو أداة عندك مش بتدعم 3.2 لسه، اكتب x.3.1.	`openapi: 3.2.0`

2 نسخة **الـ API بتاعك**. انت اللي بتحددها.	`info:`
	`  title: Content Library API`
	`  version: 1.0.0`

3 `identifier` = كود SPDX (من 3.1).	`  license:`
	`  name: MIT`
	`  identifier: MIT`

4 الـ Base URL. كل الـ Paths بتتحط بعده.	`servers:` `  - url: http://localhost:3001/v1`
5 "مفيش Auth" بشكل صريح (هنشوف ليه تحت).	`security: []`

6 كل Path وتحته الـ Methods. كل واحد اسمه **Operation**.	`paths:`
	`  /articles:`
	`  get: ...`
	`  post: ...`

7 الحاجات اللي بتتكرر، بتتعرّف مرة وتتشاور عليها بـ `$ref`.	`components:`
	`  schemas: ...`
	`  parameters: ...`
	`  responses: ...`

21.3 Operation كامل

POST /articles

```yaml
/articles:
  post:
    operationId: createArticle
    summary: Create an article
    tags: [articles]
    parameters:
      - name: Idempotency-Key
        in: header
        description: A unique key (UUID) so retries do not create duplicates.
        schema: { type: string, maxLength: 255 }
    requestBody:
      required: true
      content:
        application/json:
          schema: { $ref: "#/components/schemas/ArticleInput" }
          example: { title: REST naming, authorId: "2" }
    responses:
      "201":
        description: Created
        headers:
          Location:
            description: URL of the new article
            schema: { type: string }
        content:
          application/json:
            schema: { $ref: "#/components/schemas/Article" }
      "415": { $ref: "#/components/responses/Problem" }
      "422": { $ref: "#/components/responses/ValidationProblem" }
```

تشريح الأمر

```yaml
operationId: createArticle
```

1 `operationId` اسم فريد للعملية. الأدوات بتعمل منه اسم Function (`api.createArticle()`). اختاره بعناية.

2	`tags`	تجميع في الـ Docs (القسم "articles").
3	`in: header`	مكان الـ Parameter: `query` أو `path` أو `header` أو `cookie` (و 3.2 زوّد `querystring` للـ Query كلها كـ string واحد).
4	`requestBody.content`	مفتاحه الـ Content-Type. والـ PATCH عندنا مفتاحه `application/merge-patch+json` (فصل 15).
5	`"201"`	الـ Status codes بين علامات تنصيص (YAML بيحتاجها علشان متتقريش كرقم).
6	`headers: Location`	هيدرز الرد بتتوصف كمان.
7	`$ref`	JSON Pointer لحتة في نفس الملف. `#` = أول الملف.

الـ Custom method والـ Path parameters

```yaml
/articles/{articleId}:publish:
  parameters:
    - $ref: "#/components/parameters/ArticleId"
  post:
    operationId: publishArticle
    responses:
      "200": ...
      "404": { $ref: "#/components/responses/Problem" }
      "409": { $ref: "#/components/responses/Problem" }
```

`{articleId}` بين أقواس معوجة = Path parameter (في Express كانت `:id`). والـ `:publish` هنا حرفي عادي. والـ Parameters اللي تحت الـ Path نفسه بتتطبق على كل الـ Methods بتاعته.

21.4 الـ Components

components/schemas (جزء)

```yaml
    ArticleInput:
      type: object
      required: [title, authorId]
      properties:
        title: { type: string, minLength: 3 }
        status: { $ref: "#/components/schemas/Status", default: draft }
        authorId: { type: string }
        tags: { type: array, items: { type: string }, default: [] }
    Article:
      allOf:
        - $ref: "#/components/schemas/ArticleInput"
        - type: object
          required: [id, createdAt]
          properties:
            id: { type: string, readOnly: true }
```

```yaml
            createdAt: { type: string, format: date-time, readOnly: true }
    ArticlePage:
      type: object
      required: [data, nextCursor]
      properties:
        data: { type: array, items: { $ref: "#/components/schemas/Article" } }
        nextCursor: { type: [string, "null"] }
```

تشريح الأمر

```
allOf (2) / readOnly (3) / type: [string, "null"] (4)
```

1. **JSON Schema** — من 3.1، الـ Schemas دي JSON Schema 2020-12 بالظبط (فصل 7). كل اللي اتعلمته هناك شغال هنا.
2. **allOf** — Article = كل حاجة في ArticleInput + `id` و `createdAt`. بنكتب الـ Fields مرة واحدة.
3. **readOnly** — السيرفر بيحطه، والـ Client ميبعتهوش. (الـ Types اللي هتطلع هتعلّمه `readonly`).
4. **[string, "null"]** — الطريقة الجديدة لـ "String أو null" في 3.1+ (بدل `nullable: true` بتاعة 3.0).

components/responses: الـ Errors مرة واحدة

```yaml
    Problem:
      description: Error (RFC 9457)
      content:
        application/problem+json:
          schema: { $ref: "#/components/schemas/Problem" }
```

وكل Operation بيقول `"404": { $ref: "#/components/responses/Problem" }`. الشكل اللي اتفقنا عليه في فصل 17 بقى مكتوب ومتوثّق في حتة واحدة.

المصدر OpenAPI Specification 3.2.0 · OpenAPI Initiative · Learn OpenAPI

21.5 الـ Lint بـ Redocly CLI

الـ Lint بيقرا الـ Spec ويقولك: ده مش Valid، أو ده Valid بس مش Best practice. Redocly CLI مجاني ومفتوح المصدر، وبيشتغل بـ `npx` من غير تسطيب:

Terminal

```
user@MacBook ~ % npx -y @redocly/cli@latest --version
2.70.0
user@MacBook ~ % npx -y @redocly/cli@latest lint openapi.yaml
No configurations were provided -- using built in recommended configuration by default.

[1] openapi.yaml:11:10 at #/servers/0/url
Server `url` should not point to example.com or localhost.
Warning was generated by the no-server-example.com rule.
```

```
[2] openapi.yaml:119:7 at #/paths/~1articles~1{articleId}/delete/responses
Operation must have at least one `4XX` response.
Warning was generated by the operation-4xx-response rule.

Woohoo! Your API description is valid. 🎉
You have 2 warnings.
```

تشريح الأمر

```
at #/paths/~1articles~1{articleId}/delete/responses
```

1. `#/paths/…` مكان المشكلة كـ JSON Pointer. والـ `/` اللي جوه اسم الـ Path بقت `~1` (قاعدة RFC 6901: `~1` = `/`، و `~0` = `~`).
2. `[1] localhost` تحذير منطقي لـ API حقيقي، بس سيرفرنا فعلاً Local. فقفلنا القاعدة دي في `redocly.yaml` بتعليق بيقول ليه.
3. `[2] 4XX` الـ DELETE مكانش موثّق ليه أي Error. بس الـ Rate limiter ممكن يرجّع 429 لأي طلب، فزوّدناه.

ولما شيلنا `security` من الـ Spec (لأن `library.js` مفيهوش Auth)، الـ Lint طلّع **6 Errors** جديدة:

Terminal

```
[1] openapi.yaml:18:5 at #/paths/~1articles/get
Every operation should have security defined on it or on the root level.
Error was generated by the security-defined rule.
...
❌ Validation failed with 6 errors.
```

القاعدة دي بتقول: "لازم تقول صراحة. نسيت الـ Auth ولا قاصد؟". فكتبنا `security: []` (Array فاضية = مفيش Auth، عن قصد):

Terminal

```
user@MacBook ~ % npx -y @redocly/cli@latest lint openapi.yaml
validating openapi.yaml...
openapi.yaml: validated in 44ms

Woohoo! Your API description is valid. 🎉
```

redocly.yaml

```
extends:
  - recommended
rules:
  no-server-example.com: off   # our lab server really is on localhost
```

المصدر Redocly · recommended rules Redocly CLI · lint

21.6 ملف واحد، حاجات كتير

Terminal

```
(صفحة واحدة تفتحها في المتصفح) Docs HTML #
user@MacBook ~ % npx -y @redocly/cli@latest build-docs openapi.yaml -o docs.html
🎉 bundled successfully in: docs.html (262 KiB) [⏱️ 96ms].

(فصل 11) Frontend لـ TypeScript types #
user@MacBook ~ % npx -y openapi-typescript@latest spec/openapi.yaml -o spec/schema.d.ts
✨ openapi-typescript 7.13.0
🚀 spec/openapi.yaml → spec/schema.d.ts [46.7ms]
```

اللي اتعمل لوحده schema.d.ts جزء من

```
Article: components["schemas"]["ArticleInput"] & {
    readonly id: string;
    /** Format: date-time */
    readonly createdAt: string;
    /** Format: date-time */
    readonly publishedAt?: string;
};
```

لاحظ الـ `allOf` بقى `&`، والـ `readOnly` بقى `readonly`. لما تشتغل React بـ TypeScript، الـ Editor هيكمّل أسماء الـ Fields لوحده ويقولك لو كتبت `createdat` غلط.

21.7 Contract test: مين بيكدب؟

الـ Spec وعد. هل السيرفر ملتزم بيه؟ لأن الـ Schemas دي JSON Schema، نقدر نستخدم Ajv (فصل 7) نفسه نقارن ردود حقيقية بيها:

contract.mjs (الجزء المهم)

```
const spec = parse(readFileSync("spec/openapi.yaml", "utf8"));
const ajv = new Ajv2020({ allErrors: true, strict: false });
addFormats(ajv);
ajv.addSchema({ $id: "spec", components: spec.components });
const check = (name) => ajv.getSchema(`spec#/components/schemas/${name}`);

await test("library.js list", "http://localhost:3001/v1/articles?limit=2", "ArticlePage");
await test("library.js one ", "http://localhost:3001/v1/articles/4", "Article");
await test("library.js 404 ", "http://localhost:3001/v1/articles/999", "Problem");
await test("old server.js  ", "http://localhost:3000/v1/articles/2", "Article");
```

Terminal

```
user@MacBook ~ % node contract.mjs
PASS  library.js list  (200 vs ArticlePage)
PASS  library.js one   (200 vs Article)
PASS  library.js 404   (404 vs Problem)
FAIL  old server.js    (200 vs Article)
      / must have required property 'authorId'
      / must have required property 'createdAt'
      /id must be string
```

السيرفر القديم بتاع فصل 13 (`server.js`) بيرجّع `"id": 2` Number ومفيش `authorId` ولا `createdAt`. الـ Contract test مسكه في ثانية. وده اللي بيتعمل في الـ CI: أي تغيير في الكود يكسر الوعد، الـ Build يقع قبل ما يوصل للناس.

لو حابب تتعمق

تقدر تعدّيه في أول قراية

أدوات بتعمل ده بشكل أكمل: Schemathesis (بيولّد طلبات من الـ Spec ويدوّر على Bugs)، و Prism (Mock server) من الـ Spec علشان الـ Frontend يشتغل قبل ما الـ Backend يخلص). اتأكد من دعمها لـ 3.2 قبل ما تستخدمها، أو اكتب الـ Spec 3.1.

ملخص الفصل

- الـ Spec = مصدر الحقيقة الوحيد. Design-first = تتفق قبل ما تكتب كود.
- `openapi` و `info` و `servers` و `paths` (Operations بـ operationId) و `components` و `$ref`.
- الـ Schemas في OpenAPI 3.1 = JSON Schema 2020-12: `allOf` و `readOnly` و `[string, "null"]`.
- Redocly lint طلّع 2 تحذير و 6 Errors (security)، وكلهم اتصلّحوا بسبب واضح.
- من نفس الملف: Docs HTML، و TypeScript types، و Contract test مسك السيرفر القديم.

صح ولا غلط؟

1. `openapi: 3.2.0` هي نسخة الـ API بتاعك. صح / غلط
2. `security: []` معناها إن الـ Auth اتنسى. صح / غلط
3. في JSON Pointer، الـ `/` اللي جوه اسم بتتكتب `~1`. صح / غلط

أسئلة مراجعة

1. إيه فايدة `operationId`؟
2. ليه `Article` معمول بـ `allOf` بدل ما نكتب كل الـ Fields تاني؟
3. إيه الفرق بين Lint و Contract test؟
4. Design-first ولا Code-first لفريق فيه Frontend مستني؟ وليه؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل — زوّد `example` لـ `Article` واعمل `build-docs` وشوفه في الصفحة.
2. متوسط — زوّد `GET /authors/{authorId}` للـ Spec بـ Schema جديد، وخلّي الـ Lint يعدّي.
3. صعب — زوّد `PUT` في الـ Spec، وزوّد على `contract.mjs` اختبار لرد الـ 422 مقابل `ValidationProblem`.

22

المرحلة 7 · الجسر والمشروع

الفصل التاني والعشرين

الجسر لـ Express: كل اللي فات في كود

كورس Maximilian هيعلّمك Node و Express بالتفصيل. الفصل ده مش بديل عنه، ده خريطة: كل مفهوم اتعلمته في الكتاب ده (Status codes، و Problem details، و Pagination، و ETag، و Idempotency، و Rate limits) بيتعمل فين في Express، بالإشارة لـ library.js سطر بسطر. ولما توصل للكورس، هتعرف كل سطر بيعمل إيه وليه.

بعد الفصل ده هتكون عارف

- الصورة الكبيرة: Request بيعدّي على سلسلة Middleware
- ترتيب library.js وليه الترتيب ده بالظبط
- req: params و query و body و get و is
- res: status و json و set و location و type و end
- Express 5: الـ async errors، و req.body undefined، والـ Path syntax
- الـ Error middleware بـ 4 Arguments
- خريطة: كل مفهوم في الكتاب ← فين في الكود
- إيه اللي ناقص علشان يبقى Production
- الكود كامل

22.1 الصورة الكبيرة

Express في الأساس حاجة واحدة: **Request بيعدّي على سلسلة Functions بالترتيب** (Middleware). كل واحدة يا إما ترد وتقفل، يا إما تنادي `next()` علشان اللي بعدها. والـ Route نفسه Middleware بيشتغل بس لو الـ Method والـ Path ماشيين.



شكل 22.1 ترتيب library.js. الترتيب هو المنطق: الـ Rate limiter قبل الـ Parser علشان الطلبات البايظة تتعد، والـ 404 بعد كل الـ Routes، والـ Error handler في الآخر خالص.

أصغر Express app

```
import express from "express";
const app = express();
app.use(express.json());                    // middleware: parse JSON bodies
app.get("/v1/ping", (req, res) => {         // route: method + path + handler
  res.json({ ok: true });                   // send JSON (status 200 by default)
});
app.listen(3001);
```

22.2 الـ req والـ res

الكود	بيعمل إيه	المفهوم في الكتاب
`req.params.id`	من الـ Path: `/v1/articles/:id`	Resources (فصل 14)
`req.query.status`	من الـ Query string، ودايماً String	Filtering (فصل 18)
`req.body`	الـ JSON بعد `express.json()`. من غيره = `undefined` في Express 5	Bodies (فصل 16)
`req.get("If-Match")`	أي Header (مش حساس للحروف)	ETag (فصل 19)
`req.is("application/json")`	هل الـ Content-Type كده؟	415 (فصل 16)

الكود	بيعمل إيه	المفهوم في الكتاب
`res.status(201)`	الـ Status code (وبيرجّع `res` علشان تكمّل)	فصل 16
`res.json(obj)`	`JSON.stringify` + `Content-Type` + يبعت	فصل 5
`res.type("application/problem+json")`	يغيّر الـ Content-Type	RFC 9457 (فصل 17)
`res.set("ETag", tag)`	يحط Header	فصل 19
`res.location(url)`	هيدر `Location`	201 (فصل 16)
`res.status(204).end()`	يرد من غير Body	DELETE (فصل 15)

جدول 22.1 اللي استخدمناه في library.js

> ⚠ **req.query دايماً Strings**
>
> `?limit=5` بتوصل `"5"` مش `5`. علشان كده في `library.js`: `Number(req.query.limit) || 10` و `Math.min(…, 50)`. وأي Input من برّه (Query، أو Body، أو Headers) **متثقش فيه**: Validate بـ zod أو Schema.

المصدر Express 5 · API reference — Express · Using middleware

22.3 Express 5: اللي اتغير

Express 5 نزل رسمي بعد سنين على 4، والنسخة دلوقتي 5.3.0 (9 أكتوبر 2026). أغلب الفيديوهات القديمة Express 4، فخلّي بالك من الفروق دي:

الحاجة	Express 4	Express 5
Error في `async` handler	لازم `try/catch` و `next(err)` بإيدك، وإلا الطلب بيعلّق	بيتمسك لوحده ويروح للـ Error handler (جربناه بـ `/v1/debug/crash`)
`req.body` من غير Parser	`{}`	`undefined` (علشان كده `req.body ?? {}` في `server.js`)
الـ Wildcard	`/*`	لازم اسم: `/*splat`
Optional param	`/:ext?`	`{/:ext}` بأقواس
حروف محجوزة	—	`( ) [ ] ? + ! \` محتاجة `\`. والـ `:` بتاعة `:publish` كمان (فصل 15)
`app.del()` و `req.param()`	موجودين	اتشالوا

الحاجة	Express 4	Express 5

جدول 22.2 Express 4 ← 5

المصدر Express · Migrating to Express 5 Express · Error handling

22.4 الـ Error middleware

JavaScript

```
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) {
    const id = crypto.randomUUID();
    console.error(id, err); // full details in the server log only
    return problem(res, 500, "Internal Server Error", { instance: `urn:uuid:${id}` });
  }
  // about:blank => title is the standard status phrase (RFC 9457 §4.2.1)
  const titles = { 400: "Bad Request", 413: "Content Too Large" };
  problem(res, status, titles[status] ?? "Bad Request", { detail: err.message });
});
```

تشريح الأمر

```
(err, req, res, next) ⇒ { ... }
```

1. `4 Arguments` دي الطريقة الوحيدة اللي Express يعرف بيها إن ده Error handler. حتى لو مش هتستخدم `next`، لازم تكتبه.
2. `err.status` الـ Errors اللي جاية من `express.json()` فيها `status` (400 للـ JSON البايظ، 413 للكبير). أي حاجة تانية = Bug = 500.
3. `≥ 500` الـ Log فيه كل حاجة، والـ Client ياخد ID بس (فصل 17).

22.5 الخريطة: الكتاب ← الكود

المفهوم	الفصل	فين في الكود
Resource names و IDs Strings	14	`/v1/articles/:id`، و `id: String(i)`
Custom method	14، 15	`"/v1/articles/:id\\:publish"`
PUT ضد PATCH، و Merge Patch	15	`app.put`، و `mergePatch()`، و `{ type: [..., "application/merge-patch+json"] })`
Idempotency-Key	15	`idem` و Map `hash` للـ Body
201 + Location، و 204، و 409، و 415، و 422	16	`res.status(201).location(…)` وغيرها

المفهوم	الفصل	فين في الكود
Problem details	17	`problem()` و `P + "validation"`
Validation بـ pointer	17	`ArticleIn` (zod) و `validate()`
Filter و Sort و Fields و Cursor و Link	18	`app.get("/v1/articles")`
Rate limit و 429	19	`app.use` أول
ETag و 304 و If-Match و 412	19	`etagOf()`، و GET و PATCH
Sunset و Deprecation	19	`app.get("/v1/posts")`
CORS	13	`server.js` : `cors({ origin })`
Raw body و Webhooks	20	`hook.js` : `express.raw()`

جدول 22.3 كل مفهوم فين في library.js

22.6 إيه اللي ناقص علشان Production؟

`library.js` معمول علشان تتعلم منه، مش علشان ينزل. ودي الحاجات اللي هتتعلمها في الكورس أو بعده:

1. **Database** بدل الـ Arrays (PostgreSQL أو MongoDB)، فالبيانات متضيعش مع الـ Restart.
2. **Auth**: JWT أو Sessions (فصل 12)، و 401 و 403 على كل Route.
3. **تقسيم الملفات**: `express.Router()` لكل Resource (`routes/articles.js`)، و Controllers، و Services.
4. **Idempotency و Rate limit** في Redis مش في الذاكرة، أو مكتبة زي `express-rate-limit`.
5. **Security headers**: `helmet`، و `app.disable("x-powered-by")` (عملناها)، و Limit لـ Body (الـ Default 100kb).
6. **Logging** منظم (JSON logs بـ Request ID)، و Health check endpoint.
7. **Tests**: Contract test زي `contract.mjs` (فصل 21) في الـ CI.
8. **Config** من Environment variables (`PORT`) و Secrets)، زي `LIMIT` و `PORT` اللي جربناهم.

المصدر Express · Security best practices · Express · Routing (express.Router)

22.7 الكود كامل: library.js

220 سطر، Express 5.3.0 و zod 4 على Node 24. كل رد في فصول 15 لـ 21 طالع منه. (اتعدّل خلال الفصول: الـ Problem types في 17، والـ Deprecation link في 19. والردود اللي في الكتاب من النسخة اللي كانت شغالة وقتها.)

lab/library.js

```
// Content Library API — lab server for chapters 15-22
import express from "express";
import crypto from "node:crypto";
import { z } from "zod";

const app = express();
app.disable("x-powered-by");
app.set("etag", false); // we set our own ETags below

// ---------- helpers ----------
function problem(res, status, title, extra = {}) {
  return res.status(status).type("application/problem+json").json({
    type: extra.type ?? "about:blank", title, status, ...extra,
  });
}
const etagOf = (obj) =>
  '"' + crypto.createHash("sha256").update(JSON.stringify(obj)).digest("base64url").slice(0,
16) + '"';

const P = "https://example.com/problems/"; // our problem type URIs

// ---------- data ----------
const articles = [];
for (let i = 1; i <= 25; i++) {
  articles.push({
    id: String(i),
    title: `Article ${i}`,
    status: i % 3 === 0 ? "draft" : "published",
    authorId: String((i % 4) + 1),
    tags: i % 2 ? ["api"] : ["json"],
    createdAt: new Date(Date.UTC(2026, 9, i, 9, 0, 0)).toISOString(),
  });
}
let nextId = 26;

// ---------- rate limit: fixed window per IP ----------
const LIMIT = Number(process.env.LIMIT ?? 100);
const WINDOW = 60; // seconds
const hits = new Map();
app.use((req, res, next) => {
  const now = Math.floor(Date.now() / 1000);
  let w = hits.get(req.ip);
  if (!w || now >= w.reset) w = { count: 0, reset: now + WINDOW };
  w.count++;
  hits.set(req.ip, w);
  const remaining = Math.max(0, LIMIT - w.count);
  res.set("RateLimit-Policy", `"default";q=${LIMIT};w=${WINDOW}`);
  res.set("RateLimit", `"default";r=${remaining};t=${w.reset - now}`);
  if (w.count > LIMIT) {
```

```
    res.set("Retry-After", String(w.reset - now));
    return problem(res, 429, "Too Many Requests", {
      detail: `Limit is ${LIMIT} requests per ${WINDOW} seconds`,
    });
  }
  next();
});

// body parser AFTER the rate limiter, so even bad JSON is counted
app.use(express.json({ type: ["application/json", "application/merge-patch+json"] }));

// ---------- validation ----------
const ArticleIn = z.object({
  title: z.string().min(3),
  status: z.enum(["draft", "review", "published"]).default("draft"),
  authorId: z.string(),
  tags: z.array(z.string()).default([]),
});
function validate(body, res) {
  const r = ArticleIn.safeParse(body);
  if (r.success) return r.data;
  problem(res, 422, "Validation failed", {
    type: P + "validation",
    errors: r.error.issues.map((i) => ({ pointer: "/" + i.path.join("/"), detail: i.message
})),
  });
  return null;
}

// ---------- list: filter, sort, cursor pagination, fields ----------
app.get("/v1/articles", (req, res) => {
  const { status, authorId, sort = "id", cursor, fields } = req.query;
  const limit = Math.min(Number(req.query.limit) || 10, 50);
  let list = articles.filter(
    (a) => (!status || a.status === status) && (!authorId || a.authorId === authorId));
  const desc = sort.startsWith("-");
  const key = desc ? sort.slice(1) : sort;
  if (!["id", "createdAt", "title"].includes(key))
    return problem(res, 400, "Invalid sort field", { type: P + "invalid-sort", detail: `Cannot
sort by '${key}'` });
  const val = (a) => (key === "id" ? Number(a.id) : a[key]);
  list = [...list].sort((a, b) => (val(a) > val(b) ? 1 : -1) * (desc ? -1 : 1));
  let start = 0;
  if (cursor) {
    const lastId = Buffer.from(cursor, "base64url").toString();
    start = list.findIndex((a) => a.id === lastId) + 1;
  }
  const page = list.slice(start, start + limit);
  const hasMore = start + limit < list.length;
  const nextCursor = hasMore ? Buffer.from(page.at(-1).id).toString("base64url") : null;
  if (nextCursor) {
```

```
    const u = new URL(req.originalUrl, "http://localhost:3001");
    u.searchParams.set("cursor", nextCursor);
    res.set("Link", `<${u.pathname}${u.search}>; rel="next"`);
  }
  const pick = fields ? fields.split(",") : null;
  const data = pick ? page.map((a) => Object.fromEntries(pick.map((f) => [f, a[f]]))) : page;
  res.json({ data, nextCursor });
});

// ---------- read one: ETag + 304 ----------
app.get("/v1/articles/:id", (req, res) => {
  const a = articles.find((x) => x.id === req.params.id);
  if (!a) return problem(res, 404, "Article not found", { type: P + "article-not-found",
detail: `No article with id '${req.params.id}'` });
  const tag = etagOf(a);
  res.set("ETag", tag);
  if (req.get("If-None-Match") === tag) return res.status(304).end();
  res.json(a);
});

// ---------- create: Idempotency-Key ----------
const idem = new Map(); // key -> { hash, status, body, location }
app.post("/v1/articles", (req, res) => {
  if (!req.is("application/json"))
    return problem(res, 415, "Unsupported Media Type", { detail: "Use Content-Type:
application/json" });
  const key = req.get("Idempotency-Key");
  const hash = crypto.createHash("sha256").update(JSON.stringify(req.body)).digest("hex");
  if (key && idem.has(key)) {
    const saved = idem.get(key);
    if (saved.hash !== hash)
      return problem(res, 422, "Idempotency-Key reused", { type: P + "idempotency-key-reused",
detail: "Same key was used with a different body" });
    res.set("Idempotent-Replayed", "true");
    return res.status(saved.status).location(saved.location).json(saved.body);
  }
  const data = validate(req.body, res);
  if (!data) return;
  const a = { id: String(nextId++), ...data, createdAt: new Date().toISOString() };
  articles.push(a);
  const location = `/v1/articles/${a.id}`;
  if (key) idem.set(key, { hash, status: 201, body: a, location });
  res.status(201).location(location).json(a);
});

// ---------- full replace: PUT ----------
app.put("/v1/articles/:id", (req, res) => {
  const i = articles.findIndex((x) => x.id === req.params.id);
  if (i === -1) return problem(res, 404, "Article not found", { type: P + "article-not-found"
});
  const data = validate(req.body, res);
```

```
  if (!data) return;
  articles[i] = { id: articles[i].id, ...data, createdAt: articles[i].createdAt };
  res.json(articles[i]);
});

// ---------- partial update: JSON Merge Patch + If-Match ----------
function mergePatch(target, patch) {
  if (patch === null || typeof patch !== "object" || Array.isArray(patch)) return patch;
  const out = { ...target };
  for (const [k, v] of Object.entries(patch)) {
    if (v === null) delete out[k];
    else out[k] = mergePatch(out[k], v);
  }
  return out;
}
app.patch("/v1/articles/:id", (req, res) => {
  const i = articles.findIndex((x) => x.id === req.params.id);
  if (i === -1) return problem(res, 404, "Article not found", { type: P + "article-not-found"
});
  const ifMatch = req.get("If-Match");
  if (ifMatch && ifMatch !== etagOf(articles[i]))
    return problem(res, 412, "Precondition Failed", { detail: "The article changed since you
read it" });
  const { id, createdAt, ...rest } = articles[i];
  const data = validate(mergePatch(rest, req.body), res);
  if (!data) return;
  articles[i] = { id, ...data, createdAt };
  res.set("ETag", etagOf(articles[i])).json(articles[i]);
});

// ---------- custom method: publish ----------
app.post("/v1/articles/:id\\:publish", (req, res) => {
  const a = articles.find((x) => x.id === req.params.id);
  if (!a) return problem(res, 404, "Article not found", { type: P + "article-not-found" });
  if (a.status === "published")
    return problem(res, 409, "Already published", { type: P + "already-published", detail:
`Article '${a.id}' is already published` });
  a.status = "published";
  a.publishedAt = new Date().toISOString();
  res.json(a);
});

// ---------- delete ----------
app.delete("/v1/articles/:id", (req, res) => {
  const i = articles.findIndex((x) => x.id === req.params.id);
  if (i !== -1) articles.splice(i, 1);
  res.status(204).end();
});

// ---------- deprecated alias ----------
app.get("/v1/posts", (req, res) => {
```

```
  res.set("Deprecation", "@1790812800"); // 2026-10-01T00:00:00Z
  res.set("Sunset", "Fri, 01 Jan 2027 00:00:00 GMT");
  res.set("Link", '</v1/articles>; rel="successor-version", ' +
    '<https://example.com/docs/deprecations/posts>; rel="deprecation"; type="text/html"');
  res.json({ data: articles.slice(0, 2) });
});

// ---------- a route that crashes (to see the 500 handler) ----------
app.get("/v1/debug/crash", async () => {
  throw new Error("DB password=hunter2 rejected"); // secret must NOT reach the client
});

// ---------- 404 + errors ----------
app.use((req, res) => problem(res, 404, "Not Found", { detail: `No route for ${req.method}
${req.path}` }));
app.use((err, req, res, next) => {
  const status = err.status ?? 500;
  if (status >= 500) {
    const id = crypto.randomUUID();
    console.error(id, err); // full details in the server log only
    return problem(res, 500, "Internal Server Error", { instance: `urn:uuid:${id}` });
  }
  // about:blank => title is the standard status phrase (RFC 9457 §4.2.1)
  const titles = { 400: "Bad Request", 413: "Content Too Large" };
  problem(res, status, titles[status] ?? "Bad Request", { detail: err.message });
});

const PORT = Number(process.env.PORT ?? 3001);
app.listen(PORT, () => console.log(`Library API on http://localhost:${PORT}`));
```

ملخص الفصل

- Express = سلسلة Middleware بالترتيب، والترتيب جزء من المنطق.
- `req.params` و`req.query` (Strings) و`req.body` و`req.get` و`res.status().json()` و`res.set` و`res.location`.
- Express 5: الـ async errors بتتمسك لوحدها، و`req.body` ممكن يبقى `undefined`، و Paths أدق.
- Error handler = 4 Arguments، وفي الآخر خالص.
- كل مفهوم في الكتاب ليه سطر في `library.js`، والناقص للـ Production: DB و Auth و Router و Redis و Security و Tests.

صح ولا غلط؟

1. `req.query.limit` بييجي Number لو بعت `?limit=5`. صح / غلط
2. الـ Error handler في Express ممكن يبقى بـ 3 Arguments. صح / غلط
3. في Express 5، Error في `async` handler بيروح للـ Error handler لوحده. صح / غلط

؟ أسئلة مراجعة

1. ليه الـ handler 404 لازم يبقى بعد كل الـ Routes؟
2. إيه اللي هيحصل لو حطيت `app.use(express.json())` قبل Webhook route؟
3. ليه `library.js` قفل الـ ETag الأوتوماتيك بتاع Express؟
4. اختار 3 حاجات من قايمة الـ Production ورتّبها حسب الأهمية.

⚑ تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `library.js` وجرّب كل الـ Endpoints بـ HTTPie (فصل 8).
2. متوسط انقل الـ Routes بتاعة المقالات لـ `express.Router()` في ملف لوحده.
3. صعب زوّد `GET /v1/authors` و `GET /v1/authors/:id` بنفس المعايير (Problem، و Pagination، و ETag)، وحدّث الـ Spec، وخلّي `contract.mjs` يختبرهم.

23

المرحلة 7 · الجسر والمشروع

الفصل التالت والعشرين

المشروع 1: تستكشف API حقيقي من الأول للآخر

وقت التطبيق كمستخدم. هتاخد API حقيقي متعرفهوش وتمشي عليه بخطوات ثابتة: الـ Docs والـ Spec، وأول طلب، والهيدرز، والـ Auth، والـ Pagination، والأخطاء، والحدود، وبعدين تكتب Script صغير بيعمل حاجة مفيدة، وتسلّم ملف ملاحظات. عملناه مع بعض على GitHub REST API (Repo بتاع Express نفسه)، وبعدين دورك على API تختاره.

بعد الفصل ده هتكون عارف

- خطة استكشاف من 8 خطوات تنفع مع أي API
- GitHub: الـ Docs والـ OpenAPI description الرسمي
- أول طلب وقراية كل Header مهم
- Rate limits مختلفة لكل Resource: core و search
- Pagination بـ Link و rel=last
- مفاجأة: Issues endpoint بيرجّع PRs
- Script كامل: report.mjs بـ timeout و Error handling
- ملف الملاحظات اللي هتسلّمه
- APIs بديلة تعمل عليها المشروع

23.1 الخطة: 8 خطوات

1. **الـ Docs والـ Spec**: Base URL، والـ Auth، والـ Versioning، وفي OpenAPI ولا لأ (فصل 11).
2. **أول طلب** بـ `curl -i` أو HTTPie: اقرا الـ Status والهيدرز قبل الـ Body (فصل 3 و 8).
3. **شكل البيانات** بـ jq: إيه الـ Fields المهمة، وأسلوب الأسماء، والتواريخ، والـ null (فصل 6 و 16).
4. **الـ Auth**: من غيره بيحصل إيه، ومعاه بيتغير إيه (فصل 12).
5. **الـ Pagination والفلترة**: Offset ولا Cursor، وفين اللينك (فصل 18).
6. **الأخطاء**: ابعت طلبات غلط عن قصد واقرا الشكل (فصل 16 و 17).
7. **الحدود**: Rate limit، و Versioning، و Deprecation (فصل 19).
8. **Script** صغير بيعمل حاجة مفيدة، بـ Error handling (فصل 10)، و**ملف ملاحظات**.

23.2 الخطوة 1: الـ Docs والـ Spec

GitHub REST API: الـ Base URL `https://api.github.com`، و Versioning بالهيدر (فصل 19)، و Auth بـ Bearer token (فصل 12)، ومن غير توكن 60 طلب في الساعة. وعندهم OpenAPI description رسمي ومفتوح في Repo اسمه `github/rest-api-description`. ده ملف ضخم، بس تقدر تدوّر فيه بـ jq عن أي Endpoint.

المصدر `GitHub · REST API docs` `github/rest-api-description (OpenAPI)`

23.3 الخطوات 2 و 3: أول طلب وشكل البيانات

Terminal

```
user@MacBook ~ % curl -s -D h1.txt 'https://api.github.com/repos/expressjs/express/releases?
per_page=3' \
    -H 'User-Agent: apis-book' | jq -r '.[] | "\(.tag_name)\t\(.published_at)\t\
(.author.login)"'
v5.3.0  2026-10-09T12:50:31Z    UlisesGascon
v4.22.2 2026-05-11T18:54:02Z    jonchurch
v5.2.1  2025-12-01T20:54:44Z    jonchurch

user@MacBook ~ % grep -iE '^(link|x-ratelimit-remaining|etag)' h1.txt
etag: W/"d84dfca22a26a66490a5146176ee8aaed7d2218078b40647e66020153695bcb3"
link: <https://api.github.com/repositories/237159/releases?per_page=3&page=2>; rel="next",
      <https://api.github.com/repositories/237159/releases?per_page=3&page=56>; rel="last"
x-ratelimit-remaining: 56
```

تشريح الأمر

```
-D h1.txt   \(.tag_name)\t   rel="last"   page=56
```

1. `-D h1.txt` — احفظ الهيدرز في ملف، والـ Body يروح لـ jq. كده عندك الاتنين من طلب واحد (والـ Rate limit غالي).
2. `\(…)` — String interpolation في jq، و`\t` Tab، و`-r` من غير علامات تنصيص (فصل 6).
3. **ملاحظة 1** — الترتيب مش بالنسخة: v4.22.2 نزلت بعد v5.2.1. الـ Releases مترتبة بالتاريخ، و Express بيصلّح الـ x.4 لسه.
4. **ملاحظة 2** — `rel="last"` بـ `page=56` و`per_page=3`: يعني حوالي 166 لـ 168 Release. عرفنا العدد من غير ما نجيبهم كلهم.
5. **ملاحظة 3** — التواريخ RFC 3339 بالـ UTC (`Z`)، والأسماء snake_case.

23.4 الخطوة 7 بدري: الحدود مختلفة

Terminal

```
user@MacBook ~ % curl -s 'https://api.github.com/search/issues?
q=repo:expressjs/express+is:issue+is:open&per_page=1' \
    -H 'User-Agent: apis-book' | jq '{total_count, incomplete_results}'
{
  "total_count": 112,
  "incomplete_results": false
}
user@MacBook ~ % curl -s -D h2.txt -o /dev/null 'https://api.github.com/search/issues?
q=repo:expressjs/express' -H 'User-Agent: apis-book'
user@MacBook ~ % grep -i '^x-ratelimit' h2.txt
x-ratelimit-limit: 10
```

```
x-ratelimit-remaining: 9
x-ratelimit-used: 1
x-ratelimit-resource: search
x-ratelimit-reset: 1792756774
```

الـ **Search** ليه عدّاد لوحده (`resource: search`): 10 في **الدقيقة** من غير توكن، مش 60 في الساعة زي `core`. والـ Docs بتأكد ده. ده من نوع الحاجات اللي **متعرفهاش غير لما تبص على الهيدرز**.

23.5 مفاجأة: الـ Issues فيها PRs

طلبنا `/repos/expressjs/express/issues?state=open&per_page=50` وجالنا 50. بس الـ Search قال إن الـ Issues المفتوحة 112، فإزاي أول 50 فيهم بس 11 Issue؟ رجعنا للـ Docs:

> **من Docs بتاعة GitHub**
>
> "GitHub's REST API considers every pull request an issue, but not every issue is a pull request." يعني الـ Endpoint ده بيرجّع الاتنين، والـ PR بيتعرف بإن فيه `pull_request` key. وده درس تصميم كمان: اسم الـ Endpoint بيوعد بحاجة، والرد فيه حاجة تانية، فلازم يتكتب بوضوح في الـ Docs.

23.6 الخطوة 8: الـ Script

report.mjs

```
const [owner = "expressjs", repo = "express"] = process.argv.slice(2);
const API = "https://api.github.com";
const headers = {
  "User-Agent": "apis-book-report",
  Accept: "application/vnd.github+json",
  "X-GitHub-Api-Version": "2026-03-10",
  ...(process.env.GITHUB_TOKEN && { Authorization: `Bearer ${process.env.GITHUB_TOKEN}` }),
};

let lastRemaining = null;
async function gh(path) {
  const res = await fetch(API + path, { headers, signal: AbortSignal.timeout(10_000) });
  lastRemaining = res.headers.get("x-ratelimit-remaining");
  if (!res.ok) {
    const body = await res.json().catch(() => ({}));
    throw new Error(`${res.status} ${body.message ?? res.statusText} (${path})`);
  }
  return res.json();
}

let r;
try {
  r = await gh(`/repos/${owner}/${repo}`);
} catch (err) {
```

```
  console.error(`Cannot read ${owner}/${repo}: ${err.message}`);
  process.exit(1);
}
const releases = await gh(`/repos/${owner}/${repo}/releases?per_page=3`);
const issues = await gh(`/repos/${owner}/${repo}/issues?state=open&per_page=50`);
const onlyIssues = issues.filter((i) => !i.pull_request); // this endpoint also returns PRs
const labels = {};
for (const i of onlyIssues) for (const l of i.labels) labels[l.name] = (labels[l.name] ?? 0) +
1;
// ... console.log the report (see the output)
```

تشريح الأمر

```
...(process.env.GITHUB_TOKEN && { Authorization: ... })
```

1. `...( && {})` لو في توكن في الـ Environment، زوّد الهيدر. لو مفيش، `...false` مبيزوّدش حاجة. التوكن عمره ما يتكتب في الكود.
2. `X-GitHub-Api-Version` ثبّتنا النسخة علشان الـ Script ميتغيرش سلوكه لوحده (فصل 19).
3. `AbortSignal.timeout` متستناش للأبد (فصل 10).
4. `res.json().catch` لو الـ Error مش JSON، متقعش وانت بتقرا الـ Error.

Terminal

```
user@MacBook ~ % node report.mjs
expressjs/express #
stars 69577 · forks 25247 · license MIT
last push 2026-10-23T11:56:47Z
latest releases:
  v5.3.0   2026-10-09
  v4.22.2  2026-05-11
  v5.2.1   2025-12-01
open items in first page: 50 (issues 11, PRs 39)
top labels: [ [ 'bug', 1 ], [ '4.x', 1 ], [ '5.x', 1 ] ]
rate limit remaining: 53

user@MacBook ~ % node report.mjs expressjs nope-404
Cannot read expressjs/nope-404: 404 Not Found (/repos/expressjs/nope-404)
user@MacBook ~ % echo $?
1
```

بالتوكن على الـ Mac

اعمل Fine-grained token من GitHub Settings (Read-only) على Public repos (يكفي)، وشغّل: `GITHUB_TOKEN=github_pat_… node report.mjs`. الحد هيبقى 5000 في الساعة. ولو هتستخدمه كتير، حطه في Keychain أو ملف `.env` مش في الـ `.zshrc` اللي ممكن يترفع بالغلط.

23.7 ملف الملاحظات اللي هتسلّمه

notes.md (القالب)

```
API: GitHub REST          Date tested: 2026-10-23
Base URL: https://api.github.com
Docs / Spec: docs.github.com/rest · github/rest-api-description (OpenAPI)
Auth: none (60/h core) or Bearer token (5000/h). No User-Agent => 403.
Versioning: X-GitHub-Api-Version header; default 2022-11-28; bad value => 400
Data: snake_case, RFC 3339 UTC dates, numeric id + string node_id
Pagination: Link header (next/last); per_page; issues use cursors too
Errors: {"message","documentation_url","status"(string)} - not RFC 9457
Limits: core 60/h, search 10/min (unauth); headers x-ratelimit-*
Surprises: /issues returns PRs too (check pull_request key)
Script: report.mjs - repo stats, last releases, open issues vs PRs
Design: what I like / what I would do differently
```

23.8 دورك

اعمل نفس الـ 8 خطوات على API تاني (من جدول فصل 9)، وسلّم `notes.md` و Script:

الـ API	هيعلّمك إيه	فكرة Script
PokeAPI	Offset pagination، وعلاقات بلينكات	كل monéPok نوعه Fire وأقوى 5 فيهم
Open-Meteo	Query params كتير، و Units، وأخطاء 400 واضحة	تقرير طقس القاهرة لـ 7 أيام بـ jq أو Node
Frankfurter v2	OpenAPI 3.1 جاهز، و 422	سعر الجنيه مقابل 3 عملات آخر شهر
DummyJSON	Auth بـ JWT و Refresh	Login ثم `/auth/me` ثم Refresh لما يخلص
(GraphQL + REST) Rick and Morty	نفس البيانات بطريقتين	نفس السؤال بـ REST وبـ GraphQL وقارن عدد الطلبات

جدول 23.1 اقتراحات

ملخص الفصل

- 8 خطوات ثابتة: Docs، وأول طلب، والشكل، والـ Auth، والـ Pagination، والأخطاء، والحدود، و Script وملاحظات.
- احفظ الهيدرز مع الـ Body (`-D`) لأن الـ Rate limit غالي.
- الحدود ممكن تختلف لكل Resource (core و search).
- الـ Docs بتشرح المفاجآت (Issues فيها PRs)، فارجعلها لما الأرقام متركبش.
- Script بـ Version ثابت، و Timeout، و Token من الـ Environment، و Exit code صح.

صح ولا غلط؟

1. الـ Search في GitHub بيشارك نفس عدّاد الـ 60 في الساعة. صح / غلط
2. كل عنصر في `/issues` عند GitHub هو Issue مش PR. صح / غلط
3. تثبيت `X-GitHub-Api-Version` في الـ Script فكرة كويسة. صح / غلط

أسئلة مراجعة

1. إزاي عرفنا عدد الـ Releases التقريبي من غير ما نجيبهم؟
2. ليه الـ Script بيطلع بـ Exit code 1 لما يفشل؟
3. إيه اللي في Errors بتاعة GitHub مختلف عن RFC 9457؟
4. ليه بنقرا الـ Token من `process.env`؟

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل شغّل `report.mjs` على Repo تاني (`facebook react`) وقارن.
2. متوسط خلّي الـ Script يمشي على كل صفحات الـ Issues المفتوحة بالـ Link header (فصل 18) ويعدّ الـ Issues الحقيقية بس.
3. صعب زوّد الـ Cache بالـ ETag: احفظ الرد والـ ETag في ملف، وابعت `If-None-Match` المرة الجاية، واطبع "from cache" لو جالك 304.

24

المرحلة 7 · الجسر والمشروع

الفصل الرابع والعشرين

المشروع 2: تصمم Content Library API

آخر خطوة: تلبس طاقية الـ Provider كاملة. هتصمم API لمكتبة محتوى، من قرارات الـ Resources والأدوار لحد Spec بـ OpenAPI 3.2 فيه Auth و Webhook، وتعدّيه من الـ Lint، وتطلّع منه Docs. عملنا نسخة كاملة كمثال (317 سطر، 10 Operations، Webhook واحد)، وفي الآخر Checklist بـ 30 نقطة تراجع بيها أي API تصممه بعد كده.

بعد الفصل ده هتكون عارف

- الـ Brief: مين بيستخدم الـ API وعايز يعمل إيه
- الأدوار: viewer و author و editor وصلاحيات كل واحد
- جدول الـ Endpoints بالـ Methods والأكواد
- القرارات الصعبة ولماذا: IDs و PATCH و If-Match إجباري
- Webhook في OpenAPI: article.published
- Lint: 4 تحذيرات وتصليحها، و Docs HTML
- Checklist من 30 نقطة لأي API
- طريقة التسليم والتقييم

24.1 الـ Brief

فريق محتوى (كتّاب ومحررين) بيكتب مقالات. محتاجين API واحد يخدم 3 Clients: لوحة تحرير بـ React، وموقع عام بيعرض المنشور بس، وأداة خارجية (Slack bot مثلاً) عايزة تعرف لما مقالة تتنشر. ده بالظبط نوع الشغل اللي هتعمله بعد Maximilian و React.

الدور	يقدر	ميقدرش
viewer	يقرا المنشور، ويعلّق	يكتب مقالات
author	يعمل Draft، ويعدّل مقالاته هو	ينشر، أو يعدّل مقالات غيره، أو يمسح
editor	كل حاجة: ينشر، ويمسح، ويمسح أي تعليق	—

جدول 24.1 الأدوار (Authorization، فصل 12)

والدور جاي من الـ JWT (Bearer). مستخدم مش مسجل = **401**. author بيحاول ينشر = **403**. author بيحاول يفتح Draft بتاع حد تاني = **404** (متكشفش إنه موجود، فصل 16).

24.2 الـ Endpoints

Operation	Request	نجاح
listArticles	`GET /articles?status&authorId&tag&sort&limit&cursor`	200 + `{data, nextCursor}`
createArticle	`POST /articles` + `Idempotency-Key`	201 + Location
getArticle	`GET /articles/{id}`	200 + ETag (أو 304)
updateArticle	`PATCH /articles/{id}` + Merge Patch + `If-Match` **إجباري**	200 + ETag جديد
deleteArticle	`DELETE /articles/{id}`	204
publishArticle	`POST /articles/{id}:publish`	200
listComments	`GET /articles/{id}/comments`	200
createComment	`POST /articles/{id}/comments`	201
deleteComment	`DELETE /comments/{id}`	204
getMe	`GET /me`	200
Webhook	`POST` لـ URL المشترك: `article.published`	2xx خلال 10 ثواني

جدول 24.2 Content Library API v1

24.3 القرارات الصعبة

○ تشريح الأمر

```
Design decisions — Content Library API v1
```

1. **IDs** Strings معتمة بـ Prefix (`art_01JB8Z3K` زي Stripe): مش بتتخمن، والنوع باين من الشكل (فصل 14).
2. **`If-Match` إجباري** المحررين بيشتغلوا على نفس المقالة، فالـ Lost update متوقع (فصل 19). من غير الهيدر = **428**، و ETag قديم = **412**.
3. **`PATCH` مش `PUT`** اللوحة بتبعت الـ Fields اللي اتغيرت بس، ومفيش خطر مسح بيانات (فصل 15). و `summary: null` = امسح الملخص.

4	`Custom method النشر`	بيعمل Validation خاص (لازم `body`)، ويحط `publishedAt`، ويبعت Webhook. أكبر من `PATCH status`.
5	`Cursor`	الموقع العام Infinite scroll، والمقالات بتتنشر طول الوقت، فال Offset هيكرر (فصل 18).
6	`Envelope للقوايم`	`{data, nextCursor}` علشان نزوّد بعدين من غير كسر.
7	`Errors`	RFC 9457 و `errors[]` بـ `pointer`، و `type` لكل مشكلة متوثّق.
8	`Versioning`	`/v1` في الـ URL (بسيط للفريق)، و Deprecation/Sunset قبل أي قفل بـ 12 شهر على الأقل.
9	`camelCase`	كل الـ Clients JavaScript.

24.4 الـ Spec

الملف الكامل `lab/spec/design.yaml`. أهم جزء جديد مقارنة بفصل 21: الـ Auth، والـ Webhook (OpenAPI من 3.1 فيه `webhooks` على أعلى مستوى، جنب `paths`):

design.yaml (الـ Auth والـ Webhook)

```yaml
security:
  - bearerAuth: []
...
webhooks:
  articlePublished:
    post:
      operationId: onArticlePublished
      summary: Sent to subscribers when an article is published
      parameters:
        - name: X-Library-Signature
          in: header
          required: true
          description: "sha256=<hex HMAC of the raw body with your webhook secret>"
          schema: { type: string }
        - name: X-Library-Delivery
          in: header
          required: true
          description: Unique delivery ID. Use it to ignore duplicates.
          schema: { type: string }
      requestBody:
        content:
          application/json:
            schema:
              type: object
              required: [event, article]
              properties:
                event: { type: string, const: article.published }
                article: { $ref: "#/components/schemas/Article" }
      responses:
        "2XX": { description: Received. Answer within 10 seconds. }
components:
```

```
  securitySchemes:
    bearerAuth:
      type: http
      scheme: bearer
      bearerFormat: JWT
      description: "Roles in the token: viewer, author, editor"
```

تشريح الأمر

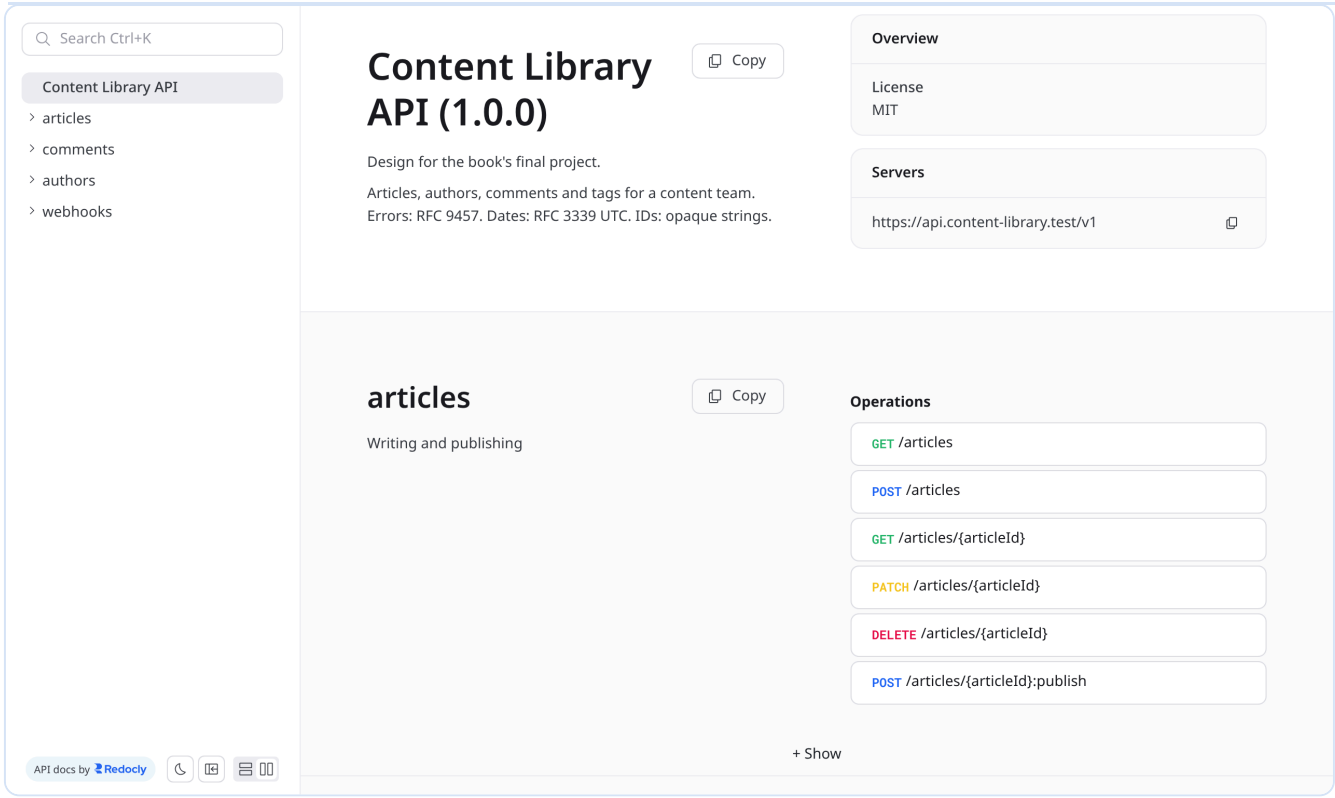
```
webhooks / "2XX" / const / securitySchemes
```

1. `webhooks` نفس شكل الـ Operation، بس انت اللي بتبعت والـ Client هو السيرفر (فصل 20).
2. `"2XX"` أي كود في الـ s200. الـ OpenAPI بيسمح بالـ Ranges دي.
3. `const` JSON Schema: القيمة دي بالظبط.
4. `Root على الـ security` كل Operation محتاج Bearer إلا لو قال غير كده.

Terminal

```
user@MacBook ~ % npx -y @redocly/cli@latest lint design.yaml
[1] design.yaml:2:1 at #/info
Info object should contain `license` field.
[2] design.yaml:14:5 at #/tags/0/description
Tag object should contain `description` field.
...
You have 4 warnings.

# بعد ما زوّدنا license و description لكل Tag:
user@MacBook ~ % npx -y @redocly/cli@latest lint design.yaml
Woohoo! Your API description is valid. 🎉
user@MacBook ~ % npx -y @redocly/cli@latest stats design.yaml
📈 Schemas: 9
🔀 Path Items: 6
🎣 Webhooks: 1
👷 Operations: 10
user@MacBook ~ % npx -y @redocly/cli@latest build-docs design.yaml -o design.html
🎉 bundled successfully in: design.html (268 KiB) [⏱️ 120ms].
```



شكل 24.1 الـ Docs اللي اتعملت من design.yaml (لقطة حقيقية من design.html في المتصفح). كل Operation وكل Webhook ليه صفحة لوحده.

24.5 الـ Checklist: 30 نقطة لأي API

#	المحور	السؤال	فصل
1	Resources	أسماء جمع، من غير أفعال، بحروف صغيرة وشرطة؟	14
2	Resources	Nesting مستوى واحد بس؟	14
3	Resources	العمليات اللي مش CRUD واضحة (`:publish`)؟	14
4	Resources	IDs Strings ومش بتتخمن؟	14
5	Methods	GET مبيغيرش حاجة أبداً؟	15
6	Methods	PATCH للتعديل، و Merge Patch موثّق؟	15
7	Methods	POST الحساس بيقبل Idempotency-Key؟	15
8	Status	201 + Location، و 204 من غير Body؟	16
9	Status	400 و 422 متفرقين بقاعدة ثابتة؟	16
10	Status	401 و 403 صح، و 404 لما متكشفش؟	16
11	Status	مفيش 200 فيها Error؟	16
12	Body	أسلوب أسماء واحد؟	16

#	المحور	السؤال	فصل
13	Body	تواريخ RFC 3339 UTC، وفلوس Integer + عملة؟	16
14	Body	null ضد مش موجود ليهم معنى موثّق؟	16
15	Body	Envelope للقوايم؟	16
16	Errors	problem+json بـ type ثابت؟	17
17	Errors	Validation بكل الأخطاء مرة واحدة بـ pointer؟	17
18	Errors	الـ 500 من غير أسرار، وبـ ID؟	17
19	Lists	Pagination من أول يوم بـ Default و Max؟	18
20	Lists	Cursor لو البيانات بتتغير؟ ومعتم؟	18
21	Lists	Sort و Filter على Fields محددة بس؟	18
22	Evolve	Versioning واضح، وقايمة Breaking changes؟	19
23	Evolve	خطة Deprecation و Sunset؟	19
24	Limits	Rate limit معلن في هيدرز، و 429 + Retry-After؟	19
25	Concurrency	ETag و If-Match للتعديل المتزامن؟	19
26	Auth	Bearer على HTTPS، والأدوار واضحة لكل Operation؟	12، 13
27	Auth	CORS بقايمة Origins مش `*` مع Credentials؟	13
28	Webhooks	توقيع HMAC، و Delivery ID، وسياسة إعادة؟	20
29	Docs	OpenAPI بيعدّي الـ Lint، وفيه Examples؟	21
30	Docs	Contract test بيتأكد إن السيرفر ملتزم؟	21

جدول 24.3 راجع بيها أي تصميم (بتاعك أو بتاع غيرك)

24.6 التسليم

1. `design.yaml` بيعدّي `redocly lint` من غير Errors.
2. `decisions.md`: كل قرار صعب وسببه (زي الجدول فوق) بكلامك انت.
3. `design.html` من `build-docs`.
4. Checklist الـ 30 نقطة: ✓ أو ✗ مع سبب لكل ✗.
5. **Bonus**: نفّذ 3 Operations في Express (ابدأ من `library.js`) واعمل لهم Contract test.

الخطوة الجاية

لما تخلص كورس Maximilian، ارجع للمشروع ده ونفّذه كامل بـ Database و Auth حقيقي. ولما تبدأ React، اعمل لوحة التحرير على الـ API بتاعك بالـ Types اللي `openapi-typescript` بيطلّعها. كده هتكون عملت Full stack من التصميم للواجهة.

ملخص الفصل

- ابدأ من الـ Clients والأدوار، مش من الـ Database.
- جدول Endpoints فيه الأكواد قبل أي كود.
- كل قرار صعب مكتوب بسببه: IDs، و If-Match إجباري، و PATCH، و Cursor، و Custom method.
- الـ Spec فيه Auth و Webhooks، وبيعدّي الـ Lint، وبيطلّع Docs.
- Checklist الـ 30 نقطة = الكتاب كله في صفحة.

صح ولا غلط؟

1. author بيحاول ينشر مقالة: الرد 401. صح / غلط
2. OpenAPI 3.1+ يقدر يوصف Webhooks. صح / غلط
3. PATCH من غير If-Match في التصميم ده يرجّع 428. صح / غلط

أسئلة مراجعة

1. ليه author بيشوف 404 مش 403 على Draft حد تاني؟
2. ليه النشر مش مجرد `PATCH {"status": "published"}` هنا؟
3. إيه اللي هيتغير في التصميم لو ضفنا تطبيق موبايل؟
4. اختار 3 نقاط من الـ Checklist شايفها الأهم واشرح ليه.

تحديات الفصل

جرّبها بإيدك، والحلول في الملحق

1. سهل زوّد Tags للـ Spec: `GET /tags` و `GET /tags/{slug}`.
2. متوسط زوّد Webhook تاني `comment.created` وعدّي الـ Lint.
3. صعب طبّق الـ Checklist على GitHub REST API (من ملاحظات مشروع 1): كام ✓ وكام ✗؟

ملحق أ

Cheat Sheet وجدول الـ Flags وجدول الرموز

ورقة مراجعة لأهم أوامر curl و HTTPie و jq وسطور fetch و Express والأدوات اللي في الكتاب، مقسّمة حسب الموضوع، وبعدها جدول الـ Flags وجدول لكل رمز ممكن تقابله في طلب أو رد أو Spec.

curl

curl	الطلبات
`curl -s URL | jq`	اطلب واقرا JSON
`curl -i URL`	الهيدرز + الـ Body
`curl -sv URL`	كل حاجة على stderr
`curl -s -D h.txt URL`	احفظ الهيدرز في ملف
`curl --json '{"a":1}' URL`	POST بـ JSON (والـ Headers)
`curl -X PATCH -H 'Content-Type: application/merge-patch+json' -d @p.json URL`	PATCH بنوع خاص
`curl -G URL --data-urlencode 'q=a b'`	Query مع Encoding
`curl --fail-with-body URL`	Body + Exit 22 لو 4xx/5xx
`curl -w '%{http_code}\n' -o /dev/null URL`	الـ Status بس
`curl -u user:pass URL`	Basic auth
`curl -H 'Authorization: Bearer $T' URL`	Bearer
`curl -H 'If-None-Match: "…"' URL`	(304) Conditional
`curl --data-binary @f.json URL`	ابعت الملف زي ما هو

HTTPie

HTTPie	HTTPie
`http URL`	GET ملوّن
`http POST URL title=Hi n:=5 ok:=true`	JSON values / String
`http URL q==term`	Query param
`http URL Header:value`	Header
`http --offline POST URL a=1`	شوف الطلب من غير ما يتبعت
`http -A bearer -a $T URL`	Bearer
`--ignore-stdin`	في Scripts (مش tty)

jq

jq	jq
`.a.b / .[0] / .[]`	المسارات
`.a? / .x // "default"`	أمان / بديل
`map(select(.done))`	فلترة
`{id, title}`	Object جديد
`length / keys / sort_by(.x) / group_by(.x)`	Functions
`-r / -c / -s / -n / -e`	Flags
`--arg k v / --argjson k 5`	متغيرات
`@csv / @tsv / @uri / @base64d`	تحويلات

fetch — JavaScript

```
const res = await fetch(url, { signal: AbortSignal.timeout(10_000) })
```
طلب بـ Timeout

```
if (!res.ok) throw ...
```
xx/5xx4 مش بترمي لوحدها

```
await res.json() / res.text()
```
Body مرة واحدة

```
res.headers.get("link")
```
Header

```
fetch(url, { method: "POST", headers: {"Content-Type": "application/json"}, body: JSON.stringify(o) })
```
POST

```
new URLSearchParams({ q, limit: 10 })
```
Query

```
JSON.stringify(o, null, 2)
```
JSON منسّق

```
JSON.parse(s, reviver)
```
Parse بتحويل

Express 5 — Express

```
app.use(express.json())
```
اقرا JSON

```
app.get("/v1/articles/:id", (req, res) => …)
```
Route

```
req.params / req.query / req.body / req.get(h)
```
الـ Input

```
res.status(201).location(u).json(o)
```
Location + 201

```
res.status(204).end()
```
من غير Body

```
res.type("application/problem+json")
```
Problem

```
app.use((err, req, res, next) => …)
```
(args 4) Error handler

```
express.raw({ type: "application/json" })
```
Raw body لـ Webhooks

```
"/v1/articles/:id\\:publish"
```
publish: حرفي

Specs & tools — أدوات

```
npx -y @redocly/cli@latest lint openapi.yaml
```
Lint

```
npx -y @redocly/cli@latest build-docs openapi.yaml -o docs.html
```
Docs

```
npx -y openapi-typescript@latest openapi.yaml -o schema.d.ts
```
Types

```
new Ajv2020({ allErrors: true })
```
Validation

```
openssl dgst -sha256 -hmac SECRET -hex < body
```
توقيع HMAC

```
date -r 1792758101
```
Unix time ← تاريخ (Mac)

1.أ جدول الـ Flags

نفس الحرف ممكن يبقى ليه معنى مختلف من أداة للتانية (`-s` في curl غير `-s` في jq). ولما تشك: `man curl` أو `jq --help` أو `http --help`.

الـ Flag	في	المعنى
`-s`	curl	اسكت (من غير Progress)
`-i`	curl	اطبع هيدرز الرد
`-I`	curl	HEAD بس
`-v`	curl	اطبع كل الحوار
`-D file`	curl	احفظ الهيدرز

الـ Flag	في	المعنى
`-o file`	curl	احفظ الـ Body
`-L`	curl	امشي ورا الـ Redirects
`-X`	curl	الـ Method
`-H`	curl	Header
`-d`	curl	Body (form، وبيشيل Newlines من @file)
`--json`	curl	Headers + Body JSON
`-G`	curl	حوّل -d لـ Query
`-f / --fail-with-body`	curl	Exit 22 على Errors
`-w`	curl	اطبع متغيرات (http_code)
`-u`	curl	Basic auth
`-r`	jq	نص خام
`-c`	jq	سطر واحد
`-s`	jq	اقرا كله كـ Array
`-n`	jq	من غير Input
`-e`	jq	Exit code حسب النتيجة
`--arg / --argjson`	jq	متغير JSON / String
`==`	HTTPie	Query param
`:=`	HTTPie	قيمة JSON (رقم/Boolean)
`:`	HTTPie	Header
`--offline`	HTTPie	اطبع الطلب بس

جدول أ.1 الـ Flags ومعانيها حسب الأداة

أ.2 جدول الرموز

الرمز	المعنى	مثال
`{ }`	Object في JSON / Block في JS	`{"a": 1}`
`[ ]`	Array	`[1, 2]`
`" "`	String في JSON (مفيش ')	`"title"`

الرمز	المعنى	مثال
`\`	Escape	`"a\"b"`
`?`	بداية الـ Query	`/articles?limit=5`
`&`	فاصل Query (وفي Shell: خلفية!)	`a=1&b=2`
`#`	JSON Pointer / Fragment في URI	`#/components/schemas`
`/`	فاصل المسار / JSON Pointer	`/title`
`~1 ~0`	/ و ~ جوه JSON Pointer	`#/paths/~1articles`
`:id`	Path param في Express	`/articles/:id`
`{id}`	Path param في OpenAPI	`/articles/{articleId}`
`:publish`	Custom method	`/articles/42:publish`
`-createdAt`	ترتيب تنازلي	`?sort=-createdAt`
`<…>; rel="next"`	Link header	`<url>; rel="next"`
`W/"…"`	Weak ETag	`W/"113-…"`
`@1790812800`	Structured field في Date	`Deprecation: @…`
`q= w= r= t=`	RateLimit params	`"default";r=0;t=60`
`sha256=`	بادئة التوقيع	`X-Hub-Signature-256`
`$ref`	مرجع في Schema/Spec	`{"$ref": "#/$defs/x"}`
`. | //`	jq: المسار / Pipe / بديل	`.a | .b // 0`
`Bearer`	نوع التوكن	`Authorization: Bearer …`
`eyJ`	بداية أي JWT (Base64url لـ {")	`eyJhbGciOi…`
`2XX`	أي كود في الـ 200s (OpenAPI)	`"2XX":`
`...`	بقية محذوفة في أمثلة الكتاب	`{"id":"4",...}`

جدول أ.2 كل رمز في الكتاب ومعناه

ملحق ب

قاموس المصطلحات

كل مصطلح مهم في الكتاب في سطر واحد، مترتب أبجدياً. ارجعه كل ما تقابل كلمة ومش فاكر معناها.

#

201 / 204
Created + Location / نجاح من غير Body.

400 / 422
طلب مش مقري / مقري بس محتواه مش مقبول.

401 / 403
مين انت؟ / عارفك بس لأ.

409 / 412
متعارض مع الحالة / الشرط (If-Match) فشل.

429
Too Many Requests: استنى (Retry-After) وأعد.

A

Access token
توكن قصير العمر بيتبعت مع كل طلب (غالباً Bearer).

API
عقد بين برنامجين: تبعت إيه، وترجع إيه، والأخطاء شكلها إيه.

API key
مفتاح بيعرّف مشروعك. مكانه Header، مش Query ولا كود Frontend.

Authentication
إثبات انت مين (401 لو فشل).

Authorization
مسموحلك تعمل إيه (403 لو فشل).

B

Base64url
Base64 بـ `-` و `_` ومن غير `=`، آمن في URLs (JWT والـ Cursors).

Basic auth
`user:pass` بـ Base64 في Header. مش تشفير.

Bearer
"اللي معاه التوكن يعدّي": `Authorization: Bearer …`.

Breaking change
تغيير بيكسر Clients شغالة: شيل Field، أو غيّر نوعه أو معناه.

C

Client Credentials
OAuth flow لسيرفر بيكلم API باسمه من غير مستخدم.

Consumer / Provider
اللي بيستخدم الـ API / اللي بيبنيه.

Content negotiation
اختيار الشكل بـ `Accept` و `Content-Type`.

Contract test
اختبار إن ردود السيرفر الحقيقية مطابقة للـ Spec.

CORS
قواعد المتصفح لطلبات من Origin تاني، والسيرفر بيسمح بهيدرز.

Cursor
علامة معتمة لـ "بعد العنصر ده" في الـ Pagination.

Custom method
عملية مش CRUD زي `POST /articles/42:publish`.

D

Deprecation
هيدر (RFC 9745) بيقول الحاجة دي قديمة من تاريخ معين.

Design-first
تكتب الـ Spec وتتفق عليه قبل الكود.

E

Endpoint
URL + Method بيعمل حاجة معينة.

Envelope
غلاف حوالين البيانات: `{data, nextCursor}`.

ETag
بصمة لنسخة الـ Resource، للـ Cache (304) وللـ If-Match (412).

Express
Framework لـ Node يبني APIs بسلسلة Middleware.

F

fetch
دالة HTTP المدمجة في المتصفح و Node.

G

GraphQL
API بـ Endpoint واحد والـ Client بيحدد الـ Fields اللي عايزها.

gRPC
RPC سريع بـ Protocol Buffers و HTTP/2، غالباً بين السيرفرات.

H

HATEOAS
الرد فيه لينكات للخطوات الجاية (مستوى 3 في Richardson).

HMAC
Hash بسر مشترك، بيثبت إن الرسالة مجتش متغيرة ومن صاحب السر.

HTTPie
أداة سطر أوامر لـ HTTP بصيغة أبسط من curl.

I

I-JSON
JSON بقيود أمان (RFC 7493): UTF-8، ومفيش Keys مكررة، وأرقام آمنة.

Idempotency-Key
هيدر بمفتاح فريد علشان إعادة POST متعملش حاجة مرتين.

Idempotent
مرة زي عشرة في أثرها على السيرفر.

If-Match
"نفّذ بس لو النسخة لسه دي"، وإلا 412.

J

jq
أداة سطر أوامر لقراية وتحويل JSON.

JSON
صيغة بيانات نصية: Object و Array و String و Number و Boolean و null.

JSON Merge Patch
PATCH بشكل الـ Resource، و null = امسح (RFC 7396).

JSON Patch
PATCH كقايمة عمليات op/path (RFC 6902).

JSON Pointer
مسار لقيمة جوه JSON: `/tags/0` (RFC 6901).

JSON Schema
وصف شكل JSON لـ Validation والتوثيق (2020-12).

JSONL
سطر = JSON. لـ Logs والبيانات الكبيرة.

JWT
توكن فيه Header و Payload وتوقيع، مقروء لأي حد ومختوم.

L

Link header
لينكات في هيدر (RFC 8288) زي `rel="next"`.

Lint
فحص الـ Spec ضد قواعد صحة و Best practices.

Lost update
تعديل بيمسح تعديل حد تاني من غير ما حد ياخد باله.

M

Middleware
Function في سلسلة Express بترد أو تنادي `next()`.

O

OAuth 2.0
إطار لإعطاء App صلاحية محددة على حسابك من غير الباسورد.

Offset pagination
"عدّي N وهات M". بسيط، بس بيكرر لما البيانات تتغير.

OIDC
OpenID Connect: طبقة Login فوق OAuth بـ ID token.

OpenAPI
مواصفة لوصف HTTP APIs في ملف YAML/JSON (اسمها القديم Swagger).

operationId
اسم فريد لعملية في OpenAPI.

Over-fetching
الرد فيه بيانات أكتر من اللي محتاجه.

P

PKCE
حماية للـ Authorization code بـ verifier و challenge.

Polling
تسأل كل شوية "في جديد؟".

Preflight
طلب OPTIONS المتصفح بيبعته قبل طلب CORS "مش بسيط".

Problem Details
شكل Errors موحّد `application/problem+json` (RFC 9457).

Q

QUERY
Method جديد (RFC 10008): بحث بـ Body وهو Safe.

R

Rate limit
حد لعدد الطلبات في فترة، و 429 لما تعدّيه.

Refresh token
توكن طويل بيجيب Access token جديد.

Representation
شكل الـ Resource اللي بيتبعت (JSON مثلاً).

Resource
"الحاجة" اللي ليها URL: `/articles/42` .

REST
أسلوب معماري بقيود (Stateless و Uniform interface...) من رسالة Fielding.

Retry-After
هيدر بيقول تستنى قد إيه قبل ما تعيد.

RFC 3339
صيغة تواريخ `2026-10-23T11:43:20Z` .

S

Safe
Method مبيغيرش حاجة (GET و HEAD).

Scope
صلاحية محددة في OAuth (`repo:read`).

SDK
مكتبة جاهزة بتلف الـ API في Functions.

SSE
Server-Sent Events: سيرفر بيبعت Events على اتصال HTTP مفتوح.

Stateless
كل طلب فيه كل اللي السيرفر محتاجه.

Status code
رقم 3 خانات بيلخّص نتيجة الطلب للآلة.

Sunset
هيدر (RFC 8594) بتاريخ وقف الحاجة نهائياً.

Swagger UI
أداة بتعرض OpenAPI كصفحة تجرّب منها.

T

timingSafeEqual
مقارنة بوقت ثابت ضد الـ Timing attacks.

U

Under-fetching
محتاج طلبات كتير علشان تجمع اللي عايزه.

URLSearchParams
كلاس JS لبناء وقراية Query strings بـ Encoding صح.

V

Versioning
طريقة تعيش بيها نسخ من الـ API: `/v1` أو Header بتاريخ.

W

Webhook
الـ Provider بيبعتلك POST لما حاجة تحصل.

WebSocket
اتصال مفتوح في الاتجاهين.

Y

YAML
صيغة زي JSON بالمسافات، شائعة لملفات OpenAPI.

ملحق ج

ملحق الروابط الرسمية

كل صفحة رسمية أو Spec اعتمد عليها الكتاب، مجمّعة حسب المصدر، وجنب كل لينك الفصول اللي استخدمته. اللينكات دي اتفتحت كلها واتأكدنا إنها شغالة يوم 23 أكتوبر 2026. الـ APIs العامة بتتغير، فلما تلاقي تعارض بين الكتاب والصفحة، الصفحة هي الصح.

ج.1 IETF: RFCs والـ Drafts

JSON · RFC 8259 · فصل 1، 4
https://www.rfc-editor.org/rfc/rfc8259

WebSocket · RFC 6455 · فصل 2
https://www.rfc-editor.org/rfc/rfc6455

Date and Time on the Internet · RFC 3339 · فصل 3
https://www.rfc-editor.org/rfc/rfc3339

§6 Numbers · RFC 8259 · فصل 4
https://www.rfc-editor.org/rfc/rfc8259#section-6

§8.1 Character Encoding · RFC 8259 · فصل 4
https://www.rfc-editor.org/rfc/rfc8259#section-8.1

I-JSON · RFC 7493 · فصل 4
https://www.rfc-editor.org/rfc/rfc7493

§5.6 Internet Date/Time Format · RFC 3339 · فصل 4
https://www.rfc-editor.org/rfc/rfc3339#section-5.6

JSON Pointer · RFC 6901 · فصل 6
https://www.rfc-editor.org/rfc/rfc6901

JSONPath · RFC 9535 · فصل 6
https://www.rfc-editor.org/rfc/rfc9535

Basic Authentication · RFC 7617 · فصل 12
https://www.rfc-editor.org/rfc/rfc7617

Bearer Token Usage · RFC 6750 · فصل 12
https://www.rfc-editor.org/rfc/rfc6750

JWT · RFC 7519 · فصل 12
https://www.rfc-editor.org/rfc/rfc7519

§4.1 Registered Claims · RFC 7519 · فصل 12
https://www.rfc-editor.org/rfc/rfc7519#section-4.1

§5 base64url · RFC 4648 · فصل 12
https://www.rfc-editor.org/rfc/rfc4648#section-5

JWT Best Current Practices · RFC 8725 · فصل 12
https://www.rfc-editor.org/rfc/rfc8725

RFC 6749 · §4.1 Authorization Code Grant · فصل 13
https://www.rfc-editor.org/rfc/rfc6749#section-4.1

RFC 7636 · PKCE · فصل 13
https://www.rfc-editor.org/rfc/rfc7636

RFC 9700 · OAuth 2.0 Security BCP · فصل 13
https://www.rfc-editor.org/rfc/rfc9700

OAuth 2.1 (draft) · فصل 13
https://datatracker.ietf.org/doc/draft-ietf-oauth-v2-1/

RFC 8628 · Device Grant · فصل 13
https://www.rfc-editor.org/rfc/rfc8628

RFC 9110 §9.2 · Common Method Properties · فصل 15
https://www.rfc-editor.org/rfc/rfc9110.html#name-common-method-properties

RFC 5789 · PATCH · فصل 15
https://www.rfc-editor.org/rfc/rfc5789.html

RFC 7396 · JSON Merge Patch · فصل 15
https://www.rfc-editor.org/rfc/rfc7396.html

RFC 6902 · JSON Patch · فصل 15
https://www.rfc-editor.org/rfc/rfc6902.html

RFC 6901 · JSON Pointer · فصل 15
https://www.rfc-editor.org/rfc/rfc6901.html

IETF Draft · The Idempotency-Key HTTP Header Field · فصل 15
https://datatracker.ietf.org/doc/draft-ietf-httpapi-idempotency-key-header/

RFC 10008 · The HTTP QUERY Method · فصل 15
https://www.rfc-editor.org/rfc/rfc10008.html

RFC 9110 §15 · Status Codes · فصل 16
https://www.rfc-editor.org/rfc/rfc9110.html#name-status-codes

RFC 6585 · 428 / 429 · فصل 16
https://www.rfc-editor.org/rfc/rfc6585.html

RFC 3339 · Date and Time on the Internet · فصل 16
https://www.rfc-editor.org/rfc/rfc3339.html

RFC 9457 · Problem Details for HTTP APIs · فصل 17
https://www.rfc-editor.org/rfc/rfc9457.html

IANA · HTTP Problem Types registry · فصل 17
https://www.iana.org/assignments/http-problem-types/http-problem-types.xhtml

RFC 8288 · Web Linking · فصل 18
https://www.rfc-editor.org/rfc/rfc8288.html

RFC 9745 · Deprecation Header · فصل 19
https://www.rfc-editor.org/rfc/rfc9745.html

RFC 8594 · Sunset Header · فصل 19
https://www.rfc-editor.org/rfc/rfc8594.html

IETF Draft · RateLimit header fields for HTTP · فصل 19
https://datatracker.ietf.org/doc/draft-ietf-httpapi-ratelimit-headers/

RFC 9110 · If-Match · فصل 19
https://www.rfc-editor.org/rfc/rfc9110.html#name-if-match

ج.2 JSON و JSON Schema و jq

json.org · فصل 1، 4
https://www.json.org/json-en.html

ECMA-404 · فصل 4
https://ecma-international.org/publications-and-standards/standards/ecma-404/

jsonlines.org · فصل 4
https://jsonlines.org/

jqlang.org · Download · فصل 6
https://jqlang.org/download/

jq 1.8 Manual · فصل 6
https://jqlang.org/manual/

jq Manual · Basic filters · فصل 6
https://jqlang.org/manual/#basic-filters

jq Manual · Builtin functions · فصل 6
https://jqlang.org/manual/#builtin-operators-and-functions

jq Manual · @csv @uri · فصل 6
https://jqlang.org/manual/#format-strings-and-escaping

json-schema.org · Specification (2020-12) · فصل 7
https://json-schema.org/specification

Understanding JSON Schema · فصل 7
https://json-schema.org/understanding-json-schema/

Understanding JSON Schema · type · فصل 7
https://json-schema.org/understanding-json-schema/reference/type

DummyJSON · Docs · فصل 9
https://dummyjson.com/docs

DummyJSON · Auth · فصل 12
https://dummyjson.com/docs/auth

JSON:API · Sorting · فصل 18
https://jsonapi.org/format/#fetching-sorting

JSON:API · Sparse Fieldsets · فصل 18
https://jsonapi.org/format/#fetching-sparse-fieldsets

ج.3 OpenAPI والأدوات حواليه

OpenAPI Specification (latest) · فصل 11
https://spec.openapis.org/oas/latest.html

OpenAPI Initiative · What is OpenAPI · فصل 11
https://www.openapis.org/what-is-openapi

Swagger UI · فصل 11
https://swagger.io/tools/swagger-ui/

openapi-typescript · فصل 11
https://openapi-ts.dev/

OpenAPI Specification 3.2.0 · فصل 21
https://spec.openapis.org/oas/v3.2.0.html

OpenAPI Initiative · Learn OpenAPI · فصل 21
https://learn.openapis.org/

Redocly CLI · lint · فصل 21
https://redocly.com/docs/cli/commands/lint

Redocly · recommended rules · فصل 21
https://redocly.com/docs/cli/rules/recommended

ج.4 MDN

MDN · Web APIs (بتوع المتصفح) · فصل 1
https://developer.mozilla.org/en-US/docs/Web/API

MDN Glossary · API · فصل 1
https://developer.mozilla.org/en-US/docs/Glossary/API

MDN · WebSockets API · فصل 2
https://developer.mozilla.org/en-US/docs/Web/API/WebSockets_API

MDN · Using server-sent events · فصل 2
https://developer.mozilla.org/en-US/docs/Web/API/Server-sent_events/Using_server-sent_events

MDN · ETag · فصل 3
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/ETag

MDN · JSON.stringify() · فصل 5
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/stringify

MDN · JSON.parse() · فصل 5
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON/parse

MDN · JSON · فصل 5
https://developer.mozilla.org/en-US/docs/Web/JavaScript/Reference/Global_Objects/JSON

MDN · Using the Fetch API · فصل 10
https://developer.mozilla.org/en-US/docs/Web/API/Fetch_API/Using_Fetch

MDN · Response · فصل 10
https://developer.mozilla.org/en-US/docs/Web/API/Response

MDN · Response.ok · فصل 10
https://developer.mozilla.org/en-US/docs/Web/API/Response/ok

MDN · AbortSignal.timeout() · فصل 10
https://developer.mozilla.org/en-US/docs/Web/API/AbortSignal/timeout_static

MDN · URLSearchParams · فصل 10
https://developer.mozilla.org/en-US/docs/Web/API/URLSearchParams

MDN · HTTP authentication · فصل 12
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/Authentication

MDN · WWW-Authenticate · فصل 12
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/WWW-Authenticate

MDN · CORS · فصل 13
https://developer.mozilla.org/en-US/docs/Web/HTTP/Guides/CORS

MDN · HTTP response status codes · فصل 16
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status

MDN · Retry-After · فصل 19
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Headers/Retry-After

MDN · 412 Precondition Failed · فصل 19
https://developer.mozilla.org/en-US/docs/Web/HTTP/Reference/Status/412

ج.5 Node و Express

Node.js 24 · fetch · فصل 10
https://nodejs.org/docs/latest-v24.x/api/globals.html#fetch

Node.js · crypto.createHmac · فصل 12
https://nodejs.org/docs/latest-v24.x/api/crypto.html#cryptocreatehmacalgorithm-key-options

Express · cors middleware · فصل 13
https://expressjs.com/en/resources/middleware/cors.html

Node.js · crypto.timingSafeEqual · فصل 20
https://nodejs.org/api/crypto.html#cryptotimingsafeequala-b

Express 5 · API reference · فصل 22
https://expressjs.com/en/5x/api/

Express · Using middleware · فصل 22
https://expressjs.com/en/guide/using-middleware/

Express · Migrating to Express 5 · فصل 22
https://expressjs.com/en/guide/migrating-5/

Express · Error handling · فصل 22
https://expressjs.com/en/guide/error-handling/

Express · Security best practices · فصل 22
https://expressjs.com/en/advanced/best-practice-security/

Express · Routing (express.Router) · فصل 22
https://expressjs.com/en/guide/routing/

ج.6 GitHub

GitHub GraphQL API · فصل 2
https://docs.github.com/en/graphql

GitHub · Getting started with the REST API · فصل 3
https://docs.github.com/en/rest/using-the-rest-api/getting-started-with-the-rest-api

GitHub · Troubleshooting (User-Agent required) · فصل 3
https://docs.github.com/en/rest/using-the-rest-api/troubleshooting-the-rest-api

GitHub · Rate limits for the REST API · فصل 3، 19
https://docs.github.com/en/rest/using-the-rest-api/rate-limits-for-the-rest-api

GitHub · Best practices · فصل 3
https://docs.github.com/en/rest/using-the-rest-api/best-practices-for-using-the-rest-api

GitHub · Get a repository · فصل 3، 11
https://docs.github.com/en/rest/repos/repos#get-a-repository

GitHub · List repository issues · فصل 11
https://docs.github.com/en/rest/issues/issues#list-repository-issues

Redoc · فصل 11
https://github.com/Redocly/redoc

Scalar · فصل 11
https://github.com/scalar/scalar

github/rest-api-description · فصل 11، 23
https://github.com/github/rest-api-description

GitHub · Token prefixes · فصل 12
https://docs.github.com/en/authentication/keeping-your-account-and-data-secure/about-authentication-to-github

jsonwebtoken · فصل 12
https://github.com/auth0/node-jsonwebtoken

jose · فصل 12
https://github.com/panva/jose

GitHub · Authorizing OAuth apps · فصل 13
https://docs.github.com/en/apps/oauth-apps/building-oauth-apps/authorizing-oauth-apps

Microsoft · API Guidelines · فصل 14
https://github.com/microsoft/api-guidelines/blob/vNext/azure/Guidelines.md

GitHub · Using pagination in the REST API · فصل 18
https://docs.github.com/en/rest/using-the-rest-api/using-pagination-in-the-rest-api

GitHub · API Versions · فصل 19
https://docs.github.com/en/rest/about-the-rest-api/api-versions

GitHub · Validating webhook deliveries · فصل 20
https://docs.github.com/en/webhooks/using-webhooks/validating-webhook-deliveries

GitHub · Best practices for webhooks · فصل 20
https://docs.github.com/en/webhooks/using-webhooks/best-practices-for-using-webhooks

GitHub · Redelivering webhooks · فصل 20
https://docs.github.com/en/webhooks/testing-and-troubleshooting-webhooks/redelivering-webhooks

GitHub · Handling webhook deliveries (smee) · فصل 20
https://docs.github.com/en/webhooks/using-webhooks/handling-webhook-deliveries

GitHub · REST API docs · فصل 23
https://docs.github.com/en/rest

ج.7 Stripe

Stripe · API keys · فصل 12
https://docs.stripe.com/keys

Stripe · Idempotent requests · فصل 15
https://docs.stripe.com/api/idempotent_requests

Stripe · Currencies (minor units) · فصل 16
https://docs.stripe.com/currencies

Stripe · Pagination · فصل 18
https://docs.stripe.com/api/pagination

Stripe · Versioning · فصل 19
https://docs.stripe.com/api/versioning

Stripe · Webhooks · فصل 20
https://docs.stripe.com/webhooks

Stripe · Signature verification · فصل 20
https://docs.stripe.com/webhooks/signature

Stripe CLI · فصل 20
https://docs.stripe.com/stripe-cli/use-cli

ج.8 أدوات ومكتبات

curl · man page · فصل 1
https://curl.se/docs/manpage.html

graphql.org · Learn · فصل 2
https://graphql.org/learn/

GraphQL Specification · فصل 2
https://spec.graphql.org/

graphql.org · Serving over HTTP · فصل 2
https://graphql.org/learn/serving-over-http/

Ajv · Getting started · فصل 7
https://ajv.js.org/guide/getting-started.html

Ajv · JSON Schema keywords · فصل 7
https://ajv.js.org/json-schema.html

curl man · --json · فصل 8
https://curl.se/docs/manpage.html#--json

curl man · -G · فصل 8
https://curl.se/docs/manpage.html#-G

curl man · --fail-with-body · فصل 8
https://curl.se/docs/manpage.html#--fail-with-body

curl man · --write-out · فصل 8
https://curl.se/docs/manpage.html#-w

Everything curl · فصل 8
https://everything.curl.dev/

HTTPie CLI docs · فصل 8
https://httpie.io/docs/cli

HTTPie · Request items · فصل 8
https://httpie.io/docs/cli/request-items

Bruno docs · فصل 8
https://docs.usebruno.com/

Postman docs · فصل 8
https://learning.postman.com/docs/introduction/overview/

Postman pricing · فصل 8
https://www.postman.com/pricing/

Postman Echo · فصل 9
https://learning.postman.com/docs/developer/echo-api/

oauth.net/2 · فصل 13
https://oauth.net/2/

OpenID · How Connect works · فصل 13
https://openid.net/developers/how-connect-works/

ج.9 APIs عامة ومراجع تانية

Frankfurter · Docs · فصل 1
https://frankfurter.dev/

Fielding (2000) · Architectural Styles... (REST) · فصل 1
https://ics.uci.edu/~fielding/pubs/dissertation/top.htm

gRPC · Introduction · فصل 2
https://grpc.io/docs/what-is-grpc/introduction/

formulae.brew.sh · jq · فصل 6
https://formulae.brew.sh/formula/jq

formulae.brew.sh · httpie · فصل 8
https://formulae.brew.sh/formula/httpie

Insomnia docs · فصل 8
https://developer.konghq.com/insomnia/

Chrome DevTools · Network features reference · فصل 8
https://developer.chrome.com/docs/devtools/network/reference

httpbin.org · فصل 9
https://httpbin.org/

JSONPlaceholder · Guide · فصل 9
https://jsonplaceholder.typicode.com/guide/

PokeAPI · Docs v2 · فصل 9
https://pokeapi.co/docs/v2

Rick and Morty API · Docs · فصل 9
https://rickandmortyapi.com/documentation

Open-Meteo · Docs · فصل 9
https://open-meteo.com/en/docs

Open-Meteo · Terms (حدود الاستخدام المجاني) · فصل 9
https://open-meteo.com/en/terms

api.nasa.gov · فصل 9
https://api.nasa.gov/

Frankfurter · openapi.json · فصل 11
https://api.frankfurter.dev/v2/openapi.json

Fetch Standard · CORS protocol · فصل 13
https://fetch.spec.whatwg.org/#http-cors-protocol

Fielding · Chapter 5: REST · فصل 14
https://ics.uci.edu/~fielding/pubs/dissertation/rest_arch_style.htm

Martin Fowler · Richardson Maturity Model · فصل 14
https://martinfowler.com/articles/richardsonMaturityModel.html

Google AIP-122 · Resource names · فصل 14
https://google.aip.dev/122

Google AIP-136 · Custom methods · فصل 14
https://google.aip.dev/136

Zalando · RESTful API Guidelines · فصل 14
https://opensource.zalando.com/restful-api-guidelines/

Google AIP-140 · Field names · فصل 16
https://google.aip.dev/140

ملحق د

المصادر ومنهجية التحقق

إزاي اتجمعت المعلومات في الكتاب ده، وإيه اللي اتجرب فعلاً وإيه اللي متجربش، علشان تعرف تثق في إيه وتراجع إيه بنفسك.

د.1 تاريخ اللقطة

كل المعلومات اتاخدت من المصادر الرسمية (RFCs، و MDN، و json.org، و OpenAPI spec، و Docs بتاعة GitHub و Stripe و Express) يوم **23 أكتوبر 2026**، وكل الـ 160 لينك اللي في ملحق الروابط اتفتحوا في نفس اليوم. الـ APIs العامة بتتغير (زي ما حصل مع REST Countries)، فلو رد عندك اختلف عن الكتاب، الرد الحقيقي هو الصح.

د.2 النسخ اللي اتجربت عليها

- Node.js 24.22.1 (LTS)، و Express 5.3.0، و zod 4، و ajv 8.20.0، و ajv-formats 3.0.1، و yaml 2.9.1.
- curl و OpenSSL 3.5.7، و jq 1.8.2، و HTTPie 3.2.4.
- Redocly CLI 2.70.0، و openapi-typescript 7.13.0 (بـ `npx`).
- OpenAPI 3.2.0 للـ Specs اللي كتبناها، و JSON Schema 2020-12.

د.3 إيه اللي اتجرب وإزاي

- **كل الطلبات للـ APIs العامة** (GitHub، و Stripe من غير Key، و JSONPlaceholder، و DummyJSON، و PokeAPI، و Open-Meteo، و Frankfurter، و NASA، و REST Countries، و GraphQL countries، و httpbin) اتبعتت فعلاً، والردود في الكتاب حقيقية (أحياناً مقصوصة بـ `...`).
- **كل السيرفرات والسكريبتات** (`server.js` و `library.js` و `hook.js` و `contract.mjs` و `report.mjs` وغيرهم) اتشغّلت، والخرج في الكتاب منها. `library.js` اتعدّل خلال الفصول، وكل رد مكتوب من النسخة اللي كانت شغالة وقتها.
- الـ JWT و HMAC و PKCE اتأكدنا منهم بالـ Test vectors الرسمية (GitHub و RFC 7636).
- **البيئة Linux مش macOS.** أوامر الـ Mac (brew، و `date -r`، و jq المدمج في macOS، و LibreSSL) مكتوبة من الدوكيومنتيشن. وفيه فروق صغيرة متوقعة، زي رسالة خطأ DNS (على الـ Mac غالباً `ENOTFOUND`، وفي بيئتنا ظهر `ECONNRESET`) وشكل خرج `openssl dgst`.

د.4 حاجات لسه Draft أو جديدة

- هيدرز `RateLimit` و `Idempotency-Key` لسه IETF Drafts، وممكن تتغير قبل ما تبقى RFC.
- OAuth 2.1 لسه Draft. والـ QUERY method (RFC 10008) جديد، والدعم في الأدوات لسه بيتبني.
- OpenAPI 3.2 جديد، وبعض الأدوات ممكن تكون لسه على 3.1. Redocly و openapi-typescript اشتغلوا معاه في تجاربنا.

د.5 ملاحظات لقيناها وإحنا بنجرّب

- GitHub مبيبعتش `WWW-Authenticate` مع الـ 401، ونسخة API مش موجودة بترجّع 400 (والـ Docs بتقول نسخة اتقفلت = 410).
- Express بيرجّع 404 مش 405 للـ Method الغلط، وبيحط Weak ETag أوتوماتيك.
- REST Countries v3.1 اتقفلت، و Frankfurter v1 مفيهاش الجنيه.

ملحق ه

الإجابات والحلول

جاوب بنفسك الأول، وبعدين قارن. لكل فصل: إجابات أسئلة المراجعة، وبعدها صح ولا غلط، وبعدها مفاتيح التحديات. أغلب التحديات تجارب عملية، فلو وصلت لنفس النتيجة بطريقة تانية فهي صح.

الفصل 1: حكاية الـ API

1. الـ Endpoint (العنوان)، والـ Input المطلوب (Parameters و Body)، وشكل الـ Output، والأخطاء الممكنة وأكوادها. (وكمان الـ Auth والحدود.)
2. لأن الـ Shell بيعتبر `&` "شغّل الأمر في الخلفية"، فالـ URL بيتقطع عندها والباقي بيتنفذ كأمر تاني. علامات التنصيص بتخلي الـ URL كله كلمة واحدة.
3. Library API بتنادي Functions جوه نفس البرنامج (`document.querySelector` أو `Array.prototype.map`). Web API بتكلّم برنامج تاني على الشبكة بـ HTTP (Frankfurter و GitHub).
4. إن الـ Consumers متفقين معاه على العقد بس (الـ URLs وشكل الـ JSON)، مش على الكود جوه. فطالما العقد ثابت، يقدر يغيّر اللغة والـ Database براحته.

صح ولا غلط: (1) غلط: أي تغيير في الـ HTML بيكسره، والـ API عقد ثابت. · (2) صح: دي Browser API (Library API). · (3) غلط: Web APIs في MDN هي APIs المتصفح (fetch و DOM)، مش بناء سيرفر.

التحديات:

1. نفس الشكل بس الأرقام بالـ EUR.
2. مثلاً `login` و `id` و `public_repos` و `followers` و `created_at`.
3. v1 مفيهاش EGP وشكل الرد مختلف (base و rates)، والـ Docs بتوصّي بـ v2.

الفصل 2: أنواع الـ APIs

1. Over-fetching: محتاج اسم الدولة بس والرد فيه 50 Field.
   Under-fetching: محتاج الدولة وعملتها وجيرانها فبتعمل 3 طلبات.
2. لأن كل Resource في REST ليه URL و GET، فالـ Cache بيخزّن بالـ URL. في GraphQL كله POST على URL واحد.
3. Polling = انت بتسأل كل شوية. Webhook = هو بيبعتلك.
   Polling لما الخدمة مفيهاش Webhooks أو سيرفرك مش متاح من برّه.
4. SSE: البيانات رايحة اتجاه واحد (من السيرفر)، وشغالة على HTTP عادي وبتعيد الاتصال لوحدها.

صح ولا غلط: (1) غلط: Endpoint واحد (`/graphql`). · (2) غلط: الأخطاء بترجع جوه `errors` مع 200. · (3) صح.

التحديات:

1. `{ country(code: "EG") { name continent { name } } }`
.
2. `query($c: ID!) { country(code: $c) { name } }` وغيّر `variables`.
3. `/countries/EG` ثم `/countries/EG/languages`... يعني طلبين أو أكتر بدل واحد.

الفصل 3: تشريح طلب ورد

1. `Accept` في الطلب: "عايز الرد بالشكل ده". `Content-Type`: "الـ Body اللي معايا بالشكل ده"، ويبتبعت في الطلب (لو فيه Body) وفي الرد.
2. لأن المحتوى ممكن يكون اتغير فعلاً، أو الطلب مختلف في Header موجود في `Vary` (زي Accept أو Authorization) فالـ ETag بتاع نسخة تانية مش هيطابق.
3. غالباً الـ URL غلط أو في Proxy/Login page أو Error page من سيرفر تاني. اقرا أول سطور النص.
4. ID للطلب ده بالذات، تبعته للـ Support علشان يلاقوه في الـ Logs.

صح ولا غلط: (1) غلط: 304 من غير Body. · (2) غلط: ده Unix timestamp لوقت التصفير. · (3) غلط: الاسم ممكن يتغير (Rename)، و `id` ثابت.

التحديات:

1. `remaining` بيقل 1 مع كل طلب.
2. `curl -i -H 'If-None-Match: "…"'` بالـ ETag من الرد الأول.
3. رجّع 415، والمتوقع حسب HTTP هو 406 Not Acceptable.

الفصل 4: JSON بالتفصيل

1. `.5` ← `0.5` ، `+1` ← `1` ، `01` ← `1` ، `NaN` ← `null` ، `0x1F` ← `31` .
2. لأن الأرقام في JS بتفقد الدقة بعد 2^{53}، فالـ ID ممكن يتغير وهو بيتقري.
3. آخر قيمة بتكسب. والـ RFC بيقول السلوك مش متوقع بين الـ Parsers، فممكن سيستم تاني ياخد الأولى.
4. `.json` قيمة واحدة. `.jsonl` كل سطر JSON لوحده، فتقدر تزوّد سطر وتقرا سطر سطر.

صح ولا غلط: (1) صح: من RFC 8259 أي قيمة ينفع تبقى JSON text. · (2) غلط: لازم `0.99` . · (3) غلط: UTF-8 عادي مسموح، و `\u` اختياري.

التحديات:

1. مثلاً
```
{"title": "...", "year": 2024, "authors": ["..."], "available": true, "price": 250, "isbn": "9780..."}
```
. والـ ISBN String لأنه كود مش رقم (وممكن يبدأ بصفر أو فيه X).
2. `"a \"q\"\nb \\ \u0645"` .
3. مسموح للـ Parser يحط حدود (حجم، وعمق، وطول أرقام)، فمتفترضش إن أي JSON صحيح هيتقري في كل مكان.

الفصل 5: JSON في JavaScript

1. `undefined` بيختفي، و `null` بيتبعت. للسيرفر: الغياب = "متغيرش"، و null = "امسح" (Merge Patch).
2.
```
JSON.stringify(user, (k, v) => k === "password" ? undefined : v)
```
.
3. الـ Body فاضي (204 أو Error من غير Body) أو الرد اتقطع.
4. لأن أي `"` أو سطر جديد في القيم هيكسر الـ JSON، وممكن حد يحقن Fields.

صح ولا غلط: (1) غلط: `'{}'` . · (2) غلط: بيرجّع String. · (3) غلط: JSON مفيهوش تعليقات (npm بيرفضه).

التحديات:

1. الرسايل بتطابق الجدول.
2.
```
if (!res.headers.get("content-type")?.includes("json")) throw new Error((await res.text()).slice(0,100))
```
.
3. Regex زي `/^\d{4}-\d{2}-\d{2}T/` ، والخطر إن نص عادي يتحول لـ Date غلط. الأحسن تحوّل Fields معروفة بالاسم.

الفصل 6: jq

1. `.[]` بيطلّع العناصر واحدة واحدة (Stream). `[ … ]` بيجمّع اللي جواه في Array.
2.
```
.[] | select(.email | endswith(".biz")) | .name
```
.
3. لأن `--arg` بيعمل Escape صح لأي قيمة، فمفيش JSON مكسور.
4. لما عايز النص من غير علامات تنصيص (علشان تستخدمه في Shell أو ملف).

صح ولا غلط: (1) غلط: ده Alternative operator (قيمة بديلة). · (2) غلط: بيرجّع null. · (3) غلط: String، والرقم بـ `--argjson` .

التحديات:

1.
```
jq -r '.abilities[].ability.name' pika.json
```
.
2.
```
group_by(.userId) | map({user: .[0].userId, percent: (map(select(.completed)) | length) * 100 / length})
```
.
3.
```
jq -r '.[] | [.number, .title, .user.login, ([.labels[].name] | join(";"))] | @csv'
```
.

الفصل 7: JSON Schema

1. توثيق، و Types، و Forms، و Mock data، وعقد بين الفرق، و Structured output للموديلات.

2. `{"type":"object","required":["email"],"properties":{"email":{"type":"string","format":"email"},"age":{"type":"integer","minimum":13,"maximum":120}}}`

.

3. علشان الـ Client ياخد كل الأخطاء مرة واحدة ويصلّحها كلها.
4. `anyOf` : واحد على الأقل يطابق. `oneOf` : واحد بس بالظبط.

صح ولا غلط: (1) غلط: الإجباري في `required` . · (2) صح. · (3) غلط: Annotation بس، إلا لو المكتبة اتظبطت تحطه (useDefaults).

التحديات:

1. زي الفصل: الملف الصح بيعدّي والغلط بيطلّع 7 أخطاء.
2. كل الـ 200 بيعدّوا.
3. هيفشل لأن `lat` String في JSONPlaceholder. يعني البيانات الوهمية مش متصممة بعناية للأنواع.

الفصل 8: curl و HTTPie والأدوات

1. `curl --json '{"title":"Hi","userId":1}' URL`

و `http POST URL title=Hi userId:=1` .

2. الاتنين Exit code 22، بس `--fail-with-body` بيطبع الـ Body (فيه رسالة الخطأ).
3. لأنه بيعمل Encoding للمسافات و `&` والعربي صح.
4. فيه Cookies وتوكنات الجلسة بتاعتك.

صح ولا غلط: (1) غلط: form-urlencoded. · (2) غلط: String، والرقم بـ `:=` . · (3) صح.

التحديات:

1. نفس البيانات، و HTTPie بيلوّن.
2. غالباً Cookie واحدة أو Header Authorization كفاية.
3. ملفات `.bru` نصية تقدر ترفعها على Git.

الفصل 9: APIs للتمرين

1. Open-Meteo بيقولك بالظبط إيه الغلط بـ `reason` . JSONPlaceholder برجّع `{}` من غير شرح.
2. لأن السيرفر بيحدد شكل اللينك وانت متعرفش هيتغير إزاي، وبتمشي وراه بس.
3. 401 (مين انت؟) مع رسالة بتقولك تجيب Key منين، أو زي ما عملوا لو مخليين DEMO_KEY.

4. Deprecation و Sunset قبلها بفترة، وإيميلات، و 410 واضح بلينك Migration.

صح ولا غلط: (1) غلط: بيقلّد بس. · (2) غلط: offset ضد page. · (3) صح.

التحديات:

1. `latitude=31.2&longitude=29.92&current=temperature_2m&timezone=Africa/Cairo`

.

2. Loop على `info.next` لحد null، وعدّ `status === "Dead"` .
3. هتلاقي كل واحد بشكل، وده سبب RFC 9457 (فصل 17).

الفصل 10: fetch في Node والمتصفح

1. بتتـ Reject لو الشبكة فشلت (DNS، أو اتصال، أو Timeout، أو CORS). مش بتتـ Reject على xx4 و xx5.
2. لأن 204 مفيهوش Body، و `res.json()` هيرمي Error.
3. `req.body` هيبقى `undefined` لأن `express.json()` مش هيقراه.
4. أي حد يقدر يفتح DevTools ويشوفه.

صح ولا غلط: (1) غلط. · (2) غلط: لازم `JSON.stringify` . · (3) غلط: Body is unusable.

التحديات:

1. `503 false` من غير Error.
2. اقرا `Retry-After` كثواني، و `await new Promise(r => setTimeout(r, s*1000))` ثم أعد.
3. `async function* paginate(url){ while(url){ const d = await (await fetch(url)).json(); yield* d.results; url = d.info.next; } }`

.

الفصل 11: قراية الـ Docs

1. لأنهم بيحددوا هتكتب الكود إزاي (Retries وتعامل مع الأخطاء) قبل أي Endpoint.
2. Path لهوية الـ Resource (`/users/42`)، و Query للفلترة والخيارات (`?status=open`).
3. اسم فريد للعملية، والأدوات بتعمل منه أسماء Functions.

4. `jq -r '.paths[][] | .operationId? // empty' fr.json`

.

صح ولا غلط: (1) غلط: Swagger الاسم القديم و OpenAPI الاسم الجديد للمواصفة، و Swagger فضل اسم أدوات. · (2) غلط: دايماً `required: true` . · (3) غلط: دي نسخة الـ API، ونسخة المواصفة في `openapi` .

التحديات:

1. Base URL، و Auth، والحدود، وأهم Endpoints، والأخطاء... إلخ.
2. `base` و `quote` الاتنين `in: path` .
3. الـ Collection بتطلع بكل الـ Endpoints جاهزة.

الفصل 12: الـ Auth

1. بيتسجل في Logs السيرفرات والـ Proxies، و History المتصفح، و Referer header.
2. HS256: Secret واحد بيوقّع ويتحقق. RS256: Private key بيوقّع و Public key أي حد يتحقق بيه.
3. علشان لو اتسرب يبوظ بسرعة. والـ Refresh token بيجيب Access token جديد من غير Login.
4. توكن بيقول `alg: none` من غير توقيع. حدّد الـ Algorithm المسموح في الـ verify ومتاخدهوش من التوكن.

صح ولا غلط: (1) غلط: Base64 بس. · (2) صح. · (3) غلط: الختم مش هيطابق.

التحديات:

1. فوراً 200، وبعد ما يخلص 401 و `{"message":"Token Expired!"}` (جربناها بـ 70 ثانية).
2. `exp` = `Math.floor(Date.now()/1000) - 60` .
3. قارن `header.alg === "HS256"` و `payload.aud === "articles-api"` قبل التوقيع.

الفصل 13: OAuth و CORS

1. `state` ضد CSRF (رد مش من طلبك). PKCE ضد سرقة الـ code (لازم الـ verifier الأصلي).
2. لأن الـ URL بيتسجل ويتسرب، والتبديل بيحصل في طلب مباشر (ومعاه PKCE أو Secret).
3. Client Credentials: مفيش مستخدم، السيرفر بيتكلم باسمه.
4. من غيره الـ JS في المتصفح مش هيقدر يقرا `Link` .

صح ولا غلط: (1) غلط: عام. · (2) صح. · (3) غلط: CORS قيد في المتصفح بس.

التحديات:

1. `createHash("sha256").update(verifier).digest("base64url")` .
2. هتلاقي `response_type=code` و `scope` و `state` وغالباً `code_challenge` .
3. GitHub بيرجّع CORS مفتوح فبتشتغل، بس الهيدر ممكن ميتقريش لو مش في Expose. و httpbin مع Authorization بيعمل Preflight.

الفصل 14: REST والتسمية

1. لأن GET المفروض Safe، فأي Crawler أو Prefetch أو Cache ممكن يبعته ويمسح بيانات.
2. Resource = المقالة 42 كفكرة. Representation = الـ JSON أو الـ HTML اللي بيوصل لها.
3. PATCH لتغيير حالة بسيط. `:publish` لما فيه شغل وآثار (Validation وإيميلات و Webhooks).
4. التعليق ليه ID لوحده، والمسار الطويل بيتكسر لو العلاقة اتغيرت.

صح ولا غلط: (1) غلط. · (2) غلط: Stateless = كل طلب فيه كل اللي محتاجه، والـ Database عادي. · (3) صح.

التحديات:

1. زي جدول "قواعد التسمية" في الفصل.
2. `/lists` و `/lists/{id}/items` و `position` في كل Item، وترتيب بـ PATCH.
3. مثلاً Endpoints فيها أفعال زي `/repos/{o}/{r}/merges` : اختاروا وضوح العملية.

الفصل 15: الـ Methods

1. PUT بيقول "خليها كده" فتكراره نفس النتيجة. POST بيقول "اعمل جديدة" فتكراره = مقالتين.
2. Merge Patch: ابعت الـ Array كلها بالـ JSON Patch. 6: `{"op":"add","path":"/tags/-","value":"x"}` .
3. 409 Conflict (حسب الـ Draft).
4. بحث بـ Body وفي نفس الوقت Safe و Idempotent ويتخزّن.

صح ولا غلط: (1) غلط: الأثر المهم مش الرد. · (2) غلط: بيمسح الـ Field. · (3) غلط: نفس المفتاح لكل الإعادات.

التحديات:

1. PATCH بيحافظ على الباقي، و PUT بيرجّع اللي بعتّه بس.
2. `[{"op":"test","path":"/status","value":"draft"},{"op":"replace","path":"/title","value":"..."},{"op":"remove","path":"/tags/0"}]`
.
3. Redis أو جدول بـ TTL 24 ساعة، وحالة "processing" ترجّع 409.

الفصل 16: Status و Body

1. (أ) 409. (ب) 415. (ج) 202. (د) 400.
2. علشان الـ Client يعرف يتابع حالة الشغل منين.
3. تزوّده جنب `data` من غير ما تكسر حد.
4. `null`: معروف إنه مفيش (مش منشورة). مش موجود: مش متاح في الرد ده.

صح ولا غلط: (1) غلط: 403. · (2) غلط: 200 و `[]`. · (3) غلط: Integer بالقروش + عملة.

التحديات:

1. مثلاً 400 و 404 و 409 و 415 و 422.
2. `{"id":"7","name":"...","role":"editor","articleCount":12,"isActive":true,"joinedAt":"2026-...Z","bio":null,"tags":[]}`
.
3. snake_case، و null كتير (`mirror_url`)، و Objects متداخلة (`owner`).

الفصل 17: الـ Errors

1. علشان لو الـ Body اتنقل أو اتسجل لوحده يفضل مفهوم.
2. `title` ثابت لكل نوع، و `detail` للمرة دي بالذات.
3. `{"type":".../insufficient-balance","title":"Not enough balance","status":403,"balance":30,"cost":50}`
.
4. علشان تربط شكوى المستخدم بالـ Log من غير ما تكشف التفاصيل.

صح ولا غلط: (1) غلط. · (2) صح. · (3) غلط.

التحديات:

1. `{"type":"about:blank","title":"Not Found","status":404,"detail":"..."}` و `application/problem+json`.
2. لو `p.status === 429` اقرا `Retry-After` واستنى وأعد مرة.
3. validation و article-not-found و already-published و precondition و rate-limited و forbidden-role.

الفصل 18: الـ Pagination

1. صفحة 1 = أول 2 (5 و 4). الـ 6 دخل في الأول، فالـ offset 2 بقى بيشاور على 4 تاني.
2. جداول صغيرة أو Admin محتاج أرقام صفحات وقفز.
3. يحمي السيرفر من `limit=1000000`.
4. في الـ Body (`next` عند PokeAPI) أو هيدر `Link` (GitHub).

صح ولا غلط: (1) صح. · (2) غلط: معتم. · (3) غلط.

التحديات:

1. `offset=0,50,100,150` و `limit=50` مع آخر واحد `limit=1`، أو `limit=151` في طلب واحد.
2. هيفوّت عنصر.
3. مقالتين بنفس `createdAt` ممكن واحدة تتكرر أو تتفوّت. الحل Cursor فيه `(createdAt, id)`.

الفصل 19: التطوير والحدود

1. Deprecation (@timestamp) = اتعلن قديم. Sunset (HTTP-date) = هيقف.
2. علشان الكود القديم ميتكسرش لوحده.
3. اتنين فتحوا المقالة، والتاني حفظ فوق الأول. `If-Match` بيرفض الحفظ القديم بـ 412.
4. كل سيرفر بيعد لوحده، فالحد الحقيقي يبقى 3 أضعاف.

صح ولا غلط: (1) غلط. · (2) غلط: ثواني من دلوقتي. · (3) صح.

التحديات:

1. `date -r 1792758101`.
2. GET ← ETag بـ PATCH ← لو 412 GET تاني واعرض.

3. زوّد `author` جنب `authorId`، و Deprecation على `authorId` في الـ Docs والهيدرز، و Sunset بعد مهلة طويلة، ثم شيله في v2.

الفصل 20: الـ Webhooks

1. 5 × 120 = 600 في الساعة، والحد 60.
2. اللي معاه السر يقدر يزوّر أي Delivery. غيّره فوراً (Rotate).
3. التوقيع بيشمل الوقت، فالرسالة القديمة بترفض بعد 5 دقايق.
4. لأن `-d` بيشيل الـ Newlines فالبايتات تتغير.

صح ولا غلط: (1) غلط: البايتات اتغيرت. · (2) غلط: Timing attack. · (3) غلط: يرد بسرعة.

التحديات:

1. `false`.
2. `if (event !== "issues" || payload.action !== "opened") return res.status(204).end()` بعد التحقق.
3. smee-client بيحوّل لـ `localhost:4000/webhooks/github`.

الفصل 21: كتابة OpenAPI

1. اسم ثابت للعملية، والأدوات بتعمل منه Functions.
2. علشان متكررش الـ Fields، وأي تعديل يحصل في مكان واحد.
3. Lint بيراجع الـ Spec نفسه. Contract test بيراجع إن السيرفر ملتزم بيه.
4. Design-first: الـ Frontend يبدأ على Mock و Types من الـ Spec.

صح ولا غلط: (1) غلط. · (2) غلط: "مفيش Auth" عن قصد. · (3) صح.

التحديات:

1. `examples` في الـ Schema وهيظهر في الـ Docs.
2. Path جديد و Schema `Author`، و `404` بـ Problem.
3. `test("422", ..., "ValidationProblem")` بطلب POST غلط.

الفصل 22: الجسر لـ Express

1. لأن Express بيمشي بالترتيب، فلو قبلهم هيرد على كل حاجة.
2. الـ Body هيتقري ويتحول، والتوقيع هيفشل.
3. لأن الـ Weak ETag مينفعش مع If-Match، فعملنا Strong بإيدنا.
4. مثلاً: Auth، ثم Database، ثم Tests.

صح ولا غلط: (1) غلط: String. · (2) غلط: 4 بالظبط. · (3) صح.

التحديات:

1. HTTPie: `http :3001/v1/articles limit==3`.
2. `const router = express.Router()` و `app.use("/v1/articles", router)`.
3. نفس `problem()` و `etagOf()`.

الفصل 23: مشروع 1

1. من `rel="last"` و `page=56` و `per_page=3`.
2. علشان أي Script أو CI يعرف إنه فشل.
3. `status` String، ومفيش `type`، والـ Content-Type `application/json`.
4. علشان التوكن ميتكتبش في الكود ولا يترفع على Git.

صح ولا غلط: (1) غلط: Search عدّاد لوحده 10 في الدقيقة. · (2) غلط: فيها PRs. · (3) صح.

التحديات:

1. أرقام مختلفة، ونفس الشكل.
2. Loop على `Link` واعمل `filter(i => !i.pull_request)`.
3. احفظ `{etag, body}` في ملف، ولو 304 اطبع "from cache".

الفصل 24: مشروع 2

1. علشان متكشفش إن الـ Draft موجود.
2. لأن فيه Validation وتواريخ و Webhook، أكبر من تغيير Field.
3. يمكن OAuth بـ PKCE، و Fields أقل (`fields=`)، ونسخ أقدم من الـ App لازم تفضل شغالة.
4. مثلاً Pagination من أول يوم، و Errors بـ RFC 9457، و If-Match.

صح ولا غلط: (1) غلط: 403. · (2) صح. · (3) صح.

التحديات:

1. Path `/tags` و `/tags/{slug}` و Schema `Tag`.
2. Webhook `commentCreated` بنفس شكل الأول.
3. GitHub بيعدّي معظم النقاط، والـ Errors مش RFC 9457.

دلوقتي الـ API مش صندوق أسود

بقيت تقرا أي طلب ورد Header بـ Header، وتفهم JSON لحد الأرقام والـ Escapes، وتكلّم أي API بـ curl و fetch، وتعرف الـ Auth و OAuth بيشتغلوا إزاي. والأهم: بقيت عارف تصمم API بنفسك، من أسماء الـ Resources لحد الـ Errors والـ Pagination والـ Webhooks، وتكتبه OpenAPI. ادخل كورس Maximilian وانت عارف كل سطر في Express بيعمل إيه وليه.

```
$ curl -si localhost:3001/v1/articles/4 → HTTP/1.1 200 OK · ETag: "39fqqzHpm_icenp6"
```